



ROBOTIC MANIPULATION

Perception, Planning, and Control

Russ Tedrake

© Russ Tedrake, 2020-2025

Last modified 2026-5-18.

[How to cite these notes, use annotations, and give feedback.](#)

Note: These are working notes used for [a course being taught at MIT](#). They will be updated throughout the Fall 2025 semester.

PDF VERSION OF THE NOTES

The PDF version of these notes are autogenerated from the HTML version. There are a few conversion/formatting artifacts that are easy to fix (please feel free to point them out). But there are also interactive elements in the HTML version are not easy to put into the PDF. When possible, I try to provide a link. But I consider the [online HTML version](#) to be the main version.

目录

- [前言](#)
- [第 1 章：简介](#)
 - [操作不仅仅是抓取与放置](#)
 - [开放世界操作](#)
 - [仿真](#)
 - [这些笔记是交互式的](#)
 - [基于模型的设计与分析](#)
 - [本笔记的组织结构](#)
 - [练习](#)
- [第 2 章：让我们为你准备一台机器人](#)
 - [机器人描述文件](#)
 - [机械臂](#)
 - 位置控制机器人
 - 位置控制。
 - 补充说明：带传动机构的连杆动力学。
 - 力矩控制机器人
 - 硬件设备的激增
 - Kuka iiwa 的仿真
 - [机械手](#)
 - 灵巧手
 - 简单夹爪
 - 柔性/欠驱动机械手
 - 其他末端执行器
 - 如果你还没见过的话.....
 - [传感器](#)
 - [整合所有组件](#)
 - HardwareStation
 - HardwareStationInterface
 - HardwareStation 独立仿真

- [更多 HardwareStation 示例](#)
 - [练习](#)
- [第 3 章：基础抓取与放置](#)
 - [字母符号表示法](#)
 - [通过空间变换实现抓取与放置](#)
 - [空间代数](#)
 - 三维旋转的表示方法
 - [正向运动学](#)
 - 运动学树
 - 用于抓取与放置的正向运动学
 - [微分运动学（雅可比矩阵）](#)
 - [微分逆运动学](#)
 - 雅可比伪逆
 - 雅可比矩阵的可逆性
 - [定义抓取与预抓取位姿](#)
 - [抓取与放置轨迹](#)
 - [整合所有组件](#)
 - [带约束的微分逆运动学](#)
 - 作为优化问题的伪逆
 - 添加速度约束
 - 添加位置与加速度约束
 - 关节居中
 - 跟踪目标位姿
 - 其他形式
 - [练习](#)
- [第 4 章：几何位姿估计](#)
 - [相机与深度传感器](#)
 - 深度传感器
 - 仿真
 - [几何表示方法](#)
 - [已知对应关系的点云配准](#)
 - [迭代最近点（ICP）](#)
 - [处理局部视角与离群点](#)
 - 检测离群点
 - 点云分割
 - 推广对应关系
 - 软对应关系
 - 非线性优化
 - 预计算距离函数
 - 全局优化
 - [非穿透与“自由空间”约束](#)
 - 将自由空间约束视为非穿透约束
 - [跟踪](#)
 - [整合所有组件](#)
 - [展望未来](#)
 - [练习](#)
- [第 5 章：料箱抓取](#)
 - [生成随机杂乱场景](#)
 - 掉落物体
 - [带摩擦接触的静力平衡](#)
 - 空间力
 - 碰撞几何

- 碰撞物体之间的接触力
 - 接触坐标系
 - (库仑) 摩擦锥
 - 作为优化问题的静力平衡
- 接触仿真
- 基于模型的抓取选择
 - 接触力矩锥
 - 共线对向抓取
- 基于点云的抓取选择
 - 点云预处理
 - 估计法向量与局部曲率
 - 评估候选抓取
 - 生成抓取候选
- 边界情况
- 任务层编程
 - 状态机与行为树
 - 任务规划
 - 大型语言模型
 - 用于“清理杂乱环境”的简单状态机
- 整合所有组件
- 练习
- 第 6 章: 运动规划
 - 逆运动学
 - 从末端执行器位姿到关节角
 - 作为约束优化的 IK
 - 全局逆运动学
 - 逆运动学与微分逆运动学
 - 使用逆运动学进行抓取规划
 - 运动学轨迹优化
 - 轨迹参数化
 - 优化算法
 - 基于采样的运动规划
 - 快速探索随机树 (RRT)
 - 概率路线图 (PRM)
 - 后处理
 - 实际中的采样规划
 - 基于凸集图 (GCS) 的运动规划
 - 凸集图
 - GCS (运动学) 轨迹优化
 - (无碰撞) 构型空间的凸分解
 - GcsTrajOpt 示例
 - 变体与扩展
 - 时间最优路径参数化
 - 练习
- 第 7 章: 移动操作
 - 全新角色登场
 - 感知有什么不同?
 - 局部视角 / 主动感知
 - 未知 (可能动态变化) 的环境
 - 机器人状态估计
 - 运动规划有什么不同?
 - 轮式机器人

- 完整约束驱动
 - 非完整约束驱动
 - 腿式机器人
- 仿真有什么不同?
- 导航
 - 建图 (除了定位之外)
 - 识别可通行地形
- 练习
- 第 8 章: 机械臂控制
 - 机械臂控制工具箱
 - 假设你的机器人是一个质点
 - 轨迹跟踪
 - (直接) 力控制
 - 间接力控制
 - 混合位置/力控制
 - 一般情况 (使用机械臂方程)
 - 轨迹跟踪
 - 关节刚度控制
 - 笛卡尔刚度与操作空间控制
 - iiwa 的一些实现细节
 - 整合所有组件
 - 孔中插销
 - 练习
- 第 9 章: 目标检测与分割
 - 迈向大数据
 - 众包标注数据集
 - 通过微调分割新类别
 - 用于操作任务的标注工具
 - 合成数据集
 - 自监督学习
 - 更大的数据集
 - 目标检测与分割
 - 整合所有组件
 - 变体与扩展
 - 使用自监督学习进行预训练
 - 利用大规模模型
 - 练习
- 第 10 章: 用于操作的深度感知
 - 位姿估计
 - 位姿表示
 - 损失函数
 - 位姿估计基准
 - 局限性
 - 抓取选择
 - (语义) 关键点
 - 稠密对应关系
 - 场景流
 - 任务层状态
 - 其他感知任务 / 表示方法
 - 练习
- 第 11 章: 强化学习
 - 强化学习软件

- [策略梯度方法](#)
 - 黑盒优化
 - 随机最优控制
 - 使用策略梯度，而不是环境梯度
 - REINFORCE、PPO、TRPO
 - 用于操作的控制应该很简单
- [基于价值的方法](#)
- [基于模型的强化学习](#)
- [练习](#)
- [第 12 章：软体机器人与触觉感知](#)
 - [为什么选择柔性？](#)
 - [软体机器人硬件](#)
 - [软体仿真](#)
 - [触觉感知](#)
 - 我们想要/需要哪些信息？
 - 视觉触觉感知
 - 全身感知
 - 触觉传感器仿真
 - [基于触觉传感器的感知](#)
 - [基于触觉传感器的控制](#)

[附录](#)

- [附录 A：空间代数](#)
 - [位置、旋转与位姿](#)
 - [空间速度](#)
 - [空间力](#)
- [附录 B：Drake](#)
 - [Pydrake](#)
 - [在线 Jupyter Notebook](#)
 - 在 Deepnote 上运行
 - 启用授权求解器
 - [在你自己的机器上运行](#)
 - [获取帮助](#)
- [附录 C：DrakeGym 环境](#)
 - [盒子翻转](#)
- [附录 D：搭建你自己的“操作工作站”](#)
 - [消息传递](#)
 - [Kuka LBR iiwa + Schunk WSG 夹爪](#)
 - [Franka Panda](#)
 - [Intel Realsense D415 深度相机](#)
- [附录 E：杂项](#)
 - [如何引用这些笔记](#)
 - [标注工具礼仪](#)
 - [一些优秀的期末项目](#)
 - [请给我反馈！](#)

You can find documentation for the source code supporting these notes [here](#).

前言

我一直都很喜欢机器人，但直到相对最近，我才把注意力转向机器人操作。我特别喜欢这样一种挑战：构建能够掌握物理规律，从而实现类似人类/动物般灵巧性与敏捷性的机器人。最早帮助我

在世界观以及机器人学研究方法中确立动力学核心地位的，是[被动动态行走器](#)以及与之相伴的优美分析。从那里开始，我又迷上了（实验）流体动力学，以及这样一个想法：拥有铰接翅膀的鸟实际上是在“操作”空气，从而实现惊人的效率和敏捷性。类人机器人以及在杂乱环境中高速飞行的飞行器，迫使我开始更深入地思考感知在动力学与控制中的作用。现在我相信，感知与动力学之间的这种相互作用是真正基础性的；而且我非常着迷于这样一个观察：操作中相对“简单”的问题（我该如何扣好我的正装衬衫纽扣？）能够非常漂亮地揭示这一问题。

我对机器人编程的方法一直都非常偏计算/算法化。我一开始主要使用机器学习中的工具（尤其是强化学习）来为简单的行走机器人开发控制系统；但随着机器人和任务变得更加复杂，我转向了基于模型的规划和基于优化的控制中更复杂的工具。在我看来，没有任何其他学科像控制理论那样深入地思考动力学；而且，最优秀的基于模型的控制算法所能获得的算法效率以及有保证的性能/鲁棒性，远远超过我们今天用学习控制所能做到的。不幸的是，与控制相关的研究在数学上的成熟性，也使得该领域在其假设和问题表述方面相对保守；而机器人操作的需求打破了这些假设。例如，鲁棒控制通常假设动力学是（近似）光滑的，并且不确定性可以由简单分布或简单集合来表示；但在机器人操作中，我们必须处理接触的非光滑力学，以及来自多样光照条件、不同数量物体、未知几何形状和未知动力学的不确定性。在实践中，据我所知，到目前为止，还没有任何最先进的机器人操作系统使用严格的控制理论，甚至来设计决定机器人何时与其操作对象建立和断开接触的低层反馈。这些笔记的一个明确目标，就是试图改变这一点。

在过去几年里，深度学习无疑对机器人感知产生了重大影响，解开了在实验室或工厂环境之外执行操作任务时一些最令人望而生畏的挑战。我们将讨论深度学习中与目标识别、分割、位姿/关键点估计、形状补全等相关的工具。如今，相对较旧的学习控制方法也正经历一股惊人的流行热潮，部分原因是大规模计算能力以及越来越容易获得的机器人硬件和仿真器。与用于感知的学习不同，学习控制算法仍远未成为一项成熟技术；其中一些看起来最令人印象深刻的结果，仍然难以理解也难以复现。但该领域近期的工作无疑凸显了控制领域所采取的保守主义的陷阱。学习研究者正在大胆地为机器人操作提出比以往更加激进、更令人兴奋的问题——在许多情况下，我们正在意识到某些操作任务实际上相当容易；而在另一些情况下，我们正在发现一些仍然从根本上很困难的问题。

最后，现在似乎正是机器人操作在世界上产生真实而巨大影响的成熟时机，其应用领域从物流到家庭机器人不一而足。在过去几年里，我们已经看到无人机从学术界的奇闻趣事转变为消费产品。甚至在更近一些时候，自动驾驶也已经从学术研究转向产业界，至少从投入的资金规模来看是如此。操作似乎会是下一个从机器人研究跃迁到实践中的重大事物。对于风险投资人来说，投资它仍然有一点风险，但一旦我们拥有了这项技术，没有人会怀疑其市场规模。我们有机会在这一转变中发挥作用，这是多么幸运啊？

所以，这些笔记就从这里开始……我们正处在学习、控制与机器人学交汇的一个令人难以置信的十字路口，有机会在工业和消费应用中产生即时影响，甚至有可能为系统理论与控制开辟全新的时代。我只是努力跟上这一切，并享受这段旅程。

一个操作工具箱

这些讲义的另一个明确目标，是为操作科学家的工具箱中最有用的工具提供高质量实现。当我必须在数学清晰性和运行时性能之间做出选择时，清晰的表述永远是我的首要优先事项；如果可能的话，我也会尽量加入高性能的表述，或者尝试给出替代方案的指引。操作研究正在快速发展，而我的目标是不断更新这些笔记以跟上它的步伐。我希望 [DRAKE](#) 以及这些笔记中提供的软件组件，能够在你的工作中直接发挥作用。

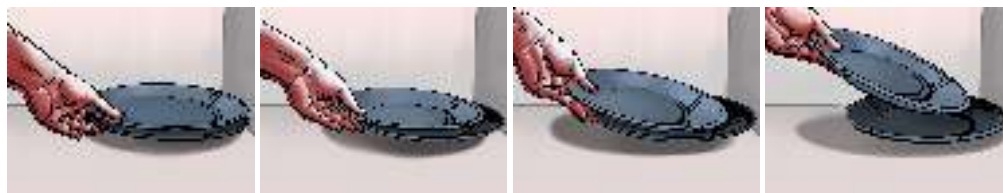
如果你想复制我们在这些笔记中使用的部分或全部硬件，可以在[附录](#)中找到相关信息和说明。

在使用这些代码时，请考虑[回馈贡献](#)（尤其是为 [DRAKE](#) 中已经成熟的代码贡献）。即便是问题/错误报告，也可能是重要的贡献。如果你对这些笔记有疑问或发现问题，请在[这里](#)提交。

CHAPTER 1

Introduction

花一点时间去欣赏人类在用双手执行任务时究竟有多么了不起，是非常值得的。那些对我们来说常常显得平凡无奇的任务——比如装洗碗机、切蔬菜、叠衣服——对于机器人而言依然是极其具有挑战性的任务，并且仍处于机器人研究的最前沿。



考虑这样一个问题：从水槽里一叠盘子中拿起一个盘子，并把它放进洗碗机。显然，你首先必须感知到水槽里有一个盘子，并且它是可接触的。让你的手移动到盘子旁边，很可能需要绕过水槽以及其他餐具的几何结构。真正拿起盘子的动作，可能需要一个相当微妙的操作：你需要先把盘子稍微翘起，让它沿着你的手指滑动，然后再沿着水槽或其他盘子滑动，从而获得一个合理的抓握姿态。大概在你把盘子从水槽中抬起时，你会希望尽量避免盘子与水槽发生碰撞，这意味着你需要对盘子的尺寸和范围有相当合理的理解（不过我实际上认为，如今的机器人对碰撞过于害怕了）。甚至把盘子放进洗碗机这个动作本身也相当微妙。你可能会认为应该先让盘子对准洗碗机的插槽，然后再把它滑进去，但我觉得人类比这更聪明。一个看起来更好的策略是：先稍微放松对盘子的抓握，以一个倾斜角度靠近，并有意让盘子的一侧接触插槽边缘，这样在你放下盘子的过程中，盘子会自然旋转到正确的位置。但这种策略背后的理由其实很微妙——它是一种在不要求盘子的位置和朝向具有很高运动学精度的情况下，实现运动学上准确任务的方法。



Figure 1.2 - 一个机器人正在从可能杂乱的水槽中拿起一个盘子（左：仿真环境中，右：真实环境中）。这个示例来自[丰田研究院的操作团队](#)。

这些问题之所以仍然如此困难，原因之一也许在于：它们需要在许多技术领域具备很强的能力，而这些领域传统上彼此之间多少有些割裂；要成为所有这些领域的专家是很有挑战性的。比起机器人建图与导航、腿式运动，或机器人学中的其他重要领域，操作中最有趣的问题更需要感知、规划和控制之间发生显著的相互作用。这既包括用于理解物体和环境局部几何结构的几何感知，也包括用于理解场景中有哪些可供操作机会的语义感知。规划通常包括对任务的运动学和动力学约束进行推理（我该如何指挥这条刚性的七自由度机械臂伸进抽屉？）。但它也包括更高层次的任务规划（要把牛奶倒进杯子里，我需要打开冰箱，然后抓住瓶子，然后拧开瓶盖，然后倾倒……也许做完之后还要把一切放回原处）。低层次的问题需要用实数来表示，但更高层次的问题则显得更加逻辑化、离散化。或许在最低层次上，我们的手会直接或通过工具与世界建立和断开接触，施加力，并快速而轻松地在粘着和滑动这两种摩擦状态之间切换——仅这些问题本身，从动力学和控制的角度来看，就已经极其丰富且困难。

我们有很多内容要讨论！

两个核心研究问题（我喜欢重点关注的）：1）有许多任务，即使是在实验室中的单个实例里，我们也不知道如何为机器人编程，使其能够稳健地完成（比如把手伸进口袋并掏出钥匙）；2）在开放世界中实现完整操作系统栈的稳健性。

1.1 操作不只是拾取与放置

操作有大量应用。把一个物体从一个箱子里拿出来，再放到另一个箱子里——这就是著名的“拾取与放置”问题的一种版本——是机器人操作的一个很好的应用。几十年来，机器人已经能够在工厂中对经过精心整理的零件完成这类任务。过去几年里，我们看到新一代拾取与放置系统使用深度学习进行感知，并且能够很好地处理更加多样化的物体集合，尤其是在放置的位置和朝向不需要非常精确的情况下。这可以通过传统机器人手来完成，也可以通过更专门用途的末端执行器来完成，例如使用吸附的末端执行器。它通常不需要非常精确地理解待拾取物体的形状、位姿、质量或摩擦特性。

然而，这些笔记的目标是考察一种比拾取与放置问题所涵盖内容宽泛得多的操作观。即使是我们关于装洗碗机的思想实验——可以说是一种更高级的拾取与放置——也对感知、规划和控制系统提出了更高要求。但人类（并且希望不久之后机器人也能做到）能够用双手完成的任务之多样性，确实令人惊叹。想看一个我喜欢的小型示例目录，可以看看 [南安普顿手部评估程序 \(SHAP\)](#)，它被设计为一种用于经验性评估假肢手的方法。Matt Mason 也在他 2018 年综述论文的开头，对操作给出了一个宽泛而深思熟虑的定义[1]。

同样重要的是要认识到，今天的操作研究与 20 世纪 80 年代和 90 年代的操作研究已经非常不同。在那个时期，研究中有一个强烈而优美的重点，即“作为抓握的操作”，其中包括一些开创性工作，例如在假设对物体具有稳定抓握的情况下，研究多指手的运动学和动力学。直到现在，我有时仍会听到有人把“抓握”这个词几乎当作操作的同义词来使用。但请意识到，今天操作研究的目标，以及这些笔记的目标，都比这要宽泛得多。当你扣衬衫纽扣时，“抓握”足以描述你的手正在做什么吗？做沙拉时呢？把花生酱抹在吐司上时呢？

1.2 开放世界中的操作

也许正因为人类在操作任务上表现得如此出色，我们对于这些任务在性能和鲁棒性方面的期望也极其之高。仅仅能够在实验室环境中可靠地把一套餐盘放进洗碗机里是不够的。我们希望系统能够处理几乎任何人放进水槽里的盘子。而且，我们希望系统能够在任何厨房中工作，不受不同几何布局、光照条件等因素的影响。要在一个包含物理、感知、规划与控制闭环的复杂操作系统中实现并验证鲁棒性，本身已经是令人望而生畏的挑战了。但我们又该如何为世界上的每一个厨房提供测试覆盖呢？

世界具有无限多样性（你永远不可能看到所有可能的厨房）的这一想法，通常被称为“开放世界”或“开放领域”问题——这个术语最早是在[电子游戏](#)的背景下被普及开的。在对操作问题的某些方面进行严谨思考，与尝试拥抱整个世界的多样性和复杂性之间取得平衡，往往并不容易。但沿着这条界线前进是非常重要的。

开放世界中的操作多样性实际上有可能让问题变得更容易。我们才刚刚进入机器人学大数据时代的黎明阶段；这一点将带来的影响再怎么强调也不过分。但我认为问题比这还更深层。例如，我们当前用于规划和控制的一些优化模型可能会陷入局部最优，因为狭窄的问题定义会产生许多古怪的解；但一旦我们要求控制器能够在极其多样化的场景中工作，那么这些古怪解就可能被自然淘汰，而优化空间反而会变得更加简单。不过，如果这种说法在某种程度上是正确的，那么我们就应该努力对其进行严格理解！

1.3 仿真

我们如今正进入操作研究黄金时代的另一个原因是：我们的软件工具（就在最近）已经变得好得多了！

我记得就在几年前（大约 2015 年），我曾和我的博士生们讨论过在操作研究中使用仿真的问题。他们都非常擅长使用仿真来开发步行机器人的控制系统。当时他们的口头禅是：“你无法在仿真中研究操作。”而且他们这样说是充分有理由的。操作中的接触力学复杂性，传统上远比只通过少量接触点与地面交互的步行机器人更难仿真。然而，更大的障碍其实是感知在操作中的核心地位；当时普遍认为，人们无法把摄像机仿真得足够真实，以至于具有实际意义。

事情变化得竟然如此之快！过去几年里，机器人学和计算机视觉社区迅速采用了达到电子游戏级别质量的渲染技术。如今逐渐形成的共识是，游戏引擎渲染器**确实能够**足够好地模拟摄像机，不仅能够仿真中**测试**感知系统，甚至还能在仿真中**训练**感知系统，并期望它们能够在现实世界中正常工作！这相当令人惊叹，因为此前我们一直非常担心：如果在仿真中训练深度学习感知系统，它可能会利用仿真图像中的某些特有缺陷，从而使问题变得更容易。

我们也已经看到接触仿真的质量和性能得到了显著提升。要实现稳定且高性能的多体接触仿真，需要处理复杂的几何查询以及刚性的（测度）微分方程。在描述这些控制方程的数学建模与数值求解方面，仍然存在进行根本性改进的空间，但如今的求解器已经足够优秀，并且极具实用价值。

1.4 这些笔记是交互式的

通过利用仿真的力量，以及如今免费在线交互式计算资源的可用性，我正在努力让这些笔记超越你在传统文本中所能看到的内容。每一章都会有可运行的代码示例，你可以在 [Deepnote](#) 上立即运行它们（无需安装），也可以下载/安装后在本地机器上运行（更多细节见[附录](#)）。它使用了一个名为 **DRAKE** 的开源库；自大约 2013 年我开始把我们的研究代码整理并更广泛地发布以来，这个库一直是我倾注心血的成果。由于所有代码都是开源的，你究竟想沿着这个兔子洞深入到什么程度，完全取决于你自己。

Mortimer Adler 有句名言：“阅读一本[伟大]的书，应当是你与作者之间的一场对话。”除了交互式图形/代码之外，我还增加了直接在这些笔记上高亮、评论和提问的功能（尽管试试看；但请阅读我关于[批注礼仪](#)的说明）。Adler 的建议是，**伟大的**写作能够把静态文本变成一场对话，跨越距离与时间；也许我是在作弊，但即使我的写作没有达到 Adler 希望的那种水准，技术也能帮助我与你交流！Adler 还建议在书上做笔记[2]；你可以打印这些页面，使用（不常更新的）[pdf](#)，或者使用同一个批注工具直接在网站上做私人批注。

我已经按章节把软件示例组织成了笔记本。每一章顶部都有一个“Launch in Deepnote”按钮；我鼓励你在阅读章节时立即打开它。你可以直接“duplicate”该章节项目，并运行其中一个笔记本的第一个单元来启动云端机器。然后在你阅读正文时，我会提供一些示例，它们在笔记本中会有对应的小节。

Example 1.1 (二维遥操作。)

在进入任何自主操作之前，我们先从感受一下在线 Jupyter 笔记本中研究操作会是什么样开始。这个示例会打开一个带有我们 3D 可视化器的新窗口，在可视化器的“Controls”菜单中，你会找到一些滑块，可以用它们驱动机器人的末端执行器四处移动。试试看吧！

Example 1.2 (三维遥操作。)

我先提供了一个二维可视化/控制示例，因为在二维中一切都更简单。但仿真实际上是完全在三维中运行的。运行第二个示例来看看吧。

1.5 基于模型的设计与分析

得益于仿真技术的进步，如今即使没有实体机器人，也可以对操作进行有意义的研究。但仿真方面的软件进展（包括对机器人动力学、传感器、执行器及其环境的仿真）还不足以支撑这些笔记中的所有主题。今天的操作研究利用了感知、规划和控制中的大量先进算法。除了提供这些单独的算法之外，这些笔记的一个主要目标，是尝试以一种系统化的方式驾驭将它们结合使用时所产生的复杂性。

你们中的许多人应该都熟悉 [ROS \(机器人操作系统\)](#)。我认为 ROS 是过去几十年里机器人领域发生的最好的事情之一。它意味着来自不同子领域的专家可以很容易地以模块化组件的形式分享他们的专业知识。组件（作为 ROS 包）只需要就它们将在网络上发送和接收的消息达成一致；即使这些包是用不同的编程语言编写的，甚至运行在不同的操作系统上，它们也可以相互协作。

虽然 ROS 让人相对容易开始进行操作研究，但它并不能满足我关于清晰思考操作问题的教学目标。以模块化方式编写组件的方法非常好，我们也会在这里采用它。但在 [DRAKE](#) 中，我们会要求每个组件多做一点事情——本质上是要求它们以一致的方式声明自己的状态、参数和时序语义——这样我们就更有机会理解系统之间复杂的关系。这也具有很大的实际价值；能够通过可重复的确定性仿真（即使其中包含随机性）来调试完整的操作系统栈，在该领域中出人意料地少见，但却极其有价值。

我们工作中的关键构建模块将是 [DRAKE Systems](#)，而系统可以以复杂的组合方式组合成 [Diagrams](#)。系统图长期以来一直是控制领域使用的建模范式；如果你使用过 Simulink、LabView 或 Modelica 等工具，那么其中的软件层面会让你非常熟悉。这些软件工具将这种框图式设计范式称为“基于模型的设计”。

Example 1.3 (系统图)

即使是上面的那些示例，虽然依赖你来执行遥操作，而不是使用自主系统栈，也仍然是由许多部分组合而成的结果。对于你在 [DRAKE](#) 中拥有的任何系统（系统图本身仍然是一个系统），你都可以直接在笔记本中可视化该图。

这个图形是交互式的。请务必放大并四处点击，以感受在这个框架中我们能够抽象掉多少复杂性。例如，试着展开 `iiwa_controller` 模块。

当你准备好进一步了解 [DRAKE](#) 中的系统框架时，我建议从 [Drake 主网站](#) 上链接的“Modeling Dynamical Systems”教程开始。

让我坦率地说：并不是每个人都喜欢把这个系统框架用于操作。有些人只是想尽快写代码，看不到放慢速度去声明状态变量等内容的好处。当你第一次编写一个新系统时，它很可能会让你觉得是一种负担。但我之所以倡导这种更严谨的方法，恰恰是[因为](#)我们正在尝试快速构建复杂系统。我相信，要让我们的开放世界操作系统达到下一个成熟度层级，就要求我们的构建模块也更加成熟。

1.6 这些笔记的组织结构

这些笔记剩余章节的组织，是围绕操作中的组件级构建模块展开的。其中许多组件本身都建立在丰富的文献基础之上（例如来自计算机视觉，或动力学与控制领域的文献）。为了避免被这些内容淹没，我选择专注于以一致且连贯的方式呈现各个领域中与操作相关的最重要思想，并提供更多文献的指引。即使只是在所有这些领域之间找到一种统一的记号表示，也可能是一项挑战！

接下来的几章将为你提供一些关于我们正在仿真的相关机器人硬件的最低限度背景知识，介绍（部分）关于如何仿真它们的细节，并介绍我们将在整本笔记中大量使用的几何和运动学基础。

在笔记的其余部分中，我决定不把章节组织成“感知”“规划”和“控制”这几个独立部分，而是以螺旋式方式穿插讲解这些主题。在第一部分，我们会学习刚好足够的感知、规划和控制内容，以构建一个基本操作系统，使其能够对已知且孤立的物体执行拾取与放置操作。随后，我会尝试通过说明我们之前的系统做不到什么，以及我们希望它在本章结束时能够做到什么，来引出每一章的新内容。

我欢迎任何反馈。也别忘了参与互动！

有一件事我们将暂时略过，那就是人类。并不是说我不喜欢人类，而是这里已经有足够多的事情要做了，没必要再尝试对人类建模。就把它称作未来工作吧。唯一的例外是当我们讨论机械臂控制时——在这个层面上，一些简单的启发式方法可以让机器人在人类周围运行时安全得多。机器人并不会理解人类，但我们希望确保它仍然不会伤害他们。也不会伤害场景中的其他机器人/智能体。

1.7 练习

Exercise 1.1 (熟悉 Drake)

Drake 是一个功能强大且成熟的软件库，能够支持许多高级机器人应用。它背后的动机在[这篇博客文章](#)中有所描述。它有非常详尽的文档，但你必须知道去哪里查找。

1. 查看不断增长的 **Drake** [教程](#)列表（也可从 **Drake** [主页](#)进入）。在 `dynamical_systems` 教程中，当我们仿真 `SimpleContinuousTimeSystem` 时，初始条件 $x(0)$ 被设置为什么值？
2. 类/函数级文档是 **Drake** 中最详尽的文档。当我使用 **Drake**（无论是 C++ 还是 Python）时，我最常打开的是 [C++ doxygen](#)。Python 文档（大多）是由它自动生成的，并没有经过同样仔细的整理；我通常只在 Python 接口与 C++ 不同的少数情况下才会去查看它。
在 C++ doxygen 中搜索“Spatial Vectors”。在代码中，我们使用哪些 ascii 字符来表示角加速度？
3. **Drake** 是开源的。这里没有黑箱算法。如果你想查看某个特定算法是如何实现的，或者想找到某个函数的使用示例，你始终可以查看源代码。如今你可以使用 [VS Code](#) 直接在浏览器中浏览代码。在[“fitted value iteration”的单元测试](#)中，我使用的 `convergence_tol` 值是多少？

在使用 **Drake** 的过程中，如果你有任何问题，请考虑使用“drake”标签在 [stackoverflow](#) 上提问。更广泛的 **Drake** 开发者社区通常能够比课程工作人员更快（和/或更好）地回答！而且在那里提问有助于建立一个可搜索的知识库，使 **Drake** 对每个人都更加有用、更加易于使用。

Exercise 1.2 (Drake 系统基础)

这个练习将向你介绍 Drake 的框图系统框架，并且我们会使用 Drake 中的一些核心概念，包括 `LeafSystem`、`Diagram`、`Context` 和 `Simulator`。在这个练习中，你将使用 Drake 的系统框架从零开始实现一个自定义动力系统。你将完全在 中完成。你需要完成以下步骤：

- 为倒立摆实现一个自定义 `LeafSystem`
- 构建系统并将其连接成用于仿真的 `Diagram`
- 使用 Drake 的内置工具运行仿真

Exercise 1.3 (物理仿真与机器人加载)

在前一个练习中的系统框架基础之上，你现在将学习如何搭建一个包含机器人和自定义资源的场景。具体来说，这个练习将向你展示如何从 [Drake 的模型数据库](#) 中加载 Kuka iiwa 机器人，以及如何通过从零开始编写 `SDF` 文件来创建你自己的自定义对象。最终，你将得到一个仿真场景：你自定义的姓名首字母会落到你创建的桌子上，场景中还包含一个 7 自由度的 Kuka iiwa 机器人！你将完全在 中完成。你需要完成以下步骤：

- 使用 `Parser` 和 `MultibodyPlant` 将 Kuka iiwa14 机器人加载到一个 `Diagram` 中
- 实现一个简单的比例控制器，作为 `LeafSystem`，用于计算关节力矩，使机器人达到期望的关节配置
- 编写一个自定义 `SDF` 文件来定义一张桌子，然后使用提供的字母生成 API 为你的姓名首字母生成 `SDF` 资源
- 组装一个包含机器人、自定义桌子以及你的姓名首字母的仿真场景

Exercise 1.4 (HardwareStation 与高级场景)

在前一个练习中，你通过创建 `DiagramBuilder`、添加 `MultibodyPlant` 和 `SceneGraph`、加载机器人并将所有组件连接起来，手动搭建了一个仿真环境。在这个练习中，你将学习如何使用 `HardwareStation` 以及在 `YAML` 配置文件中指定的模型指令，以更少的代码实现同样的结果。你将完全在 中完成。你需要完成以下步骤：

- 使用 `HardwareStation` 和场景指令搭建一个包含两个 iiwa14 机器人的场景
- 将你的自定义桌子和字母资源加载到 `HardwareStation` 中
- 将 Drake 模型仓库中的预定义对象添加到 `HardwareStation` 中

Exercise 1.5 (状态空间练习)

我们将使用状态空间模型来描述操作组件；这里有一个简单的练习题来帮助你建立思考。考虑如下系统：

$$y[n] = u[n - 3],$$

其中 y 是你感兴趣的系统输出， u 是你控制的输入，两者都是标量。在这个问题中，你需要思考如何选择状态向量 x ，以便能够在 u 变化时对输出轨迹 y 建模。

具体来说：你需要选择一个状态向量 x ，使得 $x[n + 1] = Ax[n] + Bu[n]$ 且 $y[n] = Cx[n]$ ，其中 A 、 B 和 C 都是适当大小的矩阵。

- 为了表示这个系统, $x[n]$ 的大小 (可使用的最小向量) 应该是多少? 请解释你的答案。(提示: 当你尝试用错误大小的状态来手动或计算机仿真该系统时, 会发生什么?)
- 对于你在 (a) 中选择的 状态大小, 对应的 A 、 B 和 C 矩阵分别是什么, 以表示该系统的动力学?

- 假设我们将输入与输出之间的延迟增加到 5:

$$y[n] = u[n - 5]$$

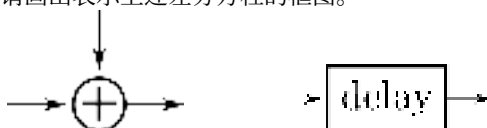
与 (a) 相比, 状态的大小会发生变化吗? 请解释原因。

- 考虑一个具有稍微不同动力学的系统:

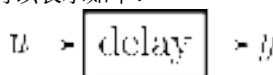
$$x[n] = x[n - 2] + u[n - 1]$$

$$y[n] = x[n] + 1$$

给定如下模块库, 请画出表示上述差分方程的框图。



例如, 系统 $y[n] = u[n - 1]$ 可以表示如下:



REFERENCES

- Matthew T Mason, 《迈向机器人操作》, 《控制、机器人与自主系统年度评论》, 第 1 卷, 第 1--28 页, 2018。
- Mortimer J Adler, 《如何在书上做标记》, 《星期六文学评论》, 7 月 6 日, 1941。[[链接](#)]

CHAPTER 2

Let's get you a robot

在本章中，我们将为你的[机甲](#)配备装备。我想确保你理解这些笔记中所使用的机器人硬件，以及它与当今其他可用硬件相比如何。你还应该在学完后理解我们如何仿真机器人，以及你可以向机器人接口发送哪些命令。

2.1 机器人描述文件

在大多数章节中，我们会反复使用一两种机器人配置。现代机器人学的一大优点是，我们将在这些笔记中开发的许多工具都相当通用，并且可以很容易地从一个机器人迁移到另一个机器人。我可以想象这些笔记未来的某个版本：你真的可以在本章中构建自己的机器人，并在后续章节中使用你的定制机器人！

之所以能够轻松地仿真/控制各种机器人，部分原因在于用于描述机器人的通用文件格式已经大量普及。遗憾的是，这个领域尚未收敛到单一的首选格式（至少目前还没有），而且每种格式都有自己的怪癖。Drake 目前可以加载 [通用机器人描述格式](#)（URDF）、[仿真描述格式](#)（SDF），并且对 [MuJoCo 格式](#)（MJCF）提供有限支持。Drake 开发者一直在尝试向上游贡献对 SDF 的改进，而不是再创建另一种格式；不过我们确实有一个非常简单的 YAML 规范，称为 [Drake 模型指令](#)，它可以非常快速、轻松地把来自这些不同文件格式的多个机器人/物体加载到同一个仿真中；你已经在中看到过这样的示例。

2.2 机械臂

市场上似乎有很多机器人机械臂可供选择。那么该如何选择呢？成本、可靠性、可用性、负载能力、运动范围，.....有许多重要的考虑因素。而我们为研究实验室所做的选择，可能会与我们作为创业公司时所做的选择非常不同。

Example 2.1 (机器人机械臂)

我整理了一个简单示例，让你可以探索当今一些流行的机器人机械臂。如果你最喜欢的机械臂还不列表中，请告诉我！

有一个特别的需求，如果我们希望机器人满足它，那么可选的平台会迅速缩减到少数几个可行方案：这个需求就是关节力矩传感与控制。在市场上的力矩控制机器人中，我在这些笔记里最常使用的是 Kuka LBR iiwa 机器人（我会尽量使用小写“iiwa”，以[与制造商保持一致](#)，不过每次看到它我都觉得有点别扭！）。



Figure 2.1 - [Kuka LBR iiwa 机器人](#)。这台机器人的负载能力为 7kg。

即使对于非常高级的应用来说，关节力矩传感与控制功能是否绝对必要，其实也并不完全明确。但作为一名非常关注机器人与世界之间接触交互的研究者，我更倾向于拥有这种能力，并去探索自己是否真的需要它，而不是事后猜想“如果当初有的话会怎样”。为了更好地理解这一点，让我们先了解一下：大多数机器人（位置控制机器人）与少数愿意承担额外成本和复杂度来提供力矩传感与控制功能的机器人之间，到底有什么区别。

2.2.1 位置控制机器人



Figure 2.2 - 两种流行的位置控制机械臂。（左）Universal Robotics 的 UR10。（右）ABB Yumi。

如今的大多数机器人机械臂都是“位置控制”的——给定一个期望关节位置（或关节轨迹），机器人会以相对较高的精度执行它。基本上，所有机械臂都可以进行位置控制——如果机器人提供了一个（带有足够高带宽的）力矩控制接口，那么我们当然也能够调节位置。实际上，把一个机器人称为“位置控制”，往往是一种比较委婉的说法，意思是它并不提供力矩控制。你知道为什么位置控制而不是力矩控制会成为主流吗？

像上面示例中的轻量级机械臂，通常由电动机驱动。对于质量相当不错的电机（例如绕组设计能够尽量减少力矩波动等），我们预期电机输出的力矩与施加的电流直接成正比：

$$\tau_{motor} = k_t i,$$

其中 τ_{motor} 是电机力矩， i 是施加的电流，而 k_t 是“[电机力矩常数](#)”。（类似地，施加电压与电机稳态速度之间也存在简单的（仿射）关系）。如果我们能够控制电流，那么为什么不能直接控制力矩呢？

简短的答案是：为了实现合理的成本与重量，我们通常会选择较小的电动机，并搭配较大的减速比，而减速机构会带来许多极难建模的动力学效应——包括齿隙、振动和摩擦。因此，电流与力矩之间的简单关系就会失效。传统观点认为，当减速比较大（例如 $\gg 10$ ）时，这些未建模项已经足够显著，以至于无法忽略，此时力矩就不再与电流简单对应。

位置控制。

在没有传动系统动力学良好模型的情况下，我们该如何克服这一挑战？调节**电机**的电流或速度，只需要在传动系统的电机侧安装传感器。要精确调节关节，我们通常需要在传动系统的输出侧增加更多传感器。重要的是，尽管由传动系统产生的力矩并不能被精确知道，但它们也不是任意的——例如，它们绝不会向系统中注入能量。最重要的是，我们可以确信，输入到电机中的电流与关节处的力矩，并最终与关节加速度之间，存在一种**单调递增**关系。请注意，我是谨慎地选择了“单调”这个词，它表示“非递减”，但**并不意味着**“严格递增”；例如，当某个关节从静止状态开始运动时，静摩擦会抵抗较小的力矩，而输出端不会产生任何加速度。

最常见的做法是在关节处添加位置传感器——通常是编码器或电位计——它们价格低廉、精度高且可靠。在实践中，我们认为这些传感器（经过一定的信号滤波/调理之后）能够提供关节位置和关节速度的精确测量；关节加速度也可以通过二次微分获得，但通常被认为噪声更大，也不太适合用于紧密反馈回路。位置传感器足以精确跟踪机械臂的期望位置轨迹。对于每个关节，如果我们将关节位置记为 q ，并给定一个期望轨迹 $q^d(t)$ ，那么我可以使用**比例-积分-微分（PID）控制**来跟踪它：

$$\tau = k_p(q^d - q) + k_d(\dot{q}^d - \dot{q}) + k_i \int (q^d - q) dt,$$

其中 k_p 、 k_d 和 k_i 分别是位置、速度和积分增益。PID 控制有丰富的理论基础，也有大量关于如何选择增益的知识，这里我不会重复展开。不过我要指出的是，当我们仿真位置控制机器人时，通常需要为实体机器人和仿真使用不同的增益。这是由于传动系统动力学造成的，同时也因为硬件中的 PID 控制器通常输出的是电压命令（通过**脉宽调制**），而不是电流命令。弥合这一建模差距，传统上并不是机器人仿真的重点——还有足够多其他细节需要做好，它们更主导“仿真到现实”的差距——但我怀疑，随着该领域逐渐成熟，主流机器人仿真器最终也会捕捉到这一点。

你们中的一些人可能正在想：“我可以训练一个神经网络来建模**任何东西**，我才不怕难以建模的传动系统！”我确实认为这种方法有值得乐观的理由；在这个方向上已经有一些初步演示（例如[1]）。这并不像拥有一个第一性原理模型那样有用，后者能够仅凭描述文件中的少量参数就泛化到新的执行器，但这种方法仍然可能非常有效。

题外话：带传动系统的连杆动力学。

有一件事可能会让人感到惊讶：尽管机械臂的关节动力学是高度耦合且依赖状态的，但 PID 增益往往仍然是针对每个关节独立选择的，并且是常数（而不是**增益调度**的）。你难道不会认为，例如，一个完全伸展开并举着牛奶壶的机械臂，与一个在竖直悬挂位置且没有负载的机械臂，所需的电机命令会有很大不同吗？令人惊讶的是，所需的增益/命令可能并没有人们想象中那么不同。

电动机在高速下效率最高（通常超过每分钟 100 或 1000 转）。即便机器人真的能够运动得那么快，我们大概也并不希望它们真的这么快！因此，几乎所有电驱机器人都带有相当大的减速比，通常在 100:1 左右；也就是说，传动输出端每转一圈，电机实际上已经转了 100 圈，而输出力矩则是电机力矩的 100 倍。对于减速比为 n 、驱动关节 q 的传动系统，我们有

$$q_{motor} = nq, \quad \dot{q}_{motor} = n\dot{q}, \quad \ddot{q}_{motor} = n\ddot{q}, \quad \tau_{motor} = \frac{1}{n}\tau.$$

有趣的是，这即使对于单个关节，也会对最终动力学（由 $f = ma$ 给出）产生相当深远的影响。先写出关节力矩与关节加速度之间的关系（暂时还不考虑电机），我们可以在旋转坐标中将 $ma = \sum f$ 写成

$$I_{arm}\ddot{q} = \tau_{gravity} + \tau,$$

其中 I_{arm} 是转动惯量。例如，对于一个**简单摆**，我们可能有

$$ml^2\ddot{q} = -mgl \sin q + \tau.$$

但施加到关节上的力矩 τ 实际上来自电机——如果我们用电机坐标来写这个方程，则得到：

$$\frac{I_{arm}}{n} \ddot{q}_{motor} = \tau_{gravity} + n\tau_{motor}.$$

如果两边同时除以 n ，并考虑到电机本身也具有惯量（例如来自大型旋转磁体），而这个惯量并不会受到减速比影响，那么我们得到：

$$\left(I_{motor} + \frac{I_{arm}}{n^2} \right) \ddot{q}_{motor} = \frac{\tau_{gravity}}{n} + \tau_{motor}.$$

值得注意的是，尽管电机的质量可能只占机器人总质量的一小部分，但对于高减速比机器人来说，它们在关节动力学中却可能起到重要作用。我们使用术语反射惯量（reflected inertia）来表示由于传动系统缩放效应，在传动系统另一侧感受到的惯性负载。机械臂在电机侧的“反射惯量”会被减速比的平方削弱；而电机在机械臂侧的“反射惯量”则会被减速比的平方放大。这会带来一些有趣的结果——当我们扩展到多连杆情况时，会发现 I_{arm} 是一个依赖状态的函数，它不仅包含被驱动连杆的惯量，也包含机械臂其他关节之间的惯性耦合。另一方面， I_{motor} 是常数，并且只影响局部关节。对于较大的减速比， I_{motor} 项会主导其他项，这会产生两个重要效果：1）它实际上使机械臂方程近似对角化（惯性耦合项相对较小）；2）动力学在整个工作空间中相对恒定（依赖状态的项相对较小）。这些效应使得我们能够相对容易地为每个关节单独调节固定反馈增益，并且在所有配置下都表现良好。

2.2.2 力矩控制机器人

虽然并不那么常见，但确实有一些机器人支持对关节力矩的直接控制。实现这种能力有几种不同的方法。

使用只需要较小减速比（例如 $\leq 10:1$ ）、且摩擦力可以忽略不计的电动机来驱动机器人，确实是可能的。过去，这些“直接驱动机器人”[2]拥有巨大的电机和有限的负载能力。近些年来，像 Barrett WAM 这样的机械臂使用了缆索传动，通过把大型电机放在基座中来保持机械臂本体轻量化。而就在最近几年，高力矩外转子电机和无框电机方面的进展，催生了新一代低成本的“准直接驱动”机器人，例如 MIT Cheetah [3]、Berkeley Blue 和 Halodi Eve。

液压执行器提供了另一种无需大型传动机构即可产生大力矩的方案。Sarcos 曾开发过一系列力矩控制机械臂（以及人形机器人），而 Boston Dynamics 的许多最著名机器人也基于液压系统（尽管如今越来越多地转向电动机）。这些机器人通常有一个集中式主泵，每个执行器都有一个（轻量级）阀门，可以让流体流经执行器或通过旁路分流；执行器两端的压差至少近似地与最终产生的力/力矩相关。

实现力矩控制的另一种方法是保留大减速比电机，但增加传感器来直接测量执行器关节侧的力矩。这正是我们在本文示例中使用的 Kuka iiwa 机器人所采用的方法；iiwa 执行器在传动系统中集成了应变片。然而，传动系统的刚度与力/力矩测量精度之间存在权衡 [4]——iiwa 的传动系统包含一个显式的“Flex Spline”，其刚度约为 5000 Nm/rad [5]。如果把这个思路推向极端，Gill Pratt 提出了“串联弹性执行器”，它在传动系统中使用刚度更低的弹簧，并提出同时测量传动系统电机侧和关节侧的位置，以估计所施加的力矩 [6]。例如，Rethink 的 Baxter 和 Sawyer 机器人就使用了串联弹性执行器；我认为他们从未公布过弹簧刚度值，但由 HEBI robotics 开发、动机类似的串联弹性执行器接近 100 Nm/rad。即使对于 iiwa 执行器来说，关节弹性也已经足够显著，以至于底层控制器需要非常仔细地显式考虑它，才能实现高性能的关节控制[7]。等我们进入关于力控制的章节时，会再讨论这些细节。

2.2.3 硬件的迅速扩增

我上面提到的低成本力矩控制机械臂，只是机器人机械臂即将大规模扩增的开端。在疫情期间，

我看到许多人在家中使用像 [xArm](#) 这样的低成本机器人。随着需求增加，成本还会继续下降。

我只想说，与研究腿式机器人相比——过去几十年里，我们一直是在实验室原型机上开展研究，而这些原型机通常是研究生（有时也有教授！）在走廊尽头的机加工车间里制造出来的——如今能够获得经过专业工程设计、高质量且高运行时间的硬件，实在是一种享受。这也意味着，我们可以在一个实验室测试算法，而另一个实验室，也许是在另一所大学，可以在几乎相同的硬件上测试算法；这带来了以前不可能实现的可重复性和共享水平。价格正在下降，这意味着会有更多相似机器人出现在更多实验室/环境中；这也是我对该领域未来几年如此乐观的重要原因之一。

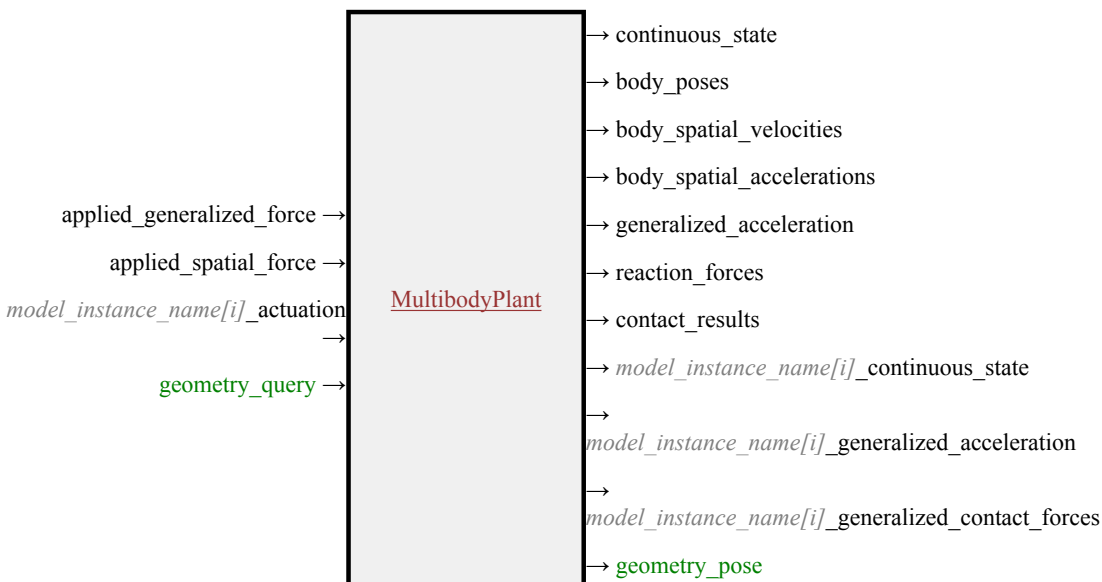
现在正是研究操作的好时代！

2.2.4 仿真 Kuka iiwa

现在是时候仿真我们选定的机器人机械臂了。第一步是获得一个机器人描述文件（通常是 URDF 或 SDF）。为方便起见，我们将包括 iiwa 在内的几个机器人模型随 Drake 一起[发布](#)。如果你有兴趣仿真其他机器人，通常可以在网上某处找到描述大多数商用机器人的 URDF 或 SDF。但需要提醒一句：这些模型的质量可能差异极大。我们甚至在运动学方面（连杆长度、几何形状等）也见过令人惊讶的错误，而动力学属性（惯量、摩擦等）尤其常常完全不准确。有时它们甚至在数学上都不一致（例如，可以在 URDF/SDF 中指定一个并不能由任何刚体在物理上实现的惯性矩阵）。如果你要求 Drake 加载含有这类违规内容的文件，Drake 会报错；我们宁愿提前提醒你，也不愿开始生成虚假的仿真结果。如今也越来越好地支持从 [Solidworks](#) 等 CAD 软件直接导出为机器人格式。

现在我们必须把这个机器人描述文件导入物理引擎。在 Drake 中，物理引擎称为 [MultibodyPlant](#)。“plant”这个术语可能看起来有些奇怪，但它非常普遍；这是控制文献中用来表示待控制物理系统的词，起源于对化工厂的控制。与控制理论的这种联系对我来说非常重要。世界上没有多少物理引擎会像 Drake 这样下如此大的功夫，使物理引擎兼容控制理论中的设计与分析。

[MultibodyPlant](#) 拥有一个类接口，其中包含丰富的方法库，可用于处理机器人的运动学和动力学。如果你需要计算质心位置、运动学雅可比矩阵，或任何类似的查询，那么你就会使用这个类接口。[MultibodyPlant](#) 还实现了可作为 [System](#) 使用的接口，在 Drake 的[系统框架](#)中具有输入端口和输出端口。为了仿真或分析 [MultibodyPlant](#) 与其他系统（例如我们的感知、规划和控制系统）之间的组合，我们将会组装[框图](#)。



正如你可能预期的那样，对于像物理引擎这样复杂而通用的东西，它有许多输入端口和输出端口；其中大多数都是可选的。我会在下面的示例中说明如何使用这些端口。

Example 2.2 (仿真被动 iiwa)

这个示例值得花几分钟仔细看看，它不仅能帮助你理解物理引擎，也能帮助你理解在 Drake 中进行仿真的一些基本机制。

可视化物理引擎结果的最佳方式，是使用 2D 或 3D 可视化器。为此，我们需要添加一个负责管理场景几何的系统；在 Drake 中，我们称它为 `SceneGraph`。一旦有了 `SceneGraph`，就可以向系统中添加多种不同的可视化器和传感器，以真正渲染场景。



Example 2.3 (可视化场景)

这个示例看起来有趣得多。现在我们有 3D 可视化！

你可能会疑惑，为什么 `MultibodyPlant` 不同时处理场景的几何信息。原因是，在许多应用中，我们希望渲染复杂场景并使用复杂传感器，但又想提供自定义动力学，而不是使用默认物理引擎。自动驾驶就是一个很好的例子；在这种情况下，我们希望用车辆和环境的所有几何信息填充一个 `SceneGraph`，但通常又希望用非常简单的车辆模型来仿真车辆，而不是把轮胎力学也加入物理引擎中。我的 [欠驱动机器人学](#) 课程中也有许多这种工作流程的示例，在那里我们大量使用“简单模型”。

现在我们已经有了 iiwa 的基本仿真，但一些微妙之处已经开始显现。物理引擎需要被告知要在关节上施加哪些力矩。在我们的示例中，我们施加零力矩，于是机器人倒下了。在现实中，这种情况永远不会发生；事实上，即使控制器关闭，实体 iiwa 机器人在关节处也基本上不会经历零力矩。和许多成熟的工业机器人机械臂一样，iiwa 的每个关节都有机械制动器，只要控制器关闭，这些制动器就会接合。若要仿真控制器关闭时的机器人，我们需要告诉物理引擎这些制动器产生的力矩。

事实上，即使控制器已经开启，并且尽管它是一台力矩控制机器人，我们也从来不能真正向电机发送零力矩。iiwa 的软件接口接受“前馈力矩”命令，但它总是会把这些命令作为附加力矩，叠加到其底层控制器之上；而这个底层控制器正在补偿重力以及电机/传动机构的力学效应。这常常让人感到沮丧，但我们大概并不真的想深入到仿真驱动机构力学细节的程度。

因此，我们能够为 iiwa 提供的最简单的合理仿真，必须包含对 Kuka 底层控制器的仿真。我们将使用 iiwa 的“关节阻抗控制”模式，并会在这些细节对让机器人表现得更好变得重要时再加以说明。现在，我们可以先把它当作已知条件，并生成我们最简单的合理 iiwa 仿真。

Example 2.4 (添加 iiwa 底层控制器)

这个示例添加了 iiwa 控制器，并将期望位置（不再是期望力矩）设置为机器人的当前状态。它是对真实机器人的更忠实仿真。很抱歉，它又一次变得无聊了！

最后补充一点：如果我们的唯一目标只是仿真机器人运动相对缓慢的操作任务，而质量、惯性和力的影响可能没有机器人（以及物体）在空间中所占据的位置那么重要，那么你可能会认为仿真机器人的**动力学**有些过度。我其实会同意你的看法。但要让一个**运动学**仿真遵守基本的交互规则，却出人意料地棘手；例如，要知道物体什么时候被拿起，什么时候没有被拿起（例如参见[8]）。目前，在 Drake 中，我们主要使用完整的物理引擎进行仿真，但在操作规划和控制中经常使用更简单的模型。

2.3 手

你可能已经注意到，iiwa 模型实际上并没有安装手；机器人出厂时带有一个安装板，让你可以连接自己选择的“末端执行器”（并且还有一些接入端口选项，使你可以将末端执行器连接到计算机，而不需要让线缆沿着机器人外部垂下来）。所以现在我们又需要做一个决定：我们应该使用哪只手？

Example 2.5 (机器人手)

我们可以在 Drake 中使用与机械臂相同类型的接口来探索不同的手部模型，不过我这里的手部模型还没有那么多。如果你最喜欢的手还不在于列表中，请告诉我！

有趣的是，当谈到机器人末端执行器时，操作研究者往往会把自己划分到几个截然不同的阵营中。

2.3.1 灵巧手

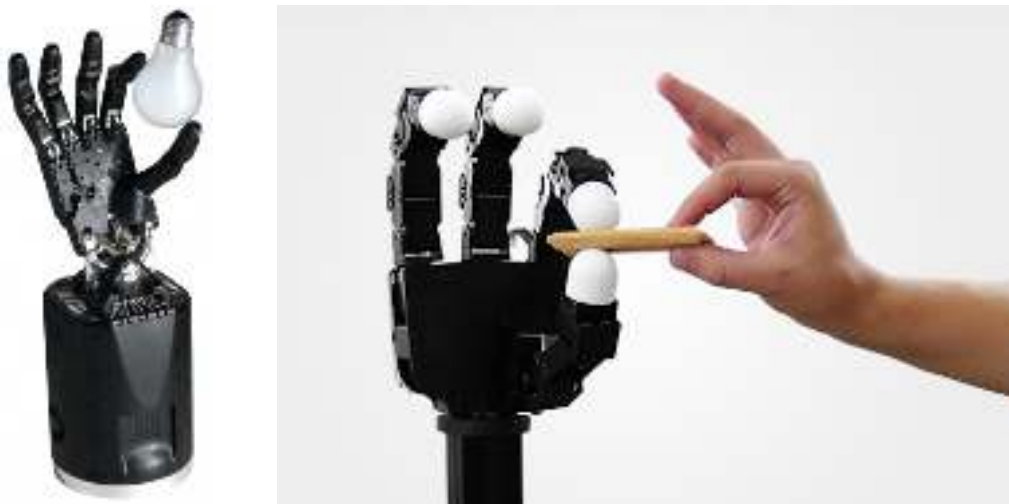


Figure 2.3 - 灵巧手。左：Shadow Dexterous Hand。右：Allegro Hand。

当然，我们对人类手的着迷是完全合理的，我们梦想着构建像人手一样灵巧且传感丰富的机器人手。但现实是，我们还没有达到那个水平。有些人选择追逐这个梦想，使用市场上最先进的灵巧手，并因此不得不面对随之而来的复杂性与缺乏鲁棒性。OpenAI 著名的“**learning dexterity**”项目就使用了 Shadow 手来操作魔方，而为了支持那些长时间学习实验，对手本身所做的大量工程工作显然也是整个故事的重要部分。新的制造技术有可能真正颠覆这一领域——像 FLLEX v2 的这个视频看起来就非常惊艳[9]——我也非常乐观地认为，在不太遥远的未来，我们将拥有能力更强、鲁棒性更高的灵巧手。

2.3.2 简单夹爪

[点击这里观看视频。](#)

Figure 2.4 - 这个由 Ken Salisbury 团队制作的 PR1 遥操作视频，如今已经成为使用非常简单的手完成极其有用任务的经典示例。查看他们的[网站](#)可以看到更多视频，包括扫地、取啤酒和卸洗碗机等任务。

另一派观点认为，其实并不需要灵巧手——我甚至可以从玩具店给你一个简单夹爪，而你依然能够在家完成许多极其有用的任务。上面的 PR1 视频就是这一点的绝佳展示。



支持简单手部的另一个重要理由，是减少复杂性所带来的优雅性与清晰性。如果对简单夹爪的深入思考，能够帮助我们更深刻地理解为什么我们需要更灵巧的手（我认为确实会如此），那就再好不过了。在这些笔记的大部分内容中，一个简单的双指夹爪最符合我们的教学目标。特别是，我选择了 Schunk WSG 050，它是我们过去几年研究中广泛使用的一款夹爪。我们也会在后续章节中探索几种不同的末端执行器，当它们有助于解释相关概念时。

需要明确的是：一个手部结构简单（自由度少）并不意味着它质量低。恰恰相反，Schunk WSG 是一款高质量夹爪，它在其单自由度上具备力控制和力测量能力，其精度甚至超过了 Kuka。要在具有许多关节的灵巧手中实现同样水平的性能，会非常困难。

2.3.3 软体/欠驱动手

最后，第三个也是最新的阵营正在推动一些巧妙的手部机械设计，这类手通常被称为“欠驱动手”。基本思想是，对于许多任务来说，你的手中也许并不需要和关节数量一样多的执行器。许多欠驱动手使用缆索驱动机构来闭合手指，其中一根肌腱就可以让手指中的多个关节弯曲。如果设计得当，这些机构可以让手指在执行器命令不变的情况下，被动地贴合被抓取物体的形状（参见[10]）。要实现这一概念，并不一定需要缆索；使用巧妙的刚性机械连杆，也可以实现性质上类似的行为。



Figure 2.5 - 欠驱动手。左：RightHand Robotics Reflex2 是 i-HY 手的后继产品[10]。右：Robotiq 三指夹爪。

后对气球施加真空会使颗粒介质发生“堵塞”，迅速在物体周围硬化，从而形成稳定抓握 [11]。

[点击这里观看视频。](#)

[这里](#)还有另一种巧妙设计，它在指尖配备了主动滚轮，以帮助实现手内重新定向。

最后，反对灵巧手的一个合理论点是：即使是人类，许多最有趣的操作也往往不是直接用手完成的，而是通过工具完成的。我特别喜欢 Matt Mason 多年来作为简单夹爪主要倡导者之一，在[我们一次机器人研讨会最后的一个问题](#)中给出的回答：他认为，例如厨房中的实用机器人，可能会配备可以快速更换的专用工具。在灵巧手的主要任务是更换工具的应用中，我们也许可以通过直接在机器人上安装一个“[工具快换装置](#)”，并使用与工具快换装置兼容的工具，从而避开复杂性。

2.3.5 如果你还没看过它……

有一次我参加一个活动，报名表问我们：“你最喜欢的机器人是什么，无论是真实的还是虚构的？”对于一个热爱机器人的人来说，这是个很难回答的问题！但我给出的答案是 [Ishikawa 团队](#) 开发的一款超级酷的“高速多指手”；这个项目早在 2004 年就开始产生令人惊叹的成果了！他们对这只手进行了“超频”——在短时间内输入比任何长时间应用中合理值更大的电流——并且还使用高速摄像机来实现这些成果。他们在 2017 年也做了一个[魔方演示](#)。

[点击这里观看视频。](#)

太棒了！

2.4 传感器

我还没有过多讨论传感器。事实上，当我们进入使用（深度）相机进行感知，以及思考[触觉传感](#)时，传感器将成为我们的一个重要主题。但我会把这些主题推迟到真正需要它们时再讲。

现在，让我们先关注机器人上的关节传感器。iiwa 和 Schunk WSG 都提供关节反馈——iiwa 驱动器会在其七个关节中的每一个上给出“测量位置”“估计速度”和“测量力矩”；请记住，关节加速度通常被认为噪声太大，不能依赖。类似地，Schunk WSG 输出“测量状态”（位置 + 速度）和“测量力”。我们可以把所有这些都作为端口提供给框图。

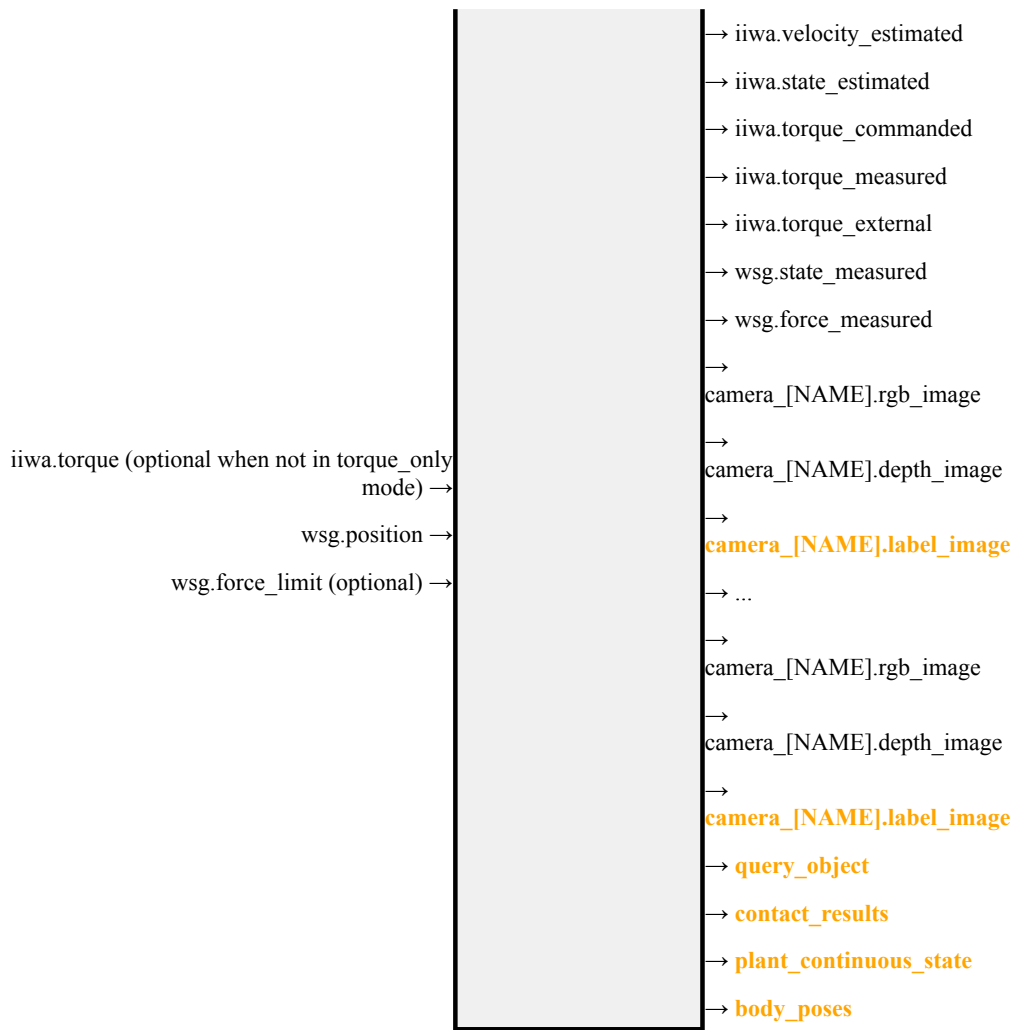
2.5 把所有内容组合起来

如果你已经完成了这些示例，就会看到，对我们机器人的恰当仿真并不只是一个物理引擎——它需要把物理模型、执行器和传感器模型，以及底层机器人控制器组装到一个共同框架中。实际上，在 Drake 中，这意味着我们正在组装越来越复杂的框图。

2.5.1 HardwareStation

框图建模范式最棒的事情之一，就是抽象和封装的力量。我们可以组装一个 `Diagram`，其中包含仿真我们的硬件平台及其环境所需的所有组件，我们会亲切地把它称为“`Hardware Station`”。`MakeHardwareStation` 方法会接收一个描述场景和机器人硬件的 YAML 文件。对于一个描述 iiwa + WSG 以及若干相机的 yaml 文件，最终得到的 `HardwareStation` 系统看起来如下：

```
iiwa.position (optional when not in  
position_only mode) → HardwareStation → iiwa.position_commanded  
→ iiwa.position_measured
```



上图中用 **橙色** 标出的输出端口是“作弊端口”——它们在仿真中可用，但在真实机器人运行时不可用（因为它们假设能够获得真实值知识）。

Example 2.6 (遥操作演示中的 Hardware station)

第一章中的遥操作笔记本使用了 `MakeHardwareStation` 接口来搭建仿真。现在你应该更清楚这个子系统内部到底发生了什么！如果你想再次查看那个示例，链接如下：

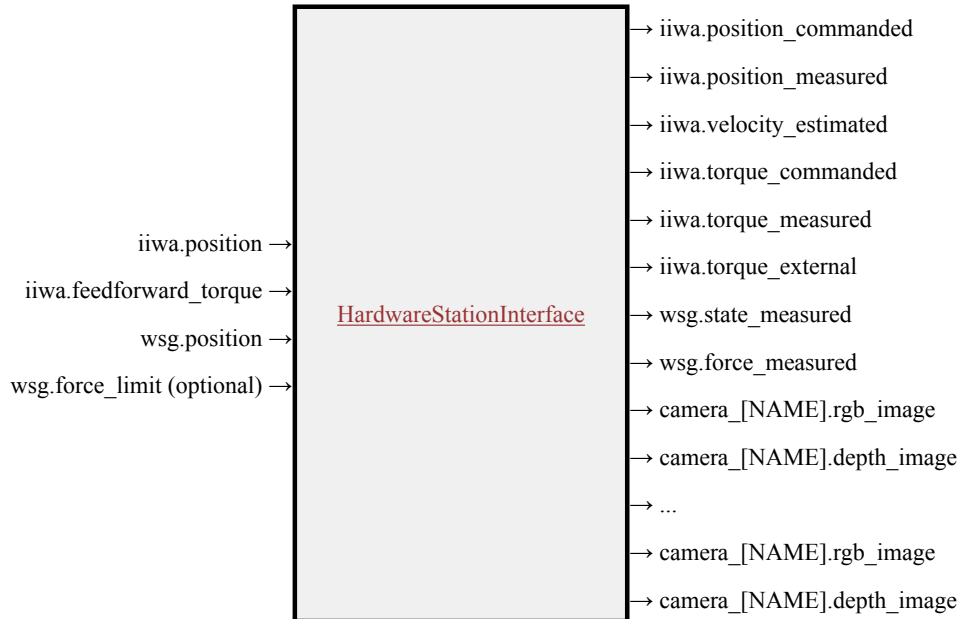
Example 2.7 (双臂操作站)

通过向 YAML 文件中再添加几行，我们可以使用同一个 `MakeHardwareStation` 方法来构建一个双臂操作站：

如果你还想仿真其他机器人/驱动器，可以直接在与这些笔记关联的 `manipulation` 仓库中对 `station.py` 文件进行本地修改，或者直接告诉我，我大概可以很快把它们加进去。

2.5.2 HardwareStationInterface

正如你在示例中看到的，`HardwareStation` 图本身旨在作为一个 `System` 用于其他图中，这些图可以包含我们的感知、规划以及更高层次的控制系统。这个模型还定义了仿真与真实硬件之间的抽象边界。只需向 `MakeHardwareStation` 方法传入 `hardware=True`，我们就会构建出一个几乎相同的系统，即 `HardwareStationInterface`。



`HardwareStationInterface` 也是一个图，但它并不是由 `MultibodyPlant` 和 `SceneGraph` 这样的仿真组件构成，而是由一些执行网络消息传递的系统组成，用来与那些和各个硬件驱动程序通信的小型可执行程序进行接口对接。如果你深入查看内部实现，就会看到我们在这里使用的是 `LCM`，而不是 ROS 消息；主要原因是，对于我们的公共仓库来说，`LCM` 是一个更轻量级的依赖（另一个原因是，在驱动层接口中，多播 UDP 比 TCP/IP 更合适）。不过，许多 Drake 开发者/用户也在 [ROS/ROS2 生态系统中使用 Drake](#)。

如果你确实拥有类似的机器人硬件，并且想在自己的机器上运行硬件接口，我已经在[附录](#)中整理一份驱动程序和物料清单。

2.5.3 HardwareStation 独立仿真

使用 `HardwareStation` 工作流程，可以很容易地从仿真中的开发过渡到真实机器人上运行。支持这一工作流程的另一个工具，是独立的 `hardware_sim` 可执行程序。这个 Python 脚本会接收同一个 YAML 文件作为输入（通过命令行），并在一个单独进程中启动仿真，其行为就像真实机器人硬件一样……发送和接收硬件侧的消息。这对于测试你的所有逻辑在加入消息传递层所带来的延迟和非确定性之后是否仍然有效很有价值；而在开发早期阶段，当我们使用 `MakeHardwareStation(..., hardware=False)` 时，会巧妙地避开这些问题。

```
python3 drake/examples/hardware_sim/hardware_sim.py
--scenario_file=station.yaml --scenario_name=Name
```

2.6 更多 `HARDWARESTATION` 示例

我非常喜欢学生们为这门课完成的项目。为了帮助实现这些项目（也希望能帮助你未来的项目），我会在这里收集更多示例，展示如何为不同的硬件配置搭建 `HardwareStation`。可以期待这个列表会随着时间不断增长！

Example 2.8 (带 Allegro 手的 iiwa)

这里有一个简单示例，展示如何仿真安装了 Allegro 手而不是 Schunk WSG 夹爪的 iiwa。（注意，Allegro 手有左手版和右手版可用。）

2.7 练习

Exercise 2.1 (反射惯量的作用)

在这个练习中，你将研究反射惯量对机器人关节空间动力学的影响，以及它如何影响简单的位置控制律。你将完全在 `中` 完成。你需要完成以下步骤：

- 推导带有电机和齿轮箱的简单摆的一阶状态空间动力学 $\dot{\mathbf{x}} = f(\mathbf{x}, \mathbf{u})$ 。
- 在相同的位置控制律下，比较直接驱动简单摆和带高减速比齿轮箱的简单摆的行为。

Exercise 2.2 (Manipulation Station 上的输入和输出端口)

在这个练习中，你将研究操作站在 Drake 系统级框架中是如何被抽象的。你将完全在 `中` 完成。你需要完成以下步骤：

- 学习如何探查操作站的输入和输出端口，并计算它们的内容。
- 通过探查端口值，探索不同端口分别对应什么。

Exercise 2.3 (Drake 中的直接关节遥操作)

在这个练习中，你将实现一种在 Drake 中控制机器人关节的方法。你将完全在 `中` 完成，并应参考 `中`。你需要完成以下步骤：

- 将第 1 章示例中的遥操作接口替换为不同的 Drake 函数，使其能够直接控制机器人的关节。

Exercise 2.4 (Drake 中的 PID 控制)

在这个练习中，你将为 Drake 中机器人的关节实现一个 PID 控制器。你将完全在 `中` 完成。你需要完成以下步骤：

- 为机器人的关节实现一个 PD 控制器。
- 将 PD 控制器扩展为完整的 PID 控制器。

REFERENCES

1. Jemin Hwangbo and Joonho Lee and Alexey Dosovitskiy and Dario Bellicoso and Vassilios Tsounis and Vladlen Koltun and Marco Hutter, "Learning agile and dynamic motor skills for legged robots", *Science Robotics*, vol. 4, no. 26, pp. eaau5872, 2019.
2. Haruhiko Asada and Kamal Youcef-Toumi, "Direct-Drive Robots - Theory and Practice", The MIT Press , 1987.
3. P. M. Wensing and A. Wang and S. Seok and D. Otten and J. Lang and S. Kim, "Proprioceptive Actuator Design in the MIT Cheetah: Impact Mitigation and High-Bandwidth Physical Interaction for Dynamic Legged Robots", *IEEE Transactions on Robotics*, vol. 33, no. 3, pp. 509-522, June, 2017.
4. Navvab Kashiri and Jörn Malzahn and Nikos Tsagarakis, "On the Sensor Design of Torque Controlled Actuators: A Comparison Study of Strain Gauge and Encoder Based Principles", *IEEE Robotics and Automation Letters*, vol. PP, 02, 2017.
5. A Wedler and M Chalon and K Landzettel and M Görner and E Krämer and R Gruber and A Beyer and HJ Sedlmayr and B Willberg and W Bertleff and others, "DLRs dynamic actuator modules for robotic space applications", *Proceedings of the 41st Aerospace Mechanisms Symposium* , May 16-18, 2012.
6. G. A. Pratt and M. M. Williamson, "Series elastic actuators", *Proceedings 1995 IEEE/RSJ International Conference on Intelligent Robots and Systems. Human Robot Interaction and Cooperative Robots* , vol. 1, pp. 399-406 vol.1, Aug, 1995.
7. Alin Albu-Schaffer and Christian Ott and Gerd Hirzinger, "A unified passivity-based control framework for position, torque and impedance control of flexible joint robots", *The international journal of robotics research*, vol. 26, no. 1, pp. 23--39, 2007.
8. Tao Pang and Russ Tedrake, "A Robust Time-Stepping Scheme for Quasistatic Rigid Multibody Systems", *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)* , 2018. [[link](#)]
9. Yong-Jae Kim and Junsuk Yoon and Young-Woo Sim, "Fluid Lubricated Dexterous Finger Mechanism for Human-Like Impact Absorbing Capability", *IEEE Robotics and Automation Letters*, vol. 4, no. 4, pp. 3971--3978, 2019.
10. Lael U. Odhner and Leif P. Jentoft and Mark R. Claffee and Nicholas Corson and Yaroslav Tenzer and Raymond R. Ma and Martin Buehler and Robert Kohout and Robert D. Howe and Aaron M. Dollar, "A Compliant, Underactuated Hand for Robust Manipulation", *International Journal of Robotics Research (IJRR)*, vol. 33, no. 5, pp. 736-752, 2014.
11. Eric Brown and Nicholas Rodenberg and John Amend and Annan Mozeika and Erik Steltz and Mitchell R Zakin and Hod Lipson and Heinrich M Jaeger, "Universal robotic gripper based on the jamming of granular material", *Proceedings of the National Academy of Sciences*, vol. 107, no. 44, pp. 18809–18814, 2010.

CHAPTER 3

基础抓取与放置

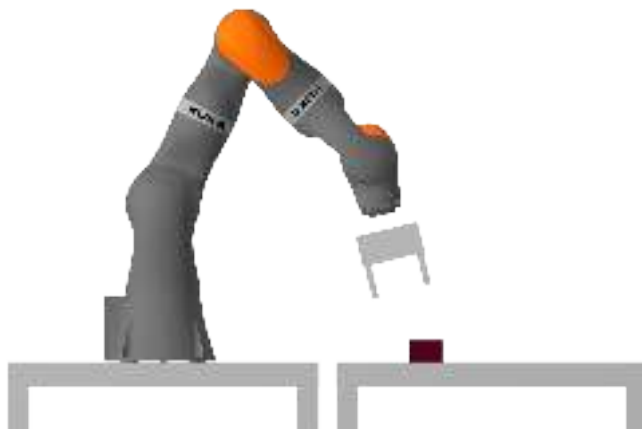


Figure 3.1 - 你的挑战：控制机器人抓起积木，并将其放置到目标位置/姿态。

舞台已经搭建完成。你拥有了自己的机器人，而我这里有一块红色泡沫砖。我会把它放在机器人前方的桌子上，而你的目标是将它移动到桌面上的指定位置/姿态。我想再推迟一个章节再讨论感知 (*perception*) 问题，因此这里你可以假设自己能够获得关于积木当前位置/姿态的完美测量值。即使不考虑感知，完成这项任务仍然需要我们建立关于几何学和运动学的一套基础工具；这也是一个非常自然的起点。

首先，我们将为运动学建立一些术语和符号表示。在这一领域，严谨的符号体系会带来巨大的收益，而随意混乱的符号则几乎必然导致困惑和程序错误。Drake 的开发者们花费了大量精力来建立并记录一套一致的 **多体系统符号体系**，我们称之为“Monogram Notation (字母标记法)”。相关文档甚至还包含了这套符号背后的动机与设计理念。在本书中，我都会统一使用这种 monogram notation。

如果你希望获得比这里更系统、更深入的运动学背景知识，我最喜欢的参考书仍然是 [1]。至于免费的在线资源，Murray 等人在 1994 年出版（现已免费开放在线阅读）的书籍中的第 2、3 章 [2] 也非常优秀；此外，Lynch 与 Park 所著 *Modern Robotics* 的前七章同样值得推荐 [3]（他们还配有非常优秀的视频课程）。不过需要注意的是，三份参考资料会使用三种（略有不同的）符号体系；我们这里使用的体系与 [1] 最为接近。Monogram notation 的详细构建过程可以参考 [4]。

请不要被如此庞大的背景知识吓倒！我个人认为，只要真正理解少数几个基本概念，就足以让你在这一领域中非常高效。更多细节会在之后需要的时候再逐步展开。

3.1 MONOGRAM NOTATION (字母标记法)

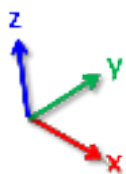
下面这些概念微妙得令人意外。我见过许多极其聪明的人自认为已经理解它们，结果却长期在符号表示上不断犯错。我自己也曾如此多年。请花一点时间认真阅读！

也许几何学中最基础的概念就是“点 (point)”。点占据空间中的某个位置，并且它们可以拥有名字，例如点 A 、 C ，或者更具描述性的名字，比如表示刚体 B 质心的 B_{cm} 。我们使用位置向量 p^A 来表示该点的位置；这里的 p 表示 position (位置)，而不是 point (点)，因为其他几何量同样也可能具有位置。

但让我们更严谨一些。位置实际上是一个相对量。 严格来说，我们只应该描述两个点之间的相对位置。 例如，我们用 ${}^A p^C$ 表示“从点 A 测量到点 C 的位置”。左上角的上标看起来确实有些奇怪， 但当我们开始进行点的坐标变换时，你会发现这种写法非常有用。

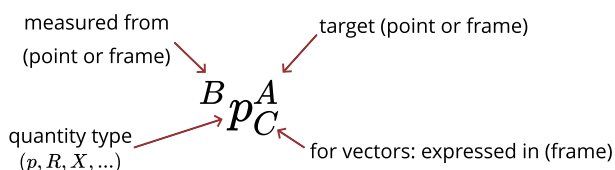
每当我们使用一个数字向量来描述（相对）位置时， 都必须明确说明所使用的参考坐标系，特别是“表达于（expressed-in）”哪个坐标系。 我们所有的坐标系都由满足“右手定则”的正交单位向量定义。 我们也会给坐标系命名，例如 F 。 如果我要表示“点 C 相对于点 A 的位置，并且该向量用坐标系 F 表达”， 我会写成 ${}^A p_F^C$ 。 如果我只想取这个向量中的某一个分量，例如 x 分量， 那么我会写成 ${}^A p_{F_x}^C$ 。 从某种意义上说，“expressed-in”坐标系更像是一种实现细节； 只有当我们需要把多体系统中的物理量表示成一个向量（例如在计算机中）时， 它才是必须明确指定的。

这套符号确实相当沉重。 我自己其实也不算特别喜欢它， 但这是目前我所知道最稳健、最不容易出错的表示方法； 当上下文足够清晰时，我们之后也会使用一些简写形式。



有几个非常特殊的坐标系。 我们使用 W 表示世界坐标系（world frame）。 在 Drake 中，我们采用车辆坐标系的约定来理解世界坐标系：正 x 指向前方，正 y 指向左侧，正 z 指向上方。 另外一类特别重要的坐标系是刚体坐标系（body frames）：多体物理引擎中的每一个刚体都会绑定一个唯一的坐标系。 我们通常使用 B_i 来表示第 i 个刚体对应的坐标系。

坐标系本身也具有位置——这个位置对应于坐标系原点。 因此，写出 ${}^W p_W^A$ 来表示“点 A 相对于世界坐标系原点的位置， 并且使用世界坐标系表达”是完全合理的。 这里就要引入简写规则了。 如果某个量的位置是“相对于某个坐标系测量”， 并且同时“也用该坐标系表达”， 那么我们可以安全地省略下标。 即： ${}^F p^A \equiv {}^F p_F^A$ 。 此外，如果“相对于谁测量（measured from）”这一部分被省略， 那么默认认为是相对于世界坐标系 W 测量， 因此 $p^A \equiv {}^W p_W^A$ 。



坐标系也具有姿态方向。我们将使用 R 表示一个旋转（rotation）， 并沿用相同的符号规则，写作 ${}^B R^A$ ，表示“坐标系 A 相对于坐标系 B 的方向”。 与向量不同，纯旋转并不额外具有一个“表达于（expressed in）”的坐标系。

一个坐标系 F 可以通过相对于另一个坐标系测得的位置和旋转被完整指定。 将位置和方向合在一起，我们称之为空间位姿（spatial pose），简称位姿（pose）。 空间变换（spatial transform），简称变换（transform）， 可以理解为 pose 的“动词形式”。 在 Drake 中，我们使用 RigidTransform 来表示位姿/变换，并用字母 X 表示它。 ${}^B X^A$ 表示“坐标系 A 相对于坐标系 B 的位姿”。 当我们谈论某个物体 O 的位姿，而没有显式提到参考坐标系时， 我们指的是 ${}^W X^O$ ，其中 O 是该物体的刚体坐标系。 对于位姿，我们不使用“表达于”坐标系的下标； 我们始终希望位姿用参考坐标系来表达。

Drake 的文档也讨论了如何在代码中使用这种符号。 简而言之， ${}^B p_C^A$ 写作 `p_BA_C`， ${}^B R^A 写作 `R_BA`， 而 ${}^B X^A$ 写作 `X_BA`。 这套规则确实可行，我保证。$

3.2 通过空间变换实现抓取与放置

现在我们已经有了符号体系，就可以表述处理基础抓取与放置问题的方法了。 我们将物体称为 O ，将夹爪称为 G 。 理想化的感知传感器会告诉我们 ${}^W X^O$ 。 让我们创建一个坐标系 O_d ，用于描述物体的“期望”位姿，即 ${}^W X^{O_d}$ 。 因此，抓取与放置操作本质上就是试图使 $X^O = X^{O_d}$ 。

为了实现这一点，我们将假设：当夹爪张开时，物体相对于世界坐标系不运动（ ${}^W X^O$ 保持不变）；当夹爪闭合时，物体相对于夹爪不运动（ ${}^G X^O$ 保持不变）。于是我们可以：

- 将世界坐标系中的夹爪 X^G 移动到一个相对于物体测得的合适位姿： ${}^O X^{G_{grasp}}$ 。
- 闭合夹爪。
- 将夹爪+物体移动到期望位姿，即 $X^O = X^{O_d}$ 。
- 张开夹爪，并将机械手撤回。

还有一个重要细节：为了在接近物体时避免与它发生碰撞，我们会在物体上方插入一个“预抓取位姿（pregrasp pose）”，即 ${}^O X^{G_{pregrasp}}$ ，作为中间步骤。当我们放下物体后撤离时，也会使用同一个变换。

显然，要编写实现这一策略的程序，我们需要一套好用的工具来处理这些变换，并将夹爪的位姿与机器人的关节角联系起来。

3.3 空间代数

这里我们开始看到前面那套繁重符号体系带来的回报：我们将定义在不同坐标系之间转换位置、旋转、位姿等量的规则。如果没有这套符号，这件事总会变成我把右手举在空中比划“右手定则”，同时脑袋在空间里跟着扭来扭去。有了这套符号，它就变成了把各个符号正确对齐的简单问题，而我们也更有可能得到正确答案！

下面是这些空间量的基本代数规则：

- 当位置向量用同一个坐标系表达，并且它们的参考符号与目标符号相匹配时，可以相加：

$${}^A p_F^B + {}^B p_F^C = {}^A p_F^C. \quad (1)$$

加法满足交换律，并且加法逆元也有明确定义：

$${}^A p_F^B = -{}^B p_F^A. \quad (2)$$

这些规则应该相当直观；请务必自己验证一下。

- 乘以一个旋转可以用来改变“表达于”的坐标系：

$${}^A p_G^B = {}^G R^{FA} {}^B p_F^B. \quad (3)$$

你可能会惊讶：仅靠一个旋转就足以改变 expressed-in 坐标系，但事实确实如此。expressed-in 坐标系的位置 **不会** 影响两个点之间的相对位置。

- 当旋转的参考符号和目标符号相匹配时，旋转可以相乘：

$${}^A R^B {}^B R^C = {}^A R^C. \quad (4)$$

其逆运算也可以简单定义为：

$$\left[{}^A R^B \right]^{-1} = {}^B R^A. \quad (5)$$

当旋转被表示为旋转矩阵时，这实际上就是矩阵逆；又因为旋转矩阵是正交归一的，所以我們还有 $R^{-1} = R^T$ 。

- 当位置是相对于某个坐标系测量的（并且也用同一个坐标系表达）时，变换会把这些内容打包成一种统一且方便的符号：

$${}^G p^A = {}^G X^{FF} p^A = {}^G p^F + {}^F p_G^A = {}^G p^F + {}^G R^{FF} p^A. \quad (6)$$

- 变换可以复合：

$${}^A X^{BB} X^C = {}^A X^C, \quad (7)$$

并且具有逆：

$$\left[{}^A X^B \right]^{-1} = {}^B X^A. \quad (8)$$

请注意，对于变换而言，我们通常 **不能** 认为 X^{-1} 等于 X^T ，尽管它仍然具有一个简单形式。

在实际实现中，变换通常使用 齐次坐标 (homogenous coordinates) 来表示，不过目前我愿意把这部分当作一种实现细节。

Example 3.1 (从相机坐标系到世界坐标系)

假设我在工作空间中固定安装了一台深度相机。我们将相机坐标系称为 C ，并用 ${}^W X^C$ 表示它在世界坐标系中的位姿。

深度相机会返回位于相机坐标系中的点。因此，我们将点 P_i 的位置写作 ${}^C p^{P_i}$ 。如果我们想把这个点转换到世界坐标系中，只需执行：

$$p^{P_i} = X^{C C} p^{P_i}.$$

这是我们最核心、最常用的操作之一。我们经常需要将来自多个相机的点融合在一起（通常是在世界坐标系中），并且总是需要以某种方式把相机坐标系与机器人坐标系联系起来。反向变换 ${}^C X^W$ （即把世界坐标投影到相机坐标系中）通常被称为相机的“外参 (extrinsics)”。

3.3.1 三维旋转的表示方法

在前面的空间代数中，我使用的是一种抽象意义上的“旋转”来书写相关规则。但为了在代码中实现这些代数运算，我们需要决定如何使用少量实数来表示这些旋转。三维旋转存在许多不同的表示方式，它们分别适用于不同类型的运算，不幸的是，并不存在一种能够完美适用于所有场景的统一表示。（这也是为什么二维问题总是更简单的众多原因之一！）常见的表示方式包括：

- 3×3 旋转矩阵，
- 欧拉角 (例如 roll-pitch-yaw)，
- 轴角表示 (axis-angle)（也称为“指数坐标”），以及
- 单位四元数 (unit quaternions)。

在 Drake 中，我们提供了所有这些表示方式，并且可以方便地在它们之间互相转换。

一个 3×3 的 旋转矩阵 是一个正交归一矩阵，它的列向量由 x -、 y -、 z - 轴构成。更具体地说，在变换 ${}^G R^F$ 中，第一列表示坐标系 F 的 x 轴在坐标系 G 中的表达，其余列依此类推。

欧拉角 (Euler angles) 通过绕 x 、 y 、 z 轴的一系列旋转来描述一个三维旋转。这些旋转的顺序非常重要；并且存在许多不同的组合方式来描述任意三维旋转。这也是为什么我们在代码中使用 `RollPitchYaw`（而不是更泛化的“Euler angle”术语），并且对它进行了 详细文档说明。Roll 表示绕 x 轴旋转，pitch 表示绕 y 轴旋转，yaw 表示绕 z 轴旋转；这种形式也被称为“外旋 X-Y-Z (extrinsic X-Y-Z) 欧拉角”。

轴角表示 (axis-angle) 使用一个任意方向的旋转轴以及绕该轴旋转的角度来描述三维旋转，总共使用三个数：向量的方向表示旋转轴，向量的模长表示旋转角度。你可以把单位四元数理解为一种经过精心归一化后的轴角表示，它被约束为单位长度，并具有许多神奇的数学性质。关于四元数，我最喜欢的一份严谨介绍大概是 [5] 的第一章。

为什么会需要这么多种表示方式？“roll-pitch-yaw”难道还不够吗？不幸的是，并不够。这个限制最容易通过观察 roll-pitch-yaw 与旋转矩阵之间的坐标变换来理解。任意的 roll-pitch-yaw 都可以转换成旋转矩阵，但反向映射却存在奇异性（singularity）。特别地，当 pitch 角等于 $\frac{\pi}{2}$ 时，roll 与 yaw 将变得无法区分。关于这一点，以及它在物理上的体现——“万向节死锁（gimbal lock）”，在[这个视频](#)中有非常精彩的解释。类似地，在轴角表示（axis-angle）中，当旋转角度为零时，向量方向并不是唯一确定的。当我们开始对旋转求导时（例如在推导运动方程时），这些奇异性就会变得非常棘手。现在人们已经充分认识到 [5]：要正确表示三维旋转群，至少需要四个数；单位四元数则是最常见的四维表示方法。

3.4 正向运动学

空间代数已经让我们距离实现抓取与放置算法非常接近了。但请记住，我们与机器人的接口所返回的是测量得到的关节位置，而机器人所接收的命令也同样是关节位置。因此，我们剩下的任务就是在关节角与笛卡尔坐标系之间进行转换。我们会分步骤完成这件事，第一步是从关节位置推导出笛卡尔坐标系；这被称为**正向运动学**（*forward kinematics*）。

在本书中，我们将使用向量 q 来表示机器人的关节位置（也称为机器人的“构型 configuration”）。如果场景中的构型不仅包含机器人，还包含环境中的物体，那么我们会使用 q 表示整个场景的构型向量，并使用例如 q_{robot} 来表示其中对应机器人关节位置的子向量。因此，正向运动学的目标是构建如下映射：

$$X^G = f_{\text{kin}}^G(q). \quad (9)$$

此外，我们希望能够对场景中定义的任意坐标系都进行正向运动学计算。我们之前建立的空间符号体系与空间代数，使得这一计算过程变得相对直接清晰。

3.4.1 运动学树

为了便于进行运动学以及相关的多体系统计算，`MultibodyPlant` 会将世界中的所有刚体组织成一种树形拓扑结构。每个刚体（除了世界刚体 `world body`）都有一个父节点，它通过一个 `Joint`（关节）或一个“浮动基座（floating base）”与父节点相连接。

Example 3.2 (检查运动学树)

Drake 提供了一些可视化支持，用于检查运动学树这一数据结构。iiwa 的运动学树与其说是一棵树，不如说更像一根藤蔓（因为它是一个串联机械臂）；不过，灵巧手的树结构就更有意思一些。我还把我们的砖块也加入到了这个示例中，这样你就可以看到，一个“自由”刚体其实只是从世界根节点分出的另一条分支。

每个 `Joint` 和“浮动基座（floating base）”都关联着若干个位置变量——也就是构型向量 q 的一个子集——并且知道如何计算跨越该关节、依赖于构型的变换：从子关节坐标系 J_C 到父关节坐标系 J_P 的变换 ${}^{J_P}X^{J_C}(q)$ 。此外，运动学树还定义了从关节坐标系到子刚体坐标系的（固定）变换 ${}^C X^{J_C}$ ，以及从关节坐标系到父坐标系的（固定）变换 ${}^P X^{J_P}$ 。将这些组合起来，我们就可以计算任意一个刚体与其父刚体之间的构型变换：

$${}^P X^C(q) = {}^P X^{J_P} {}^{J_P} X^{J_C}(q) {}^C X^{J_C}.$$

你可能会倾向于认为，每当你向 `MultibodyPlant` 添加一个关节时，你就是在增加一个自由度。但实际上情况正好相反。每当你向 `plant` 中添加一个**刚体**时，你都会增加许多自由度。随后你可以再添加关节来**移除**这些自由度；关节本质上是约束。将机器人的基座“焊接（welding）”到

世界坐标系上，会移除基座的所有浮动自由度。在子刚体与父刚体之间添加一个转动关节，则会移除除一个自由度之外的所有自由度，依此类推。

3.4.2 用于抓取与放置的正向运动学

为了计算夹爪在世界坐标系中的位姿 X^G ，我们只需查询运动学树中夹爪坐标系的父节点，并递归地复合各个变换，直到到达世界坐标系。

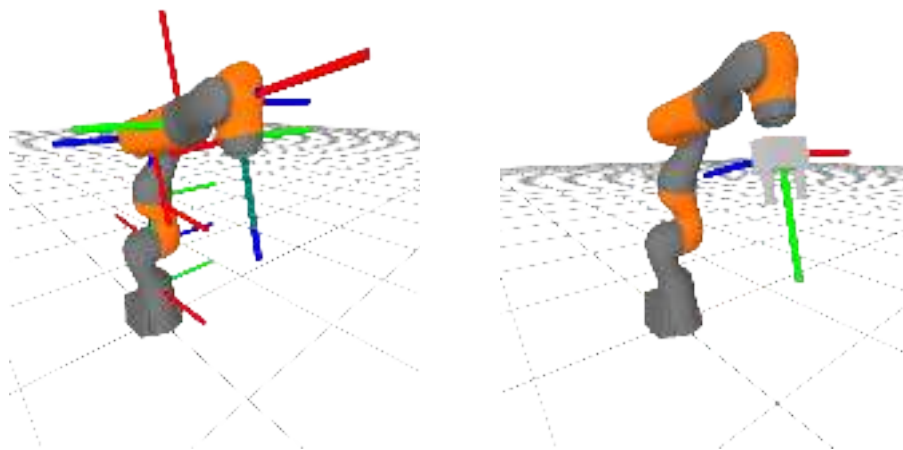


Figure 3.3 - iiwa (左) 和 WSG 夹爪 (右) 上的运动学坐标系。对于每个坐标系，正 x 轴为红色，正 y 轴为绿色，正 z 轴为蓝色。这（希望）很容易记住：XYZ $\Leftrightarrow$ RGB。

Example 3.3 (夹爪坐标系的正向运动学)

让我们计算夹爪在世界坐标系中的位姿： X^G 。我们知道它会是机器人构型的一个函数，而机器人构型只是 `MultibodyPlant` 总状态的一部分（因此它存储在 `Context` 中）。下面的示例展示了它是如何工作的。

关键代码行是：

```
gripper = plant.GetBodyByName("body", wsg)
pose = plant.EvalBodyPoseInWorld(context, gripper)
```

在幕后，`MultibodyPlant` 正在执行我们前面描述过的全部空间代数运算，从而返回该位姿（同时还会做一些巧妙的缓存，因为当你想计算同一机器人上另一个坐标系的位姿时，很多计算都可以被复用）。

Example 3.4 (“浮动基座”物体的正向运动学)

考虑一个特殊情况：某个 `MultibodyPlant` 中只添加了一个刚体，并且没有任何关节。此时运动学树只是由世界坐标系、刚体坐标系组成，它们之间通过“浮动基座（floating base）”连接。那么在这种情况下，正向运动学函数

$$X^B = f_{kin}^B(q),$$

会是什么样子？如果 q 已经表示了浮动基座的构型，那么 f_{kin}^B 是否只是恒等函数呢？

这会涉及我们如何表示变换的一些微妙问题，尤其是[我们如何表示三维旋转](#)。尽管在 `RigidTransform` 类中，为了让空间代数运算高效，我们使用的是旋转矩阵；但在构型向量 q 和 `Context` 中，为了获得紧凑表示，我们实际上使用的是单位四元数。

因此，在这个例子中，函数 f_{kin}^B 的软件实现，正是将“位置 $\times$ 单位四元数”表示转换为“位姿（位置 $\times$ 旋转矩阵）”表示的函数。

3.5 微分运动学（雅可比矩阵）

正向运动学机制使我们能够计算夹爪的位姿以及物体的位姿，并且二者都位于世界坐标系中。但如果我们的目标是[移动](#)夹爪使其到达物体位置，那么我们就需要理解：关节角的变化是如何影响夹爪位姿变化的。这通常被称为“微分运动学（differential kinematics）”。

乍一看，这似乎很直接。位姿变化与关节位置变化之间的关系，可以通过正向运动学的（偏）导数来描述：

$$dX^B = \frac{\partial f_{kin}^B(q)}{\partial q} dq = J^B(q) dq. \quad (10)$$

在许多领域中，函数的偏导数被称为“Jacobian（雅可比矩阵）”；而在机器人学中，人们几乎不会用别的名字来称呼运动学导数。

所有微妙之处，再一次地，来自于三维旋转存在多种表示方式（旋转矩阵、单位四元数等等）。尽管不存在一种对所有场景都最优的三维旋转表示，但对于[微分](#)旋转，却[确实](#)存在一种规范化表示。在无需担心奇异性、也不会损失一般性的情况下，我们可以使用一个六维向量来表示位姿变化率，即[空间速度（spatial velocity）](#)：

$${}^A V_C^B = \begin{bmatrix} {}^A \omega_C^B \\ {}^A v_C^B \end{bmatrix}. \quad (11)$$

${}^A V_C^B$ 表示坐标系 B 相对于坐标系 A 的空间速度（也称为“twist”），并且使用坐标系 C 表达；其中 ${}^A \omega_C^B \in \mathbb{R}^3$ 是[角速度（angular velocity）](#)（即坐标系 B 相对于 A 的角速度，并在坐标系 C 中表达），而 ${}^A v_C^B \in \mathbb{R}^3$ 是[平移速度（translational velocity）](#)（其简写规则与位置向量相同）。角速度是一个三维向量（包含 w_x 、 w_y 、 w_z 分量）；该向量的模长表示角速度大小，向量方向则表示（瞬时）旋转轴方向。人们很容易误以为它是 roll、pitch、yaw 的时间导数，但实际上并不是；它们之间可以通过一个[线性映射](#)方便地互相转换。它也并不是轴角表示（axis-angle）的时间导数；关于这一转换，可参考例如[这里](#)。

空间速度（spatial velocity）能够很好地融入我们的空间代数体系：

- 旋转可以用于在不同“expressed-in”坐标系之间进行转换：

$${}^A v_G^B = {}^G R^F {}^A v_F^B, \quad {}^A \omega_G^B = {}^G R^F {}^A \omega_F^B. \quad (12)$$

- 当角速度使用同一个坐标系表达时，它们可以直接相加：

$${}^A \omega_F^B + {}^B \omega_F^C = {}^A \omega_F^C, \quad (13)$$

（这一点[值得亲自验证](#)）并且其加法逆元满足 ${}^A \omega_F^C = -{}^C \omega_F^A$ ，

- 平移速度的复合则稍微复杂一些：

$${}^A v_F^C = {}^A v_F^B + {}^B v_F^C + {}^A \omega_F^B \times {}^B p_F^C. \quad (14)$$

这个式子可以通过几个步骤推导得到（[点击三角展开](#)）

对

$${}^A p^C = {}^A X^{BB} p^C = {}^A p^B + {}^A R^{BB} p^C,$$

求导, 可得

$$\begin{aligned} {}^A \dot{v}^C &= {}^A \dot{v}^B + {}^A \dot{R}^{BB} p^C + {}^A R^{BB} \dot{v}^C \\ &= {}^A \dot{v}_A^B + {}^A \dot{R}^{BB} R^{AB} p_A^C + {}^B v_A^C. \end{aligned} \quad (15)$$

允许我暂时省略坐标系标记, 用 $\dot{R}R^T = \dot{R}R^{-1}$ 来表示 ${}^A \dot{R}^{BB} R^A$ 。事实证明, $\dot{R}R^T$ 总是一个斜对称矩阵 (skew-symmetric matrix)。为了证明这一点, 对 $RR^T = I$ 求导, 得到

$$\dot{R}R^T + R\dot{R}^T = 0 \Rightarrow \dot{R}R^T = -R\dot{R}^T \Rightarrow \dot{R}R^T = -(\dot{R}R^T)^T,$$

这正是斜对称矩阵的定义。任意一个 3×3 的斜对称矩阵都可以用三个数来参数化 (对于 ${}^A \dot{R}^{BB} R^A$ 而言, 这三个数恰好就是我们的角速度向量 ${}^A \omega^B$), 并且它可以写成叉积形式, 因此:

$${}^A \dot{R}^{BB} R^{AB} p_A^C = {}^A \omega_A^B \times {}^B p_A^C.$$

随后, 对等式左右两边同时乘以 ${}^F R^A$, 即可改变 expressed-in 坐标系, 从而得到最终结果。

- 这也揭示了: 平移速度的加法逆元 并不能简单地通过交换 reference frame 与 measured-in frame 得到; 它会稍微复杂一些:

$$-{}^A v_F^B = {}^B v_F^A + {}^A \omega_F^B \times {}^B p_F^A. \quad (16)$$

如果你熟悉“螺旋理论 (screw theory)” (例如 [2] 与 [3] 中所使用的形式), 点击三角可以查看这些约定之间的关系。

螺旋理论 (例如 [2] 与 [3] 中的形式) 经常使用我们空间速度的一种特殊形式, 被称为“空间坐标系中的空间速度 (spatial velocity in the space frame)”或 “spatial twists”, 在许多计算中都非常有用。这个量写作 ${}^A V^{B_A}$, 其中 B_A 是一个刚性附着在刚体 B 上的坐标系 (因此 ${}^B V^{B_A} = 0$), 并且它在当前时刻与坐标系 A 拥有相同位姿 (因此 ${}^A p^{B_A} = 0$, ${}^A R^{B_A} = I$) [3, 3.3.2]。在这些条件下, 有:

$${}^A V^{B_A} = \begin{bmatrix} {}^A \omega^{B_A} \\ {}^A v^{B_A} \end{bmatrix} = \begin{bmatrix} {}^A \omega^{B_A} + {}^B \omega_A^B \\ {}^A v^B + {}^B v_A^B + {}^A \omega^B \times {}^B p_A^B \end{bmatrix} = \begin{bmatrix} {}^A \omega^B \\ {}^A v^B - {}^A \omega^B \times {}^A p^B \end{bmatrix}. \quad (17)$$

为什么我们只用三个分量就能表示角速度, 却不能只用三个分量表示旋转本身? 使用轴角表示 (axis-angle) 中的向量模长来表示旋转角度会出现退化问题, 因为角度本身应当以 2π 为周期循环; 但对于角速度来说, 使用向量模长则没有问题, 因为角速度的取值范围是 $(-\infty, \infty)$, 不会发生周期回绕 (wrapping)。

还需要注意另一种速度: 我将使用 v 表示 plant 的广义速度向量, 它与 q 的时间导数密切相关 (见下文)。空间速度 ${}^A V^B$ 有六个分量; 平移速度或角速度, 即 ${}^B v^C$ 或 ${}^B \omega^C$, 有三个分量; 而广义速度向量的维度则取决于它需要用多少个量来编码构型变量 q 的时间导数。对于焊接到世界坐标系上的 iiwa 来说, 这意味着它有七个分量。在这些笔记中, 我已经尽量小心地用不同排版方式区分这些不同的 v 。几乎在所有情况下, 这种区别也可以从上下文看出来。

Example 3.5 (不要假设 $\dot{q} \equiv v$)

单位四元数表示有四个分量，但这些分量必须构成一个长度为 1 的“单位向量”。旋转矩阵有 9 个分量，但它们必须构成一个满足 $\det(R) = 1$ 的正交归一矩阵。对于旋转的时间导数，我们能够使用一个无约束的三维向量，也就是我们称为角速度向量 ω 的量，这非常棒。而且你真的应该使用它；去掉这些约束会让数学推导和数值计算都变得更好。

但这会带来一个小麻烦。我们通常倾向于把广义速度理解为广义位置的时间导数。当模型中只有 iiwa，并且它被焊接到世界坐标系时，这样理解是可行的，因为所有关节都是每个只有一个自由度的转动关节。但一般情况下我们不能这样假设；尤其是当浮动基座旋转属于广义位置的一部分时。作为证据，下面这个简单示例只向 `MultibodyPlant` 中加载了一个刚体，然后打印它的 `Context`。

输出看起来如下：

```
Context
-----
Time: 0
States:
  13 continuous states
    1 0 0 0 0 0 0 0 0 0 0 0 0

plant.num_positions() = 7
plant.num_velocities() = 6

Position names: ['$world_base_link_qw', '$world_base_link_qx', '$world_base_link_qy', '$world_base_link_qz', '$world_base_link_qd']
Velocity names: ['$world_base_link_wx', '$world_base_link_wy', '$world_base_link_wz', '$world_base_link_wd']
```

你可以看到，这个系统总共有 13 个状态变量。其中 7 个是位置变量 q ，因为我们在位置向量中使用单位四元数。但我们只有 6 个速度变量 v ；在速度向量中，我们使用角速度。显然，如果两个向量的长度甚至都不一致，那么我们就不可能有 $\dot{q} = v$ 。

处理这一表示细节并不困难；Drake 提供了 `MultibodyPlant` 方法 `MapQDotToVelocity` 和 `MapVelocityToQDot`，用于在二者之间来回转换。但你必须记得使用它们！

由于三维旋转存在多种可能的表示方式，并且 $\dot{q}$ 与 v 之间也可能存在差异，实际上存在许多种不同的运动学雅可比矩阵。你可能会听到“解析雅可比矩阵（analytic Jacobian）”这一术语，它指的是正向运动学的显式偏导数（也就是 (10) 中所写的形式）；也可能会听到“几何雅可比矩阵（geometric Jacobian）”，它会将左侧的三维旋转替换为空间速度。在 Drake 的 `MultibodyPlant` 中，我们目前通过以下接口提供几何雅可比矩阵版本：

- `CalcJacobianAngularVelocity`,
- `CalcJacobianTranslationalVelocity`，以及
- `CalcJacobianSpatialVelocity`,

这些接口都会接收一个参数，用来指定你想要的是相对于 $\dot{q}$ 还是相对于 v 的雅可比矩阵。如果你确实更喜欢解析雅可比矩阵，也可以借助 Drake 对自动微分的支持来得到它（只是效率会低得多）。

Example 3.6 (用于抓取与放置的运动学雅可比矩阵)

让我们重复上面的设置，但这一次我们会打印出夹爪坐标系的雅可比矩阵：它是相对于世界坐

标系测量的，并且在世界坐标系中表达。

3.6 微分逆运动学

几何雅可比矩阵给出了广义速度与空间速度之间的线性关系：

$$V^G = J^G(q)v. \quad (18)$$

这能否给我们提供所需的信息，用来产生夹爪坐标系 G 的变化？如果我有一个期望的夹爪坐标系速度 V^{G_d} ，那么是否可以直接命令关节速度 $v = [J^G(q)]^{-1}V^{G_d}$ 呢？

3.6.1 雅可比矩阵的伪逆

每当你写下一个矩阵逆时，都很重要的一点是先检查该矩阵是否真的可逆。作为最基本的合理性检查： $J^G(q)$ 的维度是多少？我们知道空间速度有六个分量。夹爪坐标系直接焊接在 iiwa 的最后一个连杆上，而 iiwa 有七个位置变量，因此有 $J^G(q_{iiwa}) \in \mathbb{R}^{6 \times 7}$ 。这个矩阵不是方阵，所以没有普通意义下的逆。但所需自由度 **多于** 期望空间速度所要求的自由度（列数多于行数）其实是好情况，因为这意味着可能存在许多个 v 都能实现同一个期望空间速度。为了从中选择一个解（最小范数解），我们可以考虑使用 **Moore-Penrose 伪逆** J^+ 来代替普通逆：

$$v = [J^G(q)]^+ V^{G_d}. \quad (19)$$

伪逆是一个非常优美的数学概念。当 J 是方阵且满秩时，伪逆会返回该系统真正的逆。当存在多个解时（在这里，即多个关节速度都能实现同一个末端执行器空间速度），它会返回最小范数解（也就是在最小二乘意义下，最接近零、同时能够产生期望空间速度的关节速度）。当不存在精确解时，它会返回一组关节速度，使得所产生的空间速度尽可能接近期望的末端执行器速度，同样是在最小二乘意义下。真是太棒了！

Example 3.7 (我们的第一个末端执行器“控制器”)

让我们使用雅可比矩阵的伪逆来编写一个简单控制器。首先，我们会编写一个新的 `LeafSystem`，它定义一个输入端口（用于 iiwa 的测量位置），以及一个输出端口（用于 iiwa 的关节速度命令）。在该系统内部，我们会向 `MultibodyPlant` 请求夹爪的雅可比矩阵，并计算出能够实现期望夹爪空间速度的关节速度。

iiwa_position → PseudoInverseController → iiwa_velocity_command

为了让第一个示例保持简单，我们只会命令一个恒定的夹爪空间速度，并且只运行几秒钟仿真。

请注意，我们确实需要在图中再添加一个额外的系统。我们控制器的输出是期望关节速度，但 iiwa 控制器所期望的输入是期望关节位置。因此，我们会在二者之间插入一个积分器。

我并不期望你理解这个示例中的每一行代码，但找到其中的重要代码行，并确保你能够修改它们、观察会发生什么，是很值得的！

恭喜！现在系统真的开始动起来了。

3.6.2 雅可比矩阵的可逆性

有一个简单检查可以帮助我们理解：什么时候伪逆能够对任意空间速度给出精确解（也就是精确实现期望空间速度）。这个条件是：雅可比矩阵的秩（rank）必须等于其行数（有时称为“满行秩 full row rank”）。在这里，我们需要 $\text{rank}(J) = 6$ 。但仅仅给矩阵指定一个整数秩并不能说明全部问题；对于真实机器人和带噪声的浮点关节位置而言，当矩阵接近秩亏时，（伪）逆会开始“爆炸（blow-up）”。因此，一个更好的指标是观察最小奇异值（singular value）；当它趋近于零时，伪逆的范数会趋近于无穷大。

Example 3.8 (夹爪雅可比矩阵的可逆性)

你可能已经注意到，我在前面某个示例中打印出了 J^G 的最小奇异值。再运行一次试试看。看看你能否找到一些构型，使得最小奇异值接近于零。

给你一个提示：试试那些机械臂非常伸直的构型（例如，让第 2 和第 4 个关节接近零）。

另一种寻找奇异位形的好方法是：使用你的伪逆控制器发送夹爪空间速度命令，让夹爪移动到机器人工作空间的边界。试一试，看看会发生什么！事实上，在使用末端执行器命令时，这种情况在实践中很容易发生，而我们会努力避免它。

对于某个感兴趣的坐标系（例如我们的夹爪坐标系 G ），若某个构型 q 满足 $\text{rank}(J(q_{iiwa})) < 6$ ，则称该构型为运动学奇异位形（kinematic singularity）。如果你在使用末端执行器控制，请尽量避免靠近这些奇异位形！iiwa 有很多优点，但不得不承认，它的运动学工作空间并不是其中之一。相信我，如果你每天都试图让一个大型 Kuka 机械臂伸进一个小厨房水槽里工作，那么你会花费相当多的时间去思考如何避免奇异位形。

在实际中，如果我们不再把机器人底座固定在世界中的某个位置，情况通常会好很多。移动底座会增加系统复杂度，但它们对于提升机器人运动学工作空间来说非常有价值。

Example 3.9 (运动学奇异位形是真实存在的吗?)

在讨论奇异位形时，一个很自然的问题是：它们究竟是真实存在的，还是只是分析方法中的某种人为产物？机器人会爆炸吗？它会机械卡死吗？还是说只是我们的数学模型失效了？或许从一个极其简单的例子来看会更有帮助。

想象一个两连杆机械臂，每根连杆长度都为 1。那么其运动学可以写成：

$$p^G = \begin{bmatrix} c_0 + c_{0+1} \\ s_0 + s_{0+1} \end{bmatrix},$$

这里我使用了（非常常见的）简写： s_0 表示 $\sin(q_0)$ ， c_0 表示 $\cos(q_0)$ ， s_{0+1} 表示 $\sin(q_0 + q_1)$ ，等等。对应的平移雅可比矩阵为：

$$J^G(q) = \begin{bmatrix} -s_0 - s_{0+1} & -s_{0+1} \\ c_0 + c_{0+1} & c_{0+1} \end{bmatrix},$$

正如预期的那样，当机械臂完全伸直时它会失去秩（例如当 $q_0 = q_1 = 0$ 时，第一行变为零）。

[点击这里查看动画。](#)

让我们沿着 x 轴移动机器人，取 $q_0(t) = 1 - t$ ，且 $q_1(t) = -2 + 2t$ 。很明显，这条轨迹会在时

间 $t = 1$ 时经过奇异位形 $q_0 = q_1 = 0$ ，然后又顺利离开，没有任何问题。实际上，整个过程中关节速度始终保持**常数** ($\dot{q}_0 = -1, \dot{q}_1 = 2$)！对应得到的末端轨迹是：

$$p^G(t) = \begin{bmatrix} 2 \cos(1-t) \\ 0 \end{bmatrix}.$$

这里有几个重要的点需要理解。在奇异位形处，机器人确实无法让夹爪继续朝正 x 方向移动——这一奇异性是真实存在的。但同样真实的是，机器人也无法瞬时朝 $-x$ 方向移动。雅可比矩阵分析并不是某种近似，它对关节速度与夹爪速度之间的关系给出了精确描述。然而，仅仅因为你无法实现某个瞬时速度，并不意味着你永远到不了那个方向！在 $t = 1$ 时，尽管关节速度保持常数，且平移雅可比矩阵是奇异的，机器人实际上仍然在朝 $-x$ 方向**加速**： $\ddot{p}_{W_x}^G(t) = -2 \cos(1-t)$ 。

当雅可比矩阵不存在唯一逆时，还需要注意一个关于**可积性** (integrability) 的有趣微妙之处。假设机器人在时间零从某个关节构型 $q(0)$ 开始，对应的末端执行器位姿为 $X^G(0)$ 。现在让我们使用伪逆控制器执行一条末端执行器轨迹：它在任务空间中沿一条闭合路径运动，并在时间一回到原始末端执行器位姿： $X^G(1) = X^G(0)$ 。不幸的是，我们并不能保证 $q(1) = q(0)$ ；这并不只是数值实现中的数值误差造成的，而是因为众所周知，伪逆通常不会产生一个可积函数。[6] 对此进行了深入讨论，并提出了一种加权伪逆方法，可以通过一个虚拟的“柔顺性函数 (compliance function)”来恢复可积性条件。

3.7 定义抓取位姿与预抓取位姿

我会把我的红色泡沫砖放在桌子上。它的几何形状被**定义**为一个 7.5cm x 5cm x 5cm 的长方体。作为参考，我们夹爪在默认“张开”位置时，两个手指之间的距离是 10.7cm。夹爪的“手掌”距离刚体原点 3.625cm，手指长度为 8.2cm。

为了让一开始的问题更简单，我保证会把物体放在桌面上，并使物体坐标系的 z 轴朝上（与世界坐标系的 z 轴对齐），你也可以假设它安全地放置在机器人工作空间内的桌面上。但我保留给物体任意初始 yaw 角的权利。别担心，你可能已经注意到，iiwa 的第七个关节可以让你的夹爪相当灵活地旋转（远远超过我的人类手腕所能做到的范围）。

注意，从视觉上看，这个盒子具有旋转对称性——我总是可以让盒子绕其 x 轴旋转 90 度，而你无法看出区别。当我们在下一章开始使用感知时，会进一步思考这件事带来的后果。但目前，我们可以使用模拟器中全知的“作弊端口 (cheat port)”，它会为我们提供砖块的确切位姿。

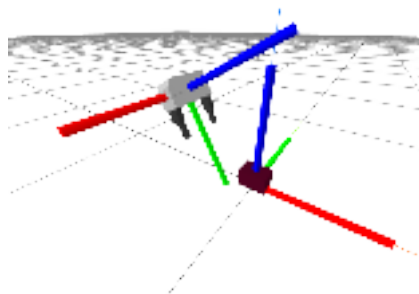


Figure 3.5 - 夹爪坐标系与物体坐标系。对于每个坐标系，**正 x 轴**为红色，**正 y 轴**为绿色，**正 z 轴**为蓝色 (XYZ $\Leftrightarrow$ RGB)。

请仔细观察上图中的夹爪坐标系，并通过颜色来理解各个坐标轴。我的思路是这样的：根据夹爪和物体的尺寸，我希望物体在夹爪坐标系中的**期望**位置（单位为米）为：

$$G_{grasp}p^O = \begin{bmatrix} 0 \\ 0.11 \\ 0 \end{bmatrix}, \quad G_{pregrasp}p^O = \begin{bmatrix} 0 \\ 0.2 \\ 0 \end{bmatrix}.$$

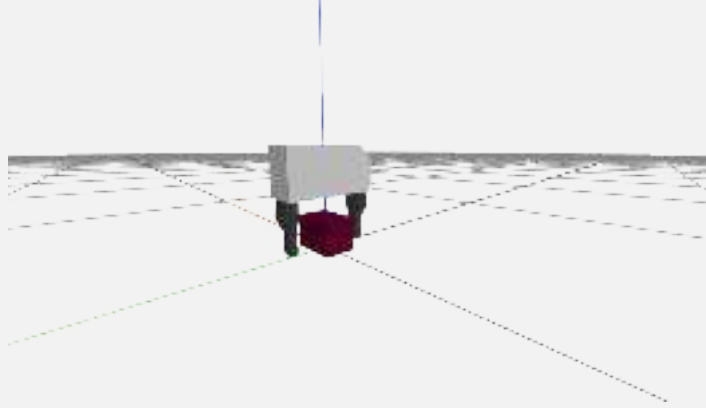
回想一下，**预抓取**位姿背后的逻辑是：先移动到物体上方一个安全的位置；如果夹爪在非常靠近物体时唯一的运动只是从预抓取位姿到抓取位姿、再返回的一段直线平移，那么这就能让我们在很大程度上暂时避免考虑碰撞问题。我希望夹爪的方向设置为：物体的正 z 轴与夹爪坐标系的负 y 轴对齐，并且物体的正 x 轴与夹爪的正 z 轴对齐。我们可以通过以下方式实现：

$$G_{grasp}R^O = \text{MakeXRotation}\left(\frac{\pi}{2}\right)\text{MakeZRotation}\left(\frac{\pi}{2}\right).$$

我承认，推这个的时候我又把右手举到空中了！我们可以一步步看这个过程（参照[上图](#)）。请把右手举到空中，摆成标准的“右手定则姿态”（食指向前，中指向左，拇指向上）；我们将这个坐标系称为 H （代表 hand），并把它附着到物体坐标系上，因此 ${}^OR^H = I$ 。我们想理解的是你的手在夹爪坐标系中的方向，即 $G_{grasp}R^H$ 。从矩阵乘法的右侧向左侧依次作用，我们首先绕当前的 z 轴（你的拇指）旋转 90 度——此时你的食指应该指向左方。接着，我们绕当前的 x 轴（仍然朝前）旋转 90 度，于是你的食指指向上方，中指指向你自己，拇指指向右方。这正是我们想要的结果：正 z 轴（你的拇指）与负 y 轴对齐，而正 x 轴（你的食指）与夹爪坐标系的正 z 轴对齐。

我们的预抓取位姿将与抓取位姿具有相同的方向。

Example 3.10 (计算抓取位姿与预抓取位姿)



下面是一个简单示例：加载一个浮动的 Schunk 夹爪和一块砖，计算抓取 / 预抓取位姿（并清晰地展示每一步变换），然后渲染夹爪相对于物体的位置关系。

我希望你已经能够体会到良好符号体系的价值。当我在编写这些笔记、思考物体相对于夹爪的合适相对位姿时，我的右手依然举在空中比划。但一旦这个位姿被确定下来，我把它输入代码后，一切就自然地正常工作了。

3.8 抓取与放置轨迹

我们已经非常接近目标了。我们知道如何生成期望的夹爪位姿，也知道如何利用空间速度命令瞬时改变夹爪位姿。现在我们需要规定：夹爪位姿应当如何随时间变化，这样我们才能将夹爪位姿转换为空间速度命令。

让我们定义所有夹爪需要经过的“关键帧（keyframe）”位姿，以及到达每个位姿的时间点。下面的示例正是在完成这件事。

Example 3.11 (一个计划“草图”)

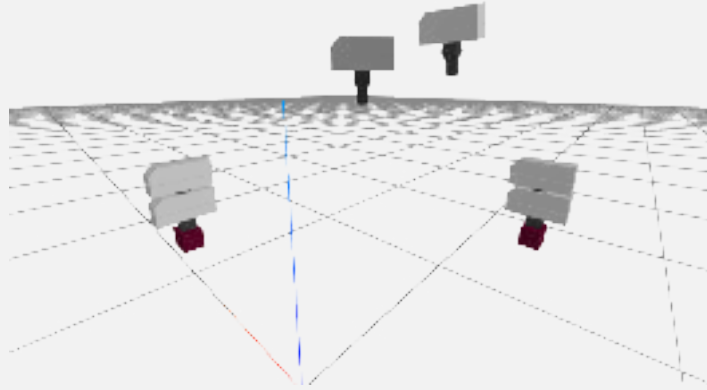


Figure 3.7 - 夹爪的关键帧。机器人的基座会位于原点，因此这里相当于从（不可见的）机器人肩膀上方向前看。手从靠近中心的“initial”位姿开始，依次移动到“prepick”、“pick”、“postpick”、“clearance”、“preplace”、“place”，最后回到“postplace”。

我是如何选择这些时间的？我让所有事情都从时间 $t = 0$ 开始，并将其余时间都列为绝对时间（也就是从零时刻开始计算的时间）。这就是机器人“醒来”并看到砖块的时刻。从起始位姿过渡到预抓取位姿应该花多久？一个非常好的答案可能取决于机器人精确的关节速度限制，但我们目前还不是在追求快速运动。因此，我选择了一个保守时间，它与手部将要移动的总欧氏距离成正比，例如 $k = 10 \text{ s/m}$ （也就是 10 cm/s ）：

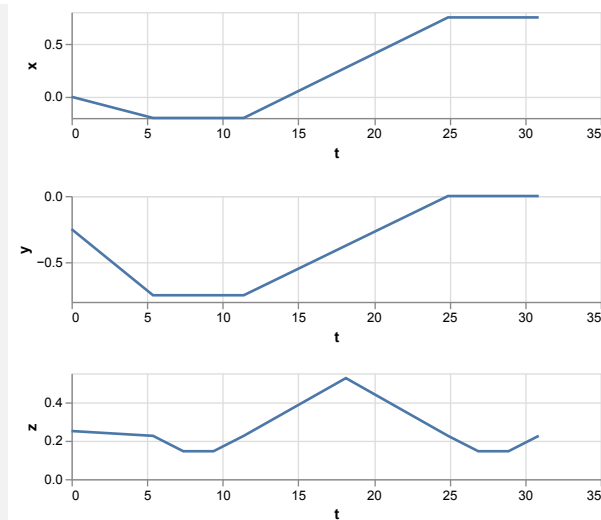
$$t_{\text{pregrasp}} = k \cdot G_0 p_{\text{pregrasp}}^2.$$

对于从预抓取到抓取以及返回的过渡，我只是选择了固定的 2 秒持续时间；对于手指需要张开/闭合的那些片段，我也让夹爪保持静止 2 秒。

在计算中表示轨迹有许多种方式。Drake 提供了一组相当不错的 [轨迹 \(trajectory\)](#) 类。其中许多轨迹类都实现为 [样条 \(splines\)](#) —— 也就是关于时间的分段多项式函数。在不同方向之间插值需要格外小心，但对于位置来说，我们可以在各个关键帧之间进行简单的线性插值。这被称为“[一阶保持 \(first-order hold\)](#)”，并且已在 Drake 的 [PiecewisePolynomial](#) 类中实现。对于旋转，我们会使用一种称为“球面线性插值 (spherical linear interpolation)”、或简称 [slerp](#) 的方法；它在 Drake 的 [PiecewiseQuaternionSlerp](#) 中实现，你也可以在[这个练习](#)中探索它。[PiecewisePose](#) 类可以方便地同时构建和处理位置轨迹与方向轨迹。

Example 3.12 (使用轨迹进行抓取)

当轨迹与三维空间相关联时，有许多种方式可以将其可视化。我在下面将[位置](#)轨迹绘制成了时间的函数。



对于三维数据，你可以在[三维中绘制它](#)。但我最喜欢的方法是把它做成 [我们三维渲染器中的动画](#)！一定要试试“Open controls>Animation”界面。你可以暂停动画，然后使用时间滑块沿着轨迹来回查看。

如果你对如何把四维四元数可视化成被困在三维空间中的生物这一话题感兴趣，你可能会喜欢[这一系列“可探索”的视频](#)。

最后还有一个细节——我们还需要一条夹爪命令轨迹（用于打开和闭合夹爪）。对此我们同样会使用一阶保持。

3.9 整合所有内容

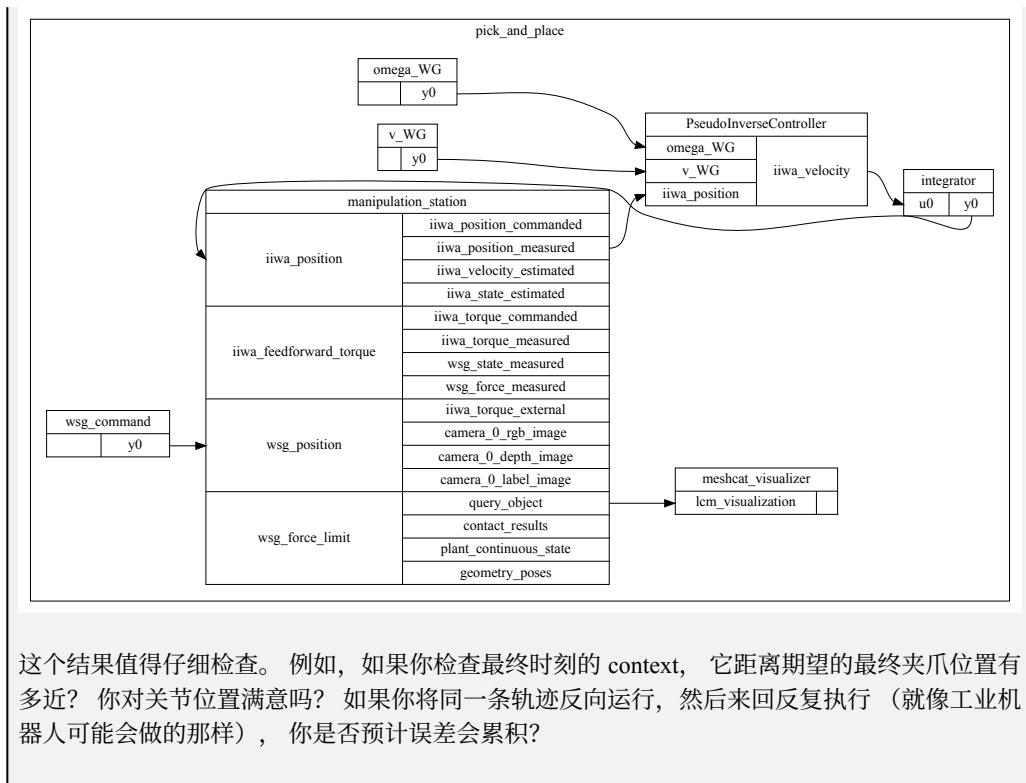
我们可以稍微泛化一下 `PseudoInverseController`，为期望夹爪空间速度 ${}^WV^G$ 添加额外的输入端口（在第一个版本中，这个速度只是硬编码在控制器里）。



我们构造出来的轨迹是一条[位姿](#)轨迹，但我们的控制器需要的是空间速度命令。幸运的是，我们使用的轨迹类支持对轨迹求导。事实上，`PiecewiseQuaternionSlerp` 足够聪明，能够将四分量四元数轨迹的导数返回为三分量角速度轨迹；而对 `PiecewisePose` 轨迹求导，会返回一条空间速度轨迹。剩下的就只是把系统图连接起来了。

Example 3.13 (完整的抓取与放置演示)

`notebook` 接下来的几个单元应该会给你一个相当令人满意的结果。 [点击这里可以不做这些工作、直接观看效果](#)。



这个结果值得仔细检查。例如，如果你检查最终时刻的 context，它距离期望的最终夹爪位置有多近？你对关节位置满意吗？如果你将同一条轨迹反向运行，然后来回反复执行（就像工业机器人可能会做的那样），你是否预计误差会累积？

3.10 带约束的微分逆运动学

上面的解法在许多情况下都能工作。我们本可以就此继续往下走。但只需要再多做一点工作，就能得到一个稳健得多的解法……一个我们在接下来许多章节中都会满意使用的解法。

那么，伪逆控制器到底有什么问题？当我说它在奇异位形附近表现不好时，你大概不会感到意外。当雅可比矩阵的最小奇异值变小时，这意味着其逆中的某些数值会变得非常大。如果你要求我们的控制器跟踪一个看起来很合理的末端执行器空间速度，那么结果可能会产生极大的速度命令。

不过，它还有其他重要限制，而且这些限制可能更微妙。真实机器人有约束——非常真实的关节角、速度、加速度和力矩约束。例如，如果你向 iiwa 控制器发送一个无法跟踪的速度命令，这个速度会被裁剪。在我们运行 iiwa 的模式中（关节阻抗模式），iiwa 并不知道你的末端执行器目标。因此，它很可能只是逐个关节独立地饱和你的速度命令。恐怕其结果并不会像一条更慢的末端执行器轨迹那样方便可控；你的末端执行器可能会严重偏离原本的路径。

既然我们知道 iiwa 的限制，更好的方法就是在控制器层面把这些约束考虑进去。将位置、速度和加速度约束纳入考虑是相对直接的；力矩约束需要完整的动力学模型，所以这里我们暂时不处理它们（但它们确实可以很自然地接入这个框架）。

3.10.1 将伪逆视为一种优化

我曾介绍过伪逆几乎像是拥有某种神奇性质：当精确解存在时，它会返回一个精确解；当精确解不存在时，它会返回最佳可能解（在最小二乘范数意义下）。这些性质都可以通过认识到一点来理解：伪逆其实只是一个最小二乘优化问题的最优解：

$$\min_v J^G(q)v - V^{G_d} \quad (20)$$

当我写出这种形式的优化问题时，我会把 v 称为决策变量，并使用 v^* 表示最优解（也就是使代价最小的决策变量取值）。这个特定的优化问题存在闭式解，而这个闭式解恰好正是伪逆所给出的结果：

$$v^* = [J^G(q)]^+ V^{G_d}.$$

更一般地，我们会期待使用数值方法来求解优化问题。

优化是一个极其丰富的主题，在本文的后续内容中，我们会使用许多优化工具。关于凸优化，如果你想看一份严谨但又易懂的精彩介绍，我强烈推荐 [7]；它可以免费在线阅读，甚至只读第一章也会非常有价值。如果你想快速了解如何在 Drake 中使用优化，请查看 [Mathematical Program 教程](#)。我几乎把“mathematical program（数学规划）”这个术语与“optimization problem（优化问题）”等同使用。如果我们实际上没有目标函数、只有约束，那么“数学规划”这个说法也许会稍微更合适一些。

3.10.2 添加速度约束

一旦我们从优化的视角理解了现有解法，就自然有了一条路径，可以将方法推广为显式考虑约束。速度约束是最直接可以加入的约束：

$$\begin{aligned} \min_v \quad & J^G(q)v - V^{G_d} \frac{v^2}{2}, \\ \text{subject to} \quad & v_{\min} \leq v \leq v_{\max}. \end{aligned} \quad (21)$$

你可以把它读作：“请找到一组关节速度，使它们尽可能接近地实现我期望的夹爪空间速度，但同时满足我的关节速度约束。”这个问题的解可能远好于先求解无约束优化问题、然后事后简单地裁剪所有超出约束的速度所得到的结果。

诚然，一般来说这是一个更难求解的问题。它的解不能仅仅用雅可比矩阵的伪逆来描述。相反，我们将在控制器每次被求值时，直接求解这个（小型）优化问题。该问题具有凸二次目标函数和线性约束，因此属于凸二次规划（Quadratic Programming, QP）问题。这是一类特别友好的优化问题，我们拥有非常强大的数值工具来求解它。

Example 3.14 (带速度约束的基于雅可比矩阵的控制)

为了帮助理解如何将微分逆运动学问题看作一个优化问题，我整理了一个二次规划优化景观的可视化示例。在下面的 notebook 中，你会看到约束以红色可视化显示，目标函数则以绿色函数的形式可视化显示。

3.10.3 添加位置约束和加速度约束

只要新增约束对于决策变量是线性的，我们就可以很容易地向 QP 中加入更多约束，而不会显著增加问题复杂度。那么，我们应该如何加入关节位置和加速度约束呢？

自然的做法是对这些约束进行一阶近似。为此，控制器需要某个特征时间步长 / 时间尺度，用来将它所做出的速度决策与位置、加速度联系起来。我们将这个时间步长记为 h 。

控制器已经接收当前测得的关节位置 q 作为输入；现在让我们再把当前测得的关节速度 v 作为第二个输入。并且我们使用 v_n 表示决策变量——也就是下一步要命令的速度。使用简单的[欧拉近似](#)来近似位置，并使用一阶导数来近似加速度，可以得到如下优化问题：

$$\begin{aligned}
& \min_{v_n} J^G(q)v_n - V^{G_d} \frac{^2}{2}, \\
& \text{subject to } v_{\min} \leq v_n \leq v_{\max}, \\
& \quad q_{\min} \leq q + hv_n \leq q_{\max}, \\
& \quad \dot{v}_{\min} \leq \frac{v_n - v}{h} \leq \dot{v}_{\max}.
\end{aligned} \tag{22}$$

3.10.4 关节居中 (Joint centering)

我们的雅可比矩阵是 6×7 ，因此实际上我们拥有比末端执行器目标更多的自由度。这不仅是一种机会，也是一种责任。当 J^G 的秩小于其列数 / 自由度数量时，我们所定义的优化问题实际上拥有无穷多个解；而我们把选择哪一个解这件事留给了求解器。通常，一个凸优化求解器会选择某种合理的解，例如取约束的“解析中心 (analytic center)”，或者在无约束问题中选择最小范数解。但为什么要把这件事交给运气呢？更好的做法是由我们自己完整定义问题，从而让它拥有唯一的全局最优解。

在机器人学基于雅可比矩阵控制的丰富历史中，有一个非常优雅的思想：通过将一个“次级 (secondary)”控制器投影到 J^G 的零空间 (null space) 中，来实现一个（在某些情况下）保证不会干扰主末端执行器空间速度控制器的附加控制。因此，为了完整定义问题，我们会提供一个试图控制所有关节的次级控制器。在这里，我们使用一个简单的关节空间控制器 $v = K(q_0 - q)$ ；这是一个比例控制器，它会驱动机器人回到其标称构型。

记 $P(q)$ 为一个将广义速度映射到 $J(q)$ 零空间中的线性映射。在传统机器人学中，我们通常使用伪逆来实现这一点： $P = (I - J^+J)$ ，但现在许多线性代数库已经能够直接获得雅可比矩阵 J 的核空间 (kernel) 的一组正交归一基。将 $Pv \approx PK(q_0 - q)$ 作为一个次级目标加入，可以写成：

$$\begin{aligned}
& \min_{v_n} J^G(q)v_n - V^{G_d} \frac{^2}{2} + \epsilon |P(q)(v_n - K(q_0 - q))|_2^2, \\
& \text{subject to constraints.}
\end{aligned} \tag{23}$$

请注意，我们在次级目标前加入了一个标量 ϵ 。这里有一个重要的点需要理解。如果不存在任何约束，那么我们甚至可以完全去掉 ϵ ——次级任务不会以任何方式干扰主任务（即跟踪空间速度）。然而，如果存在约束，那么这些约束可能会导致两个目标之间发生冲突（参见练习）。因此，我们选择 $\epsilon \ll 1$ ，从而赋予主目标相对更大的权重。但也不要把它设得太小，因为那可能会导致求解器数值表现变差。我认为 $\epsilon = 0.01$ 差不多是一个很合适的值。

如果希望在存在约束的情况下建立严格的任务优先级，还存在更复杂的方法（例如 [8, 9]），但对于我们这里构建的这种简单优先级关系，惩罚法 (penalty method) 已经是相当合理的选择。

3.10.5 跟踪期望位姿

有时，将微分 IK 的目标指定为跟踪某个期望位姿 ${}^A X^{G_d}$ ，会比显式指定期望空间速度更加自然。但我们可以很自然地将位姿命令转换为空间速度命令 ${}^A V_A^{G_d}$ 。平移分量足够简单，我们使用：

$${}^A p_A^{G_d} = {}^A p_A^G + h {}^A v_A^G \Rightarrow {}^A v_A^G = \frac{1}{h} ({}^A p_A^{G_d} - {}^A p_A^G) = \frac{1}{h} {}^G p_A^{G_d}.$$

类似地，对于角速度，我们会选择：

$${}^A \omega_A^G = \frac{1}{h} {}^G R^{G_d},$$

其中 ${}^G R^{G_d}$ 使用 轴角 (axis-angle) 形式表示。

3.10.6 其他形式

一旦我们接受了在控制回路中求解一个小型优化问题的思想，许多其他形式也都成为可能。你会在文献中找到很多这样的形式。最小化命令空间速度与实际产生速度之间的最小二乘距离，未必真的是最好的解法。我们在 Drake 中大量使用的形式会额外加入一个约束：我们的解**必须**沿着命令空间速度的相同**方向**运动。如果我们正好受到约束限制，那么机器人可以减速，但不会（在瞬时意义上）偏离命令路径。如果你花了很长时间仔细规划末端执行器的无碰撞路径，结果控制器却只把这条路径当成一个建议，那就太可惜了。不过请注意，你的计划回放系统仍然需要足够智能，能够意识到减速已经发生（开环速度命令是不够的）。

$$\begin{aligned} \max_{v_n, \alpha} \quad & \alpha, \\ \text{subject to} \quad & J^G(q)v_n = \alpha V^{G_d}, \\ & 0 \leq \alpha \leq 1, \\ & \text{additional constraints.} \end{aligned} \tag{24}$$

你应该立刻问自己：用一个标量缩放空间速度是否合理？这是另一个很好的**练习**。

当雅可比矩阵失去行秩时，这种形式会发生什么？注意， $v_n = 0, \alpha = 0$ 对这个问题来说始终是一个可行解。因此，如果机器人无法沿命令方向运动，那么它就会直接停下来。

对于有人类操作员的遥操作而言，当无法严格沿着命令方向运动时就完全停止，可能过于保守——每当机器人接近某个约束时，操作员会感觉机器人“卡住”了。例如，假设我们在机器人夹爪和桌子之间有一个避碰约束，而操作员希望沿桌面移动机械手。在严格形式下，这实际上会变得不可能。我们发现，下面这种松弛形式对操作员来说更加直观：

$$\begin{aligned} \max_{v_n, \alpha} \quad & \sum_i \alpha_i, \\ \text{subject to} \quad & J^G(q)v_n = E(q)\text{diag}(\alpha)E^{-1}(q)V^{G_d}, \\ & 0 \leq \alpha_i \leq 1, \\ & \text{additional constraints.} \end{aligned} \tag{25}$$

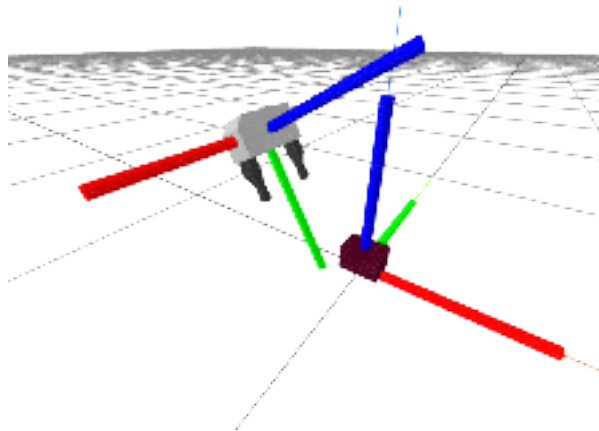
这里 $E(q)$ 是一个**线性映射**，它将欧拉角表示的导数（例如 $\frac{d}{dt}[\text{roll}, \text{pitch}, \text{yaw}, x, y, z]$ ）映射为空间速度。注意，我们也可以通过直接使用欧拉角导数来接收空间速度命令，从而避免使用可能变得奇异的 $E^{-1}(q)$ 。对熟悉优化的人来说，这看起来可能有些奇怪——这个对角矩阵是在对速度进行一种**依赖坐标系的**缩放。但这正是重点：对于人类操作员来说，在通常由机器人基座坐标系指定的 x 、 y 、 z 、 roll 、 pitch 、 yaw 坐标中，逐分量地饱和速度命令会更加直观。当你试图让机器人末端执行器沿桌面滑动时，指向桌面的速度分量会饱和，但与桌面正交的其他分量并不需要饱和。重要的是，当你命令纯平移时，你永远不会看到末端执行器发生扭转；当你命令纯旋转时，你也不会看到末端执行器发生平移。

Example 3.15 (Drake 的 DifferentialInverseKinematics)

在接下来的几章中，每当我们需要命令末端执行器时，都会使用这种微分逆运动学实现。

3.11 练习

Exercise 3.1 (空间坐标系与位置。)



我已经渲染了夹爪和砖块，并显示了它们各自对应的刚体坐标系。请注意图中表示世界坐标系 W 的非常细的彩色线。给定图中显示的构型， ${}^G p_W^O$ 的一个可能取值是什么？

- a. $[0.2, 0, -2]$
- b. $[0, 0.3, .1]$
- c. $[0, -.3, .1]$

${}^G p_W^O$ 的一个可能取值是什么？

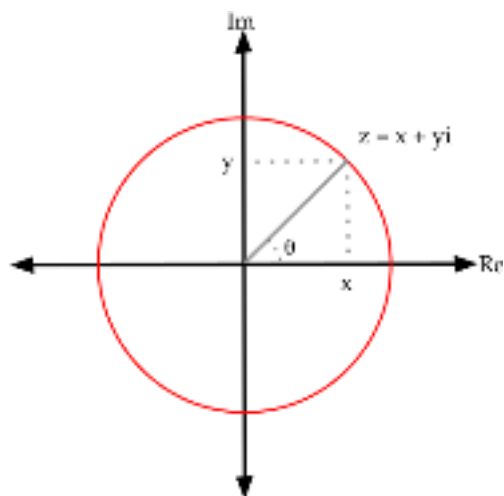
Exercise 3.2 (角速度向量的加法性质。)

TODO(russt): 补充这里。

Exercise 3.3 (球面线性插值 (slerp))

对于位置，我们可以在点之间进行线性插值，也就是“一阶保持 (first-order hold)”。在处理旋转时，我们不能简单地线性插值，而必须使用球面线性插值 (slerp)。本题的目标是深入理解 slerp 的细节。

为此，我们将考虑一个更简单的情形：旋转位于 $\mathbb{R}^2$ 中，并且可以用复数表示。游戏规则如下：一个二维向量 (x, y) 将表示为复数 $z = x + yi$ 。若要将该向量旋转 θ ，我们将其乘以 $e^{i\theta} = \cos(\theta) + i \sin(\theta)$ 。



- a. 让我们验证一下这确实可行。取二维向量 $(x, y) = (1, 1)$ 。如果你将这个向量转换为复数，使用复数乘法乘以 $z = e^{i\pi/4}$ ，然后再把结果转换回二维向量，你是否会得到预期结果？请展示你的推导过程，并解释为什么这是预期结果。

请花一点时间说服自己：这个过程（从二维向量转换为复数，乘以 $e^{i\theta}$ ，再转换回二维向量）在数学上等价于将原始向量乘以二维旋转矩阵：

$$R(\theta) = \begin{bmatrix} \cos \theta & -\sin \theta \\ \sin \theta & \cos \theta \end{bmatrix}.$$

坐标系 F 具有一个方向。我们可以使用相对于世界坐标系的旋转来表示该方向，例如 ${}^W R^F$ 。我们刚刚已经验证了，你可以用一个复数 $e^{i\theta}$ 来表示这个旋转。现在假设我们想在两个方向之间插值，这两个方向分别由坐标系 F 和 G 表示。我们应该如何在这两个坐标系之间平滑插值，例如使用 $t \in [0, 1]$ ？你将在 (b) 和 (c) 两部分中探索这个问题。

- b. 尝试 1：考虑 $z(t) = (a_F * (1 - t) + a_G * t) + i(b_F * (1 - t) + b_G * t)$ 。取 $t = .5$ 。如果你将二维向量 $(1, 1)$ 乘以 $z(t = .5)$ ，会发生什么？请展示你的推导过程，并解释哪里出了问题。
- c. 尝试 2：相反，我们可以利用另一种表示方式： $z(t) = e^{i\theta_F * (1-t) + i\theta_G * t}$ 。如果我们将二维向量 $(1, 1)$ 乘以 $z(t = .5)$ ，会发生什么？同样，请展示你的推导过程。

四元数只是这个思想在三维中的推广。在二维中，用两个数和一个约束 $a^2 + b^2 = 1$ 来表示一个单一旋转可能看起来有些低效（为什么不直接用 θ 呢！？），但我们已经看到它确实可行。在三维中，我们可以使用 4 个数 x, y, z, w 加上约束 $x^2 + y^2 + z^2 + w^2 = 1$ 来表示一个三维旋转。在二维中，只用 θ 往往也能正常工作；但在三维中，如果只使用三个数，就会遇到著名的 万向节死锁 (gimbal lock) 等问题。由 4 数组成的单位四元数为所有三维旋转提供了一种非退化映射。

就像我们在二维中看到的那样，不能简单地对四元数的 4 个数进行线性插值来插值方向。相反，我们会使用四元数 `slerp`，对两个方向之间的角度进行线性插值。细节会涉及一些四元数符号，但其概念与二维情形相同。

Exercise 3.4 (缩放空间速度)

TODO(russt)：补充这里。在此之前，下面这小段代码也许能让你相信这样做是合理的。

Exercise 3.5 (平面机械臂)

在本练习中，你将推导平面二连杆机械臂的平移正向运动学 $A_p^C = f(q)$ 以及平移雅可比矩阵 $J(q)$ 。你将完全在 中完成。你需要完成以下步骤：

- 推导该机械臂的正向运动学。
- 推导该机械臂的雅可比矩阵。
- 根据雅可比矩阵分析该机械臂的运动学奇异位形。

Exercise 3.6 (探索雅可比矩阵)

练习 3.5 要求你推导平面二连杆机械臂的平移雅可比矩阵。在本题中，我们将在多连杆机械臂的背景下更深入地探索平移雅可比矩阵。

- 对于平面二连杆机械臂，平面平移雅可比矩阵的大小是 2×2 。考虑一个 N 连杆机械臂，其关节角为 $(q_0, q_1, \dots, q_{N-1})$ ，并且其末端执行器位置由 (x, y, z) 描述。那么平移雅可比矩阵的大小是多少？
- 在考虑这些多连杆机械臂时，我们把所有可能关节角组成的空间称为**构型空间 (configuration space)**，把所有可能末端执行器位置组成的空间称为**操作空间 (operational space)**。
 - 如果平移雅可比矩阵的形状使得构型空间的维度等于操作空间的维度，那么在什么情况下可以计算其逆？（请给出比“当雅可比矩阵可逆时”更具体的回答。）如果可以计算该逆，那么有多少组关节速度可以被命令来产生一个期望末端执行器速度？
 - 考虑构型空间维度大于操作空间维度的情况。是否可能存在多于一组关节速度命令，能够产生同一个期望末端执行器速度？如果可能，这会在什么情况下发生，并且可以命令多少组这样的关节速度？如果不可能，请解释原因。
 - 继续 b.ii 的讨论，是否存在某种情况下，没有任何一组关节速度能够产生某个期望末端执行器速度？请解释。
- 下面，对于平面二连杆机械臂，我们在 $q_1 - q_2$ 平面中绘制关节速度的单位圆。这个圆随后通过平移雅可比矩阵映射到末端执行器速度空间中的一个椭圆。换句话说，这张图展示了：平移雅可比矩阵会将关节速度空间映射到末端执行器速度空间。

末端执行器速度空间中的这个椭圆称为**可操作性椭圆 (manipulability ellipsoid)**。可操作性椭圆以图形方式刻画了机器人在各个方向上移动末端执行器的能力。例如，椭圆越接近圆，末端执行器就越容易沿任意方向运动。当机器人处于奇异位形时，它无法在某些方向上产生末端执行器速度。回想你在练习 3.5 中探索过的奇异位形，在其中一个奇异位形处，可操作性椭圆会塌缩成什么形状？

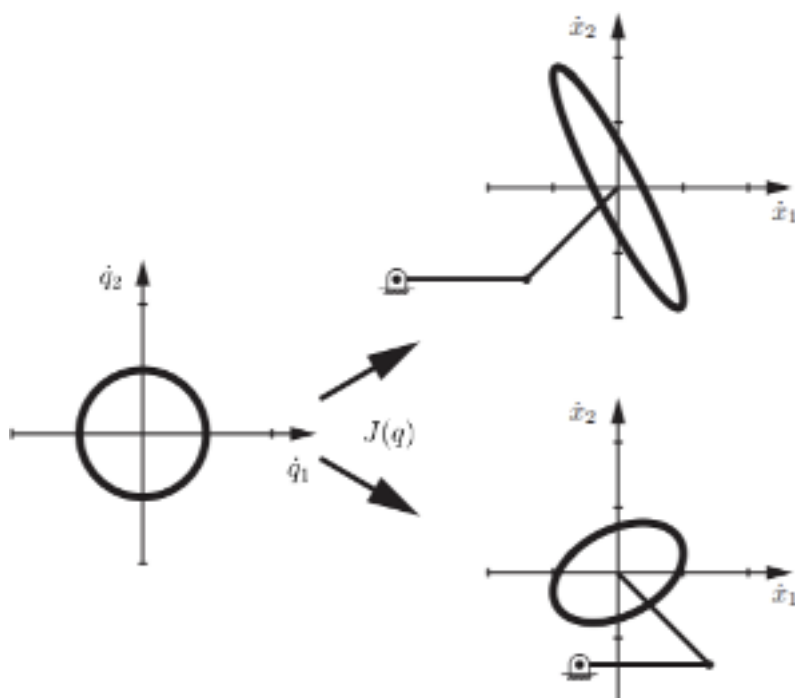


Figure 3.12 - 平面二连杆机械臂在两种不同姿态下的可操作性椭圆。改编自 [3]。

Exercise 3.7 (空间变换与抓取位姿)

在本练习中，你将运用空间代数知识，在不同参考坐标系中写出各坐标系的位姿，并自行设计一个抓取位姿。你将完全在 中完成。你需要完成以下步骤：

- 使用空间代数，在不同参考坐标系中表达各坐标系的位姿。
- 根据目标物体的构型和夹爪构型设计抓取位姿。

Exercise 3.8 (机器人画家)

在本练习中，你将为机器人设计有趣的跟踪轨迹，并观察机器人在空中虚拟作画！你将完全在 中完成。你需要完成以下步骤：

- 设计并计算指定轨迹的关键帧位姿。
- 通过对关键帧进行插值来构造轨迹。

Exercise 3.9 (QP 入门)

在本练习中，你将练习通过 Drake 的 MathematicalProgram 接口求解二次规划（Quadratic Programs, QP）的语法。你将完全在 中完成。

Exercise 3.10 (虚拟墙)

在本练习中，你将使用基于优化的微分逆运动学方法，为机器人机械臂实现一面虚拟墙。你将完全在 中完成。你需要完成以下步骤：

- 实现一个带关节速度限制的基于优化的微分 IK 控制器。
- 使用你自己的约束，通过基于优化的微分 IK 控制器，在末端执行器空间中实现一面虚拟墙。

Exercise 3.11 (相互竞争的目标)

在关于[关节居中](#)的小节中，我声称如果次级目标和主目标通过约束关联起来，那么次级目标可能会与主目标发生竞争。为了看到这一点，请考虑如下优化问题：

$$\min_{x,y} (x-5)^2 + (y+3)^2.$$

显然，最优解为 $x^* = 5, y^* = -3$ 。此外，两个目标项是可分离的。添加第二个目标项 $(y+3)^2$ 并没有以任何方式影响第一个目标的解。

现在考虑带约束的优化问题：

$$\begin{aligned} \min_{x,y} \quad & (x-5)^2 + (y+3)^2 \\ \text{subject to} \quad & x - y \leq 6. \end{aligned}$$

如果移除第二个目标，那么我们仍然会有 $x^* = 5$ 。按照现在写出的这个优化问题，最终结果是什么？（如果你愿意用代码做，也只需要几行。）我想你会发现，这些“正交”的目标实际上会相互竞争！

如果你将问题改为：

$$\begin{aligned} \min_{x,y} \quad & (x-5)^2 + \frac{1}{100}(y+3)^2 \\ \text{subject to} \quad & x - y \leq 6? \end{aligned}$$

会发生什么？我想你会发现，解会非常接近 $x^* = 5$ ，但同时 y^* 也会与完全省略“次级”目标时得到的 $y^* = 0$ 明显不同。

注意，如果约束在最优解处并未激活（例如 $x = 5, y = -3$ 满足约束），那么这些目标就不会相互竞争。

这个例子看起来可能过于简单。但我认为你会发现，它实际上与上面零空间目标形式中发生的情况非常相似。

Exercise 3.12 (运动坐标系的空间速度)

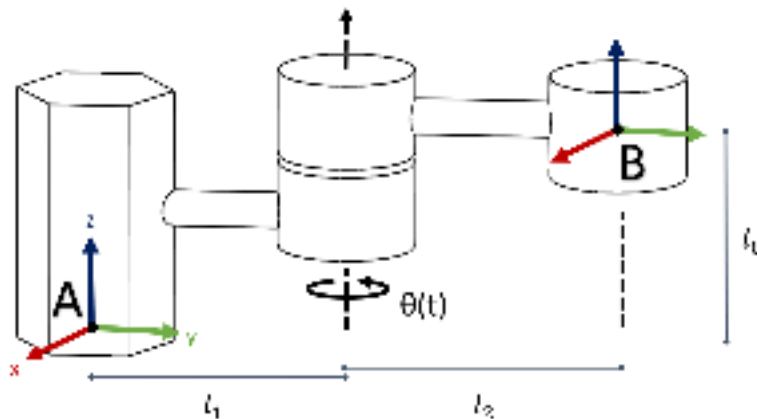


Figure 3.13 - 一个具有一个运动关节的机械臂。坐标系 A 是机器人基座坐标系，而坐标系 B 位于末端执行器上。

上图展示了一个具有一个关节的机械臂。该关节会随时间旋转，其角度是时间的函数 $\theta(t)$ 。当 $t = 0$ 时， $\theta(t) = 0$ ，并且连杆（机械臂）与坐标系 A 的 y 轴以及坐标系 B 的 y 轴对齐。在本练习中，我们将探索末端执行器的空间速度。

- 对于这个机械臂，给定 ${}^B p^C(0) = [1, 0, 0]^T$ ，写出 ${}^A p^C(0)$ 。对于一个一般点 C，推导 3x3 旋转矩阵 ${}^A R^B(t)$ 和平移向量 ${}^A p^B(t)$ ，使得 ${}^A p_A^C(t) = {}^A R^B(t) {}^B p_B^C + {}^A p^B(t)$ 。你的解应包含 l_0, l_1, l_2 ，和 $\theta(t)$ 。
- 证明对于任意旋转矩阵 R ， $\hat{\omega} \equiv \dot{R}R^{-1}$ 满足 $\hat{\omega} = -\hat{\omega}^T$ 。代入 $R = {}^A R^B(t)$ 来验证你的证明。
- 求解该机械臂的 ${}^A V^B(t)$ ，将其表示为 $l_0, l_1, l_2, \theta(t)$ 和 $\dot{\theta}(t)$ 的函数。

Exercise 3.13 (三维旋转表示中的挑战)

在关于表示三维旋转的小节中，我提到过，并不存在一种对所有事情都最好的三维旋转表示方式。在本题中，你将探索几种最常用表示方式所带来的一些意想不到的性质。

- Roll-Pitch-Yaw**：我们可以用一组“roll-pitch-yaw”角 (r, p, y) 来表示旋转。请参考 Drake 的 [RollPitchYaw](#) 类，了解这些单独角度旋转组合成一个旋转矩阵时所采用的顺序约定。给定 $p = -\pi/2$ ，请计算一个用 r 和 y 表示的旋转矩阵。在得到的矩阵中， r 的变化与 y 的变化相比有何关系？这说明“roll-pitch-yaw”作为旋转表示方式有什么问题？

提示：尝试使用[和差角三角恒等式](#)

$$\sin(\alpha + \beta) = \sin \alpha \cos \beta + \cos \alpha \sin \beta$$

$$\cos(\alpha + \beta) = \cos \alpha \cos \beta - \sin \alpha \sin \beta$$

来化简你的结果。

- 轴角 (Axis-Angle)**：考虑一个方向，它由一个右手坐标系绕其 z 轴旋转 $\pi/2$ 弧度得到。这个旋转会使正 x 轴指向旋转前正 y 轴所在的方向。表示这个旋转的一种方式轴角向量 $(0, 0, \pi/2)$ 。（回想一下，该向量描述旋转所绕的轴，而其模长表示旋转角度。）这种表示并不是唯一的。请找出另外两组能够产生相同旋转的轴角表示。
- 四元数 (Quaternion)**：单位四元数是一组四个数 (w, x, y, z) ，满足 $w^2 + x^2 + y^2 + z^2 = 1$ 。单位四元数是一种非常优秀的方向表示方式，因为它不像 roll-pitch-yaw 和轴角表示那样存在奇异性。这使它们在[插值方向](#)时非常可靠，但单位四元数仍然存在“多重表示”问题。

我们可以通过计算 $\theta = 2 \arccos(w)$ ，将单位四元数 (w, x, y, z) 转换为轴角向量 (a_x, a_y, a_z) ；对于任意整数 n ：

$$(a_x, a_y, a_z) = \begin{cases} \frac{\theta}{\sin(\theta/2)}(x, y, z) & \theta \neq 2n\pi \\ (0, 0, 0) & \theta = 2n\pi \end{cases}$$

请通过转换到轴角形式并化简所得三角表达式，证明对于任意单位四元数 (w, x, y, z) ， $(-w, -x, -y, -z)$ 表示相同的方向。

事实上，每个方向恰好对应两个单位四元数！

提示 1：对于任意 α 和 β ， $\arccos(-\alpha) = \pi - \arccos(\alpha)$ 且 $\sin(\pi - \beta) = \sin(\beta)$ 。

提示 2: 利用这样一个事实: 对于任意角度 θ , $\theta \pm 2\pi$ 表示相同的方向。

提示 3: 注意 $\frac{1}{\sin(\theta/2)}(x, y, z)$ 是一个单位向量。

Exercise 3.14 (抓取与操作字母资产)

在本练习中, 我们将基于本章开发的 [抓取与放置演示](#) 继续扩展。你将编写程序, 让机器人抓取并放置与你姓名首字母对应的字母资产。你将完全在 中完成。你需要完成以下步骤:

- a. 求解这些资产的抓取位姿和放置位姿。
- b. 补全一个 Differential IK 控制器。
- c. 构建一个 [Diagram](#), 用于仿真 IIWA 操作这些字母资产的控制过程。

REFERENCES

1. John J. Craig, "Introduction to Robotics: Mechanics and Control", Pearson Education, Inc , 2005.
2. Richard M. Murray and Zexiang Li and S. Shankar Sastry, "A Mathematical Introduction to Robotic Manipulation", CRC Press, Inc. , 1994.
3. Kevin M Lynch and Frank C Park, "Modern Robotics", Cambridge University Press , 2017.
4. Paul Mitiguy, "Advanced Dynamics & Motion Simulation: For Professional Engineers and Scientists (graduate Work) ; 3D, Computational, Guided", Prodigy Press , 2017.
5. John Stillwell, "Naive Lie theory", Springer Science & Business Media , 2008.
6. Ferdinando A. Mussa-Ivaldi and Neville Hogan, "Integrable Solutions of Kinematic Redundancy via Impedance Control", *The International Journal of Robotics Research*, vol. 10, no. 5, pp. 481-491, 1991.
7. Stephen Boyd and Lieven Vandenberghe, "Convex Optimization", Cambridge University Press , 2004.
8. Fabrizio Flacco and Alessandro De Luca and Oussama Khatib, "Control of redundant robots under hard joint constraints: Saturation in the null space", *IEEE Transactions on Robotics*, vol. 31, no. 3, pp. 637--654, 2015.
9. Adrien Escande and Nicolas Mansard and Pierre-Brice Wieber, "Hierarchical quadratic programming: Fast online humanoid-robot motion generation", *The International Journal of Robotics Research*, vol. 33, no. 7, pp. 1006--1028, 2014.

CHAPTER 4

几何位姿估计

在上一章中，我们为移动物体开发了一个初步解决方案，但我们做了一个会阻止其应用于真实机器人上的重大假设：我们假设已知物体的初始位姿。本章将首次尝试消除这一假设，通过开发工具，利用机器人深度相机获取的信息来估计该位姿。我们在这里开发的工具，在你尝试操作 **已知物体**（例如，你拥有其几何结构的网格文件）并且处于相对 **非杂乱** 环境中时，将特别有用。但它们也会成为我们后续学习更复杂方法的基础。

本章中我们采用的方法基于几何学。如今并非所有感知系统都会显式使用几何；许多深度学习方法会学习从像素到结果的更直接映射。但许多深度学习方法 **确实** 高度依赖几何，而且过去几年中，几何处理领域也经历了一场革命（稠密重建、文本生成 3D 等）。这些进展最初部分受到自动驾驶、虚拟/增强现实以及机器人操作等应用的推动，从而催生了先进的传感器和出色的新算法。

因此，我们将从几何开始，因为它能够为我们探索感知问题提供坚实基础。不过我必须说明一个注意事项——本章将“位姿估计”作为目标，因为它具体且能立即发挥作用。但我要说的是，在我们最先进的操作系统中，我会尽量 **不** 把显式位姿估计作为核心概念；除了需要假设物体已知之外，精确估计物体位姿本身也可能非常困难（由于视角不完整和/或遮挡），而且很多时候这已经超出了实现操作目标的实际需求。

4.1 相机与深度传感器

正如我们在选择机器人手臂和机械手时有许多选择一样，我们在为机器人/环境配置传感器时同样有很多选择。与机器人手臂相比，过去几年这些传感器在质量上的提升以及成本和尺寸上的降低更加惊人。这很大程度上得益于手机产业，而自动驾驶汽车的竞争也推动了高端传感器的发展。

这些硬件质量的变化，有时会导致我们的算法方法发生巨大变化。例如，当传感器的分辨率和帧率足够高，以至于世界在两帧图像之间不会发生太大变化时，估计问题会变得容易得多；这无疑促进了过去十年左右“同步定位与建图”（SLAM）领域的革命性进展。

人们可能会认为，对机器人操作来说最重要的传感器是触觉传感器（你甚至可能是对的！）。但在当前实践中，大多数重点仍然放在基于相机和/或距离感知的传感器上。至少，我们应该首先考虑这一点，因为如果我们不知道世界中需要触碰的位置，那么触觉传感器也帮不上太大忙。

传统相机——我们通常将其视为一种以某个帧率输出彩色图像的传感器——发挥着重要作用。但机器人领域大量使用能够显式测量距离（相机与世界之间的距离）或深度的传感器；有时它们会同时提供颜色信息，有时则用深度信息替代颜色。

4.1.1 深度传感器

主要包括 RGB-D（ToF、投影纹理立体视觉等）相机以及激光雷达（Lidar）

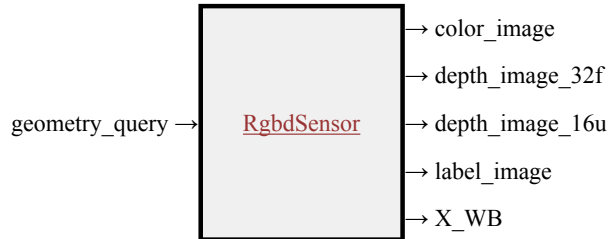
本课程中我们使用的相机是 Intel RealSense D415。

单目深度。

基于学习的立体视觉 [1]。

4.1.2 仿真

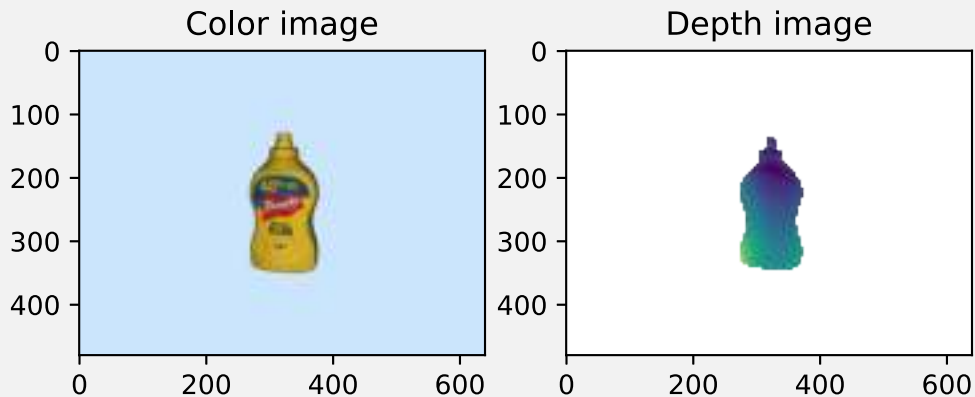
对于像 D415 这样的相机，可以在多个不同保真度层级上进行仿真。我们将在这里从一种“理想”的 RGB-D 相机仿真开始讨论——深度图像中返回的像素表示沿着每个像素坐标方向上的真实几何深度。在 **Drake** 中，这由 **RgbSensor** 系统表示，它可以直接连接到 **SceneGraph**。



这里的信号与系统抽象封装了大量复杂性。在该系统的实现内部，包含了一个完整的渲染引擎，就像你在高端电脑游戏中见到的那种。Drake 实际上支持多种**渲染引擎**；在本课程中，我们主要使用基于 OpenGL 的渲染器（构建于 **VTK** 之上），它适用于实时仿真。Drake 还能够连接高端的**基于物理的渲染（PBR）引擎**，这些引擎使用光线追踪技术，例如 **Blender 提供的 Cycles 渲染器**（参见 [Drake-Blender 项目](#)）。这些工具可能有助于渲染更高质量的图像，例如用于**训练**深度感知系统或制作精美演示。

Example 4.1 (仿真 RGB-D 相机)

作为 drake 中深度相机的一个简单示例，我构建了一个包含单个物体的场景（来自 **YCB 数据集** 中的芥末瓶），并向图中添加了一个 **RgbSensor**。完成连接后，我们只需评估输出端口，就能获得彩色图像和深度图像：



请务必花一分钟查看 MeshCat 可视化（[可在此处访问](#)）。你会看到一台相机和相机坐标系，也会像往常一样看到芥末瓶……但它可能看起来有点奇怪。这是因为我同时显示了真实的芥末瓶模型**以及**由相机渲染出的点云。你可以使用 MeshCat GUI 取消勾选真实模型可视化（右图），这样就只能看到点云。

请记住，除了需要在需要时查看源代码之外，你也始终可以检查框图，以理解“系统层面”上正在发生什么。[这里是本示例使用的框图](#)。



在 `HardwareStation` 工作流中，如果在场景数据（例如 `Yaml` 描述）中声明了相机，那么 `Rgbdsensors` 会被自动添加到 `HardwareStation` 图内部，并且它们的输出端口会作为输出暴露出来（以匹配我们与真实硬件之间的实际接口）。

4.1.3 传感器噪声与深度丢失

当然，真实的深度传感器远非理想——而且深度返回中的误差并不是简单的高斯噪声，而是取决于光照条件、物体的表面法线、物体的视觉材料属性以及其他因素。颜色通道同样也只是一种近似。即使是我们的渲染器，也只有向场景中加入足够准确的几何体、材料和光源时才能表现良好，而要捕捉真实环境的所有细微差别非常困难。在我们开始理解一些基本操作之后，我们将考察真实传感器数据，以及建模传感器与真实传感器之间的差距。

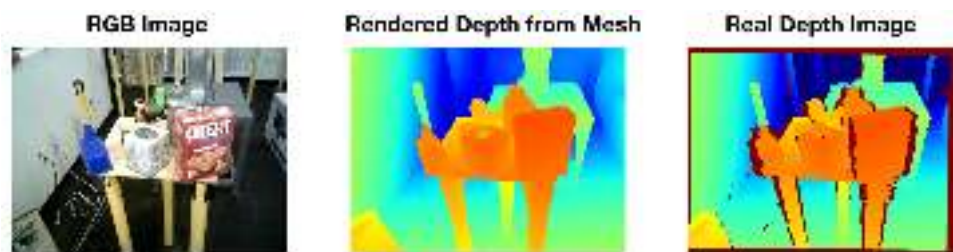


Figure 4.2 - 真实深度传感器的数据很杂乱。最右侧图像中的红色像素表示未返回测量值——深度传感器返回了“最大深度”。未返回测量值，尤其是在物体边缘（或反光物体上），在原始深度数据中很常见。本图转载自 [2]。

4.1.4 遮挡与局部视图

上面示例中芥末瓶的视图非常清楚地展示了使用相机时的一个主要挑战。相机并不能看到所有东西！它们只能看到视线范围内的内容。如果你想获得芥末瓶背面的点，就需要移动相机。而如果芥末瓶放在桌子上，那么如果你想看到底部，就必须把它拿起来。在杂乱场景中，这种情况会变得更糟，因为我们对瓶子的视图会被其他物体遮挡。即使场景并不杂乱，当机器人准备进行操作时，安装在头部或桌面上的相机也常常会被机器人的手挡住——你最希望传感器表现最佳的时刻，往往正是它们无法提供信息的时候！

为了缓解局部视图问题，将深度相机安装在机器人手腕上是相当常见的做法。但这并不能完全解决问题。我们所有的深度传感器都有最大量程和最小量程。`RealSense D415` 的匹配返回范围大约是 0.3m 到 1.5m。这意味着在接近物体的最后 30cm 内——同样也是我们希望相机表现最佳的地方——物体实际上会消失！

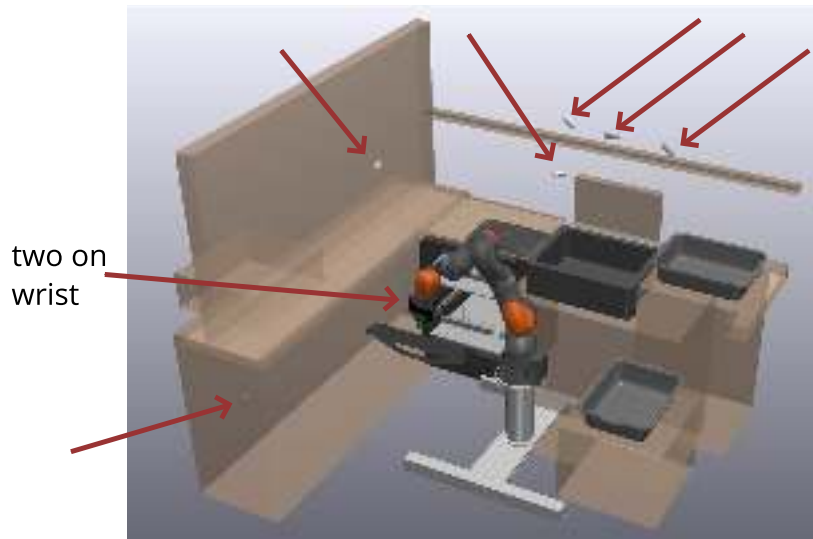


Figure 4.3 - 在条件允许的情况下，我们会尽可能在机器人的环境中放置更多相机。在 [TRI](#) 的这个装盘测试平台中，我们总共在场景中放置了八台相机（但这仍然不能解决遮挡问题）。

4.2 几何的表示方法

3D 几何有许多不同的表示方法，每一种都可能更适合某些不同的计算。我们通常可以在这些表示之间进行转换；不过有时这种转换会带来信息损失。例子包括三角化表面网格和四面体体积网格、体素/占据栅格，以及几何的隐式表示，例如有符号距离函数和神经辐射场（NeRF），它们最近又作为 3D 视觉深度学习中的表示方法变得非常流行[\[3, 4\]](#)。但我们在这里将首先使用的表示是[深度图像](#)和[点云](#)。

深度相机返回的数据采用图像的形式，其中每个像素值都是一个数字，表示沿该像素方向，相机与环境中最接近物体之间的距离。如果我们将其与相机内参的基本信息（例如镜头参数，在 [Drake](#) 中存储于 [CameraInfo](#) 类中）结合起来，那么就可以将这种深度图像表示转换为一组 3D 点 s_i 。我在这里使用 s ，是因为在下面将介绍的算法中，它们通常被称为“场景点”。这些点表示在某个由刚体变换描述的参考坐标系中，并且（可选地）附带颜色值或其他信息；这就是[点云](#)表示。在 [Drake](#) 的 [DepthImageToPointCloud](#) 系统中，默认坐标系是相机坐标系 ${}^C X^{s_i}$ ，但该系统也在一个（可选）输入端口上接受 ${}^P X^C$ ，以便轻松地将它们变换到另一个坐标系中（例如世界坐标系）。



随着深度传感器变得如此普及，该领域已经积累了许多工具库，用于对点云执行基本几何操作，并可用于在不同表示之间来回转换。我们在 [Drake](#) 中直接实现了许多基本操作。如果你需要更多功能，也可以使用诸如 [Open3D](#) 这样的开源工具库。许多较早的项目使用 [点云库 \(Point Cloud Library, PCL\)](#)，它现在已经停止维护，但仍然有一些非常有用的文档。

需要认识到的是，将深度图像转换为点云确实会丢失一些信息——具体来说，就是从相机投射射线并到达该点的相关信息。除了声明“这个点处存在几何体”之外，深度图像结合相机位姿还意味着“在相机与该点之间的直线路径上不存在几何体”。我们会在某些算法中使用这一信息，所以不要完全丢弃深度图像！更一般地说，我们会发现每种不同的几何表示都有其优点和缺点——保留多种表示并在它们之间来回转换是非常常见的做法。

4.3 处理点云

首先，让我们获取几个用于处理的仿真点云。我们可以使用 **DRAKE** 或 **Open3D** 将深度图像转换为点云；这里我们在这一步使用 **Drake**，只是因为我们已经拥有 **Drake** 格式的相机参数。虽然我们可以直接调用转换函数，但也可以在系统框架中添加一个系统，由它为我们完成转换：



从该系统的可选输入端口可以推断出，点云表示可以为每个笛卡尔点包含颜色值。下面的示例将为我们获取几个可供处理的点云。

可以对点云执行许多基本操作。这里我只给出几个例子。

4.3.1 表面法线估计

估计点云表面法线的最简单、最直接算法，是找到 k 个最近邻，并使用最小二乘法通过这些点拟合一个平面。我们将名义点（即我们要围绕其估计法线的点）记为 p_0 ，并将它的 k 个最近邻记为 $p_i, i \in [1, k]$ 。令我们构造 $k \times 3$ 的数据矩阵：

$$X = [p_1 - p_0, p_2 - p_0, \dots, p_k - p_0].$$

那么该数据矩阵的主成分由 X 的**奇异值分解**给出（或由 $X^T X$ 的特征向量给出）；如果 $X = U \Sigma V^T$ ，那么 V 的各列就是主方向，其中最后一列（对应最小奇异值）表示法线方向。长度可以归一化为一；但请记住，该向量的符号是不确定的——这 k 个最近点无法提供关于哪个方向进入物体、哪个方向离开物体的信息。这种情况下，知道原始相机方向会有所帮助——例如，**PCL** 提供了一个 **flipNormalTowardsViewpoint** 方法，用于对法线进行后处理。

Figure 4.4 - 使用 k 个最近邻进行法线估计（图片链接自 **PCL** 文档）

关于这一点，让我觉得有趣且惊讶的并不是最小二乘解有效。真正令人惊讶的是，这个操作被认为是常规且廉价的——对于点云中的每一个点，我们都找到 k 个最近邻并进行法线估计……即使点云非常大。要让这一过程高效，通常需要使用 **k-d 树** 这样的数据结构，以提高最近邻查询的效率。

另一个可以从 k 个最近邻中提取的标准特征是局部曲率。其数学形式非常相似。

4.3.2 平面分割

除了表面法线和局部曲率这样的局部属性之外，我们还可以选择将点云处理成基本形状——例如盒子或球体。另一种实用且自然的分割方式是将表面分割成平面……这可以通过一些启发式算法来完成，这些算法会将具有相似表面法线的相邻点分组。我们曾在 **DARPA** 机器人挑战赛中有效地使用过这一方法。

4.4 具有已知对应关系的点云配准

让我们开始处理本章的主要目标——机器人工作空间中的某处有一个已知物体，我们已经从深度

相机获得了一个点云。我们如何估计该物体的位姿 X^O ?



这一问题的一种非常自然且经过充分研究的表述，来自关于[点云配准](#)（也称为点集配准或扫描匹配）的文献。给定两个点云，点云配准的目标是找到一个刚体变换，使这两个点云达到最优对齐。就我们的目的而言，这意味着物体的“模型”也应当采用点云的形式（至少目前如此）。我们将用一组[模型点](#) m_i 来描述该物体，它们的位姿在物体坐标系中表示为： $^O X^{m_i}$ 。

我们的第二个点云将是来自深度相机获得的[场景点](#) s_i ，并通过相机内参将其变换到相机坐标中，即 $^C X^{s_i}$ 。我将分别使用 N_m 和 N_s 表示模型点和场景点的数量。

现在让我们假设，我们也知道 X^C 。这合理吗？对于腕部安装的相机来说，这相当于求解机械臂的正向运动学。对于安装在环境中的相机来说，这意味着需要知道相机在世界中的位姿。不过，这些位姿估计中的小误差可能会在估计出的点云中造成很大的伪影。因此，我们会非常谨慎地进行[相机外参标定](#)；我已经在[附录](#)中链接了我们的标定代码。请注意，如果你拥有一个移动操作平台（安装在移动底盘上的机械臂），那么所有这些讨论仍然适用，但你很可能在机器人坐标系中而不是世界坐标系中执行所有配准。

我们将在本节中做出的第二个[主要假设](#)是“已知对应关系”。当我在这里说“对应关系”时，我的意思是，对于场景点云中的每个点，我们都可以将它与模型点云中的某个具体点配对；例如，如果每个点都有一种唯一颜色，并且我们能够通过相机可靠地感知到这种颜色，那么就可能出现这种情况。这在实践中[并不是](#)一个合理假设，但它有助于我们入门。为了在方程中表示这种映射，我将使用一个对应向量 $c \in [1, N_m]^{N_s}$ ，其中 $c_i = j$ 表示场景点 s_i 对应于模型点 m_j 。请注意，我们并不假设这些对应关系是一一对应的。每个场景点都对应某个模型点，但反过来并不成立，而且多个场景点可能对应同一个模型点。

因此，点云配准问题其实只是一个（逆）运动学问题。我们可以在一个公共坐标系中（这里是世界坐标系）写出模型点和场景点，

$$^W p^{m_{c_i}} = ^W X^O {}^O p^{m_{c_i}} = ^W X^C {}^C p^{s_i},$$

其中只留下一个未知变换 X^O 需要求解。在上一章中，我主张使用微分运动学而不是逆运动学；为什么这里我的说法不同呢？微分运动学对感知仍然可能有用，例如当我们在获得物体初始位姿之后想要跟踪其位姿时。但与机器人情形不同，在机器人中我们可以读取关节传感器来得到当前状态，而在感知中，我们至少需要求解一次这个更困难的问题。

X^O 的解是什么样的？当 $p^{s_i} \in \mathbb{R}^3$ 时，每个模型点都会给出三个约束。未知量的确切数量取决于我们为位姿选择的具体表示方式，但几乎总是这个问题会被严重过约束。将每个点都视为相对位姿上的硬约束，也会非常容易受到测量噪声影响。因此，更好的做法是尝试找到一个能够描述数据的位姿，例如在最小二乘意义下：

$$\min_{X \in \text{SE}(3)} \sum_{i=1}^{N_s} \|X^O {}^O p^{m_{c_i}} - X^C {}^C p^{s_i}\|^2.$$

这里我使用 $\text{SE}(3)$ 这个符号表示“[特殊欧几里得群](#)”，这是表示 X 必须是一个有效刚体变换的标准记号。

为了继续，让我们选择一种具体的位姿表示来处理。我将在这里使用 3×3 旋转矩阵；下面要描述的方法也有等价的四元数形式[\[5\]](#)。为了使用旋转矩阵的系数和平移作为决策变量来写出上面的优化问题，我有：

$$\begin{aligned} \min_{p, R} \quad & \sum_{i=1}^{N_s} \|p + R^O {}^O p^{m_{c_i}} - X^C {}^C p^{s_i}\|^2, \\ \text{subject to} \quad & R^T = R^{-1}, \quad \det(R) = +1. \end{aligned} \tag{1}$$

这些约束是必要的，因为并不是每个 3×3 矩阵都是有效的旋转矩阵；满足这些约束可以确保 R 是一个有效的旋转矩阵。事实上，约束 $R^T = R^{-1}$ 本身几乎就足够了；它意味着 $\det(R) = \pm 1$ 。但满足该约束且 $\det(R) = -1$ 的矩阵称为“**非正常旋转**”，也就是一次旋转再加上关于旋转轴的反射。

让我们思考一下目前所建立的优化问题类型。鉴于我们决定使用旋转矩阵，项 $p + R^O p^{m_{c_i}} - X^{CC} p^{s_i}$ 关于决策变量是线性的（可以类比为 $Ax \approx b$ ），因此目标函数是一个凸二次函数。那么约束条件呢？第一个约束是一个二次等式约束（为了看清这一点，可以将其重写为 $RR^T = I$ ，这样会得到 9 个约束，每一个都关于 R 的元素是二次的）。行列式约束则关于 R 的元素是三次的。

Example 4.2 (二维中的纯旋转点配准)

我希望你能对这个优化问题有一种图形化的直观理解，但由于它有 $9+3$ 个决策变量，因此很难绘制其目标函数。为了将问题简化到可以绘图的核心部分，我们考虑这样一种情况：场景点与模型点之间只相差一个旋转（这就是著名的 **Wahba 问题**）。为了进一步简化，我们考虑二维问题而不是三维问题。

在二维中，我们实际上可以只用两个变量对旋转矩阵进行线性参数化：

$$R = \begin{bmatrix} a & -b \\ b & a \end{bmatrix}.$$

使用这种参数化时，约束 $RR^T = I$ 和行列式约束是等价的；它们都要求 $a^2 + b^2 = 1$ 。

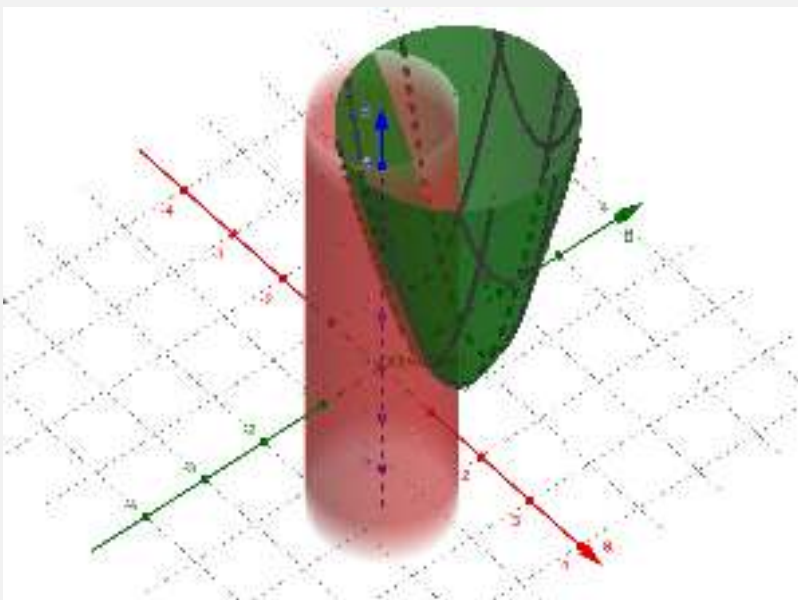


Figure 4.5 - 该图的 x 轴和 y 轴分别对应定义二维旋转矩阵 R 的参数 a 和 b 。绿色的二次碗面是目标函数 $\sum_i \|R^O p^{m_{c_i}} - p^{s_i}\|^2$ ，红色的开口圆柱是旋转矩阵约束。[点击这里查看交互版本。](#)

这和你预期的一样吗？我生成这张图时只使用了两个模型点/场景点，但增加更多点只会改变二次函数的形状，而不会改变其最小值。而且在该图的 [交互版本](#) 中，我添加了一个滑块，让你可以控制参数 θ ，它表示由 R 描述的真实旋转。试试看吧！

在测量没有噪声的情况下（例如场景点正好是经过旋转矩阵变换后的模型点），目标函数的最小值本身就已经满足约束。但如果只给点添加一点点噪声，这种情况就不再成立，此时约束便开始发挥重要作用。

在三维情况下，其几何结构应该大致相同，只是维度显然要高得多。不过，我希望这个图能让你觉得：基于奇异值分解（SVD）的优雅数值解法，或许没有那么令人意外。

那么平移呢？这里有一个极其重要的洞见，它使我们能够将旋转优化与平移优化解耦。这个洞见是：**点与点之间的相对位置会受到旋转影响，但不会受到平移影响**。因此，如果我们将点写成相对于物体上某个规范点的位置，那么就可以单独求解旋转。一旦得到旋转，就可以轻松反推出平移。对于我们的最小二乘目标函数来说，甚至存在一个“正确”的规范点——在欧氏度量下的**中心点**（类似于当所有点质量相等时的质心）。

因此，为了得到如下问题的解，

$$\min_{p \in \mathbb{R}^3, R \in \mathbb{R}^{3 \times 3}} \sum_{i=1}^{N_s} \|p + R^O p^{m_{c_i}} - p^{s_i}\|^2, \quad (2)$$

$$\text{subject to } RR^T = I, \quad (3)$$

我们首先定义模型点与场景点的中心点 $\bar{m}$ 和 $\bar{s}$ ，其位置定义为

$$^O p^{\bar{m}} = \frac{1}{N_s} \sum_{i=1}^{N_s} ^O p^{m_{c_i}}, \quad p^{\bar{s}} = \frac{1}{N_s} \sum_{i=1}^{N_s} p^{s_i}.$$

如果你好奇，这些结果来自于对**拉格朗日函数**关于 p 求梯度并令其等于零：

$$\sum_{i=1}^{N_s} 2(p + R^O p^{m_{c_i}} - p^{s_i}) = 0 \Rightarrow p^* = \frac{1}{N_s} \sum_{i=1}^{N_s} p^{s_i} - R^* \left(\frac{1}{N_s} \sum_{i=1}^{N_s} ^O p^{m_{c_i}} \right),$$

但不要忘记上面的几何解释。这只是另一种说明：我们可以将右侧表达式代入目标函数中的 p ，然后只对 R 进行求解。

为了求解 R ，构造数据矩阵

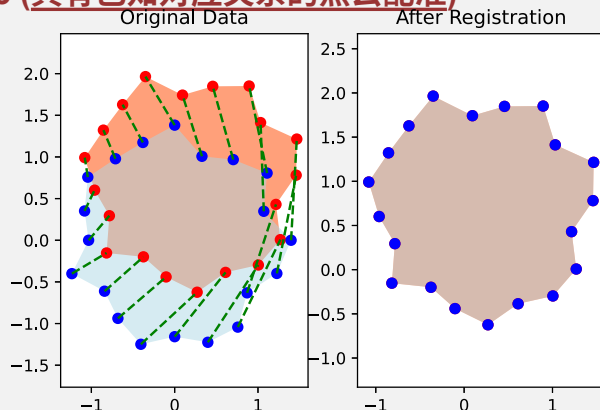
$$W = \sum_{i=1}^{N_s} (p^{s_i} - p^{\bar{s}})(^O p^{m_{c_i}} - ^O p^{\bar{m}})^T,$$

并使用 SVD 计算 $W = U\Sigma V^T$. 最优解为

$$R^* = UDV^T, \\ p^* = p^{\bar{s}} - R^* ^O p^{\bar{m}},$$

其中 D 是对角矩阵，其对角元素为 $[1, 1, \det(UV^T)]$ [6]。这看起来可能像魔法，但在 SVD 中将奇异值替换为 1，会得到回投到正交矩阵集合上的最优投影，而这个对角矩阵则确保我们不会得到非正常旋转。文献中有许多关于这一结果的推导，但很多推导会直接忽略行列式约束，而不是处理非正常旋转的问题。参见 [7]（第 3 节），这是我早期最喜欢的推导之一。

Example 4.3 (具有已知对应关系的点云配准)



在示例代码中，我创建了一个随机物体（基于 $x-y$ 平面中的一组随机点），并用一个随机二维变换对其进行了扰动。蓝色点是模型坐标系中的模型点，红色点是场景点。绿色虚线表示（完美的）对应关系。在右侧，我再次绘制了这两组点，但这一次使用估计出的位姿将它们都放到世界坐标系中。正如预期，该算法完美恢复了真实变换。

需要理解的重要一点是，一旦给定对应关系，我们就有一种高效且鲁棒的数值方法来估计位姿。

4.5 迭代最近点 (ICP)

那么我们如何获得对应关系呢？事实上，如果我们已知物体的位姿，那么找出对应关系其实很容易：当它们被变换到同一个坐标系中时，模型上对应的点就是离场景点最近的近邻点/最近点。

这一观察自然引出了一种迭代方案：我们从物体位姿的某个初始猜测开始，通过最近点计算对应关系，然后利用这些对应关系更新估计出的位姿。这就是点云配准中一种著名且经常使用的算法（尽管它有众所周知的局限性）：迭代最近点（ICP）算法。

准确地说，我们用 $\hat{X}^O$ 表示对物体位姿的估计，用 $\hat{c}$ 表示估计出的对应关系。“最近点”步骤为

$$\forall i, \hat{c}_i = \operatorname{argmin}_{j \in N_m} \|\hat{X}^O p^{m_j} - p^{s_i}\|^2.$$

换句话说，我们希望找到模型中与变换后的场景点在欧氏距离上最近的点。这就是著名的“最近邻”问题，并且我们有很好的数值解法（使用优化的数据结构）来解决它。例如，Open3D 使用 [FLANN](#)[8]。

虽然联合求解位姿和对应关系非常困难，但 ICP 利用了这样一个思想：如果我们将二者分开求解，那么两个部分都有很好的解。像这样的迭代算法，是优化中常见的方法，例如用于双线性优化或期望最大化。需要理解的是，这是一个非凸优化问题的局部解。因此，它可能会陷入局部最小值。

Example 4.4 (迭代最近点)

[点击这里查看动画。](#)

这里是在随机二维物体上运行的 ICP。蓝色是模型点，红色是场景点，绿色虚线是对应关系。我鼓励你自己运行代码。

我包含了其中一个动画，在该动画中它恰好收敛到了真正的最优解。但它更多时候会陷入局部最小值！我希望逐步查看这个过程能帮助你建立直觉。请记住，一旦对应关系正确，位姿估计就会恢复出精确解。因此，在收敛之前的每一步中，至少都有一些错误对应关系。仔细观察！

对这些局部最小值的直觉，推动了许多 ICP 变体的发展，包括点到平面 ICP、法线 ICP、使用颜色信息的 ICP、基于特征的 ICP 等等。一种基于“点对特征”的特定变体，在了一项（近乎）年度的物体位姿估计挑战赛中表现非常有效[9, 10]。

不过，也许更根本的是，我认为即使我们能够完美优化 ICP 目标函数，它本身也并不完全正确。（TODO：添加一本又长又薄的书的示例）。而且，我们拥有一些本应有助于位姿估计的信息，但这些信息并不存在于点云中（例如相机的位置）。

4.6 处理局部视图和离群点

上面的例子足以展示 ICP 可能遇到的局部最小值问题。但到目前为止，我们一直在处理过于完美的点云。点云配准是许多领域中的一个重要主题，而你在文献中看到的并非所有方法都适合处理我们在机器人中必须面对的杂乱点云。

Example 4.5 (带有杂乱点云的 ICP)

我在 notebook 中添加了许多参数供你尝试，[点击这里查看动画。](#)用来添加离群点（实际上不对应任何模型点的场景点）、添加噪声，和/或将点限制为局部视图。请分别尝试它们，也尝试组合使用它们。

Figure 4.8 - 带有离群点的 ICP 的一次示例运行（离群点建模为从工作空间上的均匀分布中额外抽取的点）。

[点击这里查看动画。](#)

Figure 4.9 - 在场景点位置中添加高斯噪声后，ICP 的一次示例运行。

[点击这里查看动画。](#)

Figure 4.10 - 场景点被限制为局部视图时，ICP 的一次示例运行。它们并不全都这么糟糕，但当我看到这个例子时，实在忍不住把它用作示例！

4.6.1 检测离群点

一旦我们有了一个合理的估计 X^O ，检测离群点就很直接了。每个场景点都会根据该场景点与其对应模型点之间的（平方）距离，为目标函数贡献一项。距离较大的场景点可以被标记为离群点并从点云中移除，使我们能够在没有这些干扰项的情况下细化位姿估计。你会发现许多 ICP 实现，例如 [Open3D 中的这个实现](#)，都包含一个用于此目的的“最大对应距离”参数。

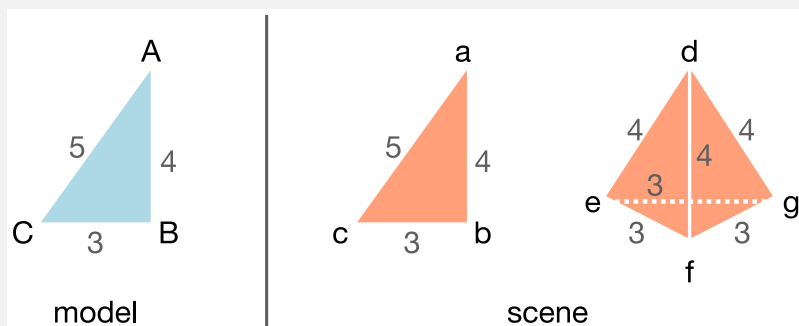
这给我们留下了一个“先有鸡还是先有蛋”的问题——我们需要一个合理的位姿估计来检测离群点，但对于非常杂乱的点云，在这些离群点存在的情况下，可能很难得到合理的估计。那么我们应该从哪里开始呢？一种传统上常与 ICP 一起使用的常见方法，是一种名为随机采样一致性（Random Sample Consensus, RANSAC）的算法 [11]。在 RANSAC 中，我们选择若干个随机的（通常相当小的）“种子”点子集，并为每个子集计算一个位姿估计。具有最多“内点”的子集——即低于最大对应距离的点——被认为是获胜集合，并可用于启动 ICP 算法。

还有另一个重要且巧妙的思想需要了解。请记住前面那个重要观察：利用点与点之间的[相对位置](#)依赖于旋转但对平移不变这一事实，我们可以将旋转和平移的求解解耦。我们可以将这个思想再

推进一步，并注意到点与点之间的**相对距离**实际上同时对旋转**和平移**都不变。传统上，这一性质被用于将尺度估计与平移和旋转估计分离（在本章中，我决定始终忽略尺度，因为它对于我们提出的问题并没有直接意义）。在 [12] 中，他们提出，这一性质也可以在我们还没有初始位姿估计之前，用来剔除离群点。如果场景中一**对点**之间的距离，不像模型中任意一对点之间的任何距离，那么这两个点中就有一个是离群点。按照这一度量寻找最大的内点集合，将对应于在图中寻找最大团；人们可以做出强假设来保证最大团就是最佳匹配，或者简单地承认这可能是一种有用的启发式方法。如果对于大型点云来说寻找最大团代价太高，也可以改用保守近似。

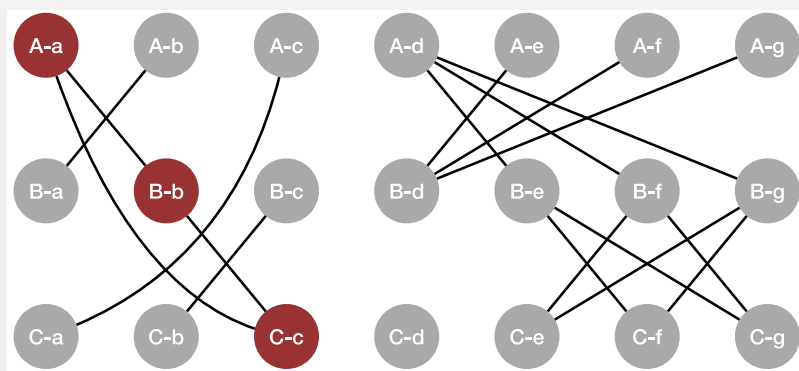
Example 4.6 (对应图中的最大团)

这里是来自 [12] 的最大团思想的一个简单示例，我与作者 Hank 一起推演了这个例子，以确保自己理解了他的想法。



模型（左侧）是一个简单三角形，其顶点标记为 $[A, B, C]$ 。场景（右侧）中有相同的三角形，点标记为 $[a, b, c]$ ，它们将恰好具有三个成对距离。但场景中还有一个金字塔，它会产生总共六个成对距离匹配（三个长度为 3，三个长度为 4）。金字塔上的虚假对应关系，会不会造成一个比真实对应关系三角形 $[A - a, B - b, C - c]$ 更大的团？

让我们使用“全对全”对应关系作为初始猜测，来构造这个成对距离图。在该图中，每一个可能的对应关系都会对应一个节点，例如我使用标签 $A - e$ 表示模型点 A 与场景点 e 之间的一种可能对应关系。然后，当且仅当 A 与 B 之间的距离等于 e 与 f 之间的距离时，我们就在图中于 $(A - e)$ 与 $(B - f)$ 之间建立一条边。



现在，检查团（clique）的价值就变得清晰了。涉及金字塔对应关系的最大团大小只有 2，而场景三角形对应的最大团大小则为 3（我已将其标为红色）。因此，对于这个例子来说，最大团**确实**恢复了真实对应关系。

以上只是用于移除离群点的大量算法/启发式方法中的两个例子。另一个值得一提的例子是 [9]，它使用了一种基于点对特征的优化投票/聚类方案。他们将其实现得非常高效，并且多年来在在线位姿估计基准测试中表现非常出色。

4.6.2 点云分割

并非所有离群点都来自深度相机的错误返回，它们很多时候其实来自场景中的其他物体。即使在最基本的情况下，我们感兴趣的物体也会放在桌子上或箱子里。如果我们预先知道桌子/箱子的几何形状，那么就可以减去该区域中的点。另一种方法是将点云裁剪 (Crop) 到一个感兴趣区域。这对于非常简单或不杂乱的场景已经足够，也足以让我们完成本章的目标。

更一般地说，如果场景中存在多个物体，我们可能仍然希望找到感兴趣的物体（比如芥末瓶？）。如果我们拥有真正鲁棒的点云配准算法，那么人们可以设想：最优配准将允许模型点仅与场景点中的一个子集对应；点云配准本身就能解决目标检测问题！遗憾的是，我们的算法还没有强大到这种程度。



Figure 4.13 - 来自 [LabelFusion](#) 的多物体场景 [13]。

因此，我们几乎总是使用某种其他算法将场景“分割”为若干个可能的物体，然后在每个分割片段上独立运行配准。有许多几何方法可用于分割[14]。但如今我们往往使用神经网络进行分割，所以我在这里不会深入这些细节。

4.6.3 推广对应关系

场景中的离群点或多个物体问题，对我们当前的对应关系处理方式提出了挑战。到目前为止，我们一直使用记号 $c_i = j$ 来表示每一个场景点到某个模型点的对应关系。但请注意，到目前为止我们的算法存在不对称性（场景 $\Rightarrow$ 模型）。我选择这个方向，是因为本章开头芥末瓶示例中的局部视图问题给了我这样的动机。不过，一旦我们开始考虑桌子以及场景中的其他物体，也许我们本该使用模型 $\Rightarrow$ 场景？你应该自己验证一下，这对 ICP 算法来说只是一个很小的改动。但如果我们同时有多个物体以及局部视图呢？

允许 c_i 取一个表示“无对应关系”的特殊值是很简单的，而且这确实有所帮助。在这种情况下，我们希望在最优解处，对应于感兴趣模型的场景点会被映射到各自的模型点，而来自离群点和其他物体的场景点会被标记为“无对应关系”。来自物体被遮挡部分的模型点，则 simply 不会有任何与之关联的对应关系。

我们可以使用一种稍微更一般的替代记号。令对应矩阵为 $C \in \{0, 1\}^{N_s \times N_m}$ ，其中当且仅当场景点 i 对应于模型点 j 时， $C_{ij} = 1$ 。使用这一记号，我们可以将估计步骤（在已知对应关系的情况下）写为：

$$\min_{X \in SE(3)} \sum_{i=1}^{N_s} \sum_{j=1}^{N_m} C_{ij} \|X^O p^{m_j} - p^{s_i}\|^2.$$

这涵盖了我们之前的方法：如果 $c_i = j$ ，则令 $C_{ij} = 1$ ，并将该行其余元素设为零。如果场景点 i 是离群点，则将整行设为零。之前的记号是对我们上面所做“最近点”计算中真实不对称性的一种更紧凑表示，并且清楚地表明我们只需要计算 N_s 个距离。但这种更一般的记号将带我们进入下一轮讨论。

4.6.4 软对应关系

如果我们放松对对应关系的严格二元定义，不再将其视为 $C_{ij} \in \{0, 1\}$ ，而是将其看作对应权重 $C_{ij} \in [0, 1]$ ，会怎样？这正是“相干点漂移”（coherent point drift, CPD）算法 [15] 背后的主要思想。如果你阅读 CPD 文献，会看到大量使用高斯混合模型（Gaussian Mixture Model, GMM）作为表示的概率论阐述。概率解释很有用，但不要让它把你弄糊涂；相信我，它本质上是同一件事。为了最大化高斯分布上的似然，第一步几乎总是取 $\log$ ，因为它是单调函数，而这会让你直接回到我们的二次目标函数。

概率解释确实为算法每次迭代中设置对应权重提供了一种自然机制，即将它们视为给定模型点时场景点的概率：

$$C_{ij} = \frac{1}{a_i} \exp \frac{-x^O O_p^{m_j - p^{s_i} 2}}{2\sigma^2}, \quad (4)$$

这就是高斯分布的标准密度函数，其中 σ 是标准差， a_i 是适当的归一化常数，用于使概率之和为一。在这种表述中编码离群点的概率也很自然（它只是修改归一化常数）。点击查看归一化常数，但它其实只是一个实现细节。归一化结果为

$$a_i = (2\pi\sigma^2)^{\frac{D}{2}} \left[\sum_{j=1}^{N_m} \exp \frac{-x^O O_p^{m_j - p^{s_i} 2}}{2\sigma^2} + \frac{w}{1-w} \frac{N_m}{N_s} \right], \quad (5)$$

其中 $D = 3$ 是欧氏空间的维度， $0 \leq w \leq 1$ 是一个样本点为离群点的概率 [15]。

CPD 算法现在与 ICP 非常相似，会在分配对应权重和更新位姿估计之间交替进行。位姿估计步骤几乎与上面的过程完全相同，只是“中心”点现在变为

$$O_p^{\bar{m}} = \frac{1}{N_C} \sum_{i,j} C_{ij} O_p^{m_j}, \quad p^{\bar{s}} = \frac{1}{N_C} \sum_{i,j} C_{ij} p^{s_i}, \quad N_C = \sum_{i,j} C_{ij},$$

而数据矩阵现在为：

$$W = \sum_{i,j} C_{ij} (p^{s_i} - p^{\bar{s}}) (O_p^{m_j} - O_p^{\bar{m}})^T.$$

其余更新步骤，包括使用 SVD 提取 R ，以及在给定 R 的情况下求解 p ，都是相同的。你不在代码中明确看到这些求和，因为如果我们把点收集到一个每列对应一个点的矩阵中，每一步都有特别漂亮的矩阵形式。

概率解释还为我们提供了一种策略，用于在每次迭代中确定高斯分布的协方差。[15] 将 σ 的估计推导为：（TODO：完成这部分推导的转换）

坊间说法是，与使用硬对应关系和二次目标函数的 ICP 相比，CPD 要鲁棒得多；高斯模型通过将离群点的对应权重设为接近零，来减轻离群点的影响。但对于大型点云来说，计算所有成对距离的代价也更高。在 [16] 中，我们指出，一个小的重构（将场景点看作在空间中按高斯分布分布，而不是围绕模型点呈高斯分布）可以让我们回到只对场景点求和的形式。它享有 CPD 的许多鲁棒性优势，同时也具有类似 ICP 这类算法的性能。

4.6.5 非线性优化

到目前为止，我们讨论的所有算法都利用了给定对应关系时通过 SVD 求解位姿估计的方法，并在估计对应关系和估计位姿之间交替进行。还有另一类重要算法试图同时求解二者。这会使优化问题变成非凸问题，这意味着它们仍然会像我们在迭代算法中看到的那样受到局部最小值的影响。但许多作者认为，使用非线性求解器的求解时间可以与迭代算法相当（例如 [17]）。

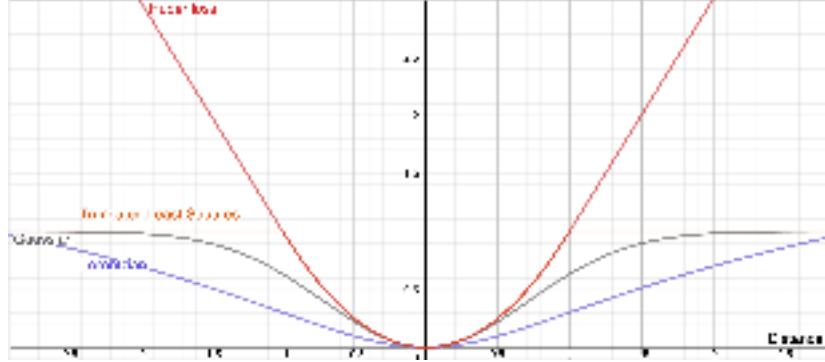


Figure 4.14 - A variety of objective functions for robust registration. [Click here for the interactive version.](#)

使用非线性优化的另一个主要优势是，这些求解器可以容纳各种各样的目标函数，从而实现类似我们在 CPD 中看到的那种对离群点的鲁棒性。我在上面绘制了几种常用的目标函数。这些函数的主要特征是它们会逐渐变平，因此更大的距离最终不会导致更高的代价。重要的是，和 CPD 中一样，这意味着我们可以在目标函数中考虑所有成对距离，因为如果离群点只是向代价中添加一个常数值（例如 1），那么它们相对于决策变量就没有梯度，也不会影响最优解。因此，对于我们喜欢的损失函数 $l(x)$ ，可以写出如下形式的目标函数，例如：

$$\min \sum_{i,j} [l(\|X^O p^{m_j} - p^{s_i}\|)].$$

那么决策变量是什么？我们还可以利用求解器的额外灵活性，使用最小参数化——例如在二维旋转中使用 θ ，或在三维旋转中使用欧拉角。目标函数会更复杂，但我们可以去掉约束。

Example 4.7 (具有已知对应关系的非线性位姿估计)

为了准确理解我们放弃了什么，让我们考虑一个热身问题：在位姿估计问题中，对最小参数化使用非线性优化。只要我们不添加任何约束，仍然可以将旋转的解与平移的解分离。因此考虑如下问题：

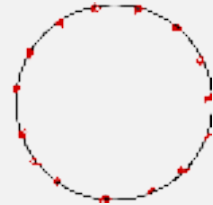
$$\min_{\theta} \sum_j \begin{bmatrix} \cos \theta & -\sin \theta \\ \sin \theta & \cos \theta \end{bmatrix} ({}^O p^{m_{c_i}} - {}^O p^{\bar{m}}) - (p^{s_i} - p^{\bar{s}})^2.$$

现在我们有了一个复杂的非线性目标函数。我们是否因此在问题中引入了局部最小值？

作为一个思想实验，考虑这样一个问题：我们所有的模型点都位于一个完美圆周上。对于任意一个场景点 i ，在仅旋转优化中，最坏情况是我们的估计 θ 与最优解相差 180 度（ π 弧度）。此时该点的代价为 $4r^2$ ，其中 r 是圆的半径。事实上，利用余弦定理，我们可以把任意误差 $\theta_{err} = \theta - \theta^*$ ，下该点的平方距离写为

$$\text{distance}^2 = r^2 + r^2 - 2r^2 \cos \theta_{err}.$$

而在圆的情况下，每个其他模型点都会贡献相同的代价。这不是一个凸函数，但每一个最小值



都是全局最优解（只是以 2π 为周期重复）。

事实上，即使我们拥有更复杂、非圆形的几何结构，这个论证仍然成立。每个点在绕圆旋转时都会产生误差，但可能具有不同的半径。对于所有模型点而言，当 $\cos \theta_{err}$ 趋近于 1 时，其误差都会减小。因此，每一个最小值都是全局最优解。我们实际上（还）没有引入局部最小值！

真正引入局部最小值的是对应关系，以及我们可能向问题中添加的其他约束。

预计算距离函数

这里有一个特别巧妙的技巧，可以加速这些非线性优化中的计算，但它要求我们重新回到“场景点到模型的最小距离”这一思路，而不是考虑点与点之间的成对距离。我认为这并不是一个很大的牺牲，因为我们现在已经不再显式枚举对应关系了。让我们稍微修改上面的目标函数，使其形式为

$$\min_i \sum \left[l \left(\min_j \|X^O p^{m_j} - p^{s_i}\| \right) \right].$$

换句话说，我们可以应用任意鲁棒损失函数，但只将其应用于场景点与模型之间的最小距离。

从计算角度来看，这种嵌套的 $\min$ 函数看起来有些吓人。但这种形式，再结合我们应用中模型点是固定这一事实，实际上使我们能够进行一些预计算，从而让在线步骤运行得很快。首先，让我们通过将内部项改写为

$$\min_j \|p^{m_j} - X^W p^{s_i}\|.$$

来把优化移到模型坐标系中。现在注意到，对于任意三维点 x ，我们都可以预先计算从 x 到模型上任意点的最小距离，记为 $\phi(x)$ 。那么上面的项其实就是 $\phi(X^W p^{s_i})$ 。这种定义在三维空间上的函数，有时被称为距离场 (distance field)，以及与其密切相关的有符号距离函数 (SDF) 和水平集 (level sets)，都是几何建模中的常见表示方式。而它们恰恰是我们快速计算大量场景点代价所需要的工具。

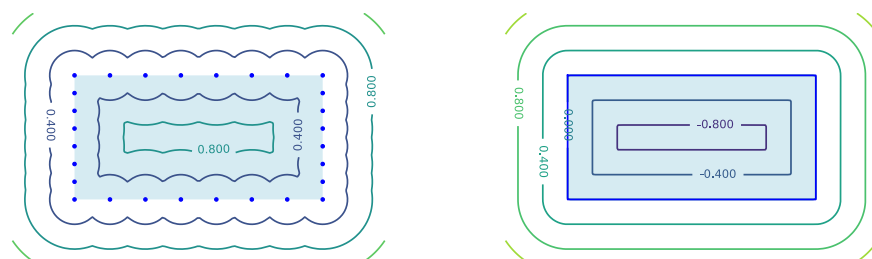


Figure 4.15 - 矩形点表示的距离函数等高线图（左），以及基于“网格”的二维矩形的有符号距离函数（右）。在基于网格的表示中，更平滑的距离等高线有助于缓解我们上面观察到的一些局部最小值问题。

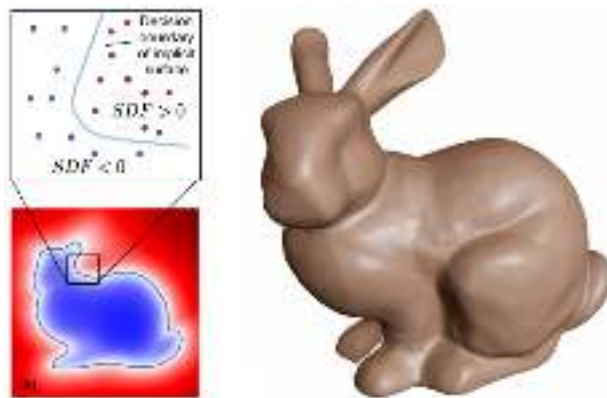


Figure 4.16 - 三维几何的“有符号距离函数”（SDF）表示的可视化。经授权转载自 [3]。

正如我上面给出的图所暗示的那样，我们可以使用预计算的距离函数来表示到网格模型的最小距离，而不仅仅是点云。并且经过适当处理，我们还可以在三维空间中定义替代性的“类距离”函数，它们具有平滑的次梯度，从而能更好地配合非线性优化。关于这一主题已有丰富的研究文献；参见 [18]，这是一个很好的参考资料。

4.6.6 全局优化

是否有希望利用我们在“已知对应关系的位姿估计”中发现的优美结构，同时使用一种能够联合搜索对应关系与位姿的算法呢？

有一些算法声称能够对 ICP 目标达到全局最优性，通常使用分支定界（branch and bound）策略 [19, 20, 21]，但我们对它们在真实点云上的表现感到失望。我们还提出了另一种方法，它使用分支定界来显式枚举对应关系，并结合截断最小二乘目标以及对 $SE(3)$ 约束的紧松弛 [22]，但该方法仍然仅限于相对较小的问题。

在最近的工作中，TEASER [12] 采用了不同的方法，利用这样一个观察：对于尺度和平移，截断最小二乘可以被高效优化。对于旋转，他们求解了截断最小二乘目标的一个半定规划（SDP）松弛。这个松弛并不能保证是紧的，但（和本章中描述的所有凸松弛一样），事后很容易验证最优解是否满足原始约束。而且，对二次旋转矩阵约束的 SDP 松弛 可能会出人意料地有效 [23]！

4.7 非穿透与“自由空间”约束

我们已经探讨了两大类位姿估计算法——一类基于优美的 SVD 解法，另一类基于非线性优化。正如我们将看到的那样，非穿透和自由空间约束在大多数情况下属于非凸约束，因此更适合采用非线性优化方法。但也存在一些凸的非穿透约束例子（例如点不能穿透半平面），并且 确实 可以将这些约束纳入我们的凸优化方法中。下面我将通过一个简单示例来展示这两种版本。

Example 4.8 (通过非线性优化实现非穿透约束)

为了简单起见，我将这个示例限制在二维空间中，在这里我们可以对旋转矩阵进行参数化：

$$R(\theta) = \begin{bmatrix} \cos(\theta) & -\sin(\theta) \\ \sin(\theta) & \cos(\theta) \end{bmatrix},$$

并求解一个已知对应关系的问题。不过请记住，我们同样也可以在非线性优化框架中加入对应关系搜索。

让我们求解如下优化问题：

$$\begin{aligned} \min_{p, \theta} \quad & \sum_i \|p + R(\theta)^O p^{m_{c_i}} - p^{s_i}\|^2, \\ \text{subject to} \quad & w p^{m_i} \geq 0, \quad \forall i \in [1, N_m]. \end{aligned}$$

为了添加非线性代价和约束，我们将一个 Python 函数以及需要与该函数绑定的决策变量传递给 `MathematicalProgram` 的 `AddCost` 和 `AddConstraint` 方法。我已经在这个 notebook 中提供了一个实现：

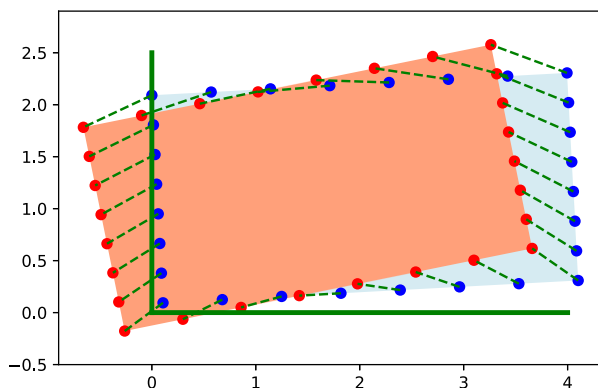


Figure 4.17 - 具有已知对应关系的受约束位姿估计问题的解。我添加了约束，要求估计位姿中所有点在世界坐标系中的 x 和 y 位置都必须大于 0（如绿色线所示）。红色场景点处于穿透状态，但求解器返回了蓝色所示的位姿，它满足这些约束。

Example 4.9 (通过凸优化实现非穿透约束)

对于这个二维示例中的旋转矩阵凸参数化，我们可以利用旋转矩阵的这种便利形式：

$$R = \begin{bmatrix} a & -b \\ b & a \end{bmatrix}.$$

请回忆一下，这种形式下的旋转矩阵约束可简化为 $a^2 + b^2 = 1$ 。这些是非凸约束，但我们可以将其松弛为 $a^2 + b^2 \leq 1$ 。这几乎与我们在[之前的示例](#)中可视化的问题完全相同，只不过这里我们还会加入平移。这个松弛本质上就是把非凸的单位圆约束变成凸的单位圆盘约束。根据之前的可视化，我们有理由对这种单位圆盘松弛可能是紧的保持乐观。

让我们求解如下优化问题：

$$\begin{aligned} \min_{p, a, b} \quad & \sum_i \|p + R^O p^{m_{c_i}} - p^{s_i}\|^2, \\ \text{subject to} \quad & a^2 + b^2 \leq 1, \\ & w p^{m_i} \geq 0, \quad \forall i \in [1, N_m], \end{aligned}$$

其中 R 如上所述依赖于 a, b 。正如上一章中添加约束迫使我们从伪逆转向基于数值优化的解一样，这个问题的解也不再简单地由 SVD 给出。按照这里的形式，该优化属于二阶锥规划（Second-Order Cone Program, SOCP）类别，不过我们需要使用一个松弛变量，将其转换为具有线性目标的标准形式。

请小心。现在我们有了一个依赖于 p 的约束，原先独立求解 R 的方法就不再有效了（至少在不作修改的情况下无效）。我在 notebook 中提供了一个简单实现。

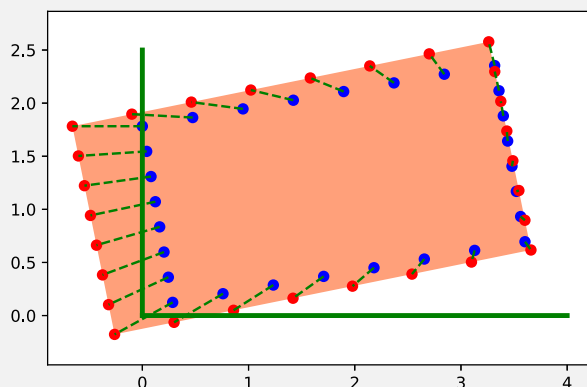


Figure 4.18 - 具有已知对应关系的受约束位姿估计问题的凸近似。我添加了约束，要求估计位姿中所有点在世界坐标系中的 x 和 y 位置都必须大于 0（如绿色线所示）。

这是一个稍微夸张的情形，其中场景点确实在将盒子拉入约束区域。我们可以看到，在这种情况下该松弛 **不是** 紧的；盒子被略微缩小以解释数据。

像这样的受约束优化可以做得相对高效，并且适合在中等规模点云的 ICP 循环中使用。但主要限制在于，我们只能将这种方法用于凸的非穿透约束（或约束的凸近似），例如这里使用的“半平面”约束。对于场景中两个物体之间的非穿透问题，它可能并不太适合。

顺便说一句，我能够轻松地在这里添加这个示例，其实是相当特别的。我不知道还有哪个工具箱能像 Drake 这样，把高级优化算法与几何 / 物理 / 动力系统结合在一起。

4.7.1 作为非穿透约束的自由空间约束

还有另一个优美的思想，我最早是在 [24] 中看到的……

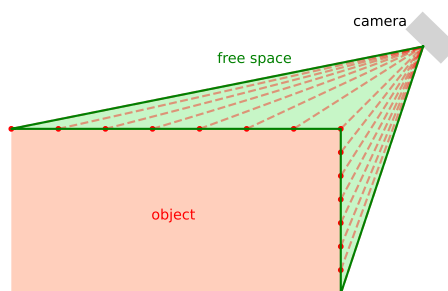


Figure 4.19 - 从相机投射到深度返回点的射线定义了一个“自由空间障碍物”，其他物体不应穿透它。

最终，尽管我非常偏爱凸形式，因为它们有可能让我们获得更深入的理解，并让算法获得更强的保证，但添加非穿透约束和自由空间约束的能力实在太有价值，不能忽视。如今，深度学习方法往往能够提供不错的初始猜测，这可以缓解一些关于局部最小值的担忧。我希望，如果你在一两年后回来看这些笔记，我能够报告我们已经在这些更复杂的形式上取得了强有力的结果。但就目前而言，在大多数应用中，我会引导你采用非线性优化方法，并充分利用这些约束。

4.8 跟踪

4.9 整合所有内容

现在我们可以把所有部分整合起来。在 `notebook` 中，我创建了一个芥末瓶位于一个箱子中的示例。首先，我使用 ICP 来定位它的位姿。然后，像上一章中那样，我规划一条简单轨迹，将它拿起并放入第二个箱子中。最后，我使用微分逆运动学来执行这条轨迹。非常令人满足！

4.10 展望未来

尽管我非常欣赏能够严谨地思考这些几何方法的能力，但我们也需要批判性地思考：我们求解的目标是否真的是正确的。

即使我们停留在纯几何领域，也并不清楚对所有点赋予相同权重的最小二乘目标就是正确的。想象一本又高又薄的书平放在桌子上——我们可能只能从书的边缘获得非常少量的返回点，但这些返回点按比例包含的信息量，远远超过我们可能从书封面获得的大量返回点。在我们这里建立的估计目标中加入相对权重并不困难，但我们还没有非常成功的基于几何的算法，能够以一种通用方式决定这些权重应该是什么。（这个方向已经有大量研究，但它是一个困难问题。）

不过请认识到，尽管几何非常优美，但到目前为止，我们几乎完全忽略了我们拥有的最重要信息：颜色值！虽然可以将颜色和其他特征放入 ICP 风格的流程中，但要在颜色空间中写出一个简单的距离度量非常困难（同一个物体的原始颜色值在不同光照条件下可能会非常不同，而不同物体的颜色值又可能看起来非常相似）。计算机视觉的进展，尤其是过去几年中基于深度学习的进展，为这个问题带来了新的活力。最近当我问学生“如果你必须放弃一个通道，深度或颜色，你会放弃哪个？”时，答案非常响亮：“我会放弃深度；不要拿走我的颜色！”这与仅仅几年前相比已经是一个很大的变化。直到 2019 年，在 6D 物体位姿估计基准测试（一项近乎每年举办的竞赛）中，几何位姿估计仍然优于基于深度学习的方法[10]。但到了 2020 年版本的挑战赛，深度学习已经追上来了。

但深度学习和几何可以（也应该？）很好地协同工作。2020 年物体位姿估计挑战赛的获胜者使用深度学习来生成位姿的初始猜测，但仍然使用几何感知（一种 ICP 变体）和深度通道来细化估计[10]。再举一个例子，识别点云之间的对应关系一直是本章的一个主要主题——而我们还没有完全解决它。这正是颜色值和深度学习大有可为的众多地方之一 [25]。我们很快就会将注意力转向它们！

4.11 练习

Exercise 4.1 (你需要多少个点?)

考虑具有已知对应关系的点云配准问题。点可以被直接映射到对应的模型点，从而指定场景中物体的位置和方向。在大多数情况下，我们拥有的点数远多于物体位姿中的决策变量数量。因此，如果将每个点对应关系都视为一个等式约束，就会使问题变得过约束。这自然引出了一个问题：

- 在二维中，能够唯一指定物体位姿所需的最少点数是多少？请给出简要理由。
- 在三维中，能够唯一指定物体位姿所需的最少点数是多少？请给出简要理由。

（提示：如果点对应关系未知，你的答案可能会如何变化？）

Exercise 4.2 (旋转对称性)

对于具有已知对应关系的点云配准问题，我们有一个优美的表述——它具有二次目标函数，并且（在放松行列式约束后）具有二次等式约束。有时我们会遇到具有旋转对称性的物体——想象一个完美立方体形状盒子。对于一个在相差 90 度的旋转处都应当具有同样好解的问题，二次目标函数如何捕捉这种情况？

Exercise 4.3 (具有随机初始化的 ICP)

如果你运行具有随机物体几何形状和真实物体位姿的 ICP 示例代码，你很难不注意到：该算法在估计平移方面做得相当合理，但（至少对于我这里生成的几何形状而言）在旋转方面表现相当糟糕。一种缓解方法是使用不同的旋转初始估计，多次运行 ICP。对于任何存在局部最小值的非凸优化问题来说，这都是一种合理策略，但在这里尤其有用，因为即使点云相当复杂，搜索空间的维度也很低。特别是，我们可以用适中数量的样本，对二维甚至三维旋转进行合理覆盖（对于三维，可考虑使用 `UniformlyRandomRotationMatrix`）。此外，从多个初始条件运行 ICP 可以并行完成。

请尝试从多个初始估计 $\hat{X}^O$ 实现 ICP，并且只在旋转上进行采样。如果你只保留其中最佳的结果（估计误差最低），那么与单次 ICP 运行相比，它能在多大程度上提升性能？

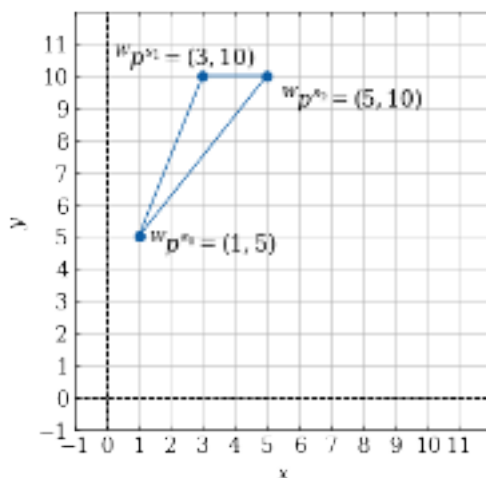
Exercise 4.4 (固定旋转的点配准)

考虑这样一种点配准情况： X^O 的旋转部分已知，但平移部分未知。（换句话说，这是示例 4.2 的相反情况。）

具体来说，假设你的场景点 ${}^W X^{CC} p^{s_i} = {}^W p^{s_i}$ 定义如下：

$$\begin{aligned} {}^W p^{s_0} &= (1, 5) \\ {}^W p^{s_1} &= (3, 10) \\ {}^W p^{s_2} &= (5, 10) \end{aligned}$$

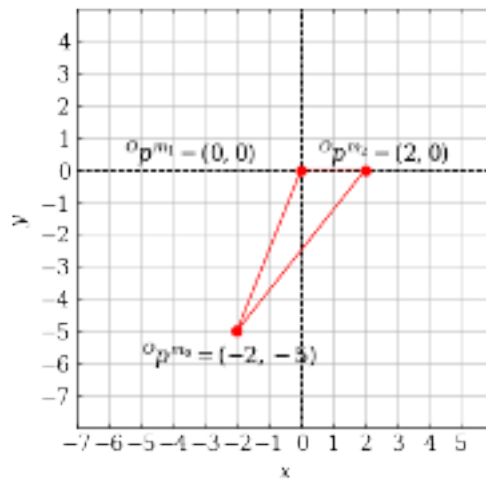
可以绘制如下：



而你的模型点定义如下：

$$\begin{aligned}o_{p^{m_0}} &= (-2, -5) \\o_{p^{m_1}} &= (0, 0) \\o_{p^{m_2}} &= (2, 0)\end{aligned}$$

可以绘制如下：



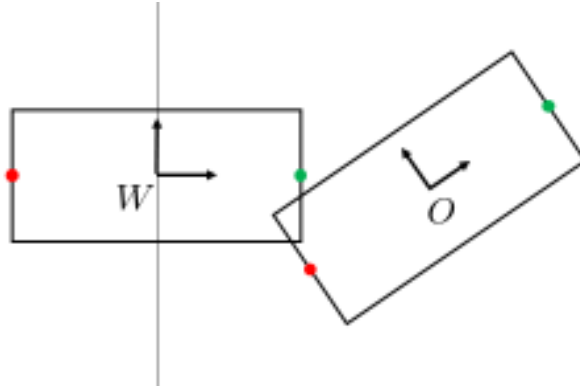
正如你所见，两个三角形具有相同的朝向，因此 $R^O = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}$ 。不过，我们仍然需要求解 X^O 的平移部分。

（在这个例子中，我们可以很容易地分离平移和旋转，因为模型的朝向已经与场景匹配，但这种解耦实际上在一般情况下同样成立。参见教材中示例 4.2 之后的解释。）

- 这个优化问题中的决策变量是什么？请具体说明。
- 当 $p^O = (0, 0)$ 时，目标函数的值是多少？当它为 $(3, 10)$ 时呢？当它为 $(6, 12)$ 时呢？
- 如果你将目标函数绘制为决策变量的函数，它会是什么形状？请解释这一点的重要意义。
- 考虑旋转固定且场景点和模型点数量相等的一般情况。我们只需要寻找平移分量。求解这个优化问题时，我们会为每个决策变量找到什么量？你的解应该在模型点和场景点无法完美对齐时（例如存在噪声）仍然正确。（提示：我们希望最小化所有变换后的场景点与其对应模型点之间的平方距离。）

Exercise 4.5 (平面两点 ICP)

我们已经看到 ICP 有许多局部最小值。但这些局部最小值是什么？在到达这些局部最小值之前，我们的初始猜测需要错到什么程度？虽然对于一般几何形状而言，分析可能会变得复杂，但让我们先从分析一个简单的平面两点 ICP 示例开始。



模型点和场景点在其对应坐标系中给定为：

$$o_{p^{m_1}} = \begin{pmatrix} 1 \\ 0 \end{pmatrix} \quad o_{p^{m_2}} = \begin{pmatrix} -1 \\ 0 \end{pmatrix} \quad w_{p^{s_1}} = \begin{pmatrix} 1 \\ 0 \end{pmatrix} \quad w_{p^{s_2}} = \begin{pmatrix} -1 \\ 0 \end{pmatrix}.$$

真实对应关系由它们的编号给出，但请注意，我们并不知道真实对应关系——ICP 只是根据最近邻来确定它。给定我们的普通 ICP 代价（成对距离平方和），我们可以用 p_x, p_y, a, b 对二维位姿 X^O 进行参数化，并写出所得优化问题如下：

$$\begin{aligned} \min_{p_x, p_y, a, b} \quad & \sum_i \left(\begin{pmatrix} p_x \\ p_y \end{pmatrix} + \begin{pmatrix} a & -b \\ b & a \end{pmatrix} o_{p^{m_{c_i}}} - w_{p^{s_i}} \right)^2 \\ \text{s.t.} \quad & a^2 + b^2 = 1. \end{aligned}$$

- 当位姿的初始猜测基于最近邻产生了正确对应关系时，证明 ICP 代价在 $p_x, p_y, b = 0$ 且 $a = 1$ 时达到最小。描述会收敛到真实解的初始位姿集合。
- 当初始猜测产生了翻转的对应关系时，证明 ICP 代价在 $p_x, p_y, b = 0$ 且 $a = -1$ 时达到最小。描述会收敛到这个错误解的初始位姿集合。
- 请记住，对应关系不需要是一一对应的。事实上，当基于最近邻计算时，它们常常不是一一对应的。通过构造数据矩阵 W ，证明当两个场景点都对应到同一个模型点时，ICP 会卡住并且无法达到零代价。ICP 的下次迭代会发生什么？（提示：你可以假设对零矩阵进行 SVD 会得到单位矩阵形式的 U 和 V 。）

Exercise 4.6 (Bunny ICP)

在本练习中，你将实现 ICP 算法，用于匹配两个 Stanford bunny 的点云。你将完全在 中完成。你需要完成以下步骤：

- 实现一种方法，用于在给定对应关系的情况下计算最小二乘变换。
- 使用第 a 部分中的最小二乘变换方法实现 ICP 算法。

Exercise 4.7 (RANSAC)

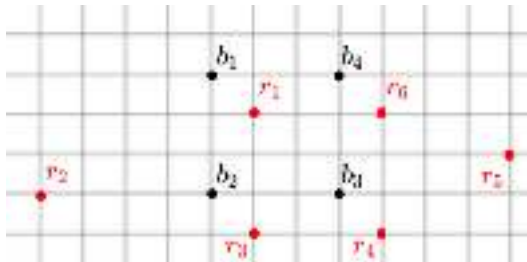
在本练习中，你将使用 RANSAC 算法从 Stanford bunny 点云中移除环境背景。你将完全在 中完成。你需要完成以下步骤：

- 实现 RANSAC 算法。
- 使用 RANSAC 算法从场景点云中移除平面表面。

Exercise 4.8 (ICP 中的离群点)

在这个问题中，你将探索一种处理 ICP 中离群点的技术，并思考距离度量如何影响 ICP 对离群点的鲁棒性。这个问题有两个部分，每个部分都有三个小问。

- a. 考虑下面可视化的设置。这里模型点用黑色给出，并标记为 b_i ；场景点用红色给出，并标记为 r_j 。



让我们考虑一种处理场景中离群点的可能算法。

1. 首先，写出每一个对应关系，即 (b_i, r_j) 对。接下来，计算 ICP 误差，即这些对应关系之间成对平方距离的总和。
2. 假设你被告知场景点中有 $\frac{1}{3}$ 是离群点。使用每个 b_i 和 r_j 之间的成对距离，你可以检测出哪两个场景点 r_j 最可能是离群点？
3. 丢弃这两个被提出的离群场景点后，计算新的 ICP 误差。

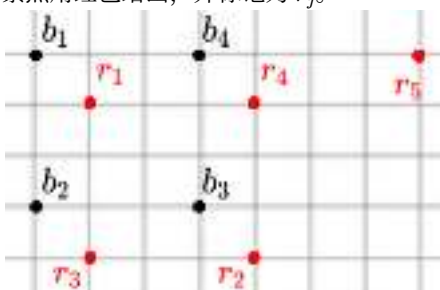
这就是 Trimmed ICP 算法背后的基本直觉！

- b. ICP 中的优化使用成对欧氏平方距离之和来刻画两组点之间的距离。作为一个思想实验：另一种可能的距离度量（虽然不太适合优化）是单向 Hausdorff 距离。简单来说，假设对手在一个形状上选择一个点，单向 Hausdorff 距离就是你被迫前往另一个形状上最近点所需经过的距离。我们可以考虑两种方案：

方案 1) 对手选择场景点，因此你选择距离该场景点最近的模型点。

方案 2) 对手选择模型点，因此你选择距离该模型点最近的场景点。

哪一种对离群点更鲁棒？让我们通过一个二维示例来探索。同样，模型点用黑色给出，并标记为 b_i ；场景点用红色给出，并标记为 r_j 。



1. 让我们考虑方案 1。作为对手，我选择场景点 r_5 。你能选择的最近模型点 b_i 是哪个？到这个模型点的距离是多少？
2. 现在在方案 2 中，我选择模型点 b_1 。你能选择的最近场景点 r_j 是哪个？到这个场景点的距离是多少？
3. 由此来看，哪种方案对离群点更鲁棒？

Exercise 4.9 (位姿估计)

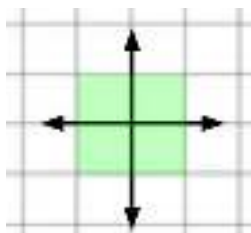
让我们回到第 3 章中的例子，但放宽“我们已知红色泡沫砖位姿”的假设。你将得到一个来自深度相机的仿真原始点云，该点云来自我们之前的设置。你的任务是对这些原始点云数据进行分割，并执行 ICP 来估计砖块的位姿。你将完全在 中完成。你需要完成以下步骤：

- 对原始点云进行分割，以移除背景。
- 使用我们的课堂 ICP 实现，正确估计红色泡沫砖的位姿。

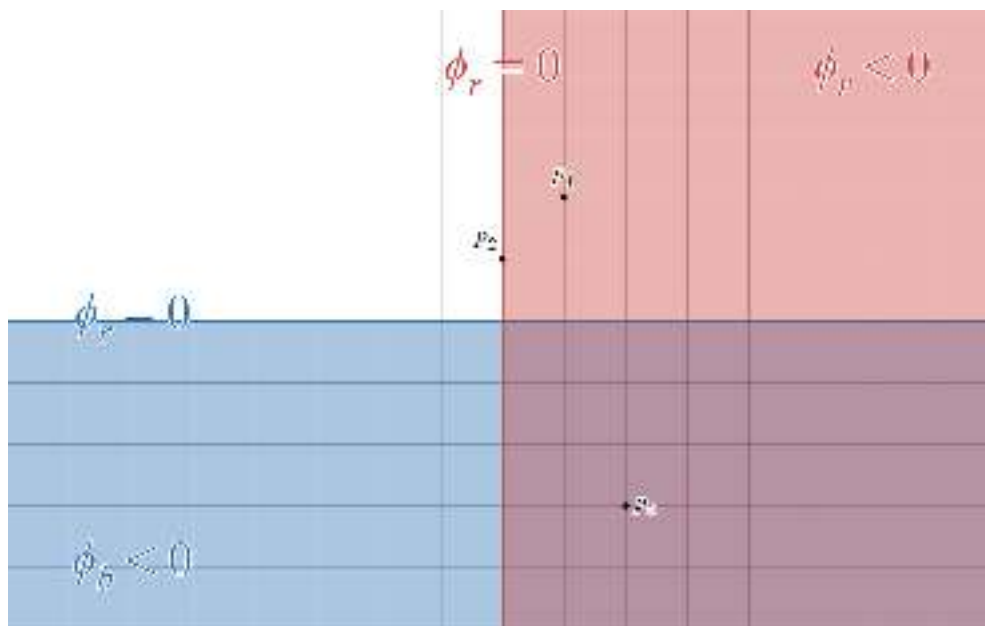
Exercise 4.10 (组合有符号距离函数 (SDF))

在本题中，我们将探索有符号距离函数及其组合！提醒一下，一个集合的有符号距离函数给出点 p 到该集合边界的最小距离，其符号取决于点 p 位于边界内部还是外部。位于边界内部的点为负值。位于边界外部的点为正值。

对于一个二维示例，考虑下面可视化的绿色盒子。盒子的中心位于原点，其有符号距离值为 -1 ，因为它到盒子任意边的距离为 1 ，并且位于盒子内部。盒子周界上的点的有符号距离值为 0 。盒子外部的任意点的有符号距离值为 $+d$ ，其中 d 是该点与盒子周界之间的最小距离。



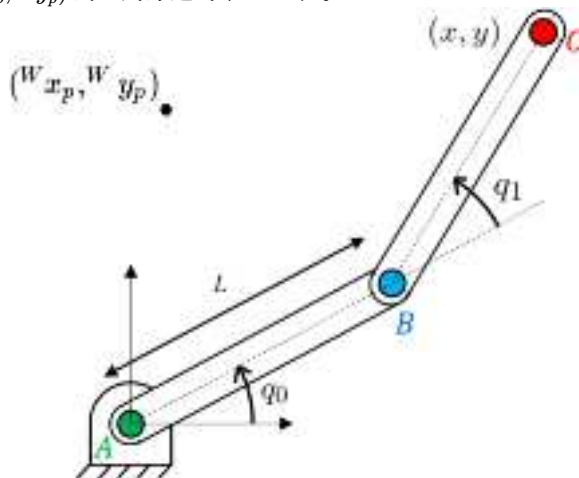
在下图中，我们绘制了两个物体：一个带阴影的红色物体和一个带阴影的蓝色物体（紫色区域是两个物体重叠的区域，并且这些物体应被视为无限半平面）。令 $\phi_r(p)$ 表示点 p 到红色物体边界的有符号距离值，令 $\phi_b(p)$ 表示点 p 到蓝色物体边界的有符号距离值。例如， $\phi_b(p_0) = -3, \phi_b(p_1) = +2$ 。



- 首先，让我们考虑一个新区域，它被定义为红色物体和蓝色物体的并集。令 $\phi_{r \cup b}(p)$ 表示点 p 到这个新边界的有符号距离值。计算： $\phi_{r \cup b}(p_0), \phi_{r \cup b}(p_1), \phi_{r \cup b}(p_2)$ 。
- 接下来，让我们考虑一个新区域，它被定义为红色物体和蓝色物体的交集（也就是紫色物体）。令 $\phi_{r \cap b}(p)$ 表示点 p 到这个新边界的有符号距离值。计算： $\phi_{r \cap b}(p_0), \phi_{r \cap b}(p_1), \phi_{r \cap b}(p_2)$ 。
- 利用你在第 (a) 部分中的经验，写出一个表达式，用于计算我们二维空间中任意点 p 到红色和蓝色物体并集的有符号距离值。你的表达式应以 $\phi_r(p)$ 和 $\phi_b(p)$ 表示。我们鼓励你根据该点所在区域分情况思考。提示：写一个按 4 个区域分类的分段表达式，然后尝试将其化简为只含 2 种情况的简洁数学表达式。
- 利用你在第 (b) 部分中的经验，写出一个类似表达式，用于计算任意点 p 到红色和蓝色物体交集（即紫色区域）的有符号距离值。你的表达式应以 $\phi_r(p)$ 和 $\phi_b(p)$ 表示，并且可以只考虑 2 种情况来写出。

一种用于计算两个物体并集 SDF 值的错误方法，是简单地取每个物体 SDF 值中的最小值。（值得花一分钟说服自己为什么这种方法是错误的！）虽然这种方法可能产生错误的距离，但它会产生具有正确符号的值（同样，请说服自己这是真的！）。因此，这种“错误”的方法可以告诉我们，对于物体集合的并集而言，我们是否处于穿透状态或非穿透状态。这使它对于计算非穿透约束很有用！我们将这种由“错误但有用”的方法生成的 SDF 称为穿透约束 SDF（Penetration-Constraint-SDF）。让我们看看如何使用它！

- 如课堂上所讨论的，对于刚体物体，我们可以预先计算该物体的 SDF，这使我们能够极大加快计算速度。然而，如果我们处理的是一个关节式物体，例如机器人手臂，其手臂构型取决于关节值，那么我们就不能再预计算 SDF。每当机器人构型发生变化时，重新计算这个“全局” SDF 会非常昂贵。（这里的全局是指整个关节式物体的 SDF。）相反，DART（Dense Articulated Real-Time Tracking，稠密关节式实时跟踪）[24] 算法认为，对于关节式物体中的每个连杆，我们都有一个“局部” SDF。局部 SDF 给出的是连杆坐标系中某个点的有符号距离值。因为我们是在连杆坐标系中操作，所以可以预先计算这些局部 SDF。考虑我们最喜欢的平面二连杆机械臂，如下图所示。给定 q_0, q_1 以及每个连杆（连杆 AB、连杆 BC）的预计算局部 SDF，请简要描述如何计算点 $(^W x_p, ^W y_p)$ 的全局穿透约束 SDF 值。



Exercise 4.11 (使用几何感知在真实环境中操作资产)

在本练习中，我们将基于上一章中的 [练习 3.14](#) 继续展开。这一次，你将通过使用深度相机和几何感知，编程让机器人拾取并放置来自你姓名首字母的字母资产，且不假设你知道这些资产

的初始位姿。你将完全在 中完成。 你需要完成以下步骤：

- a. 创建一个包含机器人、桌子、你的姓名首字母以及三个 RGB-D 传感器的场景。
- b. 使用 RGB-D 传感器获取姓名首字母的点云。
- c. 使用 ICP 估计姓名首字母在世界坐标系中的位姿。
- d. 使用估计出的位姿，结合你在练习 3.14 中的微分 IK 控制器来拾取并放置姓名首字母。

REFERENCES

1. Krishna Shankar and Mark Tjersland and Jeremy Ma and Kevin Stone and Max Bajracharya, "A learned stereo depth system for robotic manipulation in homes", *IEEE Robotics and Automation Letters*, vol. 7, no. 2, pp. 2305--2312, 2022.
2. Chris Sweeney and Greg Izatt and Russ Tedrake, "A Supervised Approach to Predicting Noise in Depth Images", *Proceedings of the IEEE International Conference on Robotics and Automation*, May, 2019. [[link](#)]
3. Jeong Joon Park and Peter Florence and Julian Straub and Richard Newcombe and Steven Lovegrove, "DeepSDF: Learning Continuous Signed Distance Functions for Shape Representation", *Proceedings of the IEEE International Conference on Computer Vision and Pattern Recognition (CVPR)*, June (To Appear), 2019. [[link](#)]
4. Ben Mildenhall and Pratul P Srinivasan and Matthew Tancik and Jonathan T Barron and Ravi Ramamoorthi and Ren Ng, "Nerf: Representing scenes as neural radiance fields for view synthesis", *Communications of the ACM*, vol. 65, no. 1, pp. 99--106, 2021.
5. Berthold K.P. Horn, "Closed-form solution of absolute orientation using unit quaternions", *Journal of the Optical Society of America A*, vol. 4, no. 4, pp. 629-642, April, 1987.
6. Andriy Myronenko and Xubo Song, "On the closed-form solution of the rotation matrix arising in computer vision problems", *arXiv preprint arXiv:0904.1613*, 2009.
7. Robert M Haralick and Hyonam Joo and Chung-Nan Lee and Xinhua Zhuang and Vinay G Vaidya and Man Bae Kim, "Pose estimation from corresponding point data", *IEEE Transactions on Systems, Man, and Cybernetics*, vol. 19, no. 6, pp. 1426--1446, 1989.
8. Marius Muja and David G Lowe, "Flann, fast library for approximate nearest neighbors", *International Conference on Computer Vision Theory and Applications (VISAPP '09)*, vol. 3, 2009.
9. Bertram Drost and Markus Ulrich and Nassir Navab and Slobodan Ilic, "Model globally, match locally: Efficient and robust 3D object recognition", *2010 IEEE computer society conference on computer vision and pattern recognition*, pp. 998--1005, 2010.
10. Tomás Hodan and Martin Sundermeyer and Bertram Drost and Yann Labbé and Eric Brachmann and Frank Michel and Carsten Rother and Jiri Matas, "BOP Challenge 2020 on 6D Object Localization", *European Conference on Computer Vision Workshops (ECCVW)*, 2020.
11. M.A. Fischler and R.C. Bolles, "Random Sample Consensus: A Paradigm for Model Fitting with Applications to Image Analysis and Automated Cartography", *Communications of the Association for Computing Machinery (ACM)*, vol. 24, no. 6, pp. 381--395, 1981.
12. Heng Yang and Jingnan Shi and Luca Carlone, "Teaser: Fast and certifiable point cloud registration", *arXiv preprint arXiv:2001.07715*, 2020.
13. Pat Marion and Peter R. Florence and Lucas Manuelli and Russ Tedrake, "A Pipeline for Generating Ground Truth Labels for Real RGBD Data of Cluttered Scenes", *International Conference on Robotics and Automation (ICRA), Brisbane, Australia*, May, 2018. [[link](#)]

14. Anh Nguyen and Bac Le, "3D point cloud segmentation: A survey", *2013 6th IEEE conference on robotics, automation and mechatronics (RAM)* , pp. 225--230, 2013.
15. Andriy Myronenko and Xubo Song, "Point set registration: Coherent point drift", *IEEE transactions on pattern analysis and machine intelligence*, vol. 32, no. 12, pp. 2262--2275, 2010.
16. Wei Gao and Russ Tedrake, "FilterReg: Robust and Efficient Probabilistic Point-Set Registration using Gaussian Filter and Twist Parameterization", *Conference on Computer Vision and Pattern Recognition (CVPR)*, June, 2019. [[link](#)]
17. Andrew W Fitzgibbon, "Robust registration of 2D and 3D point sets", *Image and vision computing*, vol. 21, no. 13-14, pp. 1145--1153, 2003.
18. Stanley Osher and Ronald Fedkiw, "Level Set Methods and Dynamic Implicit Surfaces", Springer , 2003.
19. Natasha Gelfand and Niloy J Mitra and Leonidas J Guibas and Helmut Pottmann, "Robust global registration", *Symposium on geometry processing* , vol. 2, no. 3, pp. 5, 2005.
20. Nicolas Mellado and Dror Aiger and Niloy J Mitra, "Super 4pcs fast global pointcloud registration via smart indexing", *Computer Graphics Forum* , vol. 33, no. 5, pp. 205--215, 2014.
21. Jiaolong Yang and Hongdong Li and Dylan Campbell and Yunde Jia, "Go-ICP: A globally optimal solution to 3D ICP point-set registration", *IEEE transactions on pattern analysis and machine intelligence*, vol. 38, no. 11, pp. 2241--2254, 2015.
22. Gregory Izatt and Hongkai Dai and Russ Tedrake, "Globally Optimal Object Pose Estimation in Point Clouds with Mixed-Integer Programming", *International Symposium on Robotics Research*, Dec, 2017. [[link](#)]
23. James Saunderson and Pablo A Parrilo and Alan S Willsky, "Semidefinite descriptions of the convex hull of rotation matrices", *SIAM Journal on Optimization*, vol. 25, no. 3, pp. 1314--1343, 2015.
24. Tanner Schmidt and Richard Newcombe and Dieter Fox, "DART: dense articulated real-time tracking with consumer depth cameras", *Autonomous Robots*, vol. 39, no. 3, pp. 239--258, 2015.
25. Peter R. Florence* and Lucas Manuelli* and Russ Tedrake, "Dense Object Nets: Learning Dense Visual Object Descriptors By and For Robotic Manipulation", *Conference on Robot Learning (CoRL)* , October, 2018. [[link](#)]

CHAPTER 5

料箱抓取 (Bin Picking)

在本章中，我们将考虑料箱抓取问题的最简单版本：机器人面前有一个装满随机物体的料箱，它只需要将这些物体从一个料箱移动到另一个料箱。我们不会特别关心这些物体具体是什么，也不会关心它们最终在另一个料箱中的具体位置，但我们希望我们的解决方案能够对非常广泛种类的物体都达到合理的性能水平。事实证明，这是一种非常方便的方式来构建机器人学习的训练环境——我们可以让机器人日夜不停地在两个料箱之间来回搬运物体，并且间歇性地向料箱中添加和移除物体，以提高多样性。当然，在仿真中实现这一点会更加容易！

料箱抓取在物流等行业中具有非常重要的潜在应用，而且这个问题还有许多更加精细化的版本。例如，我们可能只需要抓取某一特定类别的物体，和/或需要将物体放置到已知位置（例如用于“装箱”）。但让我们先从最基本的情况开始。

5.1 生成随机杂乱场景

如果我们的目标是测试多种多样的料箱抓取场景，那么第一项任务就是弄清楚如何生成多样化的仿真环境。我们应该如何在料箱中填充物体？到目前为止，我们一直通过仔细设置每个物体的初始位置（在 `Context` 中）来搭建每个仿真场景，但这种方法无法扩展到大规模情况。

5.1.1 下落的物体

在现实世界中，我们大概只会把随机物体直接倒进料箱里。对于仿真来说，这同样是一个不错的策略。只要我们的初始条件（以及时间步长）设置合理，我们大致可以期望仿真能够忠实地实现多刚体物理；但如果我们在初始化 `Context` 时让多个物体占据同一物理空间，那么物理行为将不再有明确意义。避免这种情况最简单、最常见的方法，就是生成随机数量的物体，并赋予它们随机位姿，同时让它们在竖直方向上的位置错开，从而保证它们在初始状态下不会发生穿透。

如果你留意的话，你可以在大多数先进多刚体模拟器的演示视频中看到大量物体下落的动画。

（这些演示有时可能会有一点误导性；因为制作一个“许多物体下落且从远处看起来不错”的仿真其实相对容易，但如果你放大并仔细观察，往往会发现许多异常现象。）对于我们的目的来说，下落动力学本身并不是重点。我们真正关心的，只是物体在完成下落之后的状态，因为这些状态将作为我们操作系统的初始条件。

Example 5.1 (二维中的泡沫砖堆)

这里是二维情况。我已经向系统中添加了许多我们最喜欢的红色泡沫砖实例。请注意，确实可以编写高度优化的模拟器，专门利用物理被限制在二维空间这一特点；但这里我并没有这么做。相反，我为每块砖添加了一个连接到世界坐标系的平面关节（planar joint），并运行我们完整的三维模拟器。这个平面关节具有三个自由度。我在这里将它们设置为 x 、 z 和 θ ，从而将物体约束在 xz 平面内。

我在 `Context` 中为每个物体设置了初始位置：其水平位置服从均匀分布、姿态具有均匀随机旋转，并且它们的初始竖直位置每隔 0.1m 错开排列。我们只需要模拟略多于一秒钟，砖块就会静止下来，并为我们提供预期中的“初始条件”。

[点击这里查看动画。](#)

对于任意随机物体来说，这样做其实并没有什么不同——下面展示的是我运行同样代码时的效果，只不过把砖块替换成了从 YCB 数据集 [1] 中随机抽取的一些物体。不知为何，即使这些物体本身有点滑稽，我们依然能够看到 [中心极限定理](#) 在努力发挥作用，这件事让我觉得很有趣。



Example 5.2 (用杂乱物体填充料箱)

同样的技术也适用于三维情况。在三维空间中设置均匀随机的姿态需要稍微多考虑一些，但 Drake 提供了 `UniformlyRandomRotationMatrix` 方法（同时也提供了用于四元数和滚转-俯仰-偏航角的方法）来替我们完成这项工作。



请注意，料箱是生成随机场景时一种特别简单的情况。例如，如果你想生成随机厨房环境，那么如果解决方案只是从均匀随机的独立同分布位姿中把冰箱、烤面包机和凳子扔下来，你大概就不会那么满意了。在这些情况下，设计合理的分布会变得有趣得多 [2]；我们将在后面的讲义中重新讨论生成式场景模型这一主题。

5.2 带摩擦接触的静态平衡

尽管上面的例子在概念上非常简单，但为了让这一切正常工作，物理引擎内部实际上发生了很多事情。为了理解其中的一些细节，让我们探索最简单的情况——所有物体都已经静止之后的（静态）力学。

我不会深入完整讨论多刚体动力学或多刚体仿真，不过我确实有更多相关笔记 [可在这里查看](#)。需要理解的重要一点是，当我们用广义位置 q 和广义速度 v 来写出熟悉的 $f = ma$ 时，它会呈现出一种特定形式：

$$M(q)\dot{v} + C(q, v)v = \tau_g(q) + \sum_i J_i^T(q)F^{c_i}.$$

我们已经理解了广义位置 q 和广义速度 v 。方程左侧只是“质量乘以加速度”的推广，其中包含质量矩阵 M 和科里奥利项 C 。右侧是（广义）力的总和，其中 $\tau_g(q)$ 表示由重力产生的项，而 F^{c_i} 是由第 i 个接触产生的空间力。 $J_i(q)$ 是第 i 个“接触雅可比矩阵”——它是将广义速度映射到第 i 个接触坐标系空间速度的雅可比矩阵。

我们这里感兴趣的是寻找这些微分方程的（稳定）稳态解，使其能够作为我们仿真的良好初始条件。在稳态时，我们有 $v = \dot{v} = 0$ ，并且很方便的是，方程左侧的所有项都为零。于是剩下的只是力平衡方程：

$$\tau_g(q) = - \sum_i J_i^T(q) F^{c_i}.$$

5.2.1 空间力

在继续之前，让我们更仔细地说明一下力的记号，并定义它的空间代数。我们通常会把力看作一个三元素向量（包含 x 、 y 和 z 方向的分量），并认为它作用在刚体上的某个点。更一般地，我们将使用 monogram 记号，为 [空间力](#) 定义一个六分量向量：

$$F_{\text{name},C}^{B_p} = \begin{bmatrix} \tau_{\text{name},C}^{B_p} \\ f_{\text{name},C}^{B_p} \end{bmatrix} \quad \text{或者，如果你更喜欢} \quad \begin{bmatrix} F_{\text{name}}^{B_p} \end{bmatrix}_C = \begin{bmatrix} \begin{bmatrix} \tau_{\text{name}}^{B_p} \end{bmatrix}_C \\ \begin{bmatrix} f_{\text{name}}^{B_p} \end{bmatrix}_C \end{bmatrix}. \quad (1)$$

$F_{\text{name},C}^{B_p}$ 是施加在点或坐标系原点 B_p 上、并在坐标系 C 中表达的具名空间力。带方括号的形式在 Drake 中更受推荐，但对于我在这些讲义中的口味来说有点过于冗长。名称是可选的；如果没有指定“在哪个坐标系中表达”，则默认为世界坐标系。尤其对于力来说，建议我们在点 B_p 的符号中包含受力的物体 B ，特别是因为我们经常会遇到大小相等、方向相反的力。在代码中，我们写作 `Fname_Bp_C`。

和空间速度一样，空间力也有一个旋转分量和一个平移分量； $\tau_C^{B_p} \in \mathbb{R}^3$ 是 [力矩](#)（作用在物体 B 上、作用点为 p 、并在坐标系 C 中表达），而 $f_C^{B_p} \in \mathbb{R}^3$ 是平移力或笛卡尔力。空间力也通常被称为 [力旋量 \(wrench\)](#)。如果你觉得把力看作具有旋转分量有些奇怪，那么可以这样理解：世界可能只会接触点施加笛卡尔力，但我们经常希望把许多作用在不同点上的笛卡尔力的合效应汇总到一个共同坐标系中。为了做到这一点，我们把每个笛卡尔力在其作用点处的等效效果表示为作用在物体另一个点上的一个力 [和](#) 一个力矩。

空间力可以自然地融入我们的空间代数：

- 当空间力作用在同一个物体上，并且在同一个坐标系中表达时，它们可以相加，例如：

$$F_{\text{total},C}^{B_p} = \sum_i F_{i,C}^{B_p}. \quad (2)$$

- 将空间力从一个作用点 B_p 平移到另一个点 B_q 时，需要使用叉积：

$$f_C^{B_q} = f_C^{B_p}, \quad \tau_C^{B_q} = \tau_C^{B_p} + B_q p_C^{B_p} \times f_C^{B_p}. \quad (3)$$

- 和所有空间向量一样，可以使用旋转在不同的“表达于”坐标系之间进行变换：

$$f_D^{B_p} = {}^D R^C f_C^{B_p}, \quad \tau_D^{B_p} = {}^D R^C \tau_C^{B_p}. \quad (4)$$

5.2.2 碰撞几何

现在我们已经有了记号，接下来需要理解接触力从何而来。

用于机器人的几何引擎，例如 **DRAKE** 中的 **SceneGraph**，会区分几何体在仿真中可以扮演的几种不同角色。在 **机器人描述文件** 中，我们区分 **视觉** 几何和 **碰撞** 几何。特别地，仿真中的每个刚体都可以关联多个 **碰撞** 几何体（它们扮演“邻近性”角色）。碰撞几何体通常比我们用于图示和感知仿真的视觉几何体简单得多——有时它们只是视觉网格的低多边形版本，有时我们实际上会使用更简单的几何体（例如盒子、球体和圆柱体）。这些更简单的几何体能让物理引擎更快、更鲁棒。

由于一些我们将在下面探讨的微妙原因，除了简化几何体之外，我们有时还会对碰撞几何进行过参数化（over-parameterize），以提高数值计算的鲁棒性。例如，在模拟红色砖块时，我们实际上为同一个物体使用了 **九个** 碰撞几何体。

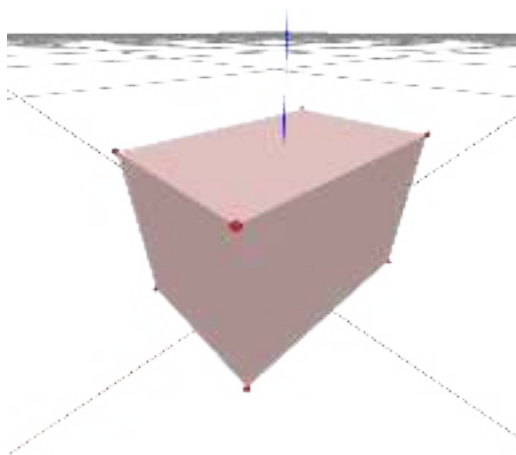


Figure 5.4 - 用于稳健模拟盒体的（经过夸张处理的）接触几何。我们在每个角点添加了接触“点”（具有 epsilon 半径的球体），并为其余接触使用了一个略微内缩的箱体。[这里](#) 提供了交互版本。

Example 5.3 (碰撞几何检查器)

Drake 提供了一个非常实用的 **ModelVisualizer** 工具，它会将 **illustration**（可视化）和 **collision**（碰撞）几何角色发布到 **Meshcat**（例如可以参考 Drake 的 [关于构建多刚体仿真的教程](#)）。这个工具对于设计新模型非常有帮助，同时也有助于理解已有模型的接触几何。

我们之前在可视化各种机器人模型时已经使用过这个工具。现在可以再次尝试；在 **Meshcat** 控制界面中导航到 **Scene > drake > proximity**，并勾选对应选项以启用对 **proximity**（邻近性）角色几何体的可视化。



Figure 5.5 - UR3e 的视觉几何与碰撞几何可视化。

`SceneGraph` 还实现了 [碰撞过滤器 \(collision filter\)](#) 的概念。例如，指定 `iiwa` 的第 5 连杆几何体不能与第 4 或第 6 连杆的几何体发生碰撞，可能会非常重要。指定某些碰撞应被忽略，不仅能够加快计算速度，还能方便我们为连杆使用简化后的碰撞几何。如果我必须确保这些球体和圆柱体在任何可行关节角度下都不会彼此重叠，那么仅用球体和圆柱体来准确近似第 4 和第 5 连杆将会极其困难。默认的碰撞过滤设置对于大多数应用来说已经足够好，但如果你愿意，也可以自行调整。

那么接触力 f^c_i 是从哪里来的呢？对于每一对没有被碰撞过滤器排除的碰撞几何体，都可能存在一对大小相等、方向相反的接触（空间）力。在 `SceneGraph` 中，[GetCollisionCandidates](#) 方法会返回所有这样的碰撞对。我们会把碰撞对中的两个物体分别称为“物体 A”和“物体 B”。

5.2.3 碰撞物体之间的接触力

在我们的数学模型中，两个物体永远不会占据同一片空间区域，但在使用数值积分的仿真中，这种情况几乎无法被完全避免。大多数模拟器会将两个发生碰撞——无论是刚好接触还是已经重叠——的物体之间的接触力，总结为作用在一个或多个接触点上的笛卡尔力。人们很容易低估这一步的复杂性！

首先，让我们认识到代码中需要正确处理的情况数量之庞大；不幸的是，开源工具在这里出现 bug 是极其常见的。但我认为，下面我创建的简单 GUI 已经非常清楚地说明了：试图用一个施加在某个点上的单一笛卡尔力，来总结两个发生深度穿透的物体之间的力，是一件充满风险的事情。当你移动这些物体时，你会发现许多不连续现象；这也是为什么刚体仿真有时会“爆炸”的一个常见原因。看起来很自然的做法是，使用多个接触点而不是单个接触点来总结这些力，某些模拟器也确实这么做，但要编写一个只依赖当前几何配置、并且能够在连续时间步之间一致地选择接触点的算法，是非常困难的。

Drake 的一项创新让它特别擅长仿真复杂接触情形，那就是我们已经不再使用点接触模型。当我们在 Drake 中启用 [水弹性接触 \(hydroelastic contact\)](#) 时，两个相互穿透物体之间的空间力是通过在一个“水弹性表面”上进行积分计算得到的，这个表面对接触斑块 (contact patch) 的概念进行了推广 [3]。在撰写本文时，你仍然必须在 Drake 中主动选择启用 hydroelastic（参见 [这个关于水弹性接触的教程](#)）；我正在努力在这些讲义的所有示例中都使用它。

Example 5.4 (接触力检查器)

我创建了一个简单的 GUI，它允许你选择任意两种基本几何类型，并检查当这些物体发生穿透时所计算出的水弹性接触信息。

5.2.4 接触坐标系

我们仍然需要决定这些空间力的大小和方向，而要做到这一点，我们需要指定空间力所施加的 [接触坐标系](#)。例如，我们 [可以](#) 用 C_B 表示物体 B 上的一个接触坐标系，力施加在该坐标系的原点处。

尽管我强烈建议你在仿真中使用水弹性接触模型，但为了便于讲解，我们这里还是先考虑点接触模型。如果碰撞引擎产生了一系列接触点位置，我们应该如何定义这些坐标系呢？

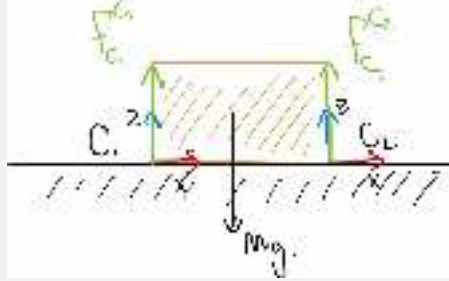
我们的约定是：将正 z 轴与“接触法向”对齐，并令正方向的力用于抵抗穿透。定义这个法向同样需要谨慎。例如，在一个盒子的角点处，法向是什么？将法向定义为两个物体之间有符号距离函数的（次）梯度，可以提供一种可靠的定义，并且这一方法还能推广到广义坐标中。接触坐标系中的 x 和 y 轴则是切向坐标中的任意一组正交基。你可以在[这里](#)找到更多图示与解释。

Example 5.5 (半平面上的砖块)

让我们通过一个简单示例来梳理这些细节——我们的泡沫砖静置在地面上。地面被焊接到世界坐标系，因此没有自由度；我们可以忽略施加在地面上的力，而只关注施加在砖块上的力。

接触力和接触坐标系在哪里？如果我们只使用盒体对盒体的几何模型，那么在这种“面-面对接触”情况下，接触力作用点的选择并不是唯一的；在三维中这种情况会更加明显（在那里三个力就已经足够）。暂时假设我们的接触引擎确定了两个接触点——分别位于盒子的两个角点。

（为了帮助接触引擎在使用点接触模型时做出鲁棒的选择，我们有时实际上会 在盒子的八个角上添加小球碰撞几何体；而在使用水弹性接触时则不需要这样做。）我在下面的示意图中将这此坐标系标记为 C_1 和 C_2 。



为了实现静态平衡，我们要求作用在砖块上的所有力和力矩都保持平衡。我们需要考虑三个力： F_g , F_1 , 和 F_2 ，它们分别是重力、接触 1 和接触 2 的简写名称。重力向量最自然的描述方式是在世界坐标系中，并作用于物体的质心处：

$$F_{g,W}^B = [0, 0, 0, 0, 0, -mg]^T.$$

接触力则最自然地在接触坐标系中描述；例如我们知道

$$f_{1,C_1,z}^{B_{C_1}} \geq 0,$$

因为接触力不可能把地面“拉起来”。为了写出力平衡方程，我们需要使用空间代数，把所有空间力都表示到同一个坐标系中，并且施加在同一个点上。

例如，如果我们要变换接触力 1，可以先变换坐标系：

$$F_{1,B}^{B_{C_1}} = {}^B R^{C_1} F_{1,C_1}^{B_{C_1}}.$$

然后可以改变它的作用点：

$$f_{1,B}^B = f_{1,C_1}^{B_{C_1}}, \quad \tau_{1,B}^B = \tau_{1,C_1}^{B_{C_1}} + {}^B p_B^{C_1} \times f_{1,C_1}^{B_{C_1}}.$$

现在我们可以求解力平衡：

$$F_{1,B}^B + F_{2,B}^B + F_{g,B}^B = 0,$$

这在三维中是六个方程（在二维中则只有三个）。平移分量告诉我

$$f_{1,B}^B + f_{2,B}^B = \begin{bmatrix} 0 \\ 0 \\ mg \end{bmatrix},$$

但真正告诉我 f_{1,B_z} 和 f_{2,B_z} 必须相等的，是 y 方向上的力矩分量，前提是质心到两个接触点的距离相等。

到目前为止，除了它们必须相互平衡之外，我们还没有讨论水平方向上的力。接下来让我们展开这一点。

5.2.5 (库仑) 摩擦锥

现在，支配接触力的规则可以开始成形了。首先也是最重要的是，我们要求不存在超距作用力。用 $\phi_i(q)$ 表示构型 q 下两个物体之间的距离，则有

$$\phi(q) > 0 \Rightarrow f^{c_i} = 0.$$

其次，我们要求法向力只用于抵抗穿透；物体永远不会被“拉入”接触：

$$f_{C_z}^{c_i} \geq 0.$$

在**刚性**接触模型中，我们会求解出能够满足非穿透约束的最小法向力（这称为最小约束原理）。在**柔性**接触模型中，我们将力定义为穿透深度和速度的函数。

切向方向上的力来自摩擦。最常用的摩擦模型是库仑摩擦，它表明

$$\sqrt{f_{C_x}^{c_i}{}^2 + f_{C_y}^{c_i}{}^2} \leq \mu f_{C_z}^{c_i},$$

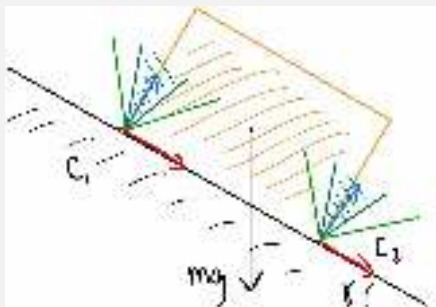
其中 μ 是一个非负标量，称为**摩擦系数**。通常我们会同时定义 μ_{static} ，它在切向速度为零时使用；以及 $\mu_{dynamic}$ ，它在切向速度非零时使用。在库仑摩擦模型中，切向接触力是在这个摩擦锥内产生最大耗散的那个力。

综合来看，这些约束的几何形状形成了一个可允许接触力的**锥**。它被著名称为“摩擦锥”，我们之后会经常提到它。

把力写成属于某个集合，对我们来说可能有点奇怪。世界当然只会选择施加一个力，不可能把集合中的所有力都施加出来。摩擦锥规定的是可能力的范围；在库仑摩擦模型下，我们说最终被选择的那个力，是这个集中能够成功抵抗接触 $x - y$ 坐标系中相对运动的力。如果摩擦锥中没有任何力能够完全抵抗这种运动，那么就会发生滑动，并且我们说世界将施加摩擦锥内具有**最大耗散**的那个力。对于圆锥形的摩擦锥而言，这个力会指向与滑动速度相反的方向。因此，尽管世界确实会从摩擦锥中“选择”一个力来施加，围绕可能被施加的力的集合进行推理仍然是有意义的，因为这些力表示摩擦能够完全抵抗的可能外力集合。例如，在重力作用下的一块砖，如果我们能够用摩擦锥内的一个力精确抵消重力，那么它就不会运动。

Example 5.6 (倾斜半平面上的砖块)

如果我们沿用上一个例子，但将桌面相对于重力方向倾斜一个角度，那么水平方向的力就开始变得重要。在推导方程之前，先检查一下你的直觉。在这种构型下，两个角点上的力的大小会保持相同吗？还是较低的那个角点上会有更大的接触力？



在上面的示意图中，你会注意到接触坐标系已经发生了旋转，使得 z 轴与接触法向对齐。我画出了两个可能的摩擦锥（深绿色和浅绿色），它们对应于两个不同的摩擦系数。我们可以通过观察立刻看出，较小的 μ 值（对应深绿色）无法产生能够完全抵消重力的接触力（摩擦锥不包含竖直线）。在这种情况下，盒子会发生滑动，并且该构型下不存在静态平衡。

如果我们将（静摩擦）摩擦系数增大到对应浅绿色区域的范围，那么我们就能找到产生平衡的接触力。事实上，对于这个问题来说，我们甚至需要一定量的摩擦才能存在平衡（我们会在练习中进一步探讨这一点）。我们还要求质心在斜坡上的竖直投影落在两个接触点之间，否则砖块会绕下边缘翻倒。我们可以通过写出同样的力/力矩平衡方程来看到这一点。假设质心位于砖块中心，砖块长度为 l 、高度为 h ，并且方程在物体坐标系 B 中表达并作用于该坐标系下，则有：

$$\begin{aligned} f_{1,B_x}^B + f_{2,B_x}^B &= -mg \sin \gamma \\ f_{1,B_z}^B + f_{2,B_z}^B &= mg \cos \gamma \\ -hf_{1,B_x}^B + lf_{1,B_z}^B &= hf_{2,B_x}^B + lf_{2,B_z}^B \\ f_{1,B_z}^B &\geq 0, \quad f_{2,B_z}^B \geq 0 \\ |f_{1,B_x}^B| &\leq \mu f_{1,B_z}^B, \quad |f_{2,B_x}^B| \leq \mu f_{2,B_z}^B \end{aligned}$$

那么，接触力的大小到底是相同还是不同呢？将第一个方程代入第三个方程，可以得到

$$f_{2,B_z}^B = f_{1,B_z}^B + \frac{mgh}{l} \sin \gamma.$$

5.2.6 作为优化问题的静态平衡

与其让物体从随机高度掉落，也许我们可以使用优化方法来初始化仿真，从而直接找到已经处于静态平衡状态的初始条件。在 DRAKE 中，[StaticEquilibriumProblem](#) 将我们上面列举的所有约束收集成一个优化问题：

$$\begin{aligned} \text{find}_q \quad & \text{subject to} \quad \tau_g(q) = - \sum_i J_i^T(q) f^{c_i} \\ & f_{C_z}^{c_i} \geq 0 \quad \forall i, \\ & |f_{C_{x,y}}^{c_i}|_2 \leq \mu f_{C_z}^{c_i} \quad \forall i, \\ & \phi_i(q) \geq 0 \quad \forall i, \\ & \phi(q) = 0 \text{ or } f_{C_z}^{c_i} = 0 \quad \forall i, \\ & \text{关节限制.} \end{aligned}$$

这是一个非线性优化问题：它包含了我们在上一章讨论过的非凸非穿透约束。倒数第二个约束（一个逻辑“或”）尤其有趣；形如 $x \geq 0, y \geq 0, x = 0 \text{ or } y = 0$ 的约束被称为互补约束（complementarity constraints），通常写作 $x \geq 0, y \geq 0, xy = 0$ 。我们可以通过将等式放宽为 $0 \leq \phi(q) f_{C_z}^{c_i} \leq \text{tol}$ ，来让非线性优化求解器更容易处理这个问题。这种做法为优化器提供了一个可以跟随的有效梯度，但代价是允许存在少量“超距作用力”。

我们还可以很容易地添加额外的代价项和约束；例如，对于初始条件，我们可以使用一个目标函数，使 q 尽量接近某个初始构型空间采样。

Example 5.7 (高塔)

那么它的效果到底如何呢？

5.3 接触仿真

静态平衡问题为我们提供了一个很好的视角，用来理解接触仿真中的一些复杂性。但当机器人和/或场景中的物体开始运动时，事情就会变得有趣得多！

更多内容即将推出！

5.4 基于模型的抓取选择

什么样的抓取才是好的抓取？这个主题在机器人学中已经被广泛研究了几十年，最初主要关注的是一个（可能具有灵巧性）的机械手如何与一个已知物体交互。[\[4\]](#) 是关于这部分文献的优秀综述；这里我只会总结其中几个关键思想。

如果抓取的目标是在机械手中稳定物体，那么“好抓取”的一种定义就是：它能够抵抗施加在物体上的、由“对抗性”力旋量（wrench）描述的扰动。

5.4.1 接触力旋量锥

上面我们介绍了摩擦锥，它表示摩擦为了抵抗运动所能够产生的力的范围。对于抓取规划而言，通过对摩擦锥中的力取加法逆元，我们可以得到所有在接触点处能够被抵抗的“对抗性”力。为了理解所有接触力（作用在多个接触点上）共同抵抗物体运动的总能力，我们希望能够以某种方式将这些力加在一起。幸运的是，空间力的空间代数可以很自然地作用于单个空间力向量，扩展到作用于整个空间力集合。

由于我们这里关心的集合都是凸锥，我将采用相对标准的记号 $\mathcal{K}$ 来表示六维的**力旋量锥**。具体来说， $\mathcal{K}_{\text{name},C}^{B_p}$ 表示与潜在空间力 $F_{\text{name},C}^{B_p}$ 对应的锥。例如，对于点接触模型（按照我们的定义，它没有扭转摩擦），其在接触坐标系中的库仑摩擦锥可以写作：

$$\mathcal{K}_C^C = \left\{ \begin{bmatrix} 0 \\ 0 \\ f_{C_x}^C \\ f_{C_y}^C \\ f_{C_z}^C \end{bmatrix} : \sqrt{(f_{C_x}^C)^2 + (f_{C_y}^C)^2} \leq \mu f_{C_z}^C \right\}. \quad (5)$$

空间力的空间代数可以直接应用于力旋量锥：

- 对于作用在同一点且在同一坐标系中表达的力旋量相加，我们所需要的解释是：由多个力旋量锥之和形成的锥，描述了这样一个力旋量集合——从每个单独的锥中各选择一个元素，然后将它们相加所能得到的所有力旋量。这个集合运算是 [闵可夫斯基和](#)，我们用 $\oplus$ 表示，例如：

$$\mathcal{K}_{\text{total},C}^{B_p} = \mathcal{K}_{0,C}^{B_p} \oplus \mathcal{K}_{1,C}^{B_p} \oplus \dots \quad (6)$$

- 将力旋量锥从一个作用坐标系 B_p 平移到另一个坐标系 B_q ，是对这些锥的线性操作；为了强调这一点，我将公式 3 写成矩阵形式：

$$\mathcal{K}_C^{B_q} = \begin{bmatrix} I_{3 \times 3} & [B_q p_C^{B_p}]_{\times} \\ 0_{3 \times 3} & I_{3 \times 3} \end{bmatrix} \mathcal{K}_C^{B_p}, \quad (7)$$

其中记号 $[p]_{\times}$ 表示与叉积对应的反对称矩阵。

- 可以使用旋转在不同的“表达于”坐标系之间进行变换：

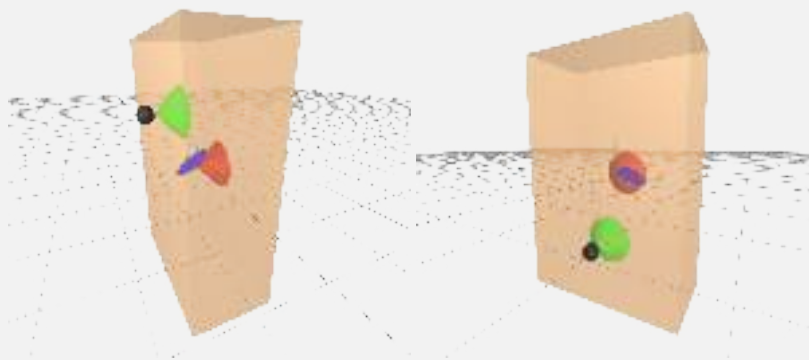
$$\mathcal{K}_D^{B_p} = \begin{bmatrix} {}^D R^C & 0_{3 \times 3} \\ 0_{3 \times 3} & {}^D R^C \end{bmatrix} \mathcal{K}_C^{B_p}. \quad (8)$$

Example 5.8 (接触力旋量锥可视化)

我制作了一个简单的交互式可视化工具，你可以用它来帮助建立对这些力旋量锥的直觉。我这里有一个固定在空间中的盒子，它只受到接触力作用（这里没有绘制重力）；我在接触点处以及物体中心处都画出了摩擦锥。请注意，我在这个例子中有意禁用了水弹性接触——这些力仅使用每个碰撞对之间的单个接触点来计算——以简化解释。

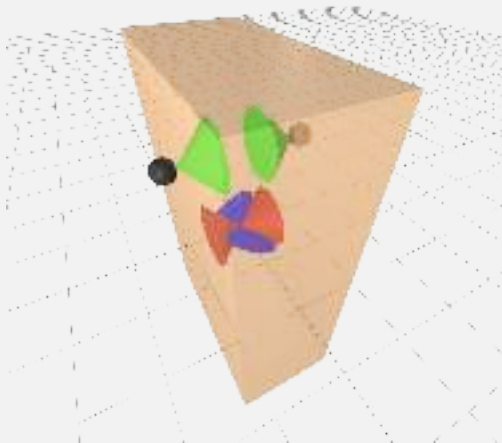
这里有一个重要的注意事项：力旋量锥位于 $\mathbb{R}^6$ 中，但我只能在 $\mathbb{R}^3$ 中绘制锥。因此，我采用了（标准的）做法，用两个锥来绘制六维锥到三维空间的投影：一个用于平移分量（接触点处用绿色表示，在物体坐标系处再次用红色表示），另一个用于旋转分量（在物体坐标系中用蓝色表示）。这可能会稍微产生误导，因为实际上不能从这两个集合中独立选择元素。

这里展示的是单个接触点的接触力旋量锥，并分别对两个不同的接触位置进行了可视化：



我希望你能够立刻注意到，力旋量锥中的旋转分量是低维的——由于叉积的存在，该空间中的所有向量都必须与向量 ${}^{B_p} p^C$ 正交。当然，最好的方式还是你自己运行 notebook，体验真正的三维交互式可视化。

这里展示的是两个彼此正对的接触点所形成的接触力旋量锥：



请注意，尽管每个旋转锥本身都是低维的，但它们张成的是不同的空间。它们结合在一起（通

过这两个集合的闵可夫斯基和来理解）后，可以抵抗施加在物体坐标系中的所有纯力矩。这种直觉大体上是正确的，但这里也体现出了把六维锥投影成两个三维锥所带来的一点误导性。有些力旋量实际上是这种抓取无法抵抗的。具体来说，如果我把力旋量锥可视化到两个接触点正中间的位置，我们会发现这些力旋量锥并不包含沿着两个接触点连线方向施加的力矩。仅靠这两个接触点，在没有扭转摩擦的情况下，是无法抵抗绕该轴的力矩的。

在实际中，我们在仿真工作流程中使用的夹爪模型采用了水弹性接触，它会模拟一个接触斑块，并且能够产生扭转摩擦——因此我们可以抵抗绕该轴的力矩。能够产生的具体大小，将与夹爪挤压物体的力度成正比。

现在，我们可以通过将所有单独的接触力旋量锥变换到同一个坐标系中并将它们相加，从而计算任意一组接触能够施加在物体上的可能力旋量锥——即接触力旋量锥（contact wrench cone）。经典的抓取评价指标会认为，一个好的抓取应当具有较大的接触力旋量锥（即能够抵抗许多对抗性的力旋量扰动）。如果接触力旋量锥覆盖了整个 $\mathbb{R}^6$ ，那么我们就说这些接触实现了力闭合（force closure）[4]。

值得一提的是，力旋量锥优雅的（线性）空间代数性质，也使得这些量非常适合用于优化问题中（例如 [5]）。

5.4.2 共线对跖抓取

这种力旋量分析之美，最初激发了一种非常基于模型的抓取分析方法：人们可以尝试优化接触位置，以最大化接触力旋量锥。但本章的目标是尽量少假设被抓取物体的信息，因此我们（暂时）会避免在物体模型表面上优化最佳抓取位置。尽管如此，我们的基于模型的抓取分析仍然为抓取未知物体提供了一些非常好的启发式方法。

特别地，对于双指夹爪而言，一个获得较大接触力旋量锥的良好启发式方法，是寻找共线的“对跖”抓取点。这里的对跖意味着接触法向量（即接触坐标系的 z 轴）正好指向相反方向。而“共线”意味着它们位于同一条直线上——也就是两个接触点之间的连线。正如你在上面的双指接触力旋量可视化中所看到的，这是一种相当有效的启发式方法，可以获得较大的总接触力旋量锥。接下来我们将看到，即使几乎不了解物体本身，我们也可以应用这个启发式方法。

5.5 从点云中选择抓取

与其观察料箱中的物体、尝试识别具体物体并估计它们的位姿，不如考虑一种方法：直接在（未分割的）点云上寻找可抓取区域。这类非常几何化的抓取选择方法在实践中可以表现得很好，也可以在仿真中用于训练基于深度学习的抓取选择系统，使其在真实世界中同样表现出色，甚至能够处理部分视角和遮挡问题 [6, 7, 8]。

5.5.1 点云预处理

为了获得料箱内部的良好视野，我们将设置多个 RGB-D 相机。在这里的所有示例中，我为每个料箱使用了三台相机。这些相机看到的不仅是料箱中的物体；它们还会看到料箱本身、其他相机，以及可视工作空间中的任何其他东西。因此，我们需要做一些处理，将来自多台相机的点云合并成一个只包含感兴趣区域的单一点云。

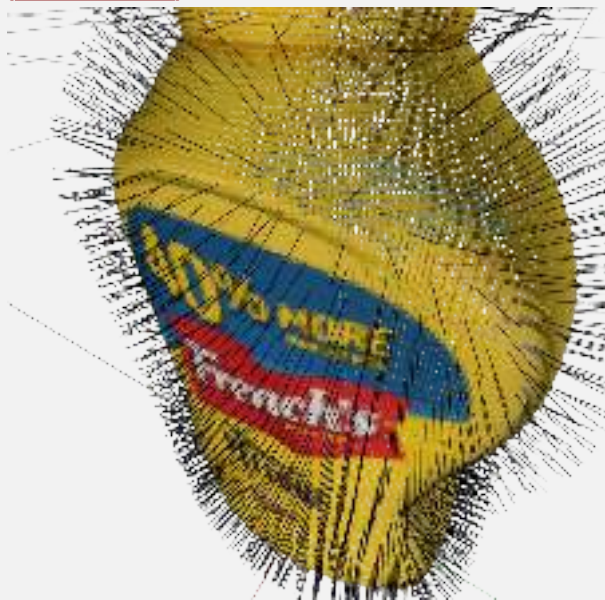
首先，我们可以对点云进行裁剪，丢弃任何来自感兴趣区域之外的点（我们会把感兴趣区域定义为已知料箱位置正上方的一个轴对齐包围盒）。

正如我们将在下面详细讨论的那样，许多抓取选择策略都会受益于对点云“**法向量**”的估计（即一个单位向量，用来估计相对于底层几何表面的法向方向）。事实上，最好是在各个单独的点云上估计法向量，并利用生成这些点的相机位置；这比在点云合并之后再估计法向量更好。

对于安装在真实世界中的传感器，**合并点云**需要高质量的相机标定，并且必须处理杂乱的深度返回。上一章中的所有工具在这里都相关，因为合并点云的任务本身就是点云配准问题的另一个实例。对于我们可以从仿真中获得的完美深度测量，并且在相机位置已知的情况下，我们可以跳过这一步，直接把多个点云中的点列表拼接在一起。

最后，得到的原始点云很可能包含了比抓取规划实际所需多得多的点。对点云进行**下采样**的一种标准方法是使用**体素**网格——也就是在三维空间中规则排列、大小一致的立方体。然后我们用每个体素中恰好一个点来概括原始点云（例如参见 [Open3D 关于体素化的说明](#)）。由于点云通常只占据三维空间中很小一部分体素，我们会使用稀疏数据结构来存储体素网格。在有噪声的点云中，这个体素化步骤也是一种有用的滤波形式。

Example 5.9 (芥末瓶点云)



我生成了一个场景，其中三台相机都观察着我们最喜欢的 YCB 芥末瓶。我取出各个单独的点云（这些点云已经由 [DepthImageToPointCloud](#) 系统转换到了世界坐标系中），对点云进行裁剪（以去除来自其他相机的几何体），估计它们的法向量，合并点云，然后再对点云进行下采样。这个顺序很重要！

我已经把所有点云都发布到了 [meshcat](#) 中，但其中许多默认设置为不可见。使用下拉菜单来打开和关闭它们，并确保你基本理解每个步骤中发生了什么。对于这个例子，如果你不想运行代码，我也可以直接给你 [meshcat 输出](#)。

5.5.2 估计法向量和局部曲率

我们将在下面发展出的抓取选择策略，将基于场景的局部几何特征（法向方向和曲率）。理解如何从点云中估计这些量，是点云处理中的一个非常好的练习，也能代表其他类似的点云算法。

让我们考虑一下如何以最小二乘的意义，将一个平面拟合到一组点上[9]。我们可以用一个位置 p 和一个单位长度的法向量 n 来描述三维空间中的一个平面。点 p^i 到该平面的距离简单地由内积的

大小给出: $(p^i - p)^T n$. 因此, 我们的最小二乘优化问题变为

$$\min_{p, n} \sum_{i=1}^N (p^i - p)^T n^2, \quad \text{subject to } |n| = 1.$$

对拉格朗日函数关于 p 求梯度并令其等于零, 可以得到

$$p^* = \frac{1}{N} \sum_{i=1}^N p^i.$$

将其代回目标函数后, 可以把问题写成

$$\min_n n^T W n, \quad \text{subject to } |n| = 1, \quad \text{其中 } W = \left[\sum_i (p^i - p^*)(p^i - p^*)^T \right].$$

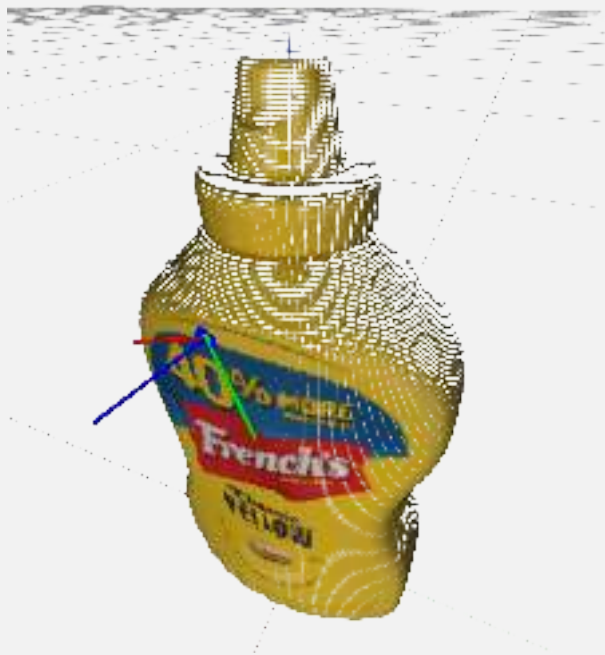
从几何上看, 这个目标函数是一个以原点为中心的二次碗形函数, 并带有单位圆约束。因此, 最优解由数据矩阵 W 的**最小**特征值所对应的 (单位长度) 特征向量给出。并且对于任意最优的 n , 其“翻转”法向量 $-n$ 同样也是最优的。我们暂时可以任意选择方向, 然后在后处理步骤中翻转法向量 (以确保所有法向量都指向相机原点)。

真正有趣的是, 第二和第三个特征值/特征向量也会告诉我们一些关于局部几何的信息。因为 W 是对称矩阵, 它具有正交特征向量, 而这些特征向量构成了点云的一个 (局部) 基。最小特征值对应的方向沿着法向, 而**最大**特征值对应于曲率最小的方向 (与该向量的点积平方最大)。这些信息对于寻找和规划抓取非常有用。[6] 以及他们之前的其他研究者, 都将这一点作为生成候选抓取的主要启发式方法。

为了近似由点云表示的网格的局部曲率, 我们可以使用快速最近邻查询来找到少量局部点, 并只在这些点上使用这个平面拟合算法。在直接基于深度图像进行法向估计时, 人们往往完全省略最近邻查询; 而是简单地采用这样一种近似: 像素坐标中的相邻点通常也是点云中的最近邻。我们可以对点云中的每一个点重复整个过程。

我还记得, 当处理点云开始成为我生活中更重要的一部分时, 我曾以为, 在某个稠密点云的每个点上执行这种中等计算量的操作, 肯定无法与在线感知兼容。但我错了! 即使在多年以前, 这类操作也经常被用于实时感知循环中。(而且与如今我们在评估大型神经网络时消耗的 **FLOPs** 数量相比, 它们简直微不足道。)

Example 5.10 (芥末瓶的法向量与局部曲率)



我已经实现了这个基础的最小二乘曲面估计算法，其中查询点用绿色表示，最近邻点用蓝色表示，而局部最小二乘估计则用我们的 $RGB \leftrightarrow XYZ$ 坐标系图形绘制出来。你绝对应该拖动它四处看看，并尝试理解这些坐标轴是如何与法向量和局部曲率对齐的。

你可能会想知道，可以去哪里阅读更多这类算法的内容。我并没有一个特别好的参考资料推荐给你。不过，Radu Rusu 是点云库（Point Cloud Library）的主要作者[10]，而他的博士论文中对 2010 年左右的点云算法做了大量很好的总结[11]。

5.5.3 评估候选抓取

现在我们已经处理好了点云，就具备了开始规划抓取所需的一切条件。我会把这部分讨论拆分成两个步骤。在本节中，我们将构建一个用于给候选抓取打分的代价函数。在下一节中，我们会讨论一些非常简单的方法，用来尝试寻找代价较低的候选抓取。

延续上面关于“基于模型”的抓取选择讨论，一旦我们抓起一个物体——或者说，当我们夹紧时，任何正好位于两指之间的东西——那么我们预期手指之间的接触力至少需要抵抗该物体的**重力力旋量**（即由重力产生的空间力）。夹爪提供的闭合力沿着夹爪的 x 轴方向，但如果我们希望在拾起物体时，它不会从手中滑落，那么接触摩擦锥内部的力就必须能够抵抗这个重力力旋量。由于我们并不知道这个力旋量具体会是什么（并且还受到手指几何形状的一些限制），一种合理的策略是：寻找点云表面上那些共线的对跖点，并且这些点还与夹爪的 x 轴方向对齐。在真实点云中，我们不太可能找到完美的对跖点对，但寻找法向量几乎相反的区域，依然是一种很好的抓取策略！

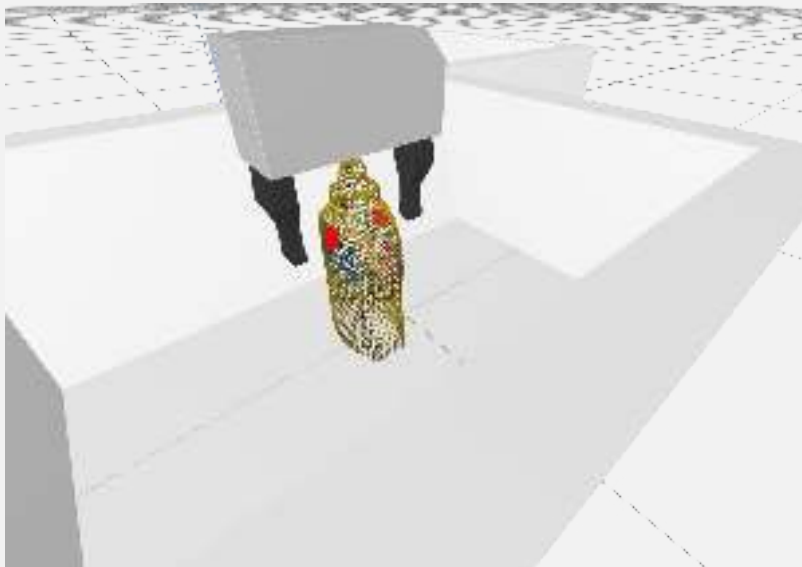
Example 5.11 (候选抓取评分)

在实践中，我们手指与物体之间的接触，更适合用**面接触**（*patch contact*）而不是点接触来描述（这是由于橡胶指尖的形变以及被抓取物体本身可能发生的形变）。因此，寻找具有一致法向

量的点区域是有意义的。实现这一点的方法有很多，我这里采用的方法是：将场景处理后的点云变换到夹爪的候选坐标系中，然后裁剪掉除指尖之间包围盒内部之外的所有点（我已经在 MeshCat 中用红色标出了这些点）。我的抓取代价函数中的第一项，仅仅是基于点云中所有点的奖励，它取决于这些点的法向量与夹爪 x 轴的对齐程度：

$$\text{cost} = - \sum_i (n_{G_x}^i)^2,$$

其中 $n_{G_x}^i$ 是裁剪后点云中第 i 个点在夹爪坐标系中表达时的 x 分量。



此外，一个好的抓取还可能涉及其他因素。对于我们这种运动学受限、需要伸入料箱中的机器人，我们可能更偏好那些能让机械臂处于有利姿态的抓取。在我代码中实现的抓取评价指标里，我加入了一个关于手部偏离竖直方向的代价项。我可以通过世界坐标系中的 $-z$ 向量 $[0, 0, -1]$ 与夹爪坐标系中的 y 轴、再经过旋转到世界坐标系之后的点积来给予奖励：

$$\text{cost} += -\alpha [0 \ 0 \ -1] R^G \begin{bmatrix} 0 \\ 1 \\ 0 \end{bmatrix} = \alpha R_{3,2}^G,$$

其中 α 是相对代价权重，而 $R_{3,2}^G$ 是旋转矩阵第三行第二列中的标量元素。

最后，我们还需要考虑候选抓取与料箱以及点云之间的碰撞。当夹爪发生碰撞时，我会直接返回无穷大的代价。我已经在 notebook 中实现了所有这些项，并给你提供了滑块来移动手部、观察代价如何变化。

5.5.4 生成候选抓取

我们已经定义了一个代价函数：给定场景中的点云以及环境模型（例如料箱的位置），它可以对候选抓取位姿 X^G 进行评分。现在我们希望求解这样一个优化问题：找到使代价最小、同时满足碰撞约束的 X^G 。

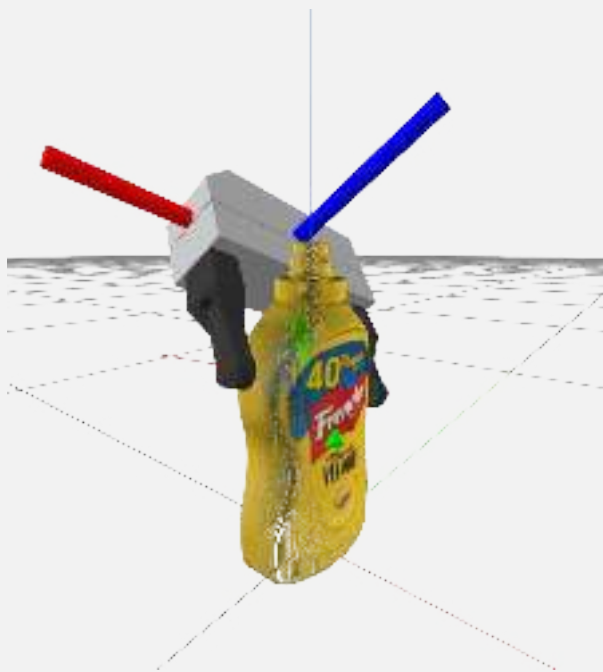
不幸的是，这是一个极其困难的优化问题，其目标函数和约束都高度非凸。此外，与对跖点相关的代价项，对于大多数 X^G 来说都为零——因为大多数随机的 X^G 都不会在两指之间包含任何点。因此，文献中的大多数方法并不会使用传统的数学规划求解器，而是转向基于随机采样的算法。而且，我们确实拥有一些很强的启发式方法来选择合理的样本。

一种启发式方法（例如 [6] 中所使用的）是利用点云的局部曲率来提出候选抓取：使点云法向量朝向手掌方向，并将手部朝向设置为让手指与最大曲率方向对齐。随后，可以沿着法向方向移动手部，直到手指脱离碰撞，甚至还能对附近点进行采样。我们已经为你编写了一个[练习](#)来探索这种启发式方法。不过对于我们的 YCB 物体来说，我不确定这是不是最好的启发式方法；因为我们有很多盒状物体，而盒子的局部曲率并不能提供太多信息。

另一种启发式方法是寻找点云中的对跖点对，然后采样那些能够使手指与这些对跖点对对齐的候选抓取。许多三维几何库至少都支持对点云体素表示进行“射线投射（ray casting）”操作；因此，一个合理的寻找对跖点的方法是：随机选择点云中的一个点，然后沿着该点法向量的反方向向点云中发射射线。如果射线投射找到的体素法向量足够对跖，并且该体素的距离小于夹爪宽度，那么我们就找到了一个合理的对跖点对。

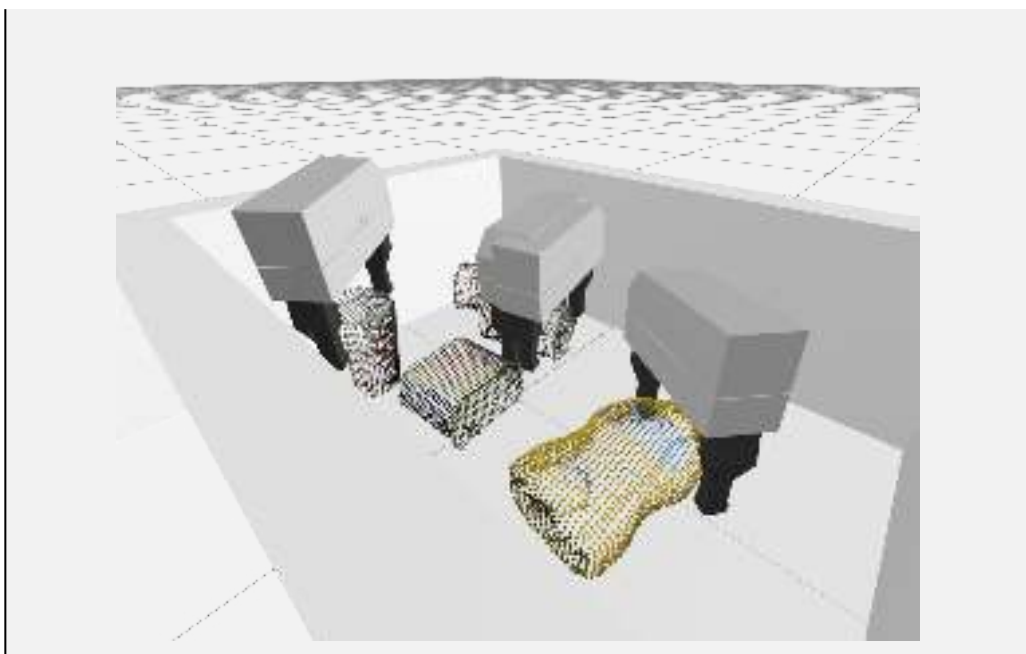
Example 5.12 (生成候选抓取)

作为射线投射的替代方案，我在代码示例中实现了一个更简单的启发式方法：我只是随机选择一个点，然后开始在不同姿态下采样抓取（从竖直方向开始），使夹爪的 x 轴与该点的法向量对齐。然后，我基本上主要依赖评分函数中的对跖项来帮我找到好的抓取。



我还实现了另一个启发式方法——一旦我找到了位于指尖之间的点云点，我就会沿着夹爪的 x 轴方向将手部向外移动，使这些点在夹爪坐标系中居中。这样可以帮助我们在闭合夹爪抓取物体时，避免把物体撞倒。

不过也就这些了！这是一个非常简单的策略。我会采样少量候选抓取，然后只绘制其中代价最低的几个。如果你多运行几次，我想你会发现它其实相当合理。每次运行时，它都会模拟物体从空中掉落；而真正的抓取评估实际上是非常快的。



5.6 边界情况

如果你稍微玩一玩我上面实现的抓取评分方法，你会发现其中存在一些缺陷。其中有些问题可以通过在代价函数中再添加几个项来比较容易地解决（虽然依然是启发式的）。例如，我并没有检查预抓取（pre-grasp）构型中的碰撞，但这一点其实可以很容易地加入。

还有一些情况下，仅靠抓取本身并不足以作为一种策略。想象一下，你把一个物体正好放在料箱的某个角落里。在不与物体或侧壁发生碰撞的情况下，可能无法让手从物体两侧包住它。上面的策略永远不会选择去尝试抓取盒子的最角落（因为它总是试图让采样点的法向量与夹爪的 x 轴对齐），而且它是否应该这样做也并不明确。对于我们这个相对较大的夹爪来说，这一点可能尤其明显。在我们曾经在 TRI 使用的设置中，我们实现了一个额外的简单“推动”启发式策略：如果水槽中存在点云，但找不到可行的候选抓取，就使用这个策略。我们不进行抓取，而是让手向下运动，并轻推那部分点云，使其朝料箱中央移动。这实际上能带来很大帮助！

我们这个简单方法还有其他一些缺陷，而这些缺陷如果只用纯几何方法，将会非常难以解决。大多数问题都归结于这样一个事实：到目前为止，我们的系统还没有“物体”的概念。例如，如果料箱中有两个物体靠得很近，这种策略导致“双重抓取”并不少见。对于较重的物体来说，靠近质心处抓取可能很重要，这样可以提高我们在保持于摩擦锥内的同时抵抗重力力矩量的机会。但这里的策略可能会仅仅夹住一把重锤手柄最远端，就把它提起来。

有趣的是，我并不认为问题一定在于点云信息不足，即使没有颜色信息也是如此。我可以给你看类似的点云，而你不会犯同样的错误。这类例子正是学习一个抓取评价指标可能有所帮助的地方。为了解决这个问题，我们并不需要实现人工通用智能；只要有一些经验，知道我们过去在某些位置尝试抓取时失败过，就足以显著改进我们的抓取启发式方法。

5.7 任务层编程

我们大多数人都习惯于按照[过程式编程](#) 范式来编写代码；代码从程序顶部开始执行，经过函数调用、分支（例如 if/then 语句）、for/while 循环等。我们很容易也想用这种方式来编写高层机器人程序。对于杂乱物体任务，我可以写成：“当第一个料箱中还有物体可以抓取时，就抓起它并移动到第二个料箱；……”。这当然没有错！但是，要成为机器人框架中的一个组件，这个高层程序必

须以某种方式在运行时与系统的其余部分通信，包括与那些以固定频率运行的低层控制器建立接口（比如 100Hz，甚至 1kHz）。那么，我们应该如何编写这种高层行为（通常称为“任务层”），才能让它与高频率的低层控制无缝集成呢？

用于编写任务层行为的编程范式有几种重要方式。广义上来说，我会将它们分成三类：

- 过程式代码（运行在单独线程中）向主机器人/仿真线程中的模块传递消息或事件。这种方式在像 ROS 这样的多进程消息传递生态系统中相当常见。
- 任务层“策略”可以直接编写成例如有限状态机（FSM）或“行为树（behavior trees）”，并在机器人/仿真循环中直接进行求值。
- 任务层“规划器”则可以根据任务层行为如何串联以实现长期目标的规则来进行规划：
 - 在最简单的情况下，规划器输出一个可以在机器人/仿真循环中执行的计划，就像我们之前用来规划夹爪运动的轨迹一样。
 - 如果规划器能够在线运行（例如根据新的传感器信息持续调整/重新规划），那么它也可以直接在机器人/仿真循环中使用。
 - 另外，一些规划器实际上可以直接将它们的计划“编译”为一个策略（例如 FSM 的形式）。[这里 给出了一个经典示例](#)。

这些范式各有其适用场景，而且有些研究者会强烈偏向其中某一种。我始终记得 Rod Brooks 写过像 [Elephants Don't Play Chess](#) 这样的论文，用来反对任务层规划[12, 13, 14]。Rod 的“包容架构（subsumption architecture）”明显属于“任务层策略”类别，并且可以被视为行为树的前身；它通过在早期 Roomba 吸尘机器人上的成功应用建立了自己的可信度。而规划领域则可能反驳说，对于真正复杂的任务，人类程序员不可能为 AI 系统可能遇到的所有排列组合/情形都编写出对应策略；真正能够扩展的方法是规划。他们还通过诸如信念空间规划（planning in belief space）和任务-运动规划（task-and-motion planning）等扩展，解决了最初关于符号落地（symbol grounding）和建模需求的一些批评；我们将在后续讲义中介绍这些内容。

5.7.1 状态机与行为树

5.7.2 任务规划

任务层规划器主要基于搜索。AI 规划作为搜索已经有很长的历史。STRIPS [15] 也许是最早、也是最著名的一种表示方法（以及算法），它使用初始状态、目标状态以及一组具有前置条件和效果的动作来描述规划问题。时序逻辑（例如线性时序逻辑 LTL、度量时序逻辑 MTL、信号时序逻辑 STL）则使得规划描述能够包含复杂的目标或随时间变化的约束，例如“事件 A 最终应该在事件 B 之后发生”。规划领域定义语言（PDDL）[16, 17] 在 STRIPS 表示基础上进行了扩展，加入了更具表达能力的特性，例如类型化变量、条件效果以及（甚至包括时序逻辑的）量词，并成为一系列长期举办、极具影响力的 [ICAPS 规划竞赛](#) 的规范语言。这些规划竞赛催生了许多高效的规划算法，例如 GraphPlan [18]、HSP [19]、Fast-Forward [20] 和 FastDownward [21]。这些规划器展示了“启发式搜索（heuristic search）”的强大能力，并且高度利用了规划描述中面向对象的特性，将因子化（factorization）作为一种强有力的启发式方法。

5.7.3 大语言模型

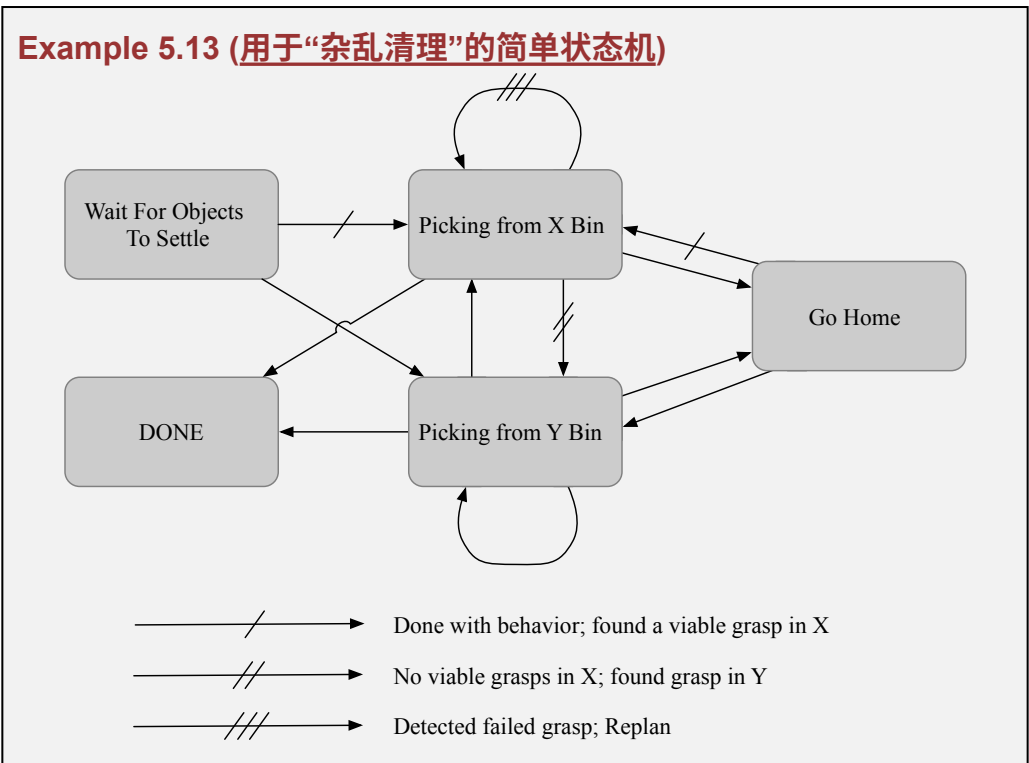
当然，在 2022 年末，随着大语言模型（LLM）的兴起，一种新的任务“规划”方法迅速爆发式登场。人们很快意识到，像 GPT-4 这样最强大的语言模型，已经能够从非常有限的自然语言描述中生成高度非平凡的计划。通过精心设计提示词，可以引导 LLM 生成机器人真正能够执行的动作，而且 LLM 似乎特别擅长编写 Python 代码。借助 Python，甚至可以提示 LLM [生成一个策略（policy）](#)，而不仅仅是一个计划 [22]。在撰写本文时，如何将机器人摄像头/传感器所感知到的当前环境自动转换为自然语言提示，仍然是一个挑战，但看起来这只是时间问题。据说，滚动发布

版的 [GPT-4V\(ision\)](#) 在这方面表现得极其出色。

一个非正式的共识是：语言模型让简单规划问题变得异常容易。重要的是，与基于搜索的规划器可能会利用你在规范中遗漏的要求/约束不同，LLM 会利用“常识”来填补空缺，从而输出非常合理的计划。当然，如今的 LLM 在长时域、多步推理任务规划方面仍然存在限制，但巧妙的提示工程已经能够带来令人印象深刻的结果 [23]。

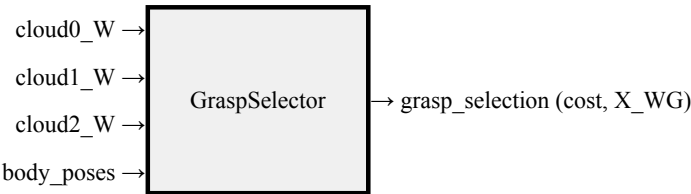
5.7.4 用于“杂乱清理”的简单状态机

对于系统框架中的杂乱清理任务而言，采用第二种方法——把任务层行为“策略”编写为一个简单状态机——会是一个很好的选择。



5.8 整合所有部分

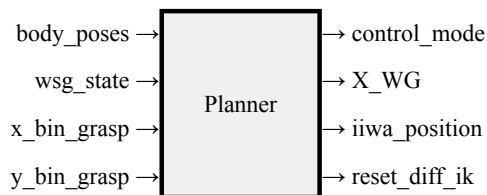
首先，我们将抓取选择算法封装成一个系统：它读取来自 3 个相机的点云，完成点云处理（包括法向量估计）以及随机采样，并将抓取选择结果输出到输出端口：



请注意，我们会实例化这个系统两次——一次用于位于正 X 轴方向上的料箱，另一次用于位于负 Y 轴方向上的料箱。由于 Drake 系统框架中的“拉取架构（pull-architecture）”，这种相对昂贵的采样过程只有在下游系统求值输出端口时才会被调用。

这个示例中的主力系统是 `Planner` 系统，它实现了基本状态机，调用规划函数，将得到的计划存

储在它的 `Context` 中，并在输出端口上对这些计划求值：



这个系统需要足够的输入/输出端口，以便将相关信息传入规划器，并向下游控制器发送命令。

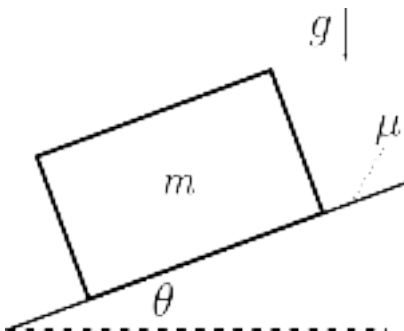
Example 5.14 (杂乱清理 (完整演示))

5.9 练习

Exercise 5.1 (评估静态平衡)

在这个问题中，我们将使用库仑摩擦模型，其中 $|f_t| \leq \mu f_n$ 。换句话说，摩擦力足够大，可以抵抗任何运动，直到所需的力超过 μf_n 为止；在这种情况下， $|f_t| = \mu f_n$ 。

- a. 考虑一个质量为 m 的盒子放在倾角为 θ 的斜坡上，盒子与斜坡之间的摩擦系数为 μ ：



对于给定角度 θ ，为了让盒子不会沿平面下滑，所需的最小摩擦系数是多少？用 g 表示重力加速度。

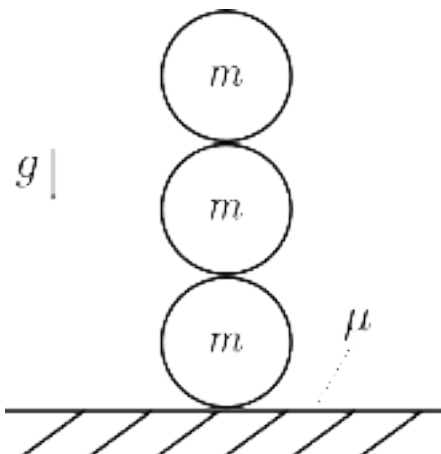
现在考虑一个平坦地面，上面放着三个实心（均匀密度）球体，每个球体半径为 r 、质量为 m 。假设球体彼此之间以及球体与地面之间具有相同的摩擦系数 μ 。

对于下面每一种构型：这些球体是否可能在某个 $\mu \in [0, 1]$ 、 $m > 0$ 、 $r > 0$ 下处于静态平衡？请解释为什么可以或为什么不可以。请记住，要让所有物体处于平衡状态，力矩和力都必须平衡。

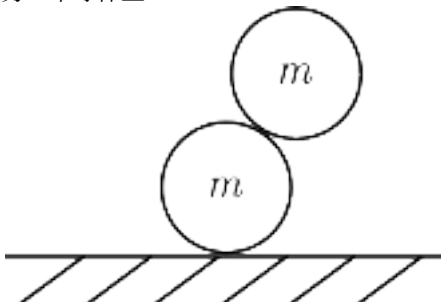
为了帮助你求解这些问题，我们提供了 来帮助你建立直觉并测试你的答案。它允许你指定球体的构型，然后使用 `StaticEquilibriumProblem` 类来求解静态平衡。请使用这个 notebook 来帮助你可视化并思考该系统，但对于每一种构型，你都应该能够为自己的答案给出物理解释。（这种物理解释的一个例子是：画出每个球体所受力和力矩的自由体图，并写出它们如何能够或不

能够相加为零的方程。这本质上就是 `StaticEquilibriumProblem` 所检查的内容。)

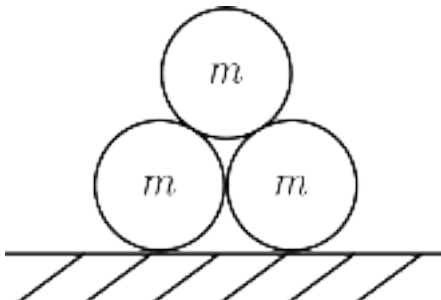
- b. 球体上下堆叠:



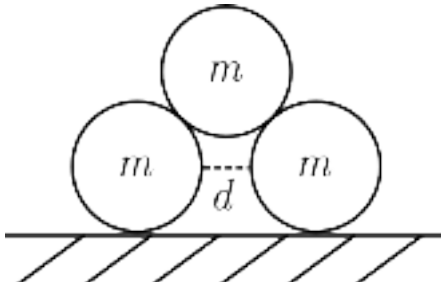
- c. 一个球体偏置地放在另一个球体上:



- d. 球体堆成金字塔:



- e. 球体堆成金字塔, 但底部两个球体之间有距离 d :



最后, 关于 `StaticEquilibriumProblem` 的几个概念性问题:

- f. 为什么我们为系统指定什么初始猜测很重要? (提示: 这是什么类型的优化问题?)
- g. 查看 Drake 文档中关于 [StaticEquilibriumProblem](#) 的说明。它列出了求解平衡时使用的

约束。其中哪两个约束可以通过自由体图来回答？

Exercise 5.2 (从深度图估计法向量)

在这个练习中，你将研究一种略有不同的法向量估计方法。具体来说，我们可以利用深度图中已经存在的空间结构，来避免计算最近邻。你将完全在 中完成本练习。你需要完成以下步骤：

- 从数学上分析如何使用由深度图中相近点簇构造的数据矩阵来计算表面法向量。
- 实现一种从深度图估计法向量的方法，且不计算最近邻。
- 思考一种场景，在该场景中，基于深度图的解决方案表现不如计算最近邻的方法。

Exercise 5.3 (解析对跖抓取)

到目前为止，我们一直使用基于采样的方法来寻找对跖抓取——但如果我们知道物体形状的方程，是否可以解析地找到一个对跖抓取呢？在这个练习中，你将分析并实现一种构造上正确的方法，使用符号微分和 `MathematicalProgram` 来寻找对跖点。你将完全在 中完成本练习。你需要完成以下步骤：

- 证明对跖点是定义在该形状上的某个能量函数的临界点。
- 证明其逆命题并不成立。
- 使用 `MathematicalProgram` 实现一种寻找这些对跖点的方法。
- 分析该能量函数的 Hessian 矩阵，以及它与对跖抓取类型之间的关系。

Exercise 5.4 (抓取候选生成)

在本章 notebook 中，我们使用对跖启发式方法生成了抓取候选。在这个练习中，我们将研究一种替代方法：基于局部曲率来生成抓取候选，类似于 [6] 中提出的方法。这个练习偏重实现，你将完全在 中完成。

Exercise 5.5 (行为树)

让我们重新介绍一下行为树（behavior trees）的术语。行为树提供了一种在不同任务之间切换的结构，它同时具备模块化和响应式特性。行为树是一棵有向有根树，其中内部节点负责管理控制流，而叶节点则是需要执行的动作或需要评估的条件。例如，一个动作可能是“抓起球（pick ball）”。这个动作可能成功，也可能失败。一个条件可能是“我的手是空的吗？（Is my hand empty?）”，它可能为真（因此条件成功），也可能为假（条件失败）。

更具体地说，每个节点都会被周期性检查。随后，它会检查自己的子节点，并根据子节点的状态报告 success（成功）、failure（失败）或 running（运行中）。控制流节点有不同类别，但我们这里只考虑两类控制流节点：

顺序节点（Sequence Node）：（→）

顺序节点从左到右依次执行每个子行为。如果任意一个子节点失败，则顺序节点返回**失败**；如果任意一个子节点正在运行，则顺序节点返回**运行中**。一种理解这个操作符的方式是：顺序节点对所有子行为取“与（and）”。

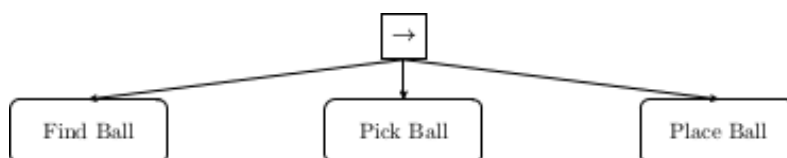
回退节点 (Fallback Node) : (?)

回退节点同样会依次执行每个子行为。然而，只要任意一个子节点成功，回退节点就会成功。一种理解这个操作符的方式是：回退节点对所有子行为取“或 (or)”。

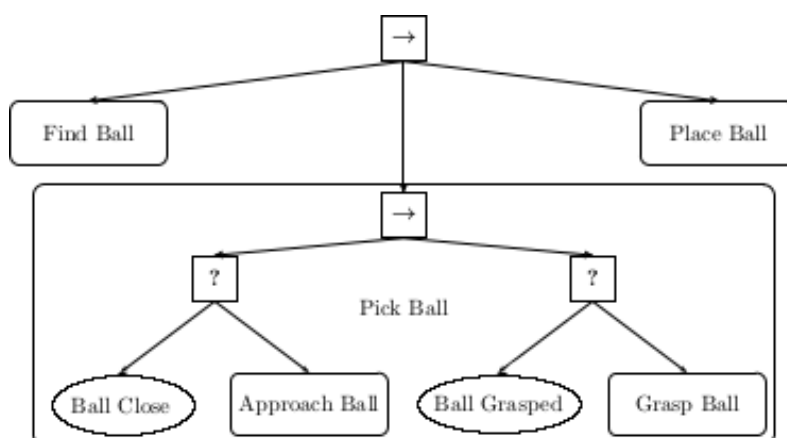
这些符号如下图所示。顺序节点用方框中的箭头表示，回退节点用方框中的问号表示，动作位于矩形框中，而条件位于椭圆框中。



让我们在一个简单任务的背景下应用对行为树的理解：机器人的目标是找到一个球，把球拿起来，并把它放进箱子中。我们可以用如下高层行为树来描述这个任务：



请确认一下，这棵小型行为树确实表达了该任务！我们还可以再深入一层，将“抓球 (Pick Ball)”行为展开：



抓球行为可以这样理解：先看左侧分支。如果球已经足够近，那么条件“球接近了吗？ (Ball Close?)”返回 true。如果球还不够近，那么我们执行“接近球 (Approach ball)”动作，从而让球变得接近。因此，要么球已经接近（即条件“Ball Close?”为真），要么我们通过成功执行“Approach ball”动作来让球接近。既然现在球已经接近了，我们就考虑抓取动作右侧的分支。如果球已经被抓住，那么条件“球已抓住？ (Ball Grasped?)”返回 true。如果球还未被抓住，我们就执行“抓住球 (Grasp Ball)”动作。

我们还可以进一步扩展行为树，以得到完整的行为：

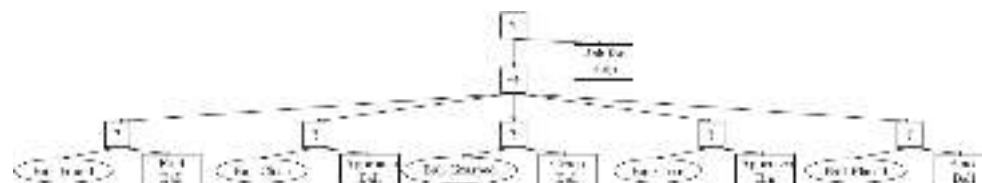
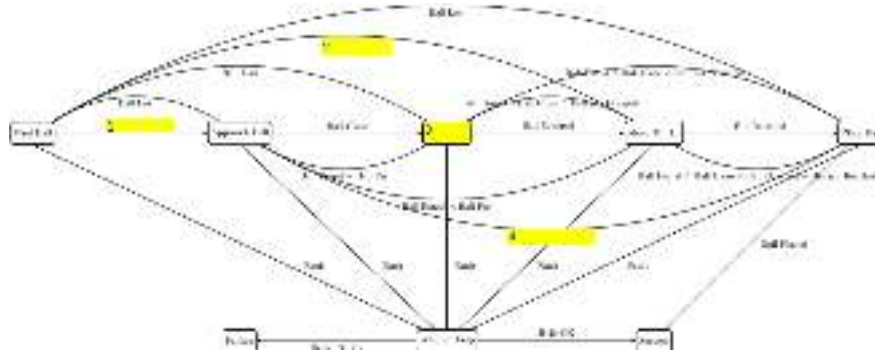


Figure 5.24 - 拾取任务的行为树

这里我们添加了这样一种行为：如果机器人无法完成任务，它可以向一位善良的人类寻求帮助。请花几分钟时间梳理这棵行为树，并确认它确实编码了我们想要的任务行为。

- a. 我们声称行为树能够实现响应式行为。让我们来探索这一点。假设机器人正在执行我们的行为树。当机器人正在执行“接近箱子（Approach Bin）”动作时，一个粗鲁的人类把球从机器人手中打掉了。球滚到了机器人依然能够看到、但距离很远的位置。根据行为树的逻辑，哪个条件会失败？又会因此执行哪个动作？
- b. 另一种对计算进行数学建模的方法是有限状态机（FSM）。有限状态机由状态、状态转移和事件组成。我们可以使用有限状态机来编码拾取任务中的行为树行为，如下图所示：



我们故意留空了一些状态和转移。你的任务是填写这 4 个值，使该有限状态机能够产生与行为树相同的行为。

从这个简单任务中我们可以看到：虽然有限状态机通常容易理解且直观，但它们也会很快变得相当混乱！

- c. 在本章和课程中，我们一直讨论的是一个料箱抓取（bin picking）任务。现在我们的目标是使用行为树来描述该料箱抓取任务的行为。我们希望编码如下行为：当 Bin 1 还没有空时，我们将选择一个可行抓取，在料箱中执行抓取，并把抓到的物体运送到 Bin 2。如果 Bin 1 中仍然有物体，但我们无法找到可行抓取，那么假设我们有一个动作可以摇晃料箱，从而打乱物体（希望这样能够产生可行抓取）。

我们定义以下条件：

- “Bin 1 空了吗？”
- “存在可行抓取吗？”
- “抓取已选择？”
- “抓取到的物体位于 Bin 2 上方？”
- “物体已抓住？”
- “物体已经在 Bin 2 中？”

以及以下动作：

- “将抓住的物体放入 Bin 2”
- “抓取物体”
- “选择抓取”
- “随机化料箱”
- “将抓住的物体运输到 Bin 2 上方”

使用这些条件、动作以及我们的两个控制流节点（顺序节点和回退节点），请你绘制一棵行为树，用来描述料箱抓取任务的行为。为了帮助你开始，我们在下面提供了一棵“草稿版”行为树，但它在多个地方是错误的（例如，某些分支可能缺失，某些控制流节点/动作/条件的类型可能不正确，等等）：

6. Andreas ten Pas and Marcus Gualtieri and Kate Saenko and Robert Platt, "Grasp pose detection in point clouds", *The International Journal of Robotics Research*, vol. 36, no. 13-14, pp. 1455--1473, 2017.
7. Jeffrey Mahler and Jacky Liang and Sherdil Niyaz and Michael Laskey and Richard Doan and Xinyu Liu and Juan Aparicio Ojea and Ken Goldberg, "Dex-net 2.0: Deep learning to plan robust grasps with synthetic point clouds and analytic grasp metrics", *arXiv preprint arXiv:1703.09312*, 2017.
8. Andy Zeng and Shuran Song and Kuan-Ting Yu and Elliott Donlon and Francois R Hogan and Maria Bauza and Daolin Ma and Orion Taylor and Melody Liu and Eudald Romo and others, "Robotic pick-and-place of novel objects in clutter with multi-affordance grasping and cross-domain image matching", *2018 IEEE international conference on robotics and automation (ICRA)*, pp. 1--8, 2018.
9. Craig M Shakarji, "Least-squares fitting algorithms of the NIST algorithm testing system", *Journal of research of the National Institute of Standards and Technology*, vol. 103, no. 6, pp. 633, 1998.
10. Radu Bogdan Rusu and Steve Cousins, "3D is here: Point Cloud Library (PCL)", *Proceedings of the IEEE International Conference on Robotics and Automation (ICRA)*, May 9-13, 2011.
11. Radu Bogdan Rusu, "Semantic 3D Object Maps for Everyday Manipulation in Human Living Environments", PhD thesis, Institut für Informatik der Technischen Universität München, 2010.
12. R. A. Brooks, "Elephants Don't Play Chess", *Robotics and Autonomous Systems*, vol. 6, pp. 3--15, 1990.
13. Rodney A. Brooks, "Intelligence Without Reason", , no. 1293, April, 1991.
14. Rodney A. Brooks, "Intelligence without representation", *Artificial Intelligence*, vol. 47, pp. 139-159, 1991.
15. Richard E Fikes and Nils J Nilsson, "STRIPS: A new approach to the application of theorem proving to problem solving", *Artificial intelligence*, vol. 2, no. 3-4, pp. 189--208, 1971.
16. Constructions Aeronautiques and Adele Howe and Craig Knoblock and ISI Drew McDermott and Ashwin Ram and Manuela Veloso and Daniel Weld and David Wilkins SRI and Anthony Barrett and Dave Christianson and others, "PDDL the planning domain definition language", *Technical Report, Tech. Rep.*, 1998.
17. Patrik Haslum and Nir Lipovetzky and Daniele Magazzeni and Christian Muise and Ronald Brachman and Francesca Rossi and Peter Stone, "An introduction to the planning domain definition language", Springer, vol. 13, 2019.
18. Avrim L Blum and Merrick L Furst, "Fast planning through planning graph analysis", *Artificial intelligence*, vol. 90, no. 1-2, pp. 281--300, 1997.
19. Blai Bonet and Héctor Geffner, "Planning as heuristic search", *Artificial Intelligence*, vol. 129, no. 1-2, pp. 5--33, 2001.
20. Jörg Hoffmann, "FF: The fast-forward planning system", *AI magazine*, vol. 22, no. 3, pp. 57--57, 2001.
21. Malte Helmert, "The fast downward planning system", *Journal of Artificial Intelligence Research*, vol. 26, pp. 191--246, 2006.
22. Jacky Liang and Wenlong Huang and Fei Xia and Peng Xu and Karol Hausman and Brian Ichter and Pete Florence and Andy Zeng, "Code as policies: Language model programs for embodied control", *2023 IEEE International Conference on Robotics and Automation (ICRA)*, pp. 9493--9500, 2023.
23. Shunyu Yao and Dian Yu and Jeffrey Zhao and Izhak Shafran and Thomas L Griffiths and Yuan Cao and Karthik Narasimhan, "Tree of thoughts: Deliberate problem solving with large language

models", *arXiv preprint arXiv:2305.10601*, 2023.

24. Michele Colledanchise and Petter Ogren, "Behavior trees in robotics and AI: An introduction", CRC Press , 2018.

CHAPTER 6

运动规划 (Motion Planning)

我们的工具箱中还缺少一些关键技能。在本章中，我们将探索一些强大的运动学轨迹运动规划方法。

事实上，直到现在的讲义才开始讲这个主题，我几乎都有点为自己感到骄傲了。就像我们在箱体抓取 (bin picking) 章节中所做的那样，为夹爪姿态编写一个相对简单的脚本，其实已经能够解决很多有趣的问题。但是，我们仍然有很多理由需要一种更加自动化的解决方案：

1. 当环境变得更加杂乱时，编写这样简单的解决方案会更加困难，我们还必须考虑机械臂与环境之间的碰撞，以及夹爪与环境之间的碰撞。
2. 如果我们在进行“移动操作 (mobile manipulation)”——即机械臂安装在移动底盘上——那么机器人可能需要在许多不同的环境中工作。即使工作空间在几何结构上并不复杂，每次到达目标时环境也可能有足够大的差异，从而需要自动化（即便可能仍然很简单）的规划。
3. 如果机器人整天都在一个已知且简单的环境中运行，那么优化它所执行的轨迹通常是有意义的；我们往往可以显著提升操作过程的速度。

事实上，如果你运行过[杂乱环境清理演示 \(clutter clearing demo\)](#)，我会认为运动规划失败是目前该方案最大的限制：手部或物体有时会与摄像头或箱体发生碰撞，或者微分逆运动学策略（它实际上忽略了关节角度）有时会导致机器人自身折叠缠绕。在本章中，我们将开发工具来大幅改善这一点！

不过我确实需要说明一个重要的保留意见。在操作任务中的运动规划领域，人们非常强调“避碰”问题。尽管我自己也在这个领域做过一些研究，但实际上我非常不喜欢“无碰撞运动规划”这种问题表述方式。我认为总体而言，机器人太害怕碰撞世界了（因为一旦碰撞，事情仍然常常会出错）。我不认为人类每次伸手时都在求解这些复杂的几何问题……即便是在非常拥挤杂乱的环境中伸手也是如此。我甚至怀疑，我们其实并不擅长求解这些问题。我更希望看到的是：即使只有非常粗糙 / 近似的穿越杂乱环境的规划，机器人依然能够表现良好；它们不害怕发生偶然接触，并且即使发生接触，仍然能够完成任务！

6.1 逆运动学 (INVERSE KINEMATICS)

本章的目标是求解运动轨迹。但我认为，如果你真正理解了如何求解逆运动学，那么你实际上已经掌握了进行轨迹规划所需的大部分内容。

我们知道，[正向运动学 \(forward kinematics\)](#) 给出了一个从关节角到例如夹爪位姿的（非线性）映射： $X^G = f_{kin}(q)$ 。因此，人们自然会认为，逆运动学 (IK) 的问题就是求解其逆映射，即 $q = f_{kin}^{-1}(X^G)$ 。但是，就像我们在微分逆运动学中所做的那样，我更倾向于把逆运动学看作一个更加通用的问题：即在丰富的代价函数与约束条件下寻找关节角。而运动学约束的可能形式实际上是非常丰富的。

例如，当我们在[为箱体抓取 \(bin picking\) 评估抓取候选 \(grasp candidates\)](#)时，我们对于手部相对于某个对跖抓取 (antipodal grasp) 的朝向只有一个软偏好 (soft preference)。在那种情况下，精确指定夹爪的 6 自由度位姿，并寻找一组能够完全满足该位姿的关节角，实际上会是一个约束过强 (overly constrained) 的描述。我会说，仅仅处理末端执行器位姿约束的情况其实很少见，我们几乎总是同时需要考虑关节空间中的代价或约束（例如关节限位），以及笛卡尔空间中的其它约束（例如非穿透约束）。

[点击此处观看视频。](#)

Figure 6.1 - 在我们关于人形机器人的研究中，我们大量使用了丰富的逆运动学规格描述（rich inverse kinematics specifications）。上面的视频展示了一个交互式逆运动学界面的示例（这里用于帮助我们研究如何让大型人形机器人进入小型 Polaris 车内）。[这里还有一个视频](#)，展示了一个工具在 Valkyrie 人形机器人上的应用，在那个例子中，我们确实指定了末端执行器位姿，但同时还加入了关节居中（joint-centering）目标以及静态稳定性约束 [1, 2]。

6.1.1 从末端执行器位姿到关节角

鉴于逆运动学在机器人学中的重要性，你可能不会惊讶于它拥有大量研究文献。但你可能也会惊讶于这些解法能够达到的广度和完整程度。正向运动学 f_{kin} 通常是一个非线性函数，但它是一个具有高度结构化特性的非线性函数。实际上，除了一些少见情况之外（例如你的机器人拥有一个螺旋关节（helical joint），也称为螺杆关节（screw joint）），描述机器人合法笛卡尔位置的方程实际上是多项式形式的。“可是等等！我的运动学方程里不是到处都有正弦和余弦吗？”你可能会这样问。三角函数项出现在我们试图关联关节角与笛卡尔坐标的时候。在 $\mathbb{R}^3$ 中，对于同一个刚体上的两个点 A 和 B ，它们之间的（平方）距离 $\|p^A - p^B\|^2$ ，是一个常数。而关节本质上只是相邻刚体之间的一种多项式约束，例如它们需要占据笛卡尔空间中的同一个点。关于这一点，可以参考 [3] 中极佳的综述。

理解多项式方程解的理论属于代数几何（algebraic geometry）的研究范畴。关于运动学理论、符号算法以及数值算法，已经存在非常深厚的文献积累。即使对于非常复杂的运动学拓扑结构，例如四杆机构（four-bar linkage）以及 Stewart-Gough 平台，我们也能够统计其解的数量，和 / 或理解其解所形成的连续流形。例如，[3] 描述了一套功能强大的数值代数几何工具箱（基于同伦方法（homotopy methods）），并在困难的运动学问题上展示了令人印象深刻的结果。

基于代数几何的运动学算法，传统上主要面向离线全局分析（offline global analysis），并且通常并不是为了控制回路中所需的快速实时逆运动学求解而设计的。如今，用于六自由度或七自由度机械臂实时逆运动学的最流行工具之一，是一个名为“IKFast”的工具，在 [4] 第 4.1 节中有所描述。它因其高效的开源实现而获得了广泛流行。IKFast 并不专注于求解的完备性（completeness），而是采用了若干近似方法，以便为“简单”运动学问题提供快速且数值鲁棒的解。它利用了这样一个事实：对于一个六自由度机械臂上的六自由度位姿约束，（对于大多数串联机械臂而言）存在“闭式解（closed-form）”，即只有有限数量的关节空间配置会产生相同的末端执行器位姿。而对于七自由度机械臂，它则在六自由度求解器的基础上，对最后一个自由度增加了一层采样过程。

这些显式解（explicit solutions）之所以重要，是因为它们能够帮助我们深入理解这些方程，同时它们也足够快，可以作为更复杂求解方法中的一个组成部分来使用。但是，这些解并不能立即提供我前面所倡导的那种丰富规格描述（rich specification）；特别是，一旦我们面对的是不等式约束而不是等式约束时，这些方法就会失效。因此，对于那些更加丰富的规格描述，我们将转向优化（optimization）方法。

6.1.2 将逆运动学视为约束优化（IK as constrained optimization）

与其将逆运动学表述为

$$q = f_{kin}^{-1}(X^G),$$

不如考虑将同一个问题作为一个优化问题来求解：

$$\min_q |q - q_0|^2, \quad (1)$$

$$\text{subject to } X^G = f_{kin}(q), \quad (2)$$

其中 q_0 是某个舒适的标称位置 (nominal position)。直接写出逆映射其实存在一些问题，特别是因为有时我们会多个（甚至无限多个）解，或者根本没有解。这种优化形式化会更加精确一些——如果有多组关节角能够实现相同的末端执行器位置，那么我们更倾向于选择那个最接近标称关节位置的解。但真正重要的是，将问题转变为优化视角之后，我们就能够连接到一整套丰富的附加代价函数与约束条件。

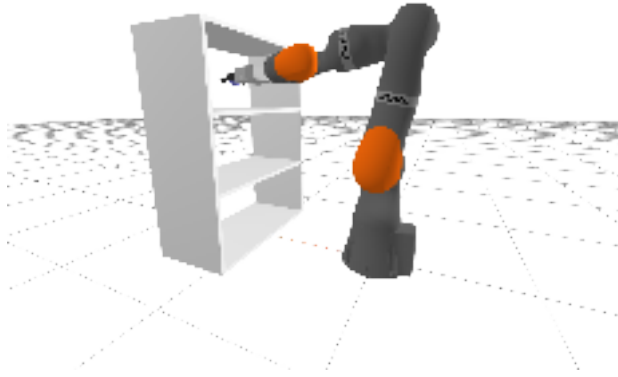


Figure 6.2 - 一个更丰富的逆运动学问题：求解关节角 q ，使机器人能够伸入货架中抓取目标物体，同时避免发生碰撞。

我们已经讨论过 将微分逆运动学 作为优化问题求解的思想。在那个工作流程中，我们一开始使用运动学雅可比矩阵 (kinematic Jacobian) 的伪逆，但随后进一步发展为从逆问题的最小二乘形式来思考问题。在更一般的最小二乘框架下，我们可以加入额外的代价函数和约束条件，以避免（近似）奇异雅可比矩阵带来的问题，同时还能考虑关节限位、关节速度限制等额外约束。我们甚至可以加入避碰约束。这些约束中的一些，关于配置变量 q 来说是高度非线性 / 非凸的函数，但在微分运动学场景中，我们只是在标称配置附近寻找一个小变化 Δq ，因此对这些非线性 / 非凸约束进行线性 / 凸近似是相当合理的。

现在，我们将考虑完整形式的问题，即直接求解这个非线性 / 非凸优化问题，而不再限制只能对初始 q 做小幅修改。从计算角度来看，这是一个困难得多的问题。使用像 SNOPT 这样强大的非线性优化求解器，我们通常仍然能够求得解，甚至能够达到交互式速度（下面的例子相当有趣）。但并不存在任何保证。即使求解器返回“不可行 (infeasible)”，也仍然可能存在真实可行解。

当然，微分逆运动学问题与完整逆运动学问题之间有着非常紧密的联系。实际上，你可以将微分逆运动学算法理解为：对完整非线性问题执行一步（投影）梯度下降，或执行一步 序列二次规划 (Sequential Quadratic Programming)。

Drake 提供了一个很方便的 InverseKinematics 类，使我们能够很容易地将许多标准的运动学 / 多体约束 组装到一个 MathematicalProgram 中。花一分钟看一下它所提供的约束。你可以添加两个物体之间相对位置和 / 或相对姿态的约束，或者要求两个物体之间的距离大于某个最小距离（例如用于非穿透约束），或者小于某个距离，等等。这正是我希望你思考 IK 问题的方式；它是一个逆问题，但这个逆问题可能包含非常丰富的代价函数和约束条件。

Example 6.1 (交互式 IK (Interactive IK))

尽管该问题具有非凸性，并且约束求值也具有不小的计算成本，但我们通常仍然可以以交互式速度求解它。我在本章 notebook 中整理了几个这样的例子：

在第一个版本中，我添加了滑块，让你控制末端执行器的期望位姿。这是 IK 问题的简单版本，适合使用更显式的解法；但我们仍然使用完整的非线性优化 IK 引擎来求解它（并且其中

确实包含关节限位约束)。这个演示看起来不会和讲义最开始的那个例子有太大不同: 在那个例子中, 你使用遥控操作 (teleop) 命令机器人拿起红色砖块。事实上, 微分 IK 也可以很好地解决这个问题。

在第二个例子中, 我试图通过在机器人正前方添加一个障碍物, 来突出非线性 IK 问题与微分 IK 问题之间的区别。由于我们的微分 IK 形式和 IK 形式都能够使用避碰约束, 两种解法都会试图防止机械臂撞到柱子上。但是, 如果你将目标末端执行器位置从柱子的一侧移动到另一侧, 完整 IK 求解器可以切换到一个新的解, 使机械臂位于柱子的另一侧; 而微分 IK 永远无法完成这种跳跃 (它会停留在柱子的第一侧, 拒绝允许发生碰撞)。

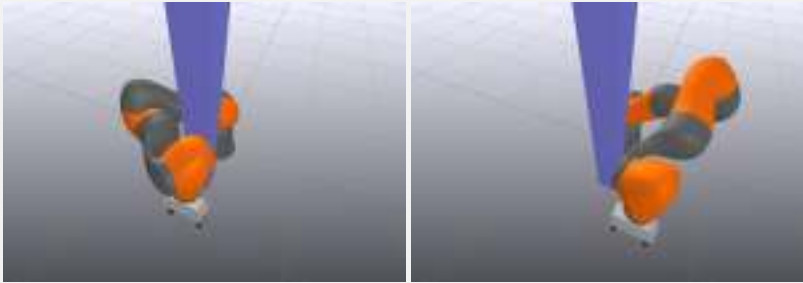


Figure 6.3 - 随着期望末端执行器位置沿正 y 方向移动, IK 求解器能够找到一个新的解, 使机械臂从柱子的另一侧绕过去。

能力越大, 责任越大。逆运动学工具箱允许你构建复杂的优化问题, 但你能否成功求解它们, 部分取决于你在选择代价函数和约束条件时是否足够谨慎。我最基本的建议如下:

1. 尽量保持目标函数 (代价函数) 简单; 我通常只会在 q 上使用“关节居中 (joint-centering)”二次代价。把本该作为约束的项以惩罚项 (penalty) 的形式加入代价函数, 往往会导致大量代价函数调参工作, 而这通常会变成一件相当棘手的事情。
2. 编写最小化约束 (minimal constraints)。你希望可行配置集合尽可能大。例如, 如果你并不需要完全约束夹爪的姿态方向, 那就不要这样做。

我会通过下面的例子进一步说明第二点。

Example 6.2 (抓取圆柱体 (Grasp the cylinder))

让我们使用 IK 来抓取一个圆柱体。如果你愿意, 也可以把它想象成一个扶手 (hand rail)。假设我们并不关心沿圆柱体哪个位置进行抓取, 也不关心以什么方向去抓取它。那么我们就应该只使用这些约束的最小形式来构建 IK 问题。

在 notebook 中, 我已经实现了其中一种方案。我现在把圆柱体的位姿放在滑块上, 因此你可以在工作空间中移动它, 并观察 IK 求解器如何决定机器人的姿态。特别地, 如果你沿 $\pm y$ 方向移动圆柱体, 你会发现机器人一开始并不会跟随..... 直到手部移动到圆柱体末端。非常漂亮!



我们可以想象出多种实现该约束的方法。以下是我的实现方式:

```

ik.AddPositionConstraint(
    frameB=gripper_frame, p_BQ=[0, 0.1, -0.02],
    frameA=cylinder_frame, p_AQ_lower=[0, 0, -0.5], p_AQ_upper=[0, 0, 0.5])
ik.AddPositionConstraint(
    frameB=gripper_frame, p_BQ=[0, 0.1, 0.02],
    frameA=cylinder_frame, p_AQ_lower=[0, 0, -0.5], p_AQ_upper=[0, 0, 0.5])

```

换句话说，我在夹爪坐标系中定义了两个点；在 `AddPositionConstraint` 方法的记号中，它们是 Bp^Q 。回忆一下，夹爪坐标系 (gripper frame) 的定义方式是：[0, .1, 0] 位于两个夹爪垫之间的正中央；你应该花一点时间确认自己真正理解 [0, .1, -0.02] 和 [0, .1, 0.02] 分别位于哪里。我们的约束要求这两个点都必须精确落在圆柱体的中心线段上。这种方式能够简洁地保留绕圆柱轴方向的姿态为“不受约束”，同时又很好地表达了圆柱位置约束。

我们已经提供了一套丰富的约束语言，用于描述 IK 问题，其中包括许多涉及机器人运动学以及机器人与环境几何关系的约束（例如最小距离约束）。现在让我们花一点时间，来欣赏一下我们要求优化器去求解的这个几何谜题。

Example 6.3 (可视化配置空间 (Visualizing the configuration space))

让我们回到 iiwa 机械臂伸入货架的例子。这个 IK 问题有两个主要约束：1) 我们希望目标球体的中心位于夹爪中心；2) 我们希望机械臂避免与货架发生碰撞。为了能够绘制这些约束，我固定了 iiwa 的三个关节，只保留了对应于 $x-z$ 平面运动的三个自由度。



Figure 6.5 - 右图展示了配置空间 (configuration space) 中“抓取球体”约束的可视化结果——可视化器中的 x 、 y 、 z 坐标轴 分别对应于平面化 iiwa 的三个关节角。

为了可视化这些约束，我在平面化 iiwa 的三个关节角空间中采样了一个稠密网格，如果某个网格点满足约束则赋值为 1，否则赋值为 0，然后运行 marching cubes (行进立方体) 算法，从而提取出该约束在配置空间中真实三维几何形状的近似表示。“抓取球体”约束生成了上图右侧所示的漂亮绿色几何体；它会被关节限位裁剪。而另一方面，避碰约束则复杂得多。想真正看清这一点，你最好打开这个 [3D 可视化](#)，自己在里面旋转观察。很可怕吧！

Example 6.4 (可视化 IK 优化问题 (Visualizing the IK optimization problem))

为了帮助你更好地理解我们所构建的问题，我制作了一个优化景观 (optimization landscape) 的可视化。先看看这个 [景观图](#)；这里只绘制了返回解附近的一小块区域。你可以使用 Meshcat 控件来显示 / 隐藏各个单独的代价函数与约束，以确保你真正理解它们。

正如我所建议的那样， 我将代价景观（即 [绿色曲面](#)） 保持为简单的二次关节居中代价

(quadratic joint-centering cost)。约束则以蓝色表示可行区域，以红色表示不可行区域：

- 关节限位约束在这里仅仅是一个简单的“边界框 (bounding-box)”约束（对于边界框约束，为了避免可视化过于杂乱，这里只绘制了红色的不可行区域）。
- 位置约束包含三个组成部分：分别对应末端执行器的 x 、 y 和 z 位置。其中， y 方向的位置约束会被平凡满足（全部为蓝色），因为该机械臂只保留了在 $x-z$ 平面内运动的关节。另外两个看起来几乎全是红色，但如果你关闭 y 方向的可视化，就会看到其中各自存在两条很窄的蓝色带状区域。这正对应于我们严格位置约束的满足条件。
- 但真正最令人印象深刻 / 可怕的，还是“最小距离 (minimum-distance)”（即非碰撞）约束。虽然我们之前已经对配置空间进行了可视化，但在这里，你可以看到：将距离作为一个实值函数 (real-valued function) 进行可视化，会直接揭示出我们提供给优化器的优化景观。

这里所有蓝色区域的交集，就定义了前面示例中的配置空间。这个可视化的全部代码都可以在 notebook 中找到，你也可以在那里查看代价函数与约束条件的精确形式：

你还应该快速看一下 [完整优化景观 \(full optimization landscape\)](#)。这里绘制的是与上面可视化相同的一组曲线，但现在它们覆盖了关节限位范围内的整个关节角定义域。像 SNOPT 这样的非线性优化器，有时候真的会令人惊叹！

6.1.3 全局逆运动学 (Global inverse kinematics)

对于没有约束且恰好具有六个自由度的逆运动学问题，我们拥有闭式解 (closed-form solutions)。而对于带有丰富代价函数与约束条件的广义逆运动学问题，我们得到的是一个在实践中表现良好、但会受到局部极小值 (local minima) 影响的非线性优化问题（因此即使存在可行解，它也可能无法找到）。如果我们放弃以交互式速度求解该优化问题，那么是否仍然有希望能够鲁棒地求解这种更丰富的 IK 形式？理想情况下，是否甚至能够达到全局最优？

这实际上是一个极其重要的问题。许多逆运动学应用都是离线运行的，并没有严格的实时性要求。想象一下，如果你想生成训练数据，用于训练一个学习逆运动学的神经网络；那么这将是全局 IK 的一个完美应用场景。又或者，如果你想进行工作空间分析 (workspace analysis)，以验证机器人是否能够到达你为其设计的工作空间中的所有目标位置，那么你也会希望使用全局 IK。我们之后将要研究的一些运动规划策略，也会将其计算过程划分为一个离线“构建 (building)”阶段，从而使在线“查询 (query)”阶段变得快得多。

根据我的经验，通用型全局非线性求解器——例如基于混合整数非线性规划 (mixed-integer nonlinear programming, MINLP) 的方法，或者使用 [可满足性模理论 \(satisfiability modulo theories, SMT\)](#) 求解器中的区间算术方法——通常无法扩展到完整机械臂问题的复杂度。但如果我们只是稍微限制所支持的代价函数与约束条件类别，那么就可以开始取得进展。

Drake 提供了 [5] 中描述的 [GlobalInverseKinematics](#) 方法的实现，它使用混合整数凸优化 (mixed-integer convex optimization)。其求解时间大约在几秒量级；它可以在远小于一分钟的时间内求解一个完整的受约束双臂问题。[6] 使用通过半正定规划松弛层级 (hierarchy of semi-definite programming relaxations) 实现的凸优化，来求解该问题的窄版本（只包含末端执行器位姿和关节限位）。理解这种方法在更大范围的代价函数与约束条件族上表现如何，将会非常有趣。

6.1.4 逆运动学与微分逆运动学 (Inverse kinematics vs differential inverse kinematics)

什么时候应该使用 IK，什么时候应该使用微分 IK？IK 求解的是一个更全局的问题，但并不保证一定成功。它也不保证当你对代价函数 / 约束条件做小幅改变时， q 只会发生小幅变化；因此你最终可能会向机器人发送很大的 Δq 命令。当你需要求解更全局的问题时，可以使用 IK；而我们将在下一节中构建的轨迹优化算法，则是生成实际 q 轨迹的自然扩展。微分 IK 对于增量式运动非常有效——例如，如果你能够在末端执行器空间中设计平滑命令，并且只需要在关节空间中对其进行跟踪。

6.1.5 使用逆运动学进行抓取规划 (Grasp planning using inverse kinematics)

在我们第一个版本的基于采样的 [抓取选择 \(grasp selection\)](#) 中，我们设置了一个目标函数，用于奖励那些手部从物体上方进行抓取的朝向。这其实只是我们真正想要解决的问题的一个（有时并不理想的）替代指标：我们希望抓取在机器人处于“舒适”姿态时是可实现的。因此，对我们的抓取评分指标进行一个简单而自然的扩展，就是为每个抓取候选求解一个逆运动学问题；并且不再对末端执行器朝向设置代价，而是直接使用关节居中代价作为目标函数。此外，如果 IK 问题返回不可行，我们就应该拒绝该样本。

一旦我们采用优化视角，这个想法还可以得到一个非常漂亮的扩展，并且它也是通向下一节轨迹规划的自然过渡。想象一下，如果任务不仅要求我们在杂乱环境中抓起物体，还要求我们同样在杂乱环境中小心地放置物体。在这种情况下，一个好的抓取方式应该对应于手相对于物体的某个位姿，并且这个位姿能够 **同时优化** 抓取配置 (pick configuration) 和放置配置 (place configuration)。我们可以构建一个优化问题，其决策变量同时包括 q_{pick} 和 q_{place} ，并加入约束，强制满足 ${}^O X^{G_{pick}} = {}^O X^{G_{place}}$ 。当然，我们依然可以加入所有其它丰富的代价函数与约束条件。在 [TRI 的餐具装载项目 \(dish-loading project\)](#) 中，这种方法被证明非常重要。因为无论是在水槽中抓起杯子，还是将其放入洗碗机架中，都存在很强的约束，因此我们需要这种联合优化 (simultaneous optimization) 才能找到成功的抓取方式。

6.2 运动学轨迹优化 (KINEMATIC TRAJECTORY OPTIMIZATION)

一旦你理解了逆运动学的优化视角，那么你其实已经走在理解运动学轨迹优化的正确道路上了。基本思想是：不再独立地求解多个逆运动学问题，而是在一次优化中同时求解一系列关节角。更进一步，让我们定义一个参数化的关节轨迹 $q_\alpha(t)$ ，其中 α 是一个参数向量。那么，对逆运动学问题的一个简单扩展就可以写成如下形式：

$$\begin{aligned} \min_{\alpha, T} \quad & T, & (3) \\ \text{subject to} \quad & X^{G_{start}} = f_{kin}(q_\alpha(0)), & (4) \\ & X^{G_{goal}} = f_{kin}(q_\alpha(T)), & (5) \\ & \forall t, \quad |\dot{q}_\alpha(t)| \leq v_{max}. & (6) \end{aligned}$$

我会将其理解为：“寻找一条轨迹 $q_\alpha(t)$ ，其中 $t \in [0, T]$ ，使夹爪能够以最短时间从起始位置移动到目标位置。”

最后一个方程 (6) 表示速度限制；这是我们告诉优化器：机器人无法瞬间从起点“传送”到终点的唯一方式。除了这一行看起来稍微有些不太标准之外，它几乎和联合求解两个逆运动学问题完全一样；只不过现在，求解器不再对 q 求梯度，而是对 α 求梯度。这一点可以很容易地通过链式法则 (chain rule) 实现。

6.2.1 轨迹参数化 (Trajectory parameterizations)

于是，一个有趣的问题就变成了：我们究竟应该如何使用参数向量 α 来参数化轨迹 $q(t)$ ？如今，你可能会想到： $q_\alpha(t)$ 可以是一个神经网络，其输入为 t ，输出为 q ，而 α 则表示网络的权重与偏置。当然你完全可以这么做，但对于这种只有标量输入的问题，我们通常会采用更加简单且更加稀疏的参数化方式，往往基于多项式。

有许多方法可以用多项式来参数化一条轨迹。例如，在动态运动规划中，[直接配点法 \(direct collocation methods\)](#) 使用 [分段三次多项式 \(piecewise-cubic polynomials\)](#) 来表示状态轨迹，而 [伪谱方法 \(pseudo-spectral methods\)](#) 则使用拉格朗日多项式 (Lagrange polynomials)。在每一种情况下，基函数的选择都是为了让算法能够利用该基函数的某种特定性质。在动态运动规划中，很大一部分重点放在动力学方程的积分精度上，以确保我们能够获得满足动力学约束的可行解。

当我们规划全驱动机械臂的运动时，通常不会那么担心动力学可行性，而是更关注运动学。对于[运动学](#)轨迹优化，所谓的 [B 样条轨迹 \(B-spline trajectory\)](#) 参数化在这里具有一些特别好的性质可供利用：

- B 样条的导数仍然是 B 样条（次数降低一阶），并且其系数是原始系数的线性函数。
- 这些基函数本身是非负且稀疏的。这使得 B 样条多项式的系数（被称为[控制点 \(control points\)](#)）具有很强的几何解释。
- 特别地，整条轨迹都保证位于活动控制点的凸包内（活动控制点是指其基函数不为零的那些控制点）。

综合来看，这意味着我们可以对有限参数化的轨迹进行优化，同时利用凸包性质，通过[线性](#)约束来确保关节位置及其任意导数的限制对所有 $\forall t \in [0, T]$ 都成立。使用大多数其它多项式基来获得这种保证，代价通常会高得多。

注意，[B 样条 \(B-splines\)](#) 与 [Bézier 曲线 \(Bézier curves\)](#) 密切相关。但“B-spline”中的“B”实际上只是代表“basis (基)”（不，我不是在开玩笑），而“[Bézier splines](#)”则略有不同。

6.2.2 优化算法 (Optimization algorithms)

Drake 中默认的 [KinematicTrajectoryOptimization](#) 类会优化一条使用 B 样条定义的轨迹，该 B 样条用于表示区间 $s \in [0, 1]$ 上的一条路径 $r(s)$ ，另外还有一个标量决策变量，对应于轨迹持续时间 T 。最终轨迹将路径与时间重标定 (time-rescaling) 结合起来： $q(t) = r(t/T)$ 。这是一种特别适合表示未知持续时间轨迹的方式，并且具有一个非常优秀的特性：仍然可以使用凸包性质。速度约束仍然是线性的；对加速度以及更高阶导数的约束会变成非线性的，但如果它们被满足，仍然意味着对所有 $\forall t \in [0, T]$ 都存在严格界限。

由于 [KinematicTrajectoryOptimization](#) 是使用 Drake 的 [MathematicalProgram](#) 编写的，因此默认情况下，它会根据当前可用的求解器，自动选择我们认为最合适的求解器。如果该优化问题只包含凸代价函数和凸约束，它就会被分派给一个凸优化求解器。但大多数时候，我们会加入来自运动学的非凸 [代价函数与约束](#)。因此在大多数情况下，默认求解器仍然是 SQP 求解器 SNOPT。你也可以自由尝试其它求解器！

我们可以添加到运动学轨迹优化问题中的一组最有趣约束之一，是 [MinimumDistanceLowerBoundConstraint](#)；当所有潜在碰撞对之间的最小距离都大于零时，我们就避免了碰撞。不过请注意，这些碰撞约束只能在路径上的离散采样点 $s_i \in [0, 1]$ 处被施加。[它们并不能保证轨迹在所有 \$\forall t \in \[0, T\]\$ 上都是无碰撞的](#)。要利用凸包性质，需要导数约束具有非常特殊的性质；对于更一般的非线性约束，我们并不具备这些性质。一种常见策略是：在优化过程中，在区间上数量适中的若干采样点处添加约束；然后在将优化后的轨迹真正发送给机器人执行之前，对该轨迹进行更密集的碰撞检查。

Example 6.5 (用于在货架之间移动的运动学轨迹优化 (Kinematic trajectory optimization for moving between shelves))

作为热身，我提供了一个简单示例：平面 iiwa 机械臂从顶部货架伸到中间货架。

如果你仔细查看代码，会发现我实际上不得不求解这个轨迹优化问题两次，才让 SNOPT 返回一个好的解（遗憾的是，由于它是局部优化方法，这种情况确实可能发生）。对于这个特定问题，有效的策略是：先在没有避碰约束的情况下求解一次，然后将得到的轨迹作为带避碰约束问题的初始猜测。

代码中另一个值得注意的地方，是我用来在优化器求解过程中为轨迹绘制一条蓝色小线的“可视化回调（visualization callback）”。可视化回调可以这样实现：例如，告诉求解器存在一个依赖于所有变量的代价函数，但它总是返回零代价；这样每当求解器评估代价函数时，它都会被调用。我在这里所做的事情确实可能拖慢优化过程，但这是获得直觉的绝佳方式，可以帮助你判断求解器什么时候正在“艰难地”进行数值求解。我认为，在这个领域中，那些拥有最快、最鲁棒求解器的人 / 论文，与那些愿意花时间可视化并仔细调理求解器数值行为的人，二者之间高度相关。

Example 6.6 (用于杂乱环境清理的运动学轨迹优化 (Kinematic trajectory optimization for clutter clearing))

我们也可以使用 `KinematicTrajectoryOptimization` 来为杂乱环境清理示例进行规划。这个优化过程更加鲁棒，并且不需要求解两次。我只用一条平凡的初始轨迹作为种子，以避免机器人试图绕其基座旋转 270 度，而不是选择更短路径的解。

在运动规划文献中，有许多与运动学轨迹优化相关的方法，它们的区别可能在于参数化方式，也可能在于求解技术，或者二者皆有。其中一些较为知名的方法包括 CHOMP [7]、STOMP [8] 以及 KOMO[9]。

例如，KOMO 是少数几种使用 增广拉格朗日方法 (Augmented Lagrangian method) 的轨迹优化技术之一。这种方法将一个带约束优化问题转写为一个无约束问题，然后使用简单但快速的基于梯度的求解器[10]。基于增广拉格朗日的方法现在似乎是最受欢迎且最成功的一类方法；我希望不久之后能在 Drake 中提供一个不错的实现！但在比较这些不同求解器时必须小心——如果 SNOPT 无法将代价函数优化到位并使约束满足默认容差（约为 $1e-6$ ），它可能会声明失败；而增广拉格朗日方法永远不会报告失败，并且也很少会以这种精度水平为目标。

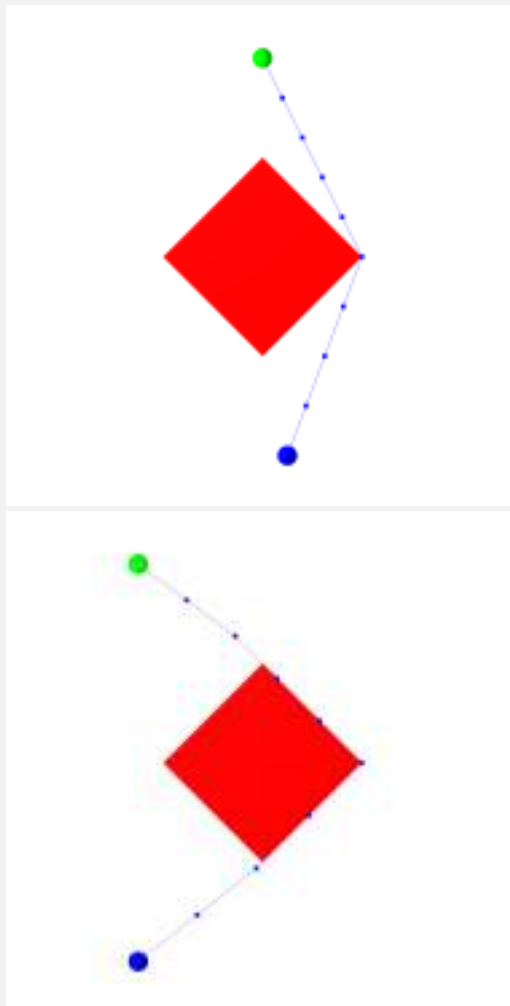
当运动学轨迹优化成功时，它们是运动规划问题中一种令人非常满意的解决方案。它们提供了一种非常自然且富有表达力的语言，用于描述期望运动（也许唯一的例外是需要对非线性约束进行采样），并且它们的求解速度足够快，可以用于在线规划。唯一的问题是：它们并不总是成功。因为它们基于非凸优化，所以容易受到局部极小值的影响，即使存在可行路径，也可能无法找到它。

Example 6.7 (无碰撞轨迹优化中的局部极小值 (Local minima in collision-free trajectory optimization))

考虑一个极其简单的例子：为一个点机器人寻找从起点（蓝色圆圈）到目标点（绿色圆圈）的最短路径，如下图所示，同时避免与红色方框发生碰撞。甚至先不考虑 B 样条的复杂性，我们可以写出如下形式的优化问题：

$$\begin{aligned}
& \min_{q_0, \dots, q_N} \sum_{n=0}^{N-1} |q_{n+1} - q_n|_2^2 \\
& \text{subject to } q_0 = q_{start} \\
& \quad q_N = q_{goal} \\
& \quad |q_n|_1 \geq 1 \quad \forall n,
\end{aligned}$$

其中最后一行是避碰约束，表示每个采样点都必须位于 ℓ_1 球之外（这是我为红色方框几何形状所做的一个方便选择）。或者，我们也可以写出一个稍微更高级的约束，用于保证每一条线段都位于障碍物之外（而不仅仅是各个顶点）。下面是这个优化问题的一些可能解：



左侧的解是该问题的（全局）最小值；右侧的解显然是一个局部极小值。一旦非线性求解器开始考虑从障碍物右侧绕行的路径，它就很不可能再找到一条从障碍物左侧绕行的解，因为在变得更好之前，该解必须先变得更差（即违反碰撞约束）。

为了解决这一限制，无碰撞运动规划领域已经明显转向了基于采样的方法。

6.3 基于采样的运动规划（SAMPLING-BASED MOTION PLANNING）

我的目标是尽快完成本节的完整草稿！在此期间，我强烈推荐 Steve LaValle 的书[11]，以及查看

开放运动规划库 (Open Motion Planning Library)。我们在 Drake 中已经实现了最常见的 基于采样的算法，并针对我们的碰撞引擎进行了优化，而且[希望很快将它们公开发布](#)。

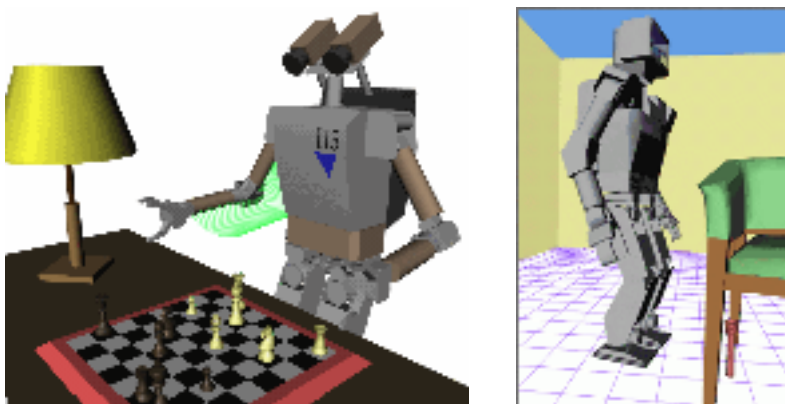


Figure 6.7 - 一些令人难以置信的早期（大约 2002 年）基于采样的运动规划成果，来自 [James Kuffner](#)（将鼠标悬停在图片上可播放动画）。这些问题在运动学上非常复杂，并且具有相当高的维度。

6.3.1 快速扩展随机树 (Rapidly-exploring random trees, RRT)

Example 6.8 (基础 RRT (The basic RRT))

Example 6.9 (RRT “Bug Trap (虫洞陷阱) ”)

6.3.2 概率路网 (Probabilistic Roadmap, PRM)

6.3.3 后处理 (Post-processing)

捷径优化 (Short-cutting)，[anytime B 样条平滑器 \(b-spline smoother\)](#)

6.3.4 实践中的基于采样的规划 (Sampling-based planning in practice)

有许多优化技巧与启发式方法 能够显著提升这些方法的效果……（优化后的碰撞检测、加权欧氏距离等……）

6.4 基于凸集图 (GRAPHS OF CONVEX SETS, GCS) 的运动规划

轨迹优化技术允许我们以丰富的方式描述代价函数与约束条件，包括连续曲线上的导数约束，并且能够扩展到高维配置空间；但它们本质上仍然依赖于局部（基于梯度）的优化，因而会受到局部极小值问题的影响。基于采样的规划器则从更全局的角度进行推理，并提供了一种（概率意义上的）完备性概念，但它们难以处理连续曲线上的导数约束，并且在高维空间中也会遇到困难。那么，是否存在一种方法，能够兼顾两者的优点呢？

我和我的学生一直在研究一种新的运动规划方法，试图弥合这一差距。它建立在一个通用优化框

架之上，该框架用于混合连续优化与组合优化（例如图上的组合优化），我们称之为“凸集图（Graphs of Convex Sets, GCS）”。

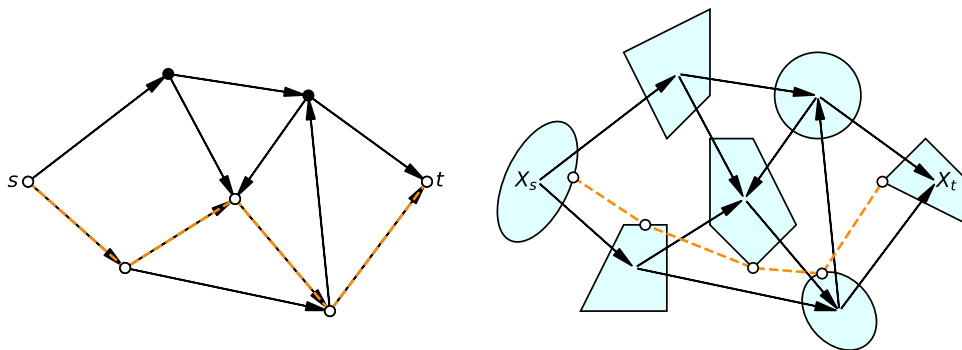
为了说明它的动机，让我们思考一下为什么 PRM 难以处理连续曲率约束。在路网构建阶段，我们会使用碰撞检测器做大量工作，以证明连接两个顶点的直线段是无碰撞的（通常只是在某种采样分辨率下成立，尽管也可以做得更好 [12]）。然后，在在线查询阶段，我们搜索离散图，以找到一条从起点到目标点的（不连续的）分段线性最短路径。一旦我们去平滑这条路径，或者用一条连续曲线来近似它，就无法保证这条路径仍然无碰撞，当然它也未必仍然是最短路径。因为我们的离线路网构建只考虑了线段，所以它没有为在线优化留下任何空间。当我们引入动力学时，这一点会变得更加令人担忧——在运动学上可行的路径，一旦加入动力学约束后可能就不再可行。虽然基于采样的规划 [可以被改造](#) 用于这些所谓的“运动-动力学规划（kino-dynamic planning）”或“基于控制的规划（control-based planning）”问题，但由于若干 [微妙原因](#)，这些算法并没有像纯运动学的基于采样规划版本那样成功。

对 PRM 工作流程做一个相对较小的改动是：每当我们选取一个样本时，不再只是把该配置空间点加入图中，而是将这个点扩展为配置空间中的一个（凸）区域。我们愿意付出一些离线计算成本，来寻找足够好的大区域，因为这会在在线查询阶段带来显著收益。首先，我们可以构建稀疏得多的图——少量区域就可以覆盖配置空间中的很大一部分——这也使我们能够扩展到更高维度。但也许更重要的是，现在我们有在线优化连续曲线的灵活性，只要这些曲线始终保持在凸区域内部即可。这需要对图搜索算法进行推广，使其能够联合优化穿过图的离散路径以及连续曲线的参数；而这正是 GCS 所提供的推广。

6.4.1 凸集图（Graphs of Convex Sets）

GCS 框架最初由 [13] 提出，并且我们已经在 [Drake 中实现了成熟版本](#)。它是一个通用优化框架，用于将组合优化（例如图搜索）与连续优化（例如我们前面研究的运动学轨迹优化）结合起来。GCS 可以为任何“网络流（network flow）”优化提供连续化扩展（参见 [14]），但对于运动规划而言，我们首先感兴趣的问题是图上的最短路径问题。

考虑一个非常经典的问题：在图中寻找从源点 s 到目标点 t 的（带权）最短路径，如左图所示。这个问题由一组顶点、一组（可能有向的）边，以及遍历每条边的代价所描述。



GCS 为该问题提供了一个简单但强大的推广：每当我们访问图中的一个顶点时，我们还可以从与该顶点相关联的一个凸集中选择一个元素。边的长度允许成为对应集合中连续变量的凸函数，同时我们也可以在边上编写凸约束（这些约束必须由任何可行路径满足）。

凸集图上的最短路径问题可以编码 NP-Hard 问题，但它们可以被表述为混合整数凸优化（mixed-integer convex optimization, MICP）。使这个框架如此强大的原因在于：这个 MICP 拥有一个非常强且高效的 [凸松弛（convex relaxation）](#)；也就是说，如果你将二值变量放松为连续变量，那么你得到的将会是（几乎）原始 MICP 的解。这意味着，相比以往的转写方式，你可以以快几个数量级的速度将 GCS 问题求解到全局最优。但在实践中，我们发现 [只求解凸松弛](#)（再加上一点

rounding) 几乎总是足以恢复最优解。如今, 在我们基于 GCS 的机器人应用中, 几乎从不再求解完整的 MICP。

你可以在我的 [欠驱动系统讲义 \(underactuated notes\)](#) 中找到更多关于 GCS 的细节与示例, 并且在 [15] 中, 可以看到对 GCS 作为一种通用混合离散-连续优化工具的非常完整且易读的论述。

6.4.2 GCS (运动学) 轨迹优化 (GCS (Kinematic) Trajectory Optimization)

GCS 提供了一套非常通用的机制, 用于优化那些自然表示在图上的混合离散 / 连续优化问题。但是, 我们该如何将运动规划问题转写为一个 GCS 问题呢? 我们最早在题为“Motion planning around obstacles with convex optimization”的论文中提出了这里将要描述的这种转写方式 [16]。既然我们在本章前面已经花了不少时间讨论运动规划问题固有的非凸性, 你应该能理解为什么我们喜欢这个标题! 我们竟然能够使用凸优化来有效地将这些问题求解到全局最优, 这一点应该会让人略感惊讶。

让我们暂时假设, 我们已经得到了无碰撞空间的一个凸分解。这是一个很强的假设, 我们将在后面解释它的合理性。

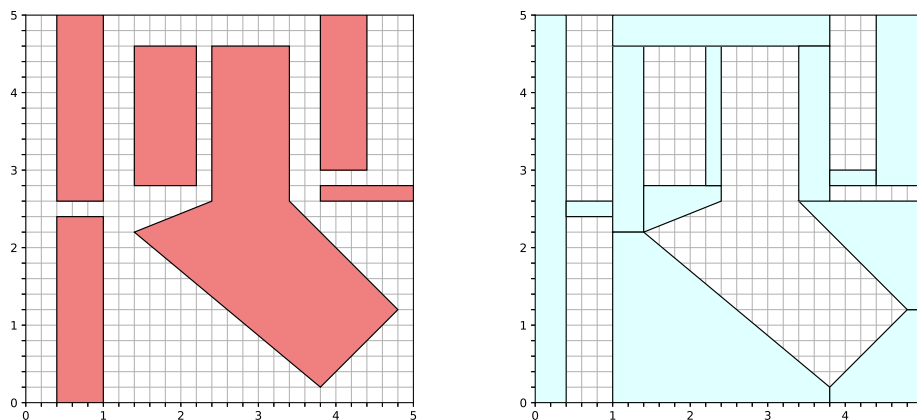


Figure 6.9 - **左图**: 一个简单的二维配置空间, 其中红色区域表示障碍物。 **右图**: 无碰撞配置空间的一个凸分解。

注意, 如果我们有俩个点位于同一个凸的无碰撞区域内, 那么连接它们的直线也一定保证无碰撞。当点位于两个 C 空间区域的交集中时, 它们就可以将穿过多个区域的连续路径连接起来。这就是我们这种转写方法的核心。这意味着, 对于 GCS 中每一次访问某个顶点, 我们都希望在一个 C 空间区域中选择 **两个点**, 从而使路径位于该区域内。因此, GCS 中的凸集并不是 C 空间区域本身, 而是这样一个集合: 对于一个 n 维配置空间, 它包含 $2n$ 个变量, 其中前 n 个变量和后 n 个变量都位于该 C 空间区域中。用集合记号来说, 它就是该集合与自身的笛卡尔积。当且仅当两个 C 空间区域相交时, 我们在这些集合之间形成无向边; 并且我们在每条边上加入一个约束, 要求第一个集合中的第二个点必须等于第二个集合中的第一个点。

Example 6.10 (简单二维配置空间中的 GcsTrajOpt (GcsTrajOpt in a simple 2D configuration space))

让我们在上图所示的简单配置空间中运行 GCS 轨迹优化。我把起点放在左下角, 把目标点放

在右上角。我会运行该算法两次：一次使用最小距离目标函数，一次使用最小时间目标函数（并加入速度约束）：

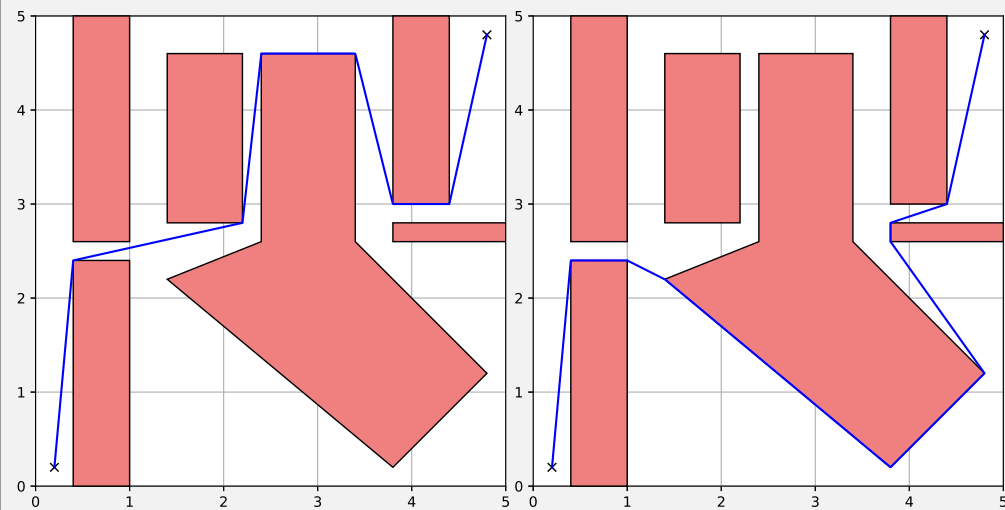


Figure 6.10 - 左图：带有最小距离代价的 GcsTrajOpt 解。右图：带有最小时间代价（以及速度约束）的 GcsTrajOpt 解。

我们可以对这一方法进行显著推广。不再只是在每个集合中放置一条线段，我们可以利用 Bezier 曲线的凸性质，来保证一整条固定次数的连续路径都位于该集合内部。利用 Bezier 曲线的导数性质，我们可以对速度添加凸约束（例如速度限制），并保证即使在区域交界处，曲线也依然是光滑的。不过，为了定义这些速度，我们需要知道沿路径前进的速率信息，因此我们也引入了时间缩放参数化，很像我们在前面的运动学轨迹优化中所做的那样，只不过这里是在每个凸集中引入时间缩放参数。

最终得到的是一个相当丰富的轨迹优化框架，它在 GCS 凸优化框架中编码了许多（当然不可否认，并不是全部）我们希望用于轨迹优化的代价函数与约束条件。例如，我们可以编写包含以下内容的目标函数：

- 时间（轨迹持续时间），
- 路径长度（用一个上界来近似），
- 路径“能量”的一个上界（例如 $\int |\dot{q}(t)|_2^2$ ），

以及包含以下内容的约束：

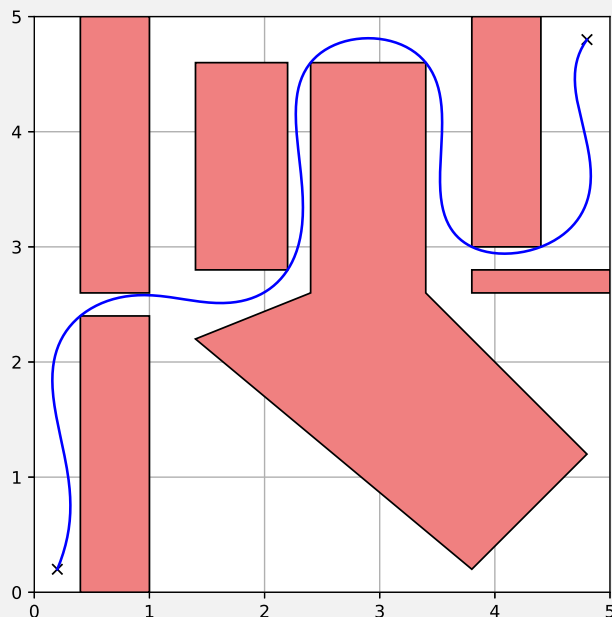
- 路径导数连续性（可达到任意阶），
- 速度约束（对所有 t 都严格施加，而不仅仅是在采样点处），
- 额外的凸位置约束和 / 或路径导数约束，例如初始与最终位置及速度。

我们会自动施加路径连续性约束，以及区域约束（即整条轨迹始终位于 C 空间区域并集内部），并且这些约束保证对所有 t 成立，而不仅仅是在采样点处成立。你可以在这里找到 [Drake 中的实现](#)。

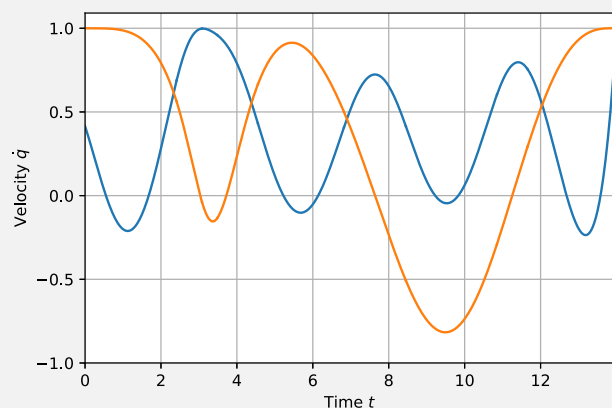
Example 6.11 (简单二维配置空间中的 GcsTrajOpt (续) (GcsTrajOpt in a simple 2D configuration space (cont.)))

这是运行与上一个例子几乎相同的最小时间优化所得到的结果，但这一次，我们用五阶 Bezier

曲线和时间缩放 来参数化每一段轨迹，并要求连续性一直达到四阶导数（“snap”）。



检查轨迹导数可以发现，速度是光滑的，并且在~~整条~~轨迹上都保持在速度界限 $[-1, 1]$ 以内；而不仅仅是在采样点处满足。



当然，非凸轨迹优化能够处理更丰富的代价函数与约束条件，但它不具备 GCS 所能提供的全局优化与完备性要素。我们正在不断向 GCS 轨迹优化框架中加入更多代价函数与约束条件——一些看似非凸的约束实际上具有精确的凸重构形式，另一些则具有不错的凸近似。这自然导致了两种技术的融合：我们可以在 GCS 凸松弛中使用更复杂代价函数与约束条件的凸近似，但一旦 GCS 选定了穿过图的离散路径，并给出了一个强初始猜测，就可以在 rounding 步骤中使用非线性优化，精确处理非凸性 [17]。在 Drake 中，这可以通过 GCS 的 `AddCost` 和 `AddConstraint` 方法中的 `use_in_transcription` 选项来实现，并且只将非凸约束添加到 `Transcription::kRestriction` 中。

6.4.3 （无碰撞）配置空间的凸分解（Convex decomposition of (collision-free) configuration space）

好吧，那么我们如何获得无碰撞配置空间的凸分解呢？让我们先认真对待与 PRM 的类比。如果我们在配置空间中采样到一个无碰撞点，那么将这个点膨胀成一个凸区域的正确方式是什么呢？

当我们处理凸几何时，答案会更简单。在我看来，用欧氏空间 $\mathbb{R}^3$ 中的凸形状来近似机器人的几何形状是合理的。尽管许多机器人（例如 iiwa）具有非凸的网格几何，但我们通常可以将它们很好地近似为更简单凸几何体的并集；有时这些凸几何体是圆柱体、球体等基本几何体，另一种做法是直接对网格进行凸分解。但是，即使这些几何体在欧氏空间中是凸的，通常也 **不合理** 将它们视为配置空间中的凸集。

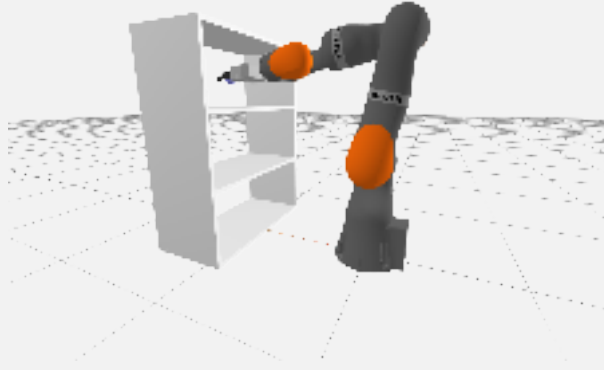
这里有少数例外。如果你的配置空间只涉及平移，或者如果你的机器人几何形状对旋转不变（例如点机器人或球形机器人），那么配置空间仍然是凸的；这是早期一些关键观察之一，它巩固了配置空间作为机器人学中核心数学对象的概念 [18]。这可以适用于移动机器人（甚至是用球体近似的四旋翼）。在这种情况下，我们可以很容易地计算采样点处任意一对凸体之间的最小欧氏距离，并且例如将该点膨胀成一个具有适当半径的球体 [19]。为了找到更大的区域，我们大量使用了 [20] 中提出的半正定规划迭代区域膨胀算法（Iterative Region Inflation by Semidefinite Program, IRIS），该算法也已 **在 Drake 中实现**。该算法会在寻找分离超平面与寻找最大体积内含椭球之间交替进行，从而“膨胀”该区域，以局部最大化该区域体积的一个高效近似。

可以将 IRIS 算法扩展到更一般的（非凸）配置空间中，用于寻找较大的凸区域。我们已经用三种方式实现了这一点：

- **使用非线性规划（nonlinear programming, NP）**。在 IRIS-NP 算法中 [21]（已 **在 Drake 中实现**），我们用类似 SNOPT 的非线性优化求解器，来替代用于寻找最近碰撞的凸优化。我们会随机采样潜在碰撞，以使该算法在概率意义上可靠；在实践中，这个算法相对较快，但并不 **保证** 该区域完全无碰撞。最近，我们通过 **IRIS-NP2** 显著提升了这一流程的性能。IRIS-NP2 由 [22] 提出，相比原始 IRIS-NP，它能够以快数个数量级的速度，创建面数更少且更大的区域。
- **使用零阶优化（zero-order optimization, ZO）**。通过用零阶优化（不需要梯度；只使用采样）替换 IRIS-NP 中的非线性优化，IRIS-ZO 会产生略小一些的区域，但可以非常容易地并行化 [22]。
- **使用代数运动学 + 平方和优化（algebraic kinematics + sums-of-squares optimization）**。利用这样一个思想：我们大多数机器人的运动学都可以用（有理）多项式来表达，我们可以使用多项式优化工具来搜索严格的非碰撞证书 [23]（已 **在 Drake 中实现**）。这种方法通常更慢，但其结果是可靠的。我们在这里必须做出的一个牺牲是：我们是在原始空间的 **立体投影（stereographic projection）** 坐标中寻找凸区域。在实践中，这意味着坐标系（因此也包括区域内的轨迹长度）会发生轻微扭曲。

Example 6.12 (配置空间中的 IRIS (IRIS in Configuration Space))

让我们在之前玩过的 iiwa 伸入货架的示例上，运行 `IrisInConfigurationSpace`（也就是 IRIS-NP）算法。我们将使用机械臂完整的 7 个自由度，以及原始碰撞几何。我希望这个例子展示两件事：（1）使用 `IrisInConfigurationSpace` 的具体机制；（2）即使面对复杂的配置空间几何，这些 IRIS 区域也能够覆盖很大的体积。



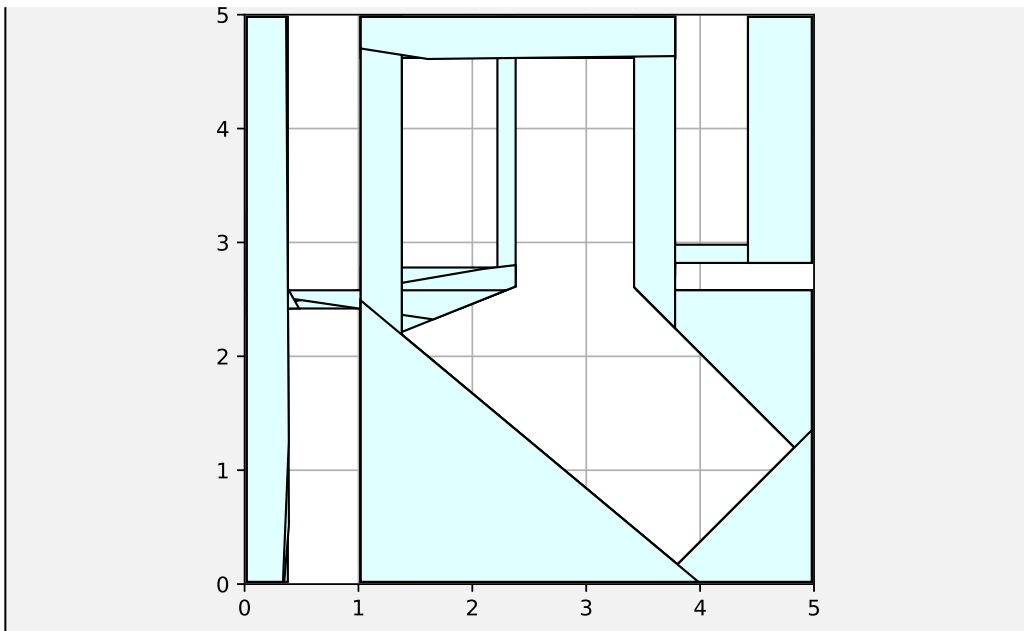
`IrisInConfigurationSpace` 可以只通过一个种子点 (seed point) 进行配置；算法的迭代会从该点附近的一个小区域开始，并不断“膨胀”它，以最大化配置空间体积的局部近似。我们可以使用 `InverseKinematics` 找到一个伸入顶部货架内部的种子配置 q 。但是如果我们从那里运行 IRIS，那么它找到的区域并不会填满货架内部空间，而是会逐渐爬出货架，去覆盖货架外部更大的配置空间。为了避免这种情况，我们将运行两次 IRIS：第一次使用机器人的“home configuration (初始配置)”作为种子，这样会生成一个位于货架外部的区域。然后我们再次运行 IRIS，使用位于货架内部的 IK 种子，但这次额外加入一个约束，将第一个区域视为障碍物（要求两个区域不能重叠）。这会迫使第二个区域使用货架内部的体积来最大化自身体积。

另一种获取这部分体积的方法，也是我现在更偏好的方法，是：使用逆运动学在感兴趣区域内收集大量配置空间样本，然后计算 `MinimumVolumeCircumscribedEllipsoid`（最小体积外接椭球），并将其作为 IRIS 的 `starting_ellipse` 使用。接着，我们只运行一次 IRIS 迭代，就可以得到一个不错的区域，并且这个区域的“体积”概念对应于覆盖该配置空间区域。这种方法避免了将先前区域视为障碍物所带来的虚假约束（因此会得到更大的区域），运行速度也更快（因为只需要运行一次 IRIS），并且更容易并行化。

如果我们继续沿用与 PRM 的类比，那么我们可能会想到：通过随机采样来对空间进行凸分解，拒绝那些处于碰撞状态、或已经位于某个现有 IRIS 区域中的样本，然后再对剩余样本进行膨胀。但实际上，我们可以做得比这更好——我们可以用相同数量的区域覆盖更大比例的自由配置空间，或者反过来，用更少的区域覆盖相近体积。我们目前用于执行这种凸分解的最佳算法，是通过计算“可见图 (visibility graph)”的“最小团覆盖 (minimum clique cover)”来实现的 [24]（已在 [Drake](#) 中实现）。

Example 6.13 (简单二维配置空间中的 GcsTrajOpt (续) (GcsTrajOpt in a simple 2D configuration space (cont.)))

如果我们在这个简单二维示例上运行“最小团覆盖 (minimum clique cover)”算法，并将覆盖阈值设为配置空间的 96%，就会得到如下这样的自动分解：



在实践中，对于 7 自由度机械臂，甚至也许直到大约 10 个自由度，试图计算配置空间中相当稠密的覆盖似乎仍然是比较可行的。但对于更高维度，我认为试图覆盖配置空间中的每个角落和缝隙是一个错误目标。我强烈更倾向于这样一种想法：使用能够代表配置空间中“重要”区域的采样点来初始化团覆盖算法。有时我们通过手动播种来做到这一点（通过调用逆运动学），或者也可以通过检查来自遥操作演示的轨迹、来自另一个规划器的轨迹（它可以生成规划，但不具备同样的保证），甚至来自强化学习策略的轨迹来实现。

Example 6.14 (IRIS 构建器 (The IRIS Builder))

这里有一个 notebook，其中包含若干不同工具，用于引导你的 IRIS 区域生成过程。

6.4.4 GcsTrajOpt 示例 (GcsTrajOpt Examples)

Example 6.15 (用于光滑曲线的正确代价函数是什么？ (What's the right cost function for smooth curves?))

Example 6.16 (iiwa 伸入货架 (iiwa reaching into shelves))

Example 6.17 (双臂 iiwa (Bimanual iiwa))

6.4.5 变体与扩展 (Variations and Extensions)

快速路径规划 (Fast path planning, FPP) [25]。许多运动规划实例并不需要 GCS 的全部能力；启发式近似通常就能找到非常好的解。FPP 并不是求解完整的 GCS 问题，即同时考虑离散变量和连续变量，而是在离散问题与连续问题之间交替求解，以找到一个局部极小值。FPP 通常比 `GcsTrajectoryOptimization` 更快；并且由于使用了交替求解，它能够处理 `GcsTrajectoryOptimization` 无法处理的导数约束（例如加速度约束）。不过，FPP 对其能

够支持的凸集 / 约束类别更加受限。（Drake 实现 [即将推出!](#)）

流形上的 GCS，包括用于移动操作的 GCS [26]，以及用于双臂操作的 GCS [27]。

带动力学约束的 GCS。对于由分段仿射动力学、凸目标函数和凸约束描述的离散时间系统，GCS 提供了迄今为止最强的形式化方法[13]。也可以更直接地在连续曲线上考虑连续动力学约束（更多内容即将推出!）。

通过接触进行规划。令人惊讶的是，我们现在已经拥有用于带接触准静态操作的紧凸松弛 [28]。最初的示例相当简单（这样我们可以仔细研究该问题，并暂时推迟求解器扩展细节），但你很快就可以期待我们在这个方向上取得更多进展。

不确定性下的规划（即将推出!）。

任务与运动规划（即将推出!）。[29] 展示了从时序逻辑到 GCS 的一种自然转写方式。

作为（反馈）策略的 GCS（即将推出!）。

6.5 时间最优路径参数化 (TIME-OPTIMAL PATH PARAMETERIZATIONS)

一旦我们有了一个运动规划，一个自然的问题就是：在速度、加速度和力矩限制约束下，我们最快能够以多快的速度执行该规划？

让我们通过 [路径参数化](#) $\mathbf{r}(s)$ 和 [时间参数化](#) $t = h(s)$ ，定义一条定义在区间 $t \in [t_0, t_f]$ 上的轨迹 $\mathbf{q}(t)$ ，使得

$$\mathbf{r}(s) = \mathbf{q}(h(s)), \text{ where } s \in [0, 1], h(0) = t_0, h(1) = t_f.$$

我们要求时间缩放是单调的，因此 $\forall s, h'(s) > 0$ ，其中 $h'(s) = \frac{\partial h}{\partial s}$ 。我们还会使用简写 $h''(s) = \frac{\partial^2 h}{\partial s^2}$ ，等等。这种参数化的优点在于：当 $\mathbf{r}(s)$ 固定时，我们可能希望施加在 $\mathbf{q}(t)$ 上的许多目标函数和约束，实际上都会变成关于 $h(s)$ 的凸约束。为了看到这一点，写出

$$\begin{aligned}\mathbf{r}'(s) &= \dot{\mathbf{q}}(t)h'(s), \\ \mathbf{r}''(s) &= \ddot{\mathbf{q}}(t)h'(s)^2 + \dot{\mathbf{q}}(t)h''(s).\end{aligned}$$

为了研究这一点，让我们再次通过路径参数化 $\mathbf{r}(s)$ 和时间参数化 $s(t)$ ，定义一条定义在区间 $t \in [t_0, t_f]$ 上的轨迹 $\mathbf{q}(t)$ ，使得

$$\mathbf{q}(t) = \mathbf{r}(s(t)), \text{ where } s(t) \in [0, 1], s(t_0) = 0, s(t_f) = 1.$$

我们将约束 $\forall t \in [t_0, t_f], \dot{s}(t) > 0$ ，从而保证从 s 到 t 的逆映射始终存在。这种参数化的优点在于：当 $\mathbf{r}(s)$ 固定时，我们可能希望施加在 $\mathbf{q}(t)$ 上的许多目标函数和约束，实际上关于 $s(t)$ 的导数是凸的。为了看到这一点，使用简写 $\mathbf{r}'(s) = \frac{\partial \mathbf{r}}{\partial s}$ ， $\mathbf{r}''(s) = \frac{\partial^2 \mathbf{r}}{\partial s^2}$ ，等等，写出

$$\begin{aligned}\dot{\mathbf{q}} &= \mathbf{r}'(s)\dot{s}, \\ \ddot{\mathbf{q}}(t) &= \mathbf{r}''(s)\dot{s}^2 + \mathbf{r}'(s)\ddot{s}.\end{aligned}$$

更重要 / 更令人兴奋的是，将这些项代入 [机械臂方程 \(manipulator equations\)](#) 可得：

$$\mathbf{u}(s) = \mathbf{m}(s)\ddot{s} + \mathbf{c}(s)\dot{s}^2 + \bar{\tau}_g(s),$$

其中 $\mathbf{u}(s)$ 是缩放时间 $s(t)$ 处的指令力矩，并且

$$\mathbf{m}(s) = \mathbf{M}(\mathbf{r}(s))\mathbf{r}'(s), \quad (7)$$

$$\mathbf{c}(s) = \mathbf{M}(\mathbf{r}(s))\mathbf{r}''(s) + \mathbf{C}(\mathbf{r}(s), \mathbf{r}'(s))\mathbf{r}'(s), \quad (8)$$

$$\bar{\tau}_g(s) = \tau_g(\mathbf{r}(s)). \quad (9)$$

下一关键步骤是引入变量替换： $b(s(t)) = \dot{s}^2(t)$ ，并注意到[†] $b'(s) = 2\ddot{s}$ 。我们将使用一条参数化轨迹来表示区间 $s \in [0, 1]$ 上的 $b(s)$ 。让我们看看，我们可以将一组极其有用的代价函数和约束，写成关于 $b(s)$ 的凸函数，并在采样时间点 $s_i = s(t_i)$ 处进行求值：

[†] 因为 $\dot{b}(s) = b'(s)\dot{s}$ 且 $\dot{b}(s) = 2\dot{s}\ddot{s}$ 。

- 单调性约束：

$$b(s_i) \geq \epsilon \Rightarrow \dot{s}(t_i) > 0.$$

- 速度限制是边界框约束：

$$\forall j, \dot{q}_{min,j}^2 \leq r_j'(s_i)^2 b(s_i) \leq \dot{q}_{max,j}^2 \Rightarrow \dot{\mathbf{q}}_{min} \leq \dot{\mathbf{q}}(t_i) \leq \dot{\mathbf{q}}_{max}.$$

- 加速度限制是线性约束：

$$\ddot{\mathbf{q}}_{min} \leq \mathbf{r}''(s_i)b(s_i) + \frac{1}{2}\mathbf{r}'(s_i)b'(s_i) \leq \ddot{\mathbf{q}}_{max} \Rightarrow \ddot{\mathbf{q}}_{min} \leq \ddot{\mathbf{q}}(t_i) \leq \ddot{\mathbf{q}}_{max}.$$

- 力矩限制是线性约束：

$$\mathbf{u}_{min} \leq \frac{1}{2}\mathbf{m}(s_i)b'(s_i) + \mathbf{c}(s_i)b(s_i) + \bar{\tau}_g(s_i) \leq \mathbf{u}_{max} \Rightarrow \mathbf{u}_{min} \leq \mathbf{u}(t_i) \leq \mathbf{u}_{max}.$$

- 最小化时间持续长度是一个凸目标：

$$t_f - t_0 = \int_{t_0}^{t_f} 1 dt = \int_0^1 \frac{1}{\sqrt{b(s)}} ds.$$

这看起来是非凸的，但可以通过松弛变量来写出： $a(s) = \dot{s}$ ，并使用洛伦兹锥（Lorentz cone）凸约束 $a(s) \geq \sqrt{b(s)}$ ，然后直接使用 $a(s)$ ；只要我们始终在把 $a(s)$ 往下压即可。 $\min \int_0^1 \sigma(s) ds, \sigma(s) \sqrt{b(s)} \geq 1. \rightarrow$

尽管我们希望有限数量的约束 就足以推出这些约束在整个时间区间 $[t_0, t_f]$ 上都成立，但在实践中，我们只是将这些约束施加在足够多的采样时间点上，并放弃严格保证。例如，如果我们使用 Bezier 曲线来参数化 $\mathbf{r}(s)$ 和 $b(s)$ ，并在控制点上施加约束（这些控制点属于由上面列出的乘法组合所生成的更高阶曲线），那么我们就可以利用 Bezier 曲线的凸包性质，以及 Bezier 曲线在求导、加法（通过升阶）和乘法下封闭这一事实。我们还必须使用机械臂方程的多项式形式。即便如此，做出这一断言仍然假设了对 $s(t) = \int_{t_0}^t \sqrt{b(s(t'))} dt'$ 的完美积分。

$$\begin{aligned} t &= h(s), \\ \dot{t} &= h'(s)\dot{s} = 1. \quad \dot{s}(t) = \frac{1}{h'(s)}. \\ \ddot{t} &= 0 = h''(s)\dot{s}^2 + h'(s)\ddot{s}. \end{aligned}$$

这些时间最优路径参数化（time-optimal path parameterizations, TOPP）在整个 20 世纪 80 年代得到了广泛研究（例如 [30]），当时使用的是专门设计的算法，例如试图找出 bang-bang 轨迹中的所有切换时刻。[31, 32, 33] 建立了我在这里所描述的与凸优化之间的联系。[34] 提出了一个流行的数值实现，称为 [TOPPRA](#)（基于可达性分析的时间最优路径参数化，time-optimal path parameterizations based on reachability analysis）。我们在 [16] 中大量使用了这些思想（并详细说明了它们与 Bezier 曲线参数化之间的联系）。你可以在 Drake 中找到 [PathParameterizedTrajectory](#) 类。

一个常见问题是：TOPP 是否可以用于对轨迹的 *jerk*（加加速度）（或更高阶导数）写出凸约束，

$$\mathbf{r}'''(s) = \ddot{\mathbf{q}}(t)h'(s)^3 + 3\dot{\mathbf{q}}(t)h'(s)h''(s) + \dot{\mathbf{q}}h'''(t).$$

$$\ddot{\mathbf{q}}(t) = \mathbf{r}'''(s)\dot{s}^3(t) + 3\mathbf{r}''(s)\dot{s}(t)\ddot{s}(t) + \mathbf{r}'(s)\ddot{s}^2(t).$$

这个问题之所以会出现，是因为许多工业机械臂都有必须遵守的 jerk 限制。[32] 讨论了这一问题 的一个版本。（更多内容即将推出……）我们可以通过额外的松弛变量 $j(s)$ 以及凸的旋转洛伦兹 锥约束 $j(s) \leq a(s)b(s)$ 来施加这些约束。

6.6 练习 (EXERCISES)

Exercise 6.1 (开门 (Door Opening))

在本练习中，你将实现一个基于优化的逆运动学求解器，用于打开橱柜门。你将完全在 中完 成工作。你需要完成以下步骤：

- 写出抓住橱柜把手这一 IK 问题的约束。
- 将 IK 问题形式化为一个优化实例。
- 使用 MathematicalProgram 实现该优化问题。

Exercise 6.2 (RRT 运动规划 (RRT Motion Planning))

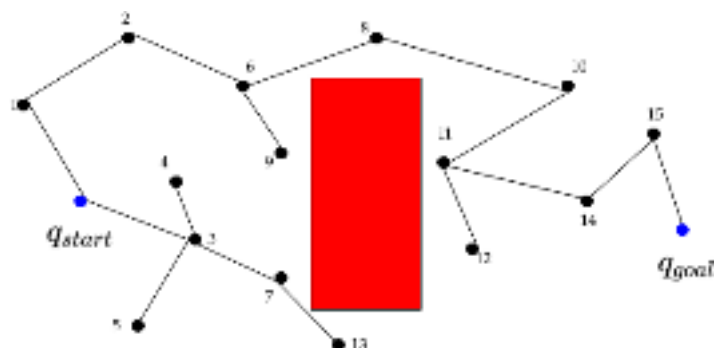
在本练习中，你将实现并分析课堂上介绍的 RRT 算法。你将完全在 中完成工作。你需要完成 以下步骤：

- 为 Kuka 机械臂实现 RRT 算法。
- 回答有关其性质的问题。
- 为 Kuka 机械臂实现 RRT-Connect 算法。

Exercise 6.3 (改进 RRT 路径质量 (Improving RRT Path Quality))

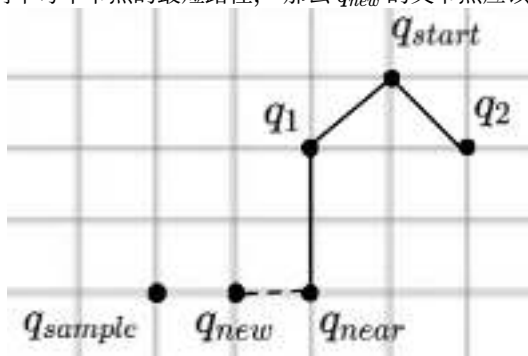
由于节点是通过随机过程生成的，RRT 输出的路径通常看起来会有些突兀、不够平滑（“RRT 舞蹈”是许多机器人学家最喜欢的舞步）。有许多策略可以改善路径质量，在本题中我们将探 索其中两种。为了本题起见，我们假设路径质量指的是路径长度，也就是说目标是找到尽可 能短的路径，其中距离度量为关节空间中的欧氏距离。

- 一种改善路径质量的策略是通过“捷径化 (shortcutting)”对路径进行后处理，它会 尝试用更短的线段替换路径中已有的部分 [35]。该方法通常使用如下算法实现：
 - 沿路径随机选择两个非连续节点。
 - 尝试用 RRT 的 extend 操作连接它们。
 - 如果得到的新路径更好，就用这条新的、更好的路径替换现有路径。
 重复步骤 1-3，直到满足终止条件（通常是有限步数或时间限制）。对于本题，我们可以假 设 extend 操作是在关节空间中的一条直线。考虑下图：RRT 已经找到了一条从 q_{start} 到 q_{goal} 的相当曲折的路径。图中有一个障碍物（红色显示），并且 q_{start} 与 q_{goal} 以蓝色高亮显示（免责声明：该图是手动生成的，用作说明性示例）。



说出一对节点，对于这对节点，捷径化算法会得到一条更短的路径（也就是说，它们是当前解路径上的两个节点，如果我们直接连接它们，就可以生成一条更短路径）。你应该假设距离度量是二维欧氏距离。

- b. 作为一种后处理技术，捷径化会基于已有路径进行推理，并允许对图进行局部“重连（re-wiring）”。它是一种启发式方法，但不能保证这棵树会编码最短路径。为了探索这一点，让我们放大观察 RRT 的一次迭代（如下图所示），其中 q_{sample} 是随机生成的配置， q_{near} 是现有树中距离最近的节点，而 q_{new} 是从 q_{near} 朝向 q_{sample} 方向进行 RRT 扩展得到的节点。当标准 RRT 算法（你在[前一个练习](#)中实现过）将 q_{new} 添加到树中时，它的父节点是什么？如果我们希望这棵树编码从起始节点 q_{start} 到树中每个节点的最短路径，那么 q_{new} 的父节点应该是哪一个节点？



这种动态“重连”以发现最小代价路径（对我们而言就是最短距离路径）的思想，是 RRT 的渐进最优变体 RRT* [36] 的关键部分。随着采样数量趋于无穷，RRT* 会找到通向目标的最优路径！这不同于“普通”RRT，后者被证明是次优的（该证明的直觉是：RRT 会“困住”自己，因为它在搜索过程中无法找到更好的路径）。

Exercise 6.4 (使用凸优化分解无障碍空间 (Decomposing Obstacle-Free Space with Convex Optimization))

在本练习中，你将实现 IRIS 算法 [20] 的一部分。该算法通过一系列凸优化，用于计算无障碍空间中的大区域。这些区域可以被各种规划方法使用，以搜索从起点到目标点且保持无碰撞的轨迹。你将完全在 中完成工作。你需要完成以下步骤：

- a. 实现一个 QP，用于寻找障碍物上距离自由空间椭圆最近的点。
- b. 实现算法中搜索一组超平面的部分，这些超平面将自由空间椭圆与所有障碍物分离开来。

Exercise 6.5 (规划与放置 (Plan and Place))

在本练习中，你将基于前几章中操作字母资产的练习继续开展工作。你将使用 RRT-Connect 来规划并仿真一条无碰撞轨迹，用于在杂乱环境中操作你名字首字母对应的物体。然后，你将实现捷径化，以改善路径质量并减少一些“RRT 舞蹈”。你将完全在 中完成工作。你需要完成以下步骤：

- 定义关键帧位姿。
- 实现一个基于优化的逆运动学函数，对于每个关键帧位姿，求解一个可行的机器人配置。
- 实现 RRT-Connect。
- 实现捷径化。

Exercise 6.6 (使用关节空间中的 IRIS 和 GCS 对 Iiwa 穿过货架进行路径规划 (Iiwa Path Planning through Shelves Using IRIS and GCS in Joint Space))

在本练习中，你将练习使用 IRIS 寻找大的无碰撞配置空间区域，并使用 GCS 在关节空间中穿过这些区域为 Kuka Iiwa 机械臂规划轨迹。你将在 中完成工作。你需要完成以下步骤：

- 围绕你期望的起点和终点，在关节空间中定义并计算 IRIS 区域。
- 通过你的 IRIS 区域构建一个 GCS 轨迹规划问题，并求解它来规划 Iiwa 的关节空间路径。

Exercise 6.7 (货架外部周围的轨迹优化 (Trajectory Optimization Around a Shelf Exterior))

通过本练习，你将更好地理解机器人周围环境如何影响你在构建与求解轨迹优化问题时所使用的技术。具体来说，我们将研究：当夹爪沿着货架外部移动时，在 Iiwa 运动学工作空间边界附近运行、以及复杂碰撞约束条件，会带来哪些影响。你将在 中完成工作。你需要完成以下步骤：

- 在 Kuka Iiwa 的完整关节空间中，定义并求解一个不包含碰撞约束的运动学轨迹优化问题。
- 在之前的形式化基础上加入碰撞约束，使用之前的“karate chop (空手刀)”轨迹作为初始猜测，并分析为什么第二次迭代无法在默认求解器参数下收敛。
- 研究更具信息量的初始猜测如何能够使求解更快收敛到解。

REFERENCES

1. Maurice Fallon and Scott Kuindersma and Sisir Karumanchi and Matthew Antone and Toby Schneider and Hongkai Dai and Claudia Pérez D'Arpino and Robin Deits and Matt DiCicco and Dehann Fourie and Twan Koolen and Pat Marion and Michael Posa and Andrés Valenzuela and Kuan-Ting Yu and Julie Shah and Karl Iagnemma and Russ Tedrake and Seth Teller, "An Architecture for Online Affordance-based Perception and Whole-body Planning", *Journal of Field Robotics*, vol. 32, no. 2, pp. 229-254, September, 2014. [[link](#)]
2. Pat Marion and Maurice Fallon and Robin Deits and Andrés Valenzuela and Claudia Pérez D'Arpino and Greg Izatt and Lucas Manuelli and Matt Antone and Hongkai Dai and Twan

- Koolen and John Carter and Scott Kuindersma and Russ Tedrake, "Director: A User Interface Designed for Robot Operation With Shared Autonomy", *Journal of Field Robotics*, vol. 1556-4967, 2016. [[link](#)]
3. Charles W. Wampler and Andrew J. Sommese, "Numerical algebraic geometry and algebraic kinematics", *Acta Numerica*, vol. 20, pp. 469-567, 2011.
 4. Rosen Diankov, "Automated Construction of Robotic Manipulation Programs", PhD thesis, Carnegie Mellon University, August, 2010.
 5. Hongkai Dai and Gregory Izatt and Russ Tedrake, "Global inverse kinematics via mixed-integer convex optimization", *International Symposium on Robotics Research*, 2017. [[link](#)]
 6. Pavel Trutman and Mohab Safey El Din and Didier Henrion and Tomas Pajdla, "Globally optimal solution to inverse kinematics of 7DOF serial manipulator", *IEEE Robotics and Automation Letters*, vol. 7, no. 3, pp. 6012--6019, 2022.
 7. Nathan Ratliff and Matthew Zucker and J. Andrew (Drew) Bagnell and Siddhartha Srinivasa, "CHOMP: Gradient Optimization Techniques for Efficient Motion Planning", *IEEE International Conference on Robotics and Automation (ICRA)*, May, 2009.
 8. Mrinal Kalakrishnan and Sachin Chitta and Evangelos Theodorou and Peter Pastor and Stefan Schaal, "STOMP: Stochastic trajectory optimization for motion planning", *2011 IEEE international conference on robotics and automation*, pp. 4569--4574, 2011.
 9. Marc Toussaint, "A tutorial on Newton methods for constrained trajectory optimization and relations to SLAM, Gaussian Process smoothing, optimal control, and probabilistic inference", *Geometric and numerical foundations of movements*, pp. 361--392, 2017.
 10. Marc Toussaint, "A Novel Augmented Lagrangian Approach for Inequalities and Convergent Any-Time Non-Central Updates", , 2014.
 11. Steven M. LaValle, "Planning Algorithms", Cambridge University Press , 2006.
 12. Amice and Alexandre and Werner and Peter and Tedrake and Russ, "Certifying Bimanual RRT Motion Plans in a Second", *International Conference on Robotics and Automation*, pp. 9293-9299, 2024. [[link](#)]
 13. Tobia Marcucci and Jack Umenberger and Pablo A. Parrilo and Russ Tedrake, "Shortest Paths in Graphs of Convex Sets", *arxiv*, 2023. [[link](#)]
 14. Ravindra K Ahuja and Thomas L Magnanti and James B Orlin, "Network flows: theory, algorithms, and applications", Prentice-Hall, Inc. , 1993.
 15. Tobia Marcucci, "Graphs of Convex Sets with Applications to Optimal Control and Motion Planning", PhD thesis, Massachusetts Institute of Technology, May, 2024. [[link](#)]
 16. Tobia Marcucci and Mark Petersen and David von Wrangel and Russ Tedrake, "Motion planning around obstacles with convex optimization", *Science Robotics*, vol. 8, no. 84, 2023.
 17. David von Wrangel and Russ Tedrake, "Using Graphs of Convex Sets to Guide Nonconvex Trajectory Optimization", *Proceedings of the IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pp. 8, 2024. [[link](#)]
 18. Tomas Lozano-Perez, "Spatial planning: A configuration space approach", Springer , 1990.
 19. Alexander Shkolnik and Russ Tedrake, "Sample-Based Planning with Volumes in Configuration Space", *arXiv:1109.3145v1 [cs.RO]*, 2011.
 20. Robin L H Deits and Russ Tedrake, "Computing Large Convex Regions of Obstacle-Free Space through Semidefinite Programming", *Proceedings of the Eleventh International Workshop on the Algorithmic Foundations of Robotics (WAFR 2014)*, 2014. [[link](#)]
 21. Mark Petersen and Russ Tedrake, "Growing convex collision-free regions in configuration space using nonlinear programming", *arXiv preprint arXiv:2303.14737*, 2023.

22. Peter Werner and Thomas Cohn and Rebecca H. Jiang and Tim Seyde and Max Simchowitz and Russ Tedrake and Daniela Rus, "Faster Algorithms for Growing Collision-Free Convex Polytopes in Robot Configuration Space", *Accepted to ISRR 2024*, 2024. [[link](#)]
23. Hongkai Dai* and Alexandre Amice* and Peter Werner and Annan Zhang and Russ Tedrake, "Certified Polyhedral Decompositions of Collision-Free Configuration Space", *The International Journal of Robotics Research (IJRR)*, vol. 43, no. 9, pp. 1322-1341, November, 2023. [[link](#)]
24. Peter Werner and Alexandre Amice and Tobia Marcucci and Daniela Rus and Russ Tedrake, "Approximating Robot Configuration Spaces with few Convex Sets using Clique Covers of Visibility Graphs", *International Conference on Robotics and Automation*, pp. 10359-10365, 2024. [[link](#)]
25. Tobia Marcucci and Parth Nobel and Russ Tedrake and Stephen Boyd, "Fast Path Planning Through Large Collections of Safe Boxes", *arXiv preprint arXiv:2305.01072*, 2023.
26. Thomas Cohn and Mark Petersen and Max Simchowitz and Russ Tedrake, "Non-Euclidean Motion Planning with Graphs of Geodesically-Convex Sets", *Robotics: Science and Systems*, 2023. [[link](#)]
27. Thomas Cohn and Seiji Shaw and Max Simchowitz and Russ Tedrake, "Constrained Bimanual Planning with Analytic Inverse Kinematics", *arXiv preprint arXiv:2309.08770*, May, 2024. [[link](#)]
28. Bernhard P Graesdal and Shao Yuan Chew Chia and Tobia Marcucci and Savva Morozov and Alexandre Amice and Pablo A Parrilo and Russ Tedrake, "Towards Tight Convex Relaxations for Contact-Rich Manipulation", *arXiv preprint arXiv:2402.10312*, 2024.
29. Vince Kurtz and Hai Lin, "Temporal Logic Motion Planning with Convex Optimization via Graphs of Convex Sets", *arXiv preprint arXiv:2301.07773*, 2023.
30. J.E. Bobrow and S. Dubowsky and J.S. Gibson, "Time-Optimal Control of Robotic Manipulators Along Specified Paths", *Int. J of Robotics Research*, vol. 4, no. 3, pp. 3--17, 1985.
31. Diederik Verscheure and Bram Demeulenaere and Jan Swevers and Joris De Schutter and Moritz Diehl, "Time-optimal path tracking for robots: A convex optimization approach", *Automatic Control, IEEE Transactions on*, vol. 54, no. 10, pp. 2318--2327, 2009.
32. Frederik DeBrouwere and Wannes Van Loock and Goele Pipeleers and Quoc Tran Dinh and Moritz Diehl and Joris De Schutter and Jan Swevers, "Time-optimal path following for robots with convex--concave constraints using sequential convex programming", *IEEE Transactions on Robotics*, vol. 29, no. 6, pp. 1485--1495, 2013.
33. Thomas Lipp and Stephen Boyd, "Minimum-time speed optimisation over a fixed path", *International Journal of Control*, vol. 87, no. 6, pp. 1297--1311, 2014.
34. Hung Pham and Quang-Cuong Pham, "A new approach to time-optimal path parameterization based on reachability analysis", *IEEE Transactions on Robotics*, vol. 34, no. 3, pp. 645--659, 2018.
35. R. Geraerts and M. Overmars, "A comparative study of probabilistic roadmap planners", *Algorithmic Foundations of Robotics V*, pp. 43--58, 2004.
36. S. Karaman and E. Frazzoli, "Sampling-based Algorithms for Optimal Motion Planning", *Int. Journal of Robotics Research*, vol. 30, pp. 846--894, June, 2011.

CHAPTER 7

移动操作（Mobile Manipulation）

到目前为止，我们主要关注的是桌面操作（table-top manipulation）。你可以把许多有趣的物体和挑战带到固定在桌子上的机器人面前，而且在接下来的许多章节中，这仍然会让我们有很多内容可以研究。但在继续深入之前，我想先停下来问一个问题——如果机器人同时还拥有轮子（甚至腿！）会怎样呢？

“移动操作（mobile manipulation）”中的许多技术挑战，与我们之前讨论的桌面操作中的挑战是相同的。但也有些挑战是全新的，或者至少在移动场景中变得更加迫切。描述这些挑战以及一些解决方案，正是本章的主要目标。

最重要的是，为我们的操作机器人增加移动能力，通常能够帮助我们扩大对机器人能力的设想范围。现在，我们可以开始思考机器人在房屋中移动并完成各种不同任务的场景。是时候把我们的造物送出去，去探索（操纵？）这个世界了！

7.1 一组新的角色

7.2 感知方面有什么不同？

7.2.1 部分视角 / 主动感知

我们曾经讨论过，当只能获得待操作物体的部分视角时所带来的一些技术挑战，详见[几何感知章节](#)。在桌面操作场景中，我们通常可以在桌子周围的各个外部角度布置摄像头进行观察，但在杂乱环境中，我们的视野仍然可能被遮挡。

当你是一个移动机器人时，我们通常不会假设你能够像固定式系统那样，对环境进行同等程度的全面布设与仪器化。通常会尝试只使用安装在机器人本体上的传感器，例如安装在头部的摄像头。如果我们只能看到每个物体的一侧，那么我们的[对向抓取（antipodal grasping）策略](#)就无法直接使用。

这个问题的主要解决方案是使用机器学习。如果你只能看到一个芥末瓶的一半，那么瞬时传感器读数本身并不包含足够的信息来规划抓取；我们必须以某种方式推断出物体背面的大致形状（也称为“形状补全（shape completion）”，虽然不一定需要显式完成）。你之所以能够较好地猜测芥末瓶后半部分的样子，是因为你以前接触过芥末瓶——正是你过去交互过的物体统计特性，提供了缺失的信息。因此，在这里学习不仅仅是一种便利手段，而是根本性的需求。我们很快就会讨论基于学习的感知！

不过，还有另一种缓解方式，而且对于移动操作机器人来说通常是可行的。现在摄像头（以及所有传感器）都是可移动的，我们可以主动移动它们，以获取更多信息并降低不确定性。关于“主动感知（active perception）”和“在不确定性下进行规划（planning under uncertainty）”已经有非常丰富的研究文献。[后续还会继续讨论]

7.2.2 未知（且可能动态变化）的环境

对于桌面操作，我们一开始假设物体是已知的，然后讨论了能够处理未知物体的方法（例如对向抓取）。但在整个讨论过程中，我们始终假设自己知道桌子 / 箱体（bins）的几何形状与位置！

例如，我们会主动裁剪掉击中已知环境的点云数据，只保留与目标物体相关的点。当机器人被固定在桌子上时，这样做是非常合理的；但一旦机器人具备移动能力，并开始与更加多样化、更加未知的环境交互时，我们就需要放宽这一假设。

一种做法是尝试对环境中的所有东西进行建模。我们可以使用目标检测来识别桌子，再通过位姿估计（甚至可能是形状估计）将参数化的桌子模型拟合到观测结果上。但这是一条非常困难的道路，尤其是在多样且大规模的场景中。是否存在一种类似于物体对向抓取的方法，能够同样减少我们对环境显式建模的需求呢？

我们希望拥有一种环境表示方式：它不需要显式的物体模型；它能够支持运动规划所需的各种查询（例如快速碰撞检测与最小距离计算）；它能够从原始传感器数据中高效更新；并且能够良好扩展到大型场景。例如，原始合并后的点云虽然易于更新、且不需要模型，但并不适合快速执行规划查询。

体素网格（voxel grids）是一种几何表示方式，它满足了我们（几乎所有的）期望条件——实际上，我们之前已经用过它们，用来高效地下采样点云。在这种表示中，我们将三维空间中的某个有限体积离散化为由固定大小立方体组成的网格；比如说，每个立方体为 1 cm^3 。我们将这些立方体中的每一个标记为已占据或未占据（或者更一般地说，我们可以为其赋予一个被占据的概率）。然后，对于无碰撞运动规划，我们把已占据的立方体视为需要避开的碰撞几何体。许多基于体素网格的碰撞查询都可以非常快速（并且很容易并行化）。特别是球体与体素之间的碰撞查询可以非常快，这也是你会在 Drake 模型中看到许多碰撞几何体被密集排列的球体近似表示的原因之一。使用点云来更新体素网格也很高效。为了让体素表示既能具有精细分辨率，又能表示非常大的场景，我们可以利用一些巧妙的数据结构。

Example 7.1 (Octomap)

在机器人学中，一种特别好用且高效的大规模体素网格实现，是一种名为 Octomap 的算法 / 软件包 [1].....

如今，将视觉场景分割为物体与环境，最好的方法是使用机器学习；我们很快就会实现分割流水线。

7.2.3 机器人状态估计

移动操作机器人在感知方面出现的最大差异，是不仅需要估计环境的状态，还需要估计机器人自身的状态。例如，在目前为止的点云处理算法中，我们一直默认自己知道 ${}^W X^C$ ，也就是相机在世界坐标系中的位姿。我们也许可以投入精细的标定流程，使用已知形状和图案，来（离线）估计这些相机外参。我们还一直假定机器人的运动学是已知的——当机器人固定在桌子上，并且世界坐标系与末端执行器之间的所有变换都能通过高精度编码器直接测量时，这是一个相当合理的假设。但一旦机器人拥有移动底盘，这些假设就会失效。

移动机器人的状态估计有着深厚的历史；[2] 是这一领域的经典参考文献。这些算法通常基于（近似的）递归贝叶斯滤波，或与之密切相关的平滑算法 [3]。通常，除了轮式编码器和惯性测量单元（IMU）之外，我们还会在移动底盘上配备距离传感器和 / 或摄像头。状态估计流水线会将这些带噪声的测量融合在一起，形成一个一致的估计。事实上，移动机器人领域最重要的经验之一是：仅靠车轮里程计几乎总是不足以完成估计；真实的轮子会打滑，而视觉 / 距离测量提供了一组关键且独立的测量信息。

自 [2] 写成以来，许多事情已经发生了变化。基于距离的传感器（激光雷达和深度相机）在量程、精度和帧率方面都有了显著提升，使得这些方法的效果大幅增强。

Example 7.2 (在 Drake 中仿真距离传感器)

不过，也许最大的变化是：估计算法已经从严重依赖基于距离的传感器 / 深度相机，转向越来越强大的、仅使用 RGB 摄像头的估计算法。这得益于视觉-惯性里程计 [4]、单目深度估计 [5]，以及稠密三维重建 / 运动恢复结构 [6, 7] 等方面的进展。

7.3 运动规划方面有什么不同？

让我们从一个例子开始：也许这是用我们目前已有的工具箱，自己构建移动操作机器人最简单的方式。

Example 7.3 (棱柱底座上的 iiwa)

我们取 iiwa 模型，并给它的底座增加几个自由度。不再把 link 0 焊接到世界坐标系上，而是插入三个棱柱关节（通过不可见、无质量的连杆连接），分别对应 x 、 y 和 z 方向上的运动。我会允许 x 和 y 不受边界限制，但会给 z 设置适度的关节限位。iiwa 的第一个关节本来就绕底座的 z 轴旋转；我只需移除该关节的限位，让它可以连续旋转。这些改动很简单，但现在我就得到了移动机器人的核心特征。

现在让我们问一个问题：为了让运动规划工具适用于这个版本的 iiwa，我需要做哪些修改？答案是：不多！我们的运动规划工具都相当通用，基本上适用于任何运动学机器人构型。我们在这里添加的移动底座，只是给这棵运动学树多增加了几个关节，除此之外可以直接纳入原有框架。

一个可能需要注意的问题是：我们增加了自由度的数量（这里从 7 个增加到了 10 个）。轨迹优化算法由于（也许可以说正因为？）是围绕某条轨迹进行局部优化的，因而很容易扩展到高维问题，所以这不是问题。基于采样的规划器在维度增加时会更加吃力，但 RRT 在 10 维空间中应该仍然没有问题。PRM 当然也可以在 10 维空间中工作，但可能需要高效的实现，并且已经开始接近它的能力边界。

另一个对于移动底座来说可能较新的问题是：连续关节的存在（绕 z 轴的旋转没有关节限位）。这在规划栈中可能需要一些谨慎处理。在基于采样的规划器中，通常需要调整距离度量和扩展操作，以考虑能够绕过 2π 的边。非凸轨迹优化本身并不一定需要算法层面的修改，但确实可能受到局部极小值的影响。对于[基于凸集图 \(Graphs of Convex Sets\) 的轨迹优化](#)，只需要注意让 IRIS 区域工作在局部坐标系中，并且在任何可环绕关节上的定义域宽度都不要超过 π [8]。如果没有正确处理这一点，就可能得到有点荒谬的规划结果，让机器人绕远路；这里有一个[例子](#)，展示了运动学轨迹优化在货架之间导航时，找到了一个次优解，该例来自 [8]。

如果我们不是用棱柱关节来实现移动操作机器人，而是使用轮子或腿，会发生什么呢？正如我们将看到的，一些轮式机器人（以及大多数腿式机器人）可以直接提供我们的棱柱关节抽象；但另一些机器人会引入一些约束，而这些约束必须在运动规划过程中加以考虑。

7.3.1 轮式机器人

尽管真实的轮子确实会打滑，但我们通常在无滑移假设下进行运动规划，然后使用反馈控制来跟踪规划出的路径 [9, Ch. 49]。

全向驱动

Example 7.4 ([麦克纳姆轮驱动运动学](#))

Example 7.5 ([在 Drake 中仿真麦克纳姆轮](#))

非完整驱动

差速驱动、Dubins 车、Reeds-Shepp。

Example 7.6 ([差速驱动运动学](#))

7.3.2 腿式机器人

如果你的工作是为机器人的腿部进行规划与控制，那么这里有[大量丰富的文献可供你探索](#)。但如果你使用的是像 [Spot](#) 这样的腿式机器人来执行移动操作，并且使用的是 Boston Dynamics 提供的 API，那么（幸运或不幸的是）你并不会深入到腿部的低层控制中。事实上，Spot 提供的 API 在一阶近似下将这个腿式平台视为一个全向底座。（还有一些次要考虑因素，例如机器人的质心会受到动态约束限制，并且可能会偏离你请求的轨迹，以保持平衡。）

Example 7.7 ([使用 Spot 进行移动操作](#))

当然，当你在崎岖或不连续的地形上执行移动操作时，情况会变得有趣得多；这正是腿式平台真正闪光的地方！

7.4 [仿真方面有什么不同？](#)

除了扩展我们的自主算法之外，移动操作还促使我们扩大仿真的范围。不再只是仿真桌面上的几个物体，现在我们开始思考如何在更广阔的环境中仿真机器人。

到目前为止，当我在这些笔记中使用“仿真（simulation）”这个术语时，我主要关注的是支撑性技术，比如物理引擎和渲染引擎。在当今机器人学中，人们使用的主流仿真器只有少数几个，例如 NVidia Isaac/Omniverse、MuJoCo 和 Drake。（PyBullet 曾经非常流行，但现在已经不再积极维护。）不过，还有另一类自称为“仿真器”的工具，它们建立在这些引擎之上（和 / 或建立在 Unity 或 Unreal 这类更传统的电子游戏引擎之上），并提供更高层次的抽象。我把它们看作内容聚合器，并认为它们发挥着非常重要的作用。

Example 7.8 ([Sapient](#))

Example 7.9 ([Behavior 1K](#))

Example 7.10 ([Habitat 3.0](#))

就我个人而言，我希望这些工具中的问题定义与资源文件，能够足够容易地导入到 Drake 中，从而让我们不仅能够使用我们的仿真引擎，还能够使用我们的感知、规划与控制工具来解决这些问题。为了实现这一点，我们正在逐步构建对其他格式的加载支持，或者实现不同格式之间的转换支持。

7.5 导航

到目前为止，我们在本章中讨论的大多数想法，都可以视为对现有操作工具箱相对温和的扩展。但与移动机器人导航相关的一些重要问题，在桌面操作中其实根本不会出现。

7.5.1 建图（以及定位）

7.5.2 识别可通行地形

7.6 练习

Exercise 7.1 (搭建仿真环境)

在这个练习中，你将把物体加载到仿真中，以构建一个有用的操作场景。你将只在 中完成本练习。你需要完成以下步骤：

- a. 分析预定义 SDF 文件中的碰撞几何体。
- b. 创建你自己的仿真环境。

Exercise 7.2 (移动底座逆运动学)

在这个练习中，你将为同时具有固定底座和移动底座的操作机器人，实现一个逆运动学优化问题。通过移除机器人底座的位置约束，我们能够更加容易地求解该优化问题。你将只在 中完成本练习。

REFERENCES

1. Armin Hornung and Kai M. Wurm and Maren Bennewitz and Cyrill Stachniss and Wolfram Burgard, "OctoMap: An Efficient Probabilistic 3D Mapping Framework Based on Octrees", *Autonomous Robots*, 2013.
2. S. Thrun and W. Burgard and D. Fox, "Probabilistic Robotics", MIT Press, 2005.
3. M. Kaess and A. Ranganathan and F. Dellaert, "iSAM: Incremental Smoothing and Mapping", *IEEE Transactions on Robotics*, vol. 24, no. 6, pp. 1365-1378, 2008.
4. Davide Scaramuzza and Friedrich Fraundorfer, "Visual odometry [tutorial]", *Robotics & Automation Magazine, IEEE*, vol. 18, no. 4, pp. 80--92, 2011.
5. Yue Ming and Xuyang Meng and Chunxiao Fan and Hui Yu, "Deep learning for monocular depth estimation: A review", *Neurocomputing*, vol. 438, pp. 14--33, 2021.
6. Ben Mildenhall and Pratul P Srinivasan and Matthew Tancik and Jonathan T Barron and Ravi Ramamoorthi and Ren Ng, "Nerf: Representing scenes as neural radiance fields for view synthesis", *Communications of the ACM*, vol. 65, no. 1, pp. 99--106, 2021.
7. Bernhard Kerbl and Georgios Kopanas and Thomas Leimkuhler and George Drettakis, "3d gaussian splatting for real-time radiance field rendering", *ACM Transactions on Graphics (ToG)*, vol. 42, no. 4, pp. 1--14, 2023.
8. Thomas Cohn and Mark Petersen and Max Simchowitz and Russ Tedrake, "Non-Euclidean

Motion Planning with Graphs of Geodesically-Convex Sets", *Robotics: Science and Systems* , 2023. [[link](#)]

9. Bruno Siciliano and Oussama Khatib, "Springer handbook of robotics", Springer , 2016.

CHAPTER 8

机械臂控制

在过去几章中，我们已经建立了一套很好的初始工具箱，用于感知场景中的物体（或一堆物体）、规划抓取，并移动机械臂完成抓取。在前几章中，我们开始设计运动规划策略，这些策略能够为机器人生成漂亮而平滑的关节轨迹，使机器人可以快速地从一次抓取移动到下一次抓取。当我们的期望轨迹越来越接近机器人的动力学极限时，我们用于在机械臂上执行这些轨迹的控制策略也需要变得稍微更复杂一些。

同时也要记住，操作远不止抓取！即使在箱内拣选应用中，我们也已经有一些例子表明，仅依靠抓取的策略可能是不够的。想象一下，你通过机器人相机向箱子里看，看到了下面这个场景：



我们到底要怎样用这个双指夹爪把那个“Cheez-it”饼干盒拿起来？（我知道你们当中有一些人正在想：对你们的吸盘夹爪来说，这个场景是多么简单；但仅靠吸盘也无法解决我们的所有问题。）

[点击这里观看视频。](#)

Figure 8.2 - 只是为了好玩，我请我女儿用我从厨房里翻出来的“双指夹爪”尝试一个类似的任务。
我们该如何编写这样的程序呢！

非抓取式操作 (nonprehensile manipulation) 这个术语的意思是“没有抓取的操作”，而人类经常这样做。当我们想到推动一个太大而无法抓取的物体（例如办公椅）时，就很容易理解这一点。但如果你仔细观察，就会发现人类即使在抓取时，也经常使用滑动和与环境接触之类的策略。这些策略只是在非抓取操作中变得更加突出。

当我们逐步建立用于抓取式 (prehensile) 和非抓取式 (nonprehensile) 操作的工具箱时，到目前为止我们的讨论中缺失的一点——而之前的讨论主要是运动学层面的——就是对力的思考。除了跟踪运动规划轨迹之外，本章还将介绍能够推理接触力的控制技术。我希望到最后你会同意：对于把那个盒子翻起来这件事，我们已经有了相当令人满意的方法！

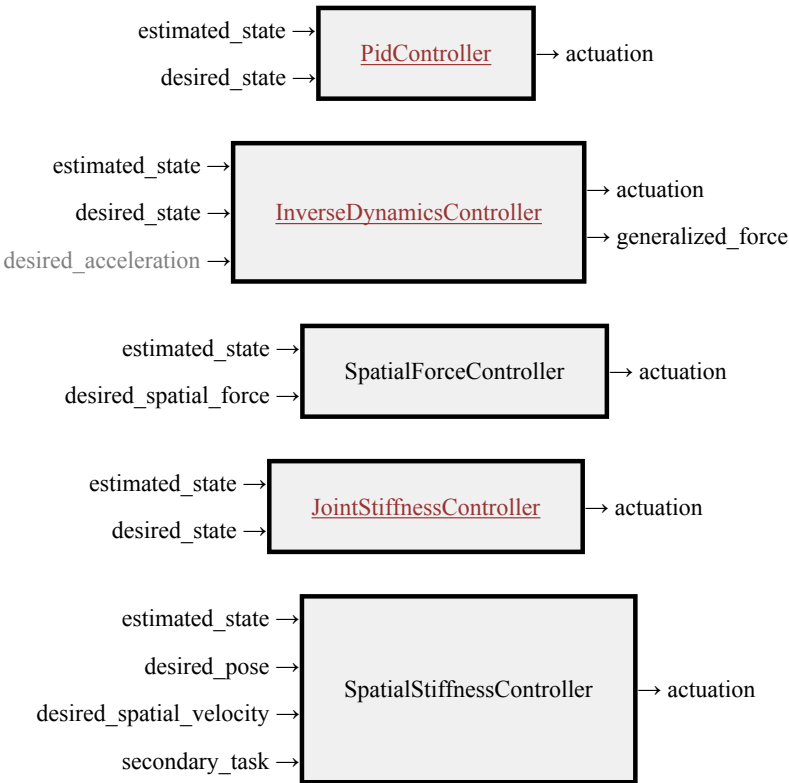
8.1 机械臂控制工具箱

当我谈到“机械臂控制”时，我讨论的是这样一类机器人控制器：它们接收（稍高层次的）命令，例如期望的关节位置和速度，或者空间力，并将这些命令转换为电机命令。在本章中，我们将始

终假设可以命令 **广义力 (generalized forces)**；对于转动关节机器人而言，这些广义力就是关节力矩。这些控制器本身并不足以完成任何有意义的任务——它们推理的是机器人自身，而不是环境中的任何物体。但它们通过提供更高层次的抽象，使编写其余控制系统变得更加容易。

通常，我们将在这里讨论的底层控制器是在直接运行于机械臂（或其控制柜）上的固件中实现的。为了使用这些硬件，理解它们如何工作以及应该如何设置其参数是很重要的。在仿真中，我们需要自己实现这些控制器，以便对机器人进行建模。

Drake 提供了许多 **机械臂控制实现**。在本章中，我们将逐一讲解其中最适合用于控制机器人手臂的实现：



到本章结束时，我希望你能够理解它们之间的差异，以及它们工作原理中的核心思想。

8.2 假设你的机器人是一个质点

像往常一样，在我们与复杂性作斗争的过程中，我希望找到一个尽可能简单（但又不过于简单）的设置！以下是我针对盒子翻转示例提出的方案。首先，我将所有运动限制在二维平面内；这样一来，我可以去掉箱子的两侧（方便你看清内部），同时也减少了我们需要考虑的自由度数量。特别地，我们可以通过给盒子添加一个 `PlanarJoint` 来避免使用四元数浮动基坐标。此外，我们先不使用完整的夹爪，而是进一步简化，只使用一个“点手指（point finger）”。我将它可视化为一个球，并将两个控制输入建模为直接在该质点的 x 和 z 坐标方向施加力。



Figure 8.4 - 这个简单模型：一个点手指、一个饼干盒以及一个二维箱体。绿色箭头表示接触力。

即使在这个简单模型中，以及在本章后续讨论中，我们仍然会关心两个动力学模型。第一个是用于仿真的完整动力学模型：它包含手指、盒子和箱体，总共有 5 个自由度（盒子的 x, z, θ ，以及手指的 x, z ）。第二个则是用于设计控制器的机器人模型：该模型只有手指的两个自由度，并受到未建模的接触力作用。按照设计，第二个模型的方程特别简单：

$$\begin{bmatrix} m & 0 \\ 0 & m \end{bmatrix} \dot{v} = m \begin{bmatrix} \ddot{x} \\ \ddot{z} \end{bmatrix} = \tau_g + \begin{bmatrix} u_x \\ u_z \end{bmatrix} + f^{F_c},$$

其中 m 是质量， τ_g 是重力“力矩”，在这里它实际上只是 $\tau_g = [0, -mg]^T$ ， u 是控制输入向量，而 f^{F_c} 是施加在手指 F 的接触点 c 上的笛卡尔接触力。

本章将大量使用我们的[空间力记号法](#)。为了确保含义清晰，上面我使用 f^{F_c} 来表示 [施加在手指 \$F\$ 的接触点 \$c\$ 上的力](#)。如果我们想表示同一个力 [施加在盒子 \$B\$ 的接触点 \$c\$ 上](#)，那么我们将其记作 f^{B_c} 。很自然地，我们有 $f^{F_c} = -f^{B_c}$ ，因为这两个力必须大小相等、方向相反。如果我们对这个记号不够谨慎，下面的符号正负号一定会变得非常混乱。

8.2.1 轨迹跟踪

在我们甚至还没有接触盒子之前，先确保我们知道如何让机器人手指在空中移动。假设你已经完成了一些运动规划，也许是使用优化方法，并且已经得到了一条漂亮的期望轨迹 $q_d(t)$ ，希望手指能够跟踪它。我们假设 $q_d(t)$ 是二次可微的。那么，为了执行这条轨迹，你应该向机器人发送什么样的广义力命令呢？

执行轨迹最早且最常见的方法之一是比例-积分-微分（PID）控制，我们在讨论[位置控制机器人](#)时已经介绍过它。[PidController](#) 使用如下控制命令：

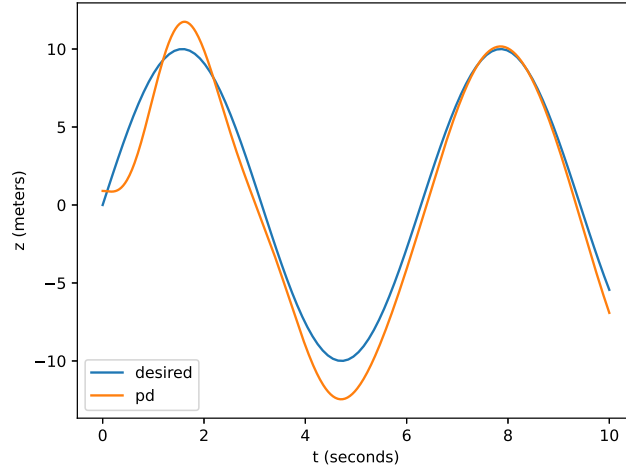
$$u(t) = K_p(q_d(t) - q(t)) + K_d(\dot{q}_d(t) - \dot{q}(t)) + K_i \int_0^t (q_d(t) - q(t)) dt,$$

其中 K_p 、 K_d 和 K_i 分别是位置、速度和积分增益。这些增益通常是对角矩阵，这样每个关节的控制命令就与其他关节相互独立。注意，在操作任务中，我们倾向于避免使用积分项，即设置 $K_i = 0$ 。你可以想象，如果机器人正顶着墙推，因而无法达到 $q(t) = q_d(t)$ ，那么积分项就会发生“积分饱和（wind-up）”，向机器人发送越来越大的控制命令，直到触发某种故障为止。

PD 控制可以非常有效；它几乎不对机器人的动力学做任何假设。但这种假设同时也是一种限制；如果我们确实知道一些关于机器人动力学的信息，那么就可以实现更好的跟踪性能。为了说明这一点，让我们写出在 PD 控制下机器人手指 z 位置的闭环动力学：

$$m\ddot{z} = -mg + k_p(z_d - z) + k_d(\dot{z}_d - \dot{z}).$$

下面是一个示例展开结果：我们命令一条正弦轨迹 $z_d(t)$ ，并让手指在 z 和 $\dot{z}$ 上稍微偏离该轨迹作为初始状态：



实际轨迹确实会跟踪期望轨迹，但会存在一些持续误差。即使期望轨迹是常数， $\dot{z}_d = 0$ ，我们也会预期存在一个稳态误差 $\tilde{z} = z_d - z$ ；可以非正式地通过令 $\dot{z} = \ddot{z} = 0$ 得到：

$$\tilde{z} = z_d - z = \frac{1}{k_p} mg.$$

在这种配置下，PD 控制器产生的回复力正好平衡了重力带来的扰动力，因而没有进一步将误差推向零。

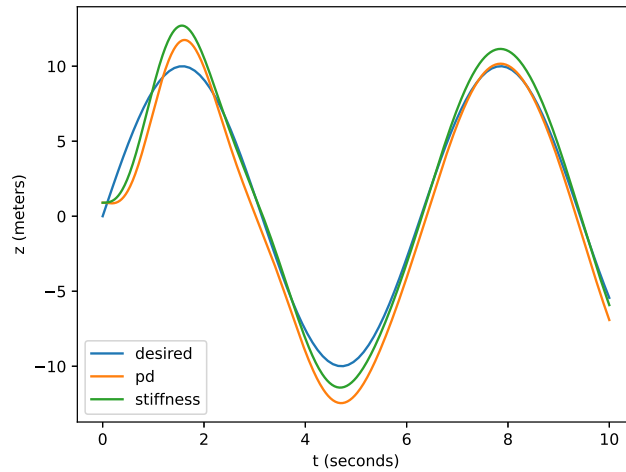
观察这些方程，很明显，如果我们把重力项告知控制器，就可以做得更好。这通常被称为“重力补偿”。出于很快就会变得清楚的原因，重力补偿 + PD 控制器在 Drake 中以 [JointStiffnessController](#) 的形式提供：

$$u = -\tau_g + K_p(q_d - q) + K_d(\dot{q}_d - \dot{q}).$$

对于我们这个点手指的 z 轴而言，这会得到如下闭环动力学：

$$m\ddot{z} = k_p(z_d - z) + k_d(\dot{z}_d - \dot{z}).$$

这个控制器没有稳态误差，并且在实践中跟踪效果更好：



虽然稳态误差消失了，但在跟踪更复杂的期望轨迹时，我们仍然可以看到误差。然而，还有一个相关的信息是我们可以获得、但尚未提供给控制器的： $\ddot{q}_d(t)$ 。让我们以一种略有不同的形式来写

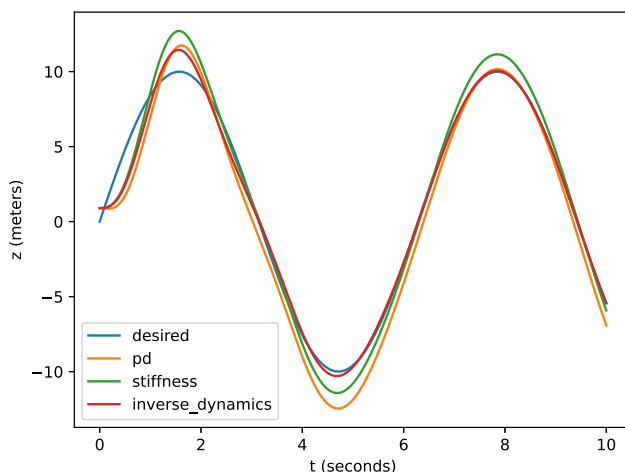
控制器输入：

$$u = -\tau_g + m [\ddot{q}_d + K_p(q_d - q) + K_d(\dot{q}_d - \dot{q})].$$

这种控制器的一般形式在 Drake 中作为 [InverseDynamicsController](#) 提供。这里有两处变化：除了在控制器中加入前馈加速度项之外，我们还将 PD 项乘以了质量。二者结合之后，使得闭环误差动力学简化为：

$$\ddot{z} + k_d \dot{z} + k_p z = 0,$$

你可能会认出它是一个简单的质量-弹簧-阻尼模型。在合理选择 k_p 和 k_d 的情况下，误差动力学会很好地收敛到零，从而给出迄今为止最好的跟踪性能：



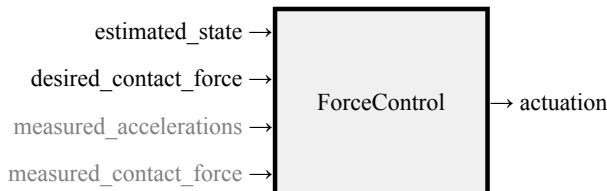
Example 8.1 (质点的轨迹跟踪)

我已经将生成这些简单图像的代码放进了一个 notebook 中，这样你就可以自己动手尝试：

8.2.2（直接）力控制

当我们开始发生接触时（在我们的例子中，就意味着手指开始接触盒子），事情就变得更有意思了。在这种情况下，控制手指的位置 / 速度 / 加速度 可能不再是唯一目标。我们还可能希望控制施加到盒子上的交互力。

在最简单的情况下，我们完全不尝试调节手指的位置，而是实现一个底层控制器：它接收期望的接触力，并产生机器人的广义力输入，使实际接触力尽可能匹配期望值。



为了调节接触力，我们需要哪些信息？当然，我们需要期望接触力 $f_{desired}^{F_c}$ 。一般来说，我们还需要机器人的状态（尽管在当前这个质点例子中，其动力学并不依赖于状态）。但我们还需要以下三者之一：（1）测量机器人加速度（我们通常尽量避免这样做，因为加速度测量往往噪声很

大)；(2) 假设机器人加速度为零；或者(3) 提供接触力测量，以便我们能够通过反馈来调节它。

让我们考虑机器人已经与盒子接触的情况。同时，我们暂时假设加速度（几乎）为零。对于大多数操作任务来说，这实际上并不是一个糟糕的假设，因为这些任务中的运动通常相对缓慢。在这种情况下，我们的运动方程简化为：

$$f^{F_c} = -mg - u.$$

我们的力控制器实现可以简单到： $u = -mg - f_{desired}^{F_c}$ 。请注意，为了实现这个控制器，我们只需要知道 **机器人本身** 的质量（而不是盒子的质量）。

当机器人没有接触时会发生什么？在这种情况下，我们就不能再合理地忽略加速度了，并且应用相同控制时会得到： $m\dot{v} = -f_{desired}^{F_c}$ 。这其实并不完全是坏事。事实上，这正是力控制最吸引人的特征之一。当你指定了一个期望力却没有得到它时，结果就是接触点会沿着期望力的（相反）方向加速。在实践中，这（局部地）往往会在未接触时把系统带入接触状态。

Example 8.2 (命令一个恒定力)

让我们看看，当我们运行一个完整仿真时会发生什么。这个仿真不仅包括非接触情况和接触情况，还包括两者之间的过渡过程（其中涉及碰撞动力学）。我将点手指放在盒子旁边，并施加一个恒定力命令，要求从盒子获得一个水平接触力。我绘制了在不同（恒定）接触力命令下，手指的 x 方向轨迹。

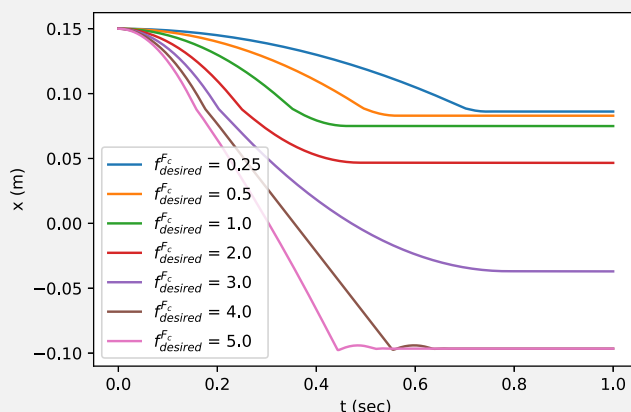


Figure 8.8 - 这是在不同恒定期望力命令下，手指水平位置随时间变化的曲线图。

对于所有严格为正的期望力命令，手指都会以恒定加速度运动，直到与盒子碰撞（在 $x = 0.089$ 处）。当 $f_{desired}^{F_c}$ 较小时，盒子几乎不会移动。对于中等范围的 $f_{desired}^{F_c}$ ，与盒子的碰撞足以让盒子开始移动，但摩擦最终会让盒子、以及因此也包括手指，一起停下来。当取值较大时，手指会持续推动盒子，直到盒子撞上远处的墙壁。

考虑一下这种行为的后果。通过命令力，我们可以编写一个控制器，使其与盒子形成良好的接触，而几乎不需要知道盒子的几何形状。（我们只需要足够的感知能力，让手指从某个位置开始，并确保沿直线接近时能够碰到盒子。）

这也是研究腿式机器人的研究人员喜欢力控制的原因之一。在具备力控制能力的步行机器人上，我们可能会在“摆动阶段（swing phase）”模拟位置控制，以便让脚大致落到我们希望踩踏的位置。然后，我们切换到力控制模式，让脚真正放到地面上。这可以显著降低对地形精确感知的要求。

Example 8.3 (一种基于力的翻起策略)

下面是我提出的一个用于翻起盒子的简单策略。一旦发生接触，我们将利用手指施加的接触力，围绕盒子的左下角产生一个力矩，从而生成旋转。但我们会对所施加的力添加约束，以确保底部角点不会滑动。



Figure 8.9 - 翻起盒子。 [点击这里](#) 查看下面示例中我们将编写的控制器动画。不过，也很值得运行代码，因为在那里你还可以看到接触力的可视化！

这些条件用力来表达非常自然。而且再一次，我们可以用非常少的盒子信息来实现这一策略。当我们施加接触力时，手指的位置会自然演化。对我来说，很难想象如何使用一个（高增益）位置控制的手指来编写同样鲁棒的策略；那很可能需要对盒子的几何形状（甚至可能还有质量）做出更多假设。

让我们把上面的文字描述编码出来，描述施加到盒子上的力。我将使用 C 表示手指与盒子之间的接触坐标系，其法向量垂直于表面并指向 **盒子内部**；使用 A 表示盒子左下角与箱体接触处的接触坐标系，其法向量沿世界坐标系正 z 方向竖直向上。

$${}^B R^C = R_y(-\frac{\pi}{2}), \quad {}^W R^A = I,$$

其中 $R_y(\theta)$ 表示绕 y 轴旋转 θ 。当然，我们还有重力，它作用在刚体质心（com）处：

$$f_{gravity,W}^{B,com} = mg.$$

正如你所见，我们将大量使用[这里描述](#)的空间力记号法 / 空间代数。

摩擦锥为我们想要施加的力提供了（线性不等式）约束。

$$\begin{aligned} f_{finger,C_z}^{B_C} &\geq 0, & |f_{finger,C_x}^{B_C}| &\leq \hat{\mu}_C f_{finger,C_z}^{B_C}, \\ f_{ground,A_z}^{B_A} &\geq 0, & |f_{ground,A_x}^{B_A}| &\leq \hat{\mu}_A f_{ground,A_z}^{B_A}. \end{aligned}$$

请务必确保你理解这个记号。在这些约束之内，我们希望将盒子旋转翻起。

为了让盒子绕 A 旋转，我们来分析一下绕 A 施加的总力矩：

$$\tau_{total,W}^{B_A} = \tau_{gravity,W}^{B_A} + \tau_{ground,W}^{B_A} + \tau_{finger,W}^{B_A},$$

但我们知道 $\tau_{ground,W}^{B_A} = 0$ ，因为力臂为零。我的目标是不在控制器中对盒子的质量和几何形状做出太多假设，因此，与其试图完美调节这个力矩，不如写成：

$$\tau_{total,W_y}^{B_A} = \tau_{finger,W_y}^{B_A} + \text{未知项}.$$

面对这些未建模项，一个合理的控制策略是对盒子的角度（记为 θ ）使用反馈控制；这个角度是 ${}^W R^B$ 的 y 分量：

$$\tau_{finger_d,W_y}^{B_A} = \text{PID}(\theta_d, \theta), \quad {}^W R^B(\theta) = R_y(\theta),$$

其中我使用 θ_d 作为期望盒子角度的简写，使用 PID 作为简单比例-积分-微分项的简写。注意， $\tau_{finger, W_y}^{B_A} \propto f_{finger, C_z}^{B_C}$ ，其中这个（常数）系数只依赖于 ${}^B\hat{p}^C$ ；它实际上并不依赖于 $\hat{\theta}$ 。

为了在满足摩擦锥约束的同时执行期望的 PID 控制，我们可以使用带约束的最小二乘：

$$\begin{aligned} \min_{f_{finger, C}^{B_C}, f_{ground, A}^{B_A}} \quad & \tau_{finger, W_y}^{B_A} - \text{PID}(\theta_d, \hat{\theta})^2, \\ \text{subject to} \quad & f_{finger, C_z}^{B_C} \geq 0, \quad |f_{finger, C_x}^{B_C}| \leq \hat{\mu}_C f_{finger, C_z}^{B_C}, \\ & f_{ground, A_z}^{B_A} \geq 0, \quad |f_{ground, A_x}^{B_A}| \leq \hat{\mu}_A f_{ground, A_z}^{B_A}, \\ & f_{ground, A}^{B_A} + \hat{f}_{gravity, A}^{B_A} + f_{finger, A}^{B_A} = 0. \end{aligned}$$

注意，一旦给定 $\hat{\theta}$ ，最后一行尽管需要一些空间代数运算，但仍然是一个线性约束。实现这一策略需要假设：

- 我们有盒子朝向的某种近似值 $\hat{\theta}$ 。这可以通过点云法向量估计获得，甚至也可以通过跟踪手指的路径获得。
- 我们有墙壁与盒子之间的静摩擦系数 $\hat{\mu}_A$ 、手指与盒子之间的静摩擦系数 $\hat{\mu}_C$ 的保守估计，以及手指相对于盒子角点的大致位置和盒子质量 $\hat{m}$ 。

你应该自己实验并判断这些估计是否可以比较粗略。我认为你会发现，即使对盒子的质量和几何形状了解不精确，这种方法也相当鲁棒。

在这个示例中，我们有多个控制器。第一个是底层力控制器：它接收一个期望接触力，并向机器人发送命令，试图调节实际接触力以匹配该期望值。第二个是更高层级的控制器：它观察盒子的朝向，并决定向底层控制器请求哪些力。

也请理解，这并不是某种用于翻起盒子的唯一策略或最优策略。我只是想展示：有时候，一些用其他方式可能很难表达的控制器，可以很容易地用力来表达！

8.2.3 间接力控制

除了通过直接指定力来控制接触交互之外，还有一种很好的哲学性替代方案。相反，我们可以将机器人编程为像一个（简单的）机械系统那样运行，以期望的方式对接触力做出反应。Ken Salisbury 在一系列重要论文中很好地描述了这种思想，并提出了**刚度控制（stiffness control）**[\[1\]](#)；随后 Neville Hogan 提出了**阻抗控制（impedance control）**[\[2, 3, 4\]](#)。

这种方法在概念上非常优雅。想象一下，我们走到 iiwa 机器人旁边，推它的末端执行器。只需要知道机器人自身的参数（而不是环境的参数），我们就可以编写一个控制器，使得当我们推动末端执行器时，末端执行器会利用整个机器人进行反推，仿佛你正在推动一个简单的弹簧-质量-阻尼系统。与其试图通过一系列位置命令刚性地移动末端执行器来实现操作，我们可以移动一个柔软虚拟弹簧的设定点（也许还可以改变其刚度），并让这个虚拟弹簧产生我们期望的接触力。

这种方法还有一个很好的优势：即使面对未建模的接触交互，它也可以帮助保证机器人不会变得不稳定。如果机器人表现得像一个耗散系统，而环境也是一个耗散系统，那么整个系统就会稳定。这种形式的论证可以为甚至非常复杂的系统保证稳定性，它建立在关于**无源性理论（passivity theory）**或更一般的 *Port-Hamiltonian* 系统的丰富文献基础之上[\[5\]](#)。

我们这个带点手指的简单模型，非常适合理解间接力控制的本质。我们系统原始的运动方程是：

$$m \begin{bmatrix} \ddot{x} \\ \ddot{z} \end{bmatrix} = mg + u + f^{F_c}.$$

我们可以编写一个简单控制器，让该系统表现得像一个（无源的）质量-弹簧-阻尼系统：

$$m \begin{bmatrix} \ddot{x} \\ \ddot{z} \end{bmatrix} + b \begin{bmatrix} \dot{x} \\ \dot{z} \end{bmatrix} + k \begin{bmatrix} x - x_d \\ z - z_d \end{bmatrix} = f^{F_c},$$

其中弹簧的平衡位置位于 (x_d, z_d) 。实现这一系统的控制器很容易写出；在点手指的情况下，它具有我们熟悉的比例-微分（PD）控制器形式，但额外加入了一个用于抵消重力的“前馈”项。

严格来说，如果我们只是像这里写的那样设置刚度和阻尼，那么这种形式的控制器通常会被称为“刚度控制（stiffness control）”，它是阻抗控制（impedance control）的一个子集。我们也可以改变系统的等效质量；那将是完整意义上的阻抗控制。不过，我的印象是，机器人控制专家的共识认为：改变等效质量通常并不值得，因为它会带来额外传感和带宽需求所导致的复杂性。

关于间接力控制的文献中，有大量术语和实现细节，这些在实践中都非常重要。例如，你的具体实现会取决于你是否能够访问力传感器，以及是否能够直接命令力/力矩。控制性能还会受到控制器带宽和传感器质量的显著影响。可以参考例如 [6] 获取更全面的综述（以及一些有趣的老视频），或者参考 [7] 来了解一种较早但很不错的观点。

如果我们有一个带力-力矩传感器的位置控制机器人，我们也可以对其响应进行编程。这被称为 **导纳控制（admittance control）**。在最简单的形式下，我们可以认为原始运动方程中已经包含了一个 PD 控制器，而我们唯一的控制输入就是调节该控制器的设定点。对于单自由度系统，我们有：

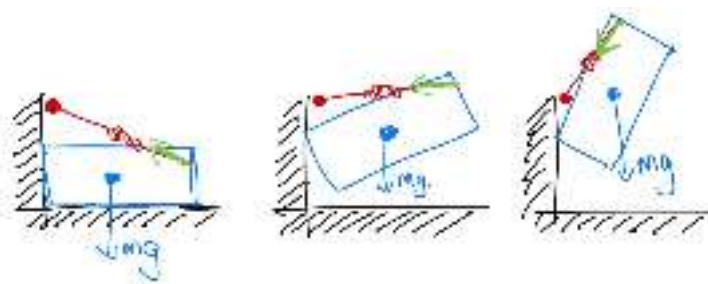
$$m\ddot{q} + \text{未建模项} = k_p(q_d - q) + k_d(\dot{q}_d - \dot{q}) + f^c.$$

Example 8.4 (使用刚度控制进行遥操作)

我没有给你提供一个带直接力控制的遥操作界面；因为那会非常难用！不过，通过调整虚拟弹簧上的设定点来移动机器人，却是非常自然的。花一点时间试着移动盒子，甚至把它翻起来吧。

为了帮助你建立直觉，我将箱体和盒子做成了略微透明的效果，并添加了一个虚拟手指/夹爪（橙色）的可视化，你可以通过滑块来移动它。

让我们进一步利用间接力控制，来提出另一种翻起盒子的方案。在箱体中央翻起盒子，需要非常明确地推理各种力，才能保证作用力保持在盒子左下角的摩擦锥内部。但还有另一种策略，它不需要对力进行如此精确的控制。我们可以先把盒子推到角落里，**然后**再把它翻起来。



为了实现这一点，我希望你想象创建一个虚拟弹簧——你可以把它想象成一根拉紧的橡皮筋——它连接在手指与靠近墙壁、略高于盒子左上角的位置之间。盒子会像一个摆一样，围绕顶部角点旋转，而这根橡皮筋会产生对应的力矩。在某个时刻，顶部角点会发生滑动，但正是同一根橡皮筋，会让手指从顶部角点向下压住盒子，从而完成整个动作。

考虑一下另一种做法：编写一个估计器和控制器，用于检测发生滑动的时刻，然后重新制定一个新计划。那绝不会是一条通向幸福的道路。仅仅通过利用机器人模型，让机器人在接触点表现得像另一个动力系统，我们就能够完成所有这些事情！

Example 8.5 (一种基于刚度控制的翻起策略)

这个控制器几乎简单得不能再简单了。我只需要命令一个轨迹，将虚拟手指移动到墙壁前方一点的位置。这样就会把盒子推向墙角，并建立起我们的支撑接触力。然后，我再让虚拟手指（也就是橡皮筋的另一端）沿着墙稍微向上移动一点，剩下的事情就可以交给力学本身来完成了！

到目前为止，我们让手指表现得像两个相互独立的质量-弹簧-阻尼系统，一个对应 x 方向，另一个对应 z 方向。我们甚至对每个方向都使用了相同的刚度 k 和阻尼 b 参数。更一般地，我们可以写成刚度矩阵和阻尼矩阵，通常分别记为 K_p 和 K_d ：

$$m \begin{bmatrix} \ddot{x} \\ \ddot{z} \end{bmatrix} + K_d \begin{bmatrix} \dot{x} \\ \dot{z} \end{bmatrix} + K_p \begin{bmatrix} x - x_d \\ z - z_d \end{bmatrix} = f^F.$$

为了强调这些刚度矩阵所属的坐标系，我们把完全相同的方程重新写一遍。注意，对于点手指而言，有 ${}^W p^F = [x, z]^T$ ，因此：

$$m {}^W \dot{v}^F + K_d {}^W v^F + K_p ({}^W p^F - {}^W p^{F_d}) = f^F.$$

有时，将刚度（以及/或者阻尼）矩阵表达在另一个坐标系 A 中会更加方便：

$$\begin{aligned} m {}^W \dot{v}^F + {}^W R^A K_d {}^W v_A^F + {}^W R^W K_p ({}^W p_A^F - {}^W p_A^{F_d}) = \\ m {}^W \dot{v}^F + {}^W R^A K_d {}^A R^W {}^W v^F + {}^W R^A K_p {}^A R^W ({}^W p^F - {}^W p^{F_d}) = f^F. \end{aligned}$$

8.2.4 混合位置/力控制

有很多应用场景中，我们希望在一个方向上显式命令力，而在另一个方向上命令位置。一个经典例子是擦拭或抛光表面——你可能关心施加在表面法向方向上的力大小，但同时又希望利用位置反馈，在表面切向方向上跟踪一条轨迹。在最简单的情况下，想象一下对于我们的点手指，在 z 方向控制力，在 x 方向控制位置：

$$u = -mg + \begin{bmatrix} k_p(x_d - x) + k_d(\dot{x}_d - \dot{x}) \\ -f_{desired, W_z}^F \end{bmatrix}.$$

如果我們希望在另一个坐标系中指定力/位置，例如接触坐标系 C ，那么可以使用

$$u = -mg + {}^W R^C \begin{bmatrix} k_p(p_{C_x}^{F_d} - p_{C_x}^F) + k_d(v_{C_x}^{F_d} - v_{C_x}^F) \\ -f_{desired, C_z}^F \end{bmatrix}. \quad (1)$$

通过命令法向力，你不仅能够控制机器人在表面上施加的压力大小，还能够对表面法向估计误差具有一定鲁棒性。如果一个位置控制机器人错误地估计了墙面的法向，那么它可能会在一个方向上完全离开墙面，而在另一个方向上又施加极大的力。如果在法向方向使用力命令，那么机器人在该方向上的位置就会自动调整为施加该力所需的位置，并能够沿着墙面运动。

[点击这里观看视频。](#)

Figure 8.11 - 一个简单的混合位置/力控制示例；机器人需要足够用力地下压，以保证手指与书之间的接触不会滑动，但同时又必须足够轻，使得书与桌子之间的接触能够滑动。这一点通过在世界坐标系 z 方向进行力控制可以自然实现。而在 x, y 平面内移动夹爪/书本，则更适合使用位置控制来描述。你可以在[一个练习](#)中更详细地探索这个例子。

位置控制还是力控制，不一定非得是一个二选一的决定。我们完全可以同时施加位置命令（就像刚度/阻抗控制那样）和一个“前馈”力命令：

$$u = -mg + K_p(p^{F_d} - p^F) + K_d(v^{F_d} - v^F) - f_{\text{feedforward}}^F.$$

显然，这种形式比式 (1) 中明确的“位置或力”模式更加一般。实际上，只要适当选择 K_p 、 K_d 和 $f_{\text{feedforward}}^F$ ，我们就能够在这个接口中实现显式的位置/力控制。正如我们将会看到的，这与 iiwa（以及许多其他力矩控制机器人）所提供的接口非常相似。

8.3 一般情况（使用机械臂方程）

使用浮动手指/夹爪是理解力控制主要概念的一种好方法，可以暂时避开许多细节。但现在是时候真正使用能够发送给机械臂的关节命令来实现这些策略了。

我们的出发点是理解：全驱动机器人机械臂的运动方程具有非常结构化的形式：

$$M(q)\ddot{q} + C(q, \dot{q})\dot{q} = \tau_g(q) + u + \sum_i J_i^T(q) f^{c_i}. \quad (2)$$

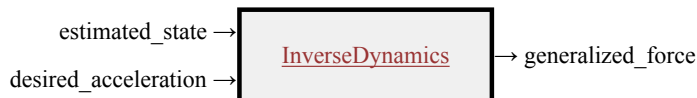
方程左侧只是“质量乘以加速度”的一种推广，其中包含质量矩阵 M 和科里奥利项 C 。右侧是（广义）力的总和： $\tau_g(q)$ 表示由重力引起的力， u 表示电机提供的关节力矩，而 f^{c_i} 是第 i 个接触产生的笛卡尔力，其中 $J_i(q)$ 是第 i 个“接触雅可比矩阵”。当我们描述将随机物体丢入箱体中的多体动力学时，我曾简要介绍过这些方程的一个版本，并且在[这里](#)有更多笔记。在本章剩余部分中，我们可以假设机器人固定在桌面上，因此没有任何浮动关节；所以我使用的是 $\dot{q}$ 和 $\ddot{q}$ ，而不是 v 和 $\dot{v}$ 。

8.3.1 轨迹跟踪

让我们暂时忽略接触项：

$$M(q)\ddot{q} + C(q, \dot{q})\dot{q} = \tau_g(q) + u.$$

正向动力学 (forward dynamics) 问题，也就是我们在仿真中使用的问题，是在给定关节力矩 u （以及 $q, \dot{q}$ ）的情况下计算加速度 $\ddot{q}$ 。**逆动力学 (inverse dynamics)** 问题，也就是我们可以在控制中使用的问题，是在给定 $\ddot{q}$ （以及 $q, \dot{q}$ ）的情况下计算 u 。作为一个系统，它看起来像这样：



这个系统实现了如下控制器：

$$u = M(q)\ddot{q}_d + C(q, \dot{q})\dot{q} - \tau_g(q),$$

从而得到的动力学为 $M(q)\ddot{q} = M(q)\ddot{q}_d$ 。由于我们知道 $M(q)$ 是可逆的，这意味着 $\ddot{q} = \ddot{q}_d$ 。

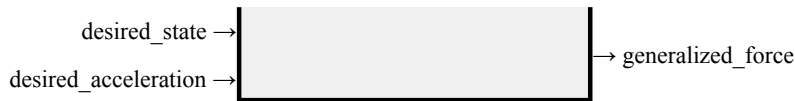
当我们把接触力重新加回来时，在实际中，我们只拥有动力学（质量矩阵等）以及状态 q 和 $\dot{q}$ 的估计值。在确定要发送的加速度命令时，我们需要考虑这些误差。例如，如果我们希望跟踪一条期望关节轨迹 $q_0(t)$ ，我们并不会仅仅对轨迹求两次导数然后直接命令 $\ddot{q}_0(t)$ ，而是可以改为命令例如：

$$\ddot{q}_d = \ddot{q}_0(t) + K_p(q_0(t) - q) + K_d(\dot{q}_0(t) - \dot{q}).$$

当然，也可以加入积分增益项。

由此得到的 **system** 还会额外增加一个用于期望状态的输入端口：





这种“逆动力学控制”在文献中还有其他几个名字，例如“计算力矩控制（computed-torque control）”，甚至“前馈加反馈线性化控制（feedforward-plus-feedback-linearizing control）” [8, 11.4.1.3]。

请自己检查： 关节刚度控制与逆动力学控制之间有什么区别？

采用关节刚度控制（取 $\tau_{ff} = 0$ ）时的闭环动力学为：

$$M(q)\ddot{q} + C(q, \dot{q})\dot{q} + K_p(q - q_d) + K_d(\dot{q} - \dot{q}_d) = \tau_{ext},$$

其中 τ_{ext} 总结了诸如接触等（未建模）力的影响。而采用逆动力学控制器时，闭环动力学为：

$$\ddot{q} + K_p(q - q_d) + K_d(\dot{q} - \dot{q}_d) = M^{-1}(q)\tau_{ext}.$$

因此，逆动力学控制器是以“加速度”而不是“力”的单位来编写反馈律；这类似于把内部矩阵塑造造成一个对角矩阵。

然而，在硬件上实现这种抵消的能力，会受到我们估计模型参数 $\hat{M}(q)$ 和 $\hat{C}(q, \dot{q})$ 准确性的限制。另一方面，刚度控制器能够实现大部分期望性能，但它只尝试抵消重力项（也许还包括摩擦项）；它并不塑造惯性。

8.3.2 关节刚度控制

在实践中，我们最常与 iiwa 交互的方式，是通过其“关节阻抗控制”模式；这一模式在 [9, 10] 中有很好的说明。就我们的目的而言，我们可以将其看作关节空间中的刚度控制：

$$u = -\tau_g(q) + K_p(q_d - q) + K_d(\dot{q}_d - \dot{q}) + \tau_{ff},$$

其中 K_p, K_d 是正的对角矩阵，而 τ_{ff} 是“前馈”力矩。在实践中，该控制器还包括一些关节摩擦补偿 [11]，但为了简洁起见，我在这里省略了这些摩擦项。这个控制器确实也会对惯性做一些塑造（因此它获得了“阻抗控制”而不仅仅是“刚度控制”的标签），但它只塑造转子惯量，而且用户并不会设置这些等效惯量。从用户的角度来看，它应该被视为刚度控制。

请自己检查： 带 PD 控制器的传统位置控制与关节刚度控制有什么区别？

二者在代数上的差别非常小。PD 控制通常具有如下形式：

$$u = K_p(q_d - q) + K_d(\dot{q}_d - \dot{q}),$$

而刚度控制则是：

$$u = -\tau_g(q) + K_p(q_d - q) + K_d(\dot{q}_d - \dot{q}).$$

换句话说，刚度控制试图抵消模型中的重力以及任何其他估计项，例如摩擦。按照这里的写法，这显然需要一个估计模型（对于 iiwa 我们有这个模型，但对于大多数带有大传动比传动装置的机械臂，我认为我们并没有这样的模型）以及力矩控制。但这种代数上的小差异，在实践中可能带来深远影响。控制器不再依赖误差来在稳态下达到关节位置。因此，我们可以把增益调得非常低，并且在实践中得到更加柔顺的响应，同时在没有外力矩时仍然实现良好的跟踪。

8.3.3 笛卡尔刚度与操作空间控制

对于我们在点手指示例中所做的控制，一个更好的类比是编写一个控制器，使机器人在世界坐标系中表现得像一个简单的动力系统。为此，我们必须在机器人上确定一个坐标系 E ，代表“末端执行器（end-effector）”，并希望在这个坐标系上施加这些简单动力学——理想情况下，该坐标系的原点应当是预期与环境接触的点。沿用我们对[运动学](#)和[微分运动学](#)的处理，我们将这个坐标系的正向运动学定义为：

$$p^E = f_{kin}(q), \quad v^E = \dot{p}^E = J(q)\dot{q}, \quad a^E = \ddot{p}^E = J(q)\ddot{q} + \dot{J}(q)\dot{q}. \quad (3)$$

我们之前其实还没有写过二阶导数，但它自然地由链式法则得到。另外，为了简单起见，我只把这里限制在笛卡尔位置上；也可以考虑末端执行器的朝向，但这需要谨慎定义三维朝向中的刚度（例如 [12]）。

机械臂控制中的一个重要思想是：我们实际上可以把机器人的动力学写在坐标系 E 中，方法是将关节力矩参数化为施加在刚体 B 的 E 点处的平移力 $f_u^{B_E}$ ： $u = J^T(q)f_u^{B_E}$ 。这来自[虚功原理](#)。通过将这一关系以及机械臂方程 (2) 代入 (3)，并假设唯一的外部接触发生在 E 处，我们可以写出如下动力学：

$$M_E(q)\ddot{p}^E + C_E(q, \dot{q})\dot{q} = f_g^{B_E}(q) + f_u^{B_E} + f_{ext}^{B_E}, \quad (4)$$

其中

$$M_E(q) = (JM^{-1}J^T)^{-1}, \quad C_E(q, \dot{q}) = M_E(JM^{-1}C - \dot{J}), \quad f_g^{B_E}(q) = M_EJM^{-1}\tau_g.$$

现在，如果我们简单地施加一个类似于关节空间中所用控制器的控制器，例如：

$$f_u^{B_E} = -f_g^{B_E}(q) + K_p(p^{E_d} - p^E) + K_d(\dot{p}^{E_d} - \dot{p}^E),$$

那么我们就可以在[末端执行器处](#)实现期望的闭环动力学：

$$M_E(q)\ddot{p}^E + C_E(q, \dot{q})\dot{q} + K_p(p^E - p^{E_d}) + K_d(\dot{p}^E - \dot{p}^{E_d}) = f_{ext}^{B_E}.$$

因此，如果我在末端执行器处推动机器人，整个机器人都会以一种方式运动，让我感觉像是在推动一个弹簧-阻尼系统。漂亮！

当 $J(q)$ 可能秩亏时（例如，如果你的自由度数量多于你试图在末端执行器处控制的自由度数量），你还会希望添加一些项，类似于用于[稳定雅可比矩阵零空间](#)的项。由于我们这里使用的是力矩而不是速度命令，自然的类比是写出一个关节居中 PD 控制器，使其在末端执行器控制的零空间中起作用：

$$u = J^T f_u^{B_E} + P[K_{p,joint}(q_0 - q) - K_{d,joint}\dot{q}].$$

这里的 P 是所谓的“动力学一致零空间投影” [13]。在微分 IK 中，我们是在速度控制器的零空间中操作；但在这里，我们必须在加速度的零空间中操作。

这种对任务空间动力学进行编程，并让次级关节空间目标在零空间中起作用的方法，由 [13] 提出，被称为[操作空间控制](#)（*operational space control*）。它可以推广为一种丰富的[优先级](#)任务空间与关节空间控制目标组合；这种优先级是通过将低优先级任务放入主任务的零空间中来实现的。与关节居中的微分逆运动学类似，这些方法也可以通过最小二乘优化的视角来理解。

8.3.4 关于 iiwa 的一些实现细节

底层阻抗控制器的实现包含许多细节，这些在 [9, 10] 中有很好的解释。尤其是，作者花了相当多篇幅来说明如何在执行器坐标中，而不是在关节坐标中实现阻抗律（记住，在两者之间存在弹性

传动)。我猜测偏好这种做法有许多微妙原因，但他们确实花了不少努力来证明：使用这个控制器可以给出真正的无源性论证。

iiwa 接口提供了一种笛卡尔阻抗控制模式。如果我们希望在末端执行器坐标中实现高性能刚度控制，那么当然应该使用它！iiwa 控制器的运行带宽（也就是更新频率）远高于我们通过所提供的网络 API 能够访问到的接口，而且其中包含许多他们花了大量精力做正确的实现细节。但在实践中我们并不使用它，因为如果不关闭机器人，我们就无法在关节阻抗控制和笛卡尔阻抗控制之间切换。唉。事实上，如果不停止机器人，我们甚至无法改变刚度增益，也无法改变坐标系 C （也就是“末端执行器位置”）。因此，我们保持在关节阻抗模式中，并在需要时通过 τ_{ff} 命令一些笛卡尔力。（如果你对驱动器细节感兴趣，那么我推荐阅读 [Franka Control Interface](#) 的文档，它更容易找到和阅读，并且与 iiwa 驱动器提供的功能非常相似。）

你可能还会注意到，我们在 `HardwareStation` 中为 `IiwaDriver` 提供的接口，接收的是机器人的期望关节位置，而不是期望关节速度。这是因为我们实际上无法向 iiwa 控制器发送期望关节速度。在实践中，我们认为它们会对我们的位置命令进行数值微分，以获得期望速度；这会增加几个控制时间步的延迟（有时还会出现非单调行为）。我认为，不允许我们发送自己的关节速度命令的原因，是为了让他们论文中的无源性论证能够成立，也可能是为了提供一个更简单/更安全的 API——他们不希望用户能够发送一系列彼此不一致的位置和速度命令（其中速度实际上并不是位置对时间的导数）。

由于该控制器试图抵消重力项，因此也可以把夹爪的集中惯量信息告诉 iiwa 固件。这一设置只会设置一次（在示教器中完成），并且在机器人运动过程中无法更改；如果你抓起了某个物体（甚至只是移动了手指），那么这些重力项的变化将不会在控制器中得到补偿。

虽然 Drake 中的 `JointStiffnessController` 是 iiwa 控制栈的最佳模型，但我们在仿真中通常改用 `InverseDynamicsController`。这是出于一个相当微妙的原因——数值性质更好。为了在物理上实现可比的刚度，微分方程中的有效刚度更小，因而可以使用更大的积分时间步长。在 iiwa 硬件上，我们通常为 `JointStiffnessController` 使用 $[800., 600, 600, 600, 400, 200, 200]$ Nm/rad 作为刚度值；但用这些值进行仿真需要很小的积分步长。

8.4 整合起来

8.5 孔中插销

机械臂力控制中的经典问题之一，受到机器人装配任务的启发，是在严格运动学公差下将两个零件配合在一起的问题。其中也许最干净、最著名的版本，就是将一个（圆形）插销插入一个（圆形）孔中的问题。如果你的插销是方的，而孔是圆的，那我就帮不上忙了。有趣的是，关于这一主题在 20 世纪 70 年代末和 80 年代初的大量文献，都来自 MIT 和 Draper Laboratory。

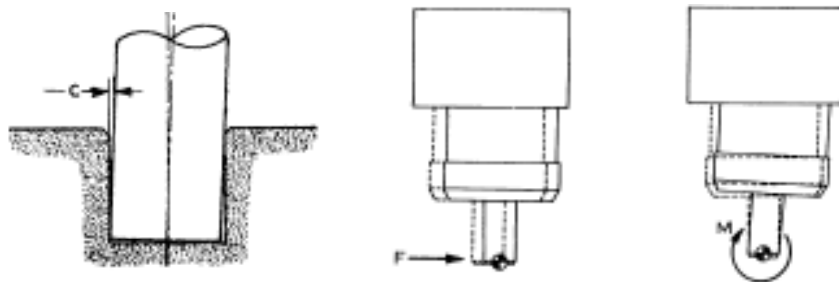


Figure 8.12 - (左) 具有运动学公差 c 的“孔中插销”插入问题。(中) 对接触力的期望响应。(右) 对接触力矩的期望响应。图像转载自 [14]（去除了一些符号）。

对于许多插销插入任务而言，在零件和/或孔的顶部添加一个小的倒角是可以接受的。这样一

来，水平方向上的小对准误差就可以通过水平方向上的简单柔顺性来处理。朝向上的未对准则更加有趣。小的朝向误差可能会导致物体被“卡住”，此时摩擦力会变得很大且含义不明确（在库仑模型下）；我们确实希望避免这种情况。同样，我们可以通过在接触处允许旋转柔顺性来避免卡住。我想你可以看出，这为什么是力控制的一个绝佳例子！

这里的关键洞见是：我们想要的旋转刚度，应该导致物体围绕零件/接触点旋转，而不是围绕夹具旋转。我们可以用刚度控制来实现这一点；但对于非常严格的公差，感知和带宽需求会成为真正的限制。力控制中最酷的结果之一是认识到：可以使用一种巧妙的物理机构，完全通过机械方式产生“远程中心柔顺性（remote-centered compliance, RCC）”，具有无限带宽且不需要任何感知[14]！

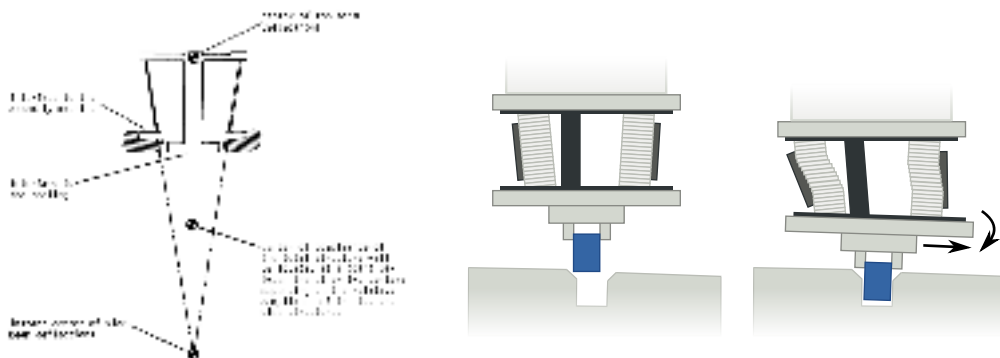


Figure 8.13 - 一种被动远程中心柔顺装置。（左）来自 [14] 的概念示意图。（右）机械实现示意图；图片来源：[Arne Nordmann, 2008](#)。一些示例产品见[这里](#)，一个不错的（较早的）视频见[这里](#)。

有趣的是，孔中插销问题不仅激发了力学与控制中的极其重要且有影响力的思想，也激发了运动规划中的重要思想 [15]。这些一般性思想对于各种以柔顺接触作为合理策略的任务都非常相关且适用，例如开门、工具使用等等。

8.6 练习

Exercise 8.1 (力与位置控制)

假设给定一个质点系统，其动力学如下：

$$m\ddot{x} = u + f^{F_c}$$

其中 x 是位置， u 是施加在质点上的输入力，而 f^{F_c} 是环境对点 F 在 c 处施加的接触力。为了同时控制机器人的位置和力，提出了如下控制器：

$$u = \underbrace{k_p(x_d - x) - k_d\dot{x}}_{\text{用于位置控制的反馈力}} - \underbrace{f_{desired}^{F_c}}_{\text{前馈力}}$$

定义该系统的误差为：

$$e = (x - x_d)^2 + (f^{F_c} - f_{desired}^{F_c})^2$$

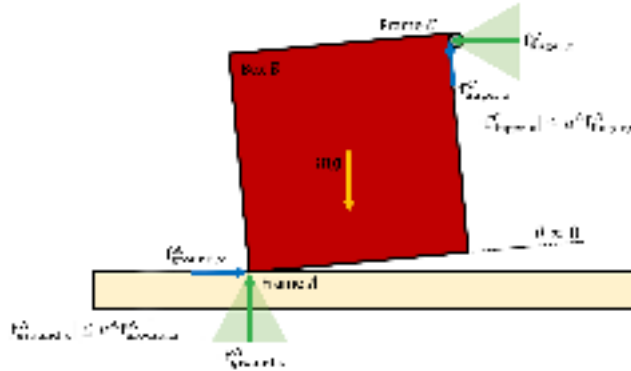
- 让我们考虑系统处于自由空间中的情况（即 $f^{F_c} = 0$ ），并且我们希望施加一个非零的期望力（ $f_{desired}^{F_c} \neq 0$ ），同时将位置驱动到零（ $x_d = 0$ ）。你将证明这是不可能的，也就是说，我们无法同时让期望力和期望位置的误差为零。你可以通过考虑稳态误差（即设置 $\dot{x} = 0, \ddot{x} = 0$ ）如何依赖于系统动力学、控制器、期望位置和期

望力来证明这一点。

- 现在考虑系统与位于 $x_d = 0$ 的墙发生刚体接触，并且命令了一个非零的期望力（ $f_{desired}^C > 0$ ）。通过考虑此时的稳态误差，证明系统可以实现零误差。

Exercise 8.2 (盒子翻起)

考虑上面示例中盒子翻起任务的初始阶段，此时 $\theta \approx 0$ 。令 B 表示盒子， A 表示盒子的支点， C 表示手指与盒子 B 之间的接触点。我们假设这些力仍然可以近似为沿坐标轴方向，但地面接触只作用在支点 A 处。



- 如果我们推得不够用力（即 $f_{finger,z}^C$ 太小），那么就无法产生足够的摩擦力（ $f_{finger,x}^C$ ）来抵消重力。通过对支点 A 求力矩和来得到 $\tau_{total,y}^A$ ，推导所需法向力 $f_{finger,z}^C$ 的下界，将其表示为 m, g 和 μ^C 的函数。你可以假设盒子是正方形的，并且重力的力臂是点 C 处力臂的一半。
- 如果我们推得太用力（即 $f_{finger,z}^C$ 太大），那么支点会向左滑动。首先，沿水平方向平衡力，推导 $f_{finger,z}^C$ 的上界。然后，沿 y 轴方向平衡力，得到一个包含 $f_{finger,z}^C$ 的方程。最后，将从水平和竖直方向力平衡中得到的关系，与第 (a) 部分中的力矩下界结合起来，完成所需法向力 $f_{finger,z}^C$ 上界的推导，将其表示为 m, g 和 μ^A 的函数。（假设 $\mu^A < 1$ ，并且我们已经实现了一个正的 $\tau_{total,y}^A$ 。）
- 结合前几个问题中的下界和上界，推导出为了使这一运动可行， μ^A, μ^C 必须满足的关系。如果 $\mu^A \approx 0.25$ （木材-金属），那么 μ^C 需要多大才能使运动可行？如果 $\mu^A = 1$ （混凝土-橡胶），又会怎样？

Exercise 8.3 (混合力-位置控制)

在这个练习中，你将分析并实现一个混合力-位置控制器来拖动一本书，就像我们在课堂上从[这个视频](#)中看到的那样。你将完全在 中完成。你将被要求完成以下步骤：

- 分析使这一运动可行的条件。
- 实现一个控制律来完成该运动。

Exercise 8.4 (孔中插销控制)

在这个练习中，你将尝试机器人操作中的孔中插销任务。具体来说，你将通过实现两种不同的刚度控制器，获得间接力控制方面的经验。你将完全在 中完成。你将被要求完成以下步骤：

- a. 为浮动插销实现刚度控制（其中构型空间和任务空间是相同的）。
- b. 为完整的平面 iiwa 实现一个操作空间刚度控制器。

REFERENCES

1. K. Salisbury, "Active stiffness control of a manipulator in Cartesian coordinates", *Proc. of the 19th IEEE Conference on Decision and Control*, 1980.
2. Neville Hogan, "Impedance Control: An Approach to Manipulation. Part I - Theory", *Journal of Dynamic Systems, Measurement and Control*, vol. 107, pp. 1--7, Mar, 1985.
3. Neville Hogan, "Impedance Control: An Approach to Manipulation. Part II - Implementation", *Journal of Dynamic Systems, Measurement and Control*, vol. 107, pp. 8--16, Mar, 1985.
4. Neville Hogan, "Impedance Control: An Approach to Manipulation. Part III - Applications", *Journal of Dynamic Systems, Measurement and Control*, vol. 107, pp. 17--24, Mar, 1985.
5. "Modeling and Control of Complex Physical Systems: The Port-Hamiltonian Approach", Springer-Verlag, 2009.
6. Luigi Villani and J De Schutter, "9", in: Force Control, Springer *Handbook of Robotics*, 2008.
7. Daniel E Whitney, "Historical perspective and state of the art in robot force control", *The International Journal of Robotics Research*, vol. 6, no. 1, pp. 3--14, 1987.
8. Kevin M Lynch and Frank C Park, "Modern Robotics", Cambridge University Press, 2017.
9. Christian Ott and Alin Albu-Schaffer and Andreas Kugi and Gerd Hirzinger, "On the passivity-based impedance control of flexible joint robots", *IEEE Transactions on Robotics*, vol. 24, no. 2, pp. 416--429, 2008.
10. Alin Albu-Schaffer and Christian Ott and Gerd Hirzinger, "A unified passivity-based control framework for position, torque and impedance control of flexible joint robots", *The international journal of robotics research*, vol. 26, no. 1, pp. 23--39, 2007.
11. Alin Albu-Schaffer and Gerd Hirzinger, "A globally stable state feedback controller for flexible joint robots", *Advanced Robotics*, vol. 15, no. 8, pp. 799--814, 2001.
12. H.J. Terry Suh and Naveen Kuppaswamy and Tao Pang and Paul Mitiguy and Alex Alspach and Russ Tedrake, "SEED: Series Elastic End Effectors in 6D for Visuotactile Tool Use", *Under review*, 2022. [[link](#)]
13. Oussama Khatib, "A Unified Approach for Motion and Force Control of Robot Manipulators: The Operational Space Formulation", *IEEE Journal of Robotics and Automation*, vol. 3, no. 1, pp. 43-53, February, 1987.
14. Samuel Hunt Drake, "Using compliance in lieu of sensory feedback for automatic assembly.", PhD thesis, Massachusetts Institute of Technology, 1978.
15. Tomas Lozano-Perez and Matthew Mason and Russell Taylor, "Automatic synthesis of fine-motion strategies for robots", *The International Journal of Robotics Research*, vol. 3, no. 1, pp. 3--24, 1984.

CHAPTER 9

目标检测与分割

我们对几何感知的研究，为估计已知物体的位姿提供了良好的工具。这些算法能够产生非常精确的估计，但仍然会受到局部极小值问题的影响。当场景变得更加杂乱/复杂，或者当我们需要处理许多不同类型的物体时，仅依靠这些方法本身已经无法提供充分的解决方案。

深度学习为我们提供了数据驱动的解决方案，与几何方法形成了极佳的互补。通过在海量数据集寻找相关性，深度学习已经被证明是一种极其有效的方式，用于解决这些更“全局性”的问题，例如检测芥末瓶是否真的存在于场景中、将图像/点云中与目标物体相关的部分分割出来，甚至还能提供一个粗略的位姿估计，并进一步通过几何方法进行精化。

互联网上已经有大量关于深度学习的资料，而我并不打算在这里重复或取代它们。不过，本章确实开启了对机器人操作中深度感知的探索，因此我觉得仍然有必要提供一点背景知识。

9.1 迈向大数据

9.1.1 众包标注数据集

现代计算机视觉革命毫无疑问是由大规模标注数据集的出现所推动的。其中最著名的无疑是 ImageNet，它凭借图像数量以及标签的准确性和实用性，远远超越了此前的数据集[1]。主导 ImageNet 创建工作的 Fei-fei Li 一直在做一些演讲，从历史角度讲述 ImageNet 是如何诞生的。[这里有一个演讲](#)，它（稍微）针对机器人学甚至机器人操作进行了调整；你也可以直接从[这里](#)开始观看。

[1] 描述了 ImageNet 中提供的标注类型：

.....标注主要分为两类：（1）图像级标注，即对图像中某类物体是否存在给出二值标签，例如“这张图像中有汽车”，但“没有老虎”；（2）物体级标注，即围绕图像中的某个物体实例提供紧密的边界框及类别标签，例如“在位置 (20,25) 附近有一把螺丝刀，其宽度为 50 像素，高度为 30 像素”。

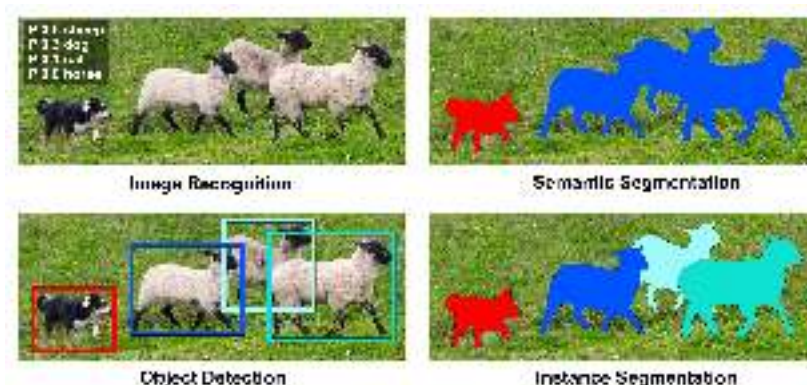


Figure 9.1 - COCO 数据集中一个带标注的示例图像，展示了图像级标注、物体级标注，以及类别/语义级或实例级分割之间的区别。

实际上，ImageNet 推动了目标检测的发展。类似地，COCO 数据集推动了像素级的实例级分割[2]，其中同一类别中的不同实例会被赋予唯一标签（同时也关联对应的类别标签）。COCO 的物

体类别数量少于 ImageNet，但每个类别包含更多实例。直到今天，我仍然觉得他们能够获得 250 万张像素级标注图像这件事令人震惊。我还记得 MIT 早期的一些项目，当时众包图像标注才刚刚起步（例如 LabelMe 项目 [3]）；Antonio Torralba 曾经开玩笑地说，他对通过众包获得的（近乎）像素级标注精度感到非常惊讶（而且他的母亲是一位特别高产且准确的标注者）！

事实证明，实例分割与机器人操作中的感知需求非常契合。在上一章中，我们面对的是一箱 YCB 物体。如果我们只想挑出芥末瓶，并且一次只抓取一个，那么就可以使用深度网络先对场景进行实例级分割，然后仅在分割后的点云上应用抓取策略。或者，如果我们确实需要估计某个物体的位姿（例如为了将它放置到指定姿态），那么对点云进行分割也能够显著提高几何位姿估计算法的成功率。

9.1.2 通过微调分割新类别

ImageNet 和 COCO 数据集包含了各种有趣类别的标签，包括奶牛、大象、熊、斑马和长颈鹿。它们也包含少量与机器人操作更相关的类别（例如盘子、叉子、刀和勺子），但并没有像 YCB 数据集中那样的芥末瓶或午餐肉罐头。那么我们该怎么办？难道我们必须重新开发同样的图像标注工具，并花钱请人帮我们标注成千上万张图像吗？

这些在实例级分割中表现出色的深度架构，最令人惊叹且近乎神奇的特性之一，就是它们能够迁移到新任务上（“迁移学习”）。一个在 ImageNet 或 COCO 这样的大型数据集上预训练过的网络，可以用相对少得多的标注数据进行微调，以适应与我们特定应用相关的一组新类别。事实上，这些架构通常被称为具有一个“骨干网络”和一个“头部”——为了训练一组新类别，通常可以直接移除已有的头部，并替换成适用于新标签的新头部。即使使用相对较小的数据集进行相对少量的训练，仍然可以获得令人惊讶的鲁棒性能。此外，似乎最初在多样化数据集（ImageNet 或 COCO）上进行训练，对于学习适用于广泛感知任务的鲁棒感知表示实际上非常重要。太不可思议了！

这是个好消息！但我们仍然需要一定数量的、针对目标物体的标注数据。过去几年中，出现了许多创业公司，它们的商业模式完全建立在帮助你完成数据集标注之上。但幸运的是，这并不是我们唯一的选择。

9.1.3 面向操作任务的标注工具

正如 LabelMe 这样的项目帮助简化了对从网络下载图像进行像素级标注的流程一样，也有许多工具被开发出来，用于简化机器人学中的标注流程。最早的例子之一是 [LabelFusion](#)，它将点云的几何感知与简单的用户界面相结合，从而能够非常快速地标注大量图像[4]。



Figure 9.2 - 来自 [LabelFusion](#) [4] 的多物体场景。（鼠标悬停可查看动画）

在 LabelFusion 中，用户提供包含若干目标物体的静态场景的多张 RGB-D 图像，以及这些物体的 CAD 模型。LabelFusion 使用一种稠密重建算法 ElasticFusion[5]，将各张图像中的点云合并为一个单一的稠密重建；这只是点云配准问题的另一个实例。该稠密重建算法还会定位相机相对于点云的位置。为了定位某个特定物体，例如上图中的电钻，LabelFusion 提供了一个简单的图形界面，要求用户在模型上点击三个点，并在场景中点击三个点，以建立“全局”对应关系，然后运行 ICP 来精化位姿估计。除了这一次配准能够在所有原始图像中提供带标签的位姿之外，还可以将 CAD 模型中的像素以已确定的位姿“渲染”到所有图像之上，从而得到精美的像素级标签。

像 LabelFusion 这样的工具可以非常快速地标注大量图像（用户点击三次，就能在许多图像中生成真实标签）。

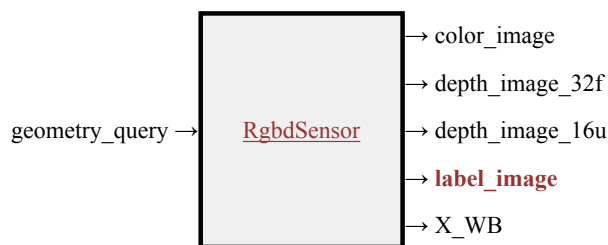
9.1.4 合成数据集

所有这些真实世界数据都极其宝贵。但我们手中还有另一个超级强大的工具：仿真！计算机视觉研究者传统上一直对使用合成图像训练感知系统非常怀疑，但随着游戏引擎级别的基于物理的渲染成为一种商品化技术，机器人学研究者已经积极地使用它来补充，甚至替代他们的真实世界数据集。如今，每年的机器人学会议都会定期围绕“sim2real”这一主题举办研讨会和/或辩论。对于任何特定场景或较窄类别的物体，我们通常可以生成足够精确的美术资源（其材料属性往往仍由美术人员调节），以及环境贴图/光照条件，使渲染图像在训练数据集中非常有效。更大的问题在于，我们是否能够生成一个足够多样化的数据集，其分布能够代表真实世界，从而像我们使用 ImageNet 训练那样训练出鲁棒的特征检测器。但对于许多严肃的机器人学团队而言，合成数据生成流水线已经显著增强，甚至取代了真实世界标注数据。

这背后有一个微妙的原因。人类对真实数据的标注虽然可以相当好，但永远不可能完美。标注错误可能会限制学习系统所能达到的总体性能上限[6]。即使我们承认渲染图像与自然图像之间存在差距，在某个阶段，能够生成任意大规模且具有完美像素级标签的数据集这一能力，实际上会使基于合成数据集的训练在真实世界测试集上的评估性能超过基于真实数据的训练。

就本章而言，我的目标是训练一个实例级分割系统，使其能够在我们的仿真图像上良好工作。对于这个用例，几乎没有争议！我将利用来自 COCO 的预训练骨干网络，并只使用合成数据进行微调。

你可能已经注意到了，我们一直在 Drake 中使用的 `RgbSensor` 实际上有一个“标签图像”输出口，只是我们还没有使用它。



这个输出端口正是为了支持我们这里的感知训练用例而存在的。它会输出一张与 RGB 图像相同的图像，只不过每个像素都会被用一个唯一的实例级标识符进行“着色”。



Figure 9.3 - 由 `RgbdSensor` 的“标签图像”输出端口提供的逐像素实例分割标签。我已经重新映射了颜色，使它们在视觉上更容易区分。

Example 9.1 (为实例分割生成训练数据)

我提供了一个简单脚本，它会运行我们在料箱抓取示例中的“杂乱生成器”，将随机 YCB 物体丢入料箱中。经过短暂仿真后，我会渲染 RGB 图像和标签图像，并将它们（连同一些包含实例和类别标识符的元数据）保存到磁盘。

我已经验证过这段代码可以在 Colab 上运行，但如果用这个未经优化的流程生成一个包含 1 万张图像的数据集，在我的大型开发台式机上大约需要一个小时。而且如果在本地运行，整理文件也更容易。因此我把这个示例作为 Python 脚本提供。

```
python3 segmentation/segmentation_data.py
```

你也可以直接跳过这一步！我已经把自己生成的 1 万张图像上传到[这里](#)。我们会在训练 notebook 中直接下载它。

9.1.5 自监督学习

9.1.6 更大规模的数据集

随着大语言模型（LLM）的兴起，一个非常自然的问题随之出现：我们如何获得一个用于计算机视觉的“基础模型”？这可以宽泛地定义为这样一种模型：它在基本上任何新图像上都具有出色的零样本预测性能，不需要提示，并且只需与非专家用户进行少量交互，就能替代在特定领域数据集上进行微调的需求。

Segment Anything [7] 于 2023 年早些时候发布；它是一个用于分割任务的基础模型。其关联数据集 SA-1B 在图像数量、图像分辨率以及带标签分割数量方面，都远远大于 COCO 等已有数据集。它

的巨大规模得益于一个“数据引擎”：该引擎使用越来越强大的 Segment Anything 模型版本来提供初始分割标签；随后这些输出会交给付费的专家图像标注人员，由他们调整/修正标签，并为模型遗漏的、越来越隐蔽的图像部分添加标签。也许，针对机器人特定数据集进行微调这件事，已经或很快就会成为过去。

9.2 目标检测与分割

关于现代目标检测与分割流水线，有很多内容值得了解。这里我只介绍最基础的部分。

对于图像识别（见图 1），可以设想训练一个标准的卷积网络，它以整张图像作为输入，并输出该图像包含羊、狗等物体的概率。事实上，这些架构甚至可以很好地用于语义分割，其中输入是一张图像，输出则是另一张图像；这一方向中一个著名的架构是全卷积网络（Fully Convolutional Network, FCN）[8]。但对于目标检测和实例分割而言，网络输出的数量甚至可能发生变化。我们该如何训练一个网络，使其输出数量可变的检测结果呢？

解决这一问题的主流方法，是首先将输入图像划分为许多（比如大约 1000 个量级的）可能代表有趣子图像的重叠区域。然后，我们可以分别在每个子图像上运行自己喜欢的图像识别和/或分割网络，并为每个被判定为具有高概率的区域输出一个检测结果。为了输出紧密的边界框，检测网络还会被训练输出一个“边界框精化”结果，用于从最终区域中选择一个子集作为边界框。最初，这些候选区域是通过更传统的图像预处理算法生成的，如 R-CNN（Regions with CNN Features）[9]。但“Fast”和“Faster”版本的 R-CNN 甚至用学习得到的“区域候选网络”替代了这些预处理步骤[10, 11]。

对于实例分割，我们将使用非常流行的 Mask R-CNN 网络，它将上述所有思想结合在一起，使用区域候选网络以及全卷积网络来完成目标检测和掩码预测 [12]。在 Mask R-CNN 中，掩码与目标检测结果并行评估，并且实际返回的只是对应于最可能检测结果的那些掩码。在撰写本文时，最新且性能最好的 Mask R-CNN 实现位于 Facebook AI Research 的 Detectron2 项目中。但那个版本并不像最初发布在 PyTorch torchvision 包中的版本那样用户友好且简洁；因此我们这里的实验将继续使用 torchvision 版本。

Example 9.2 (为料箱抓取微调 Mask R-CNN)

下面的 notebook 会加载我们的 1 万张图像数据集，以及一个在 COCO 数据集上预训练过的 Mask R-CNN 网络。然后，它会将预训练网络的头部替换为一个新的头部，使其输出数量适合我们的 YCB 识别任务，接着使用我的新数据集仅训练 10 个 epoch。

[Open in Colab](#)

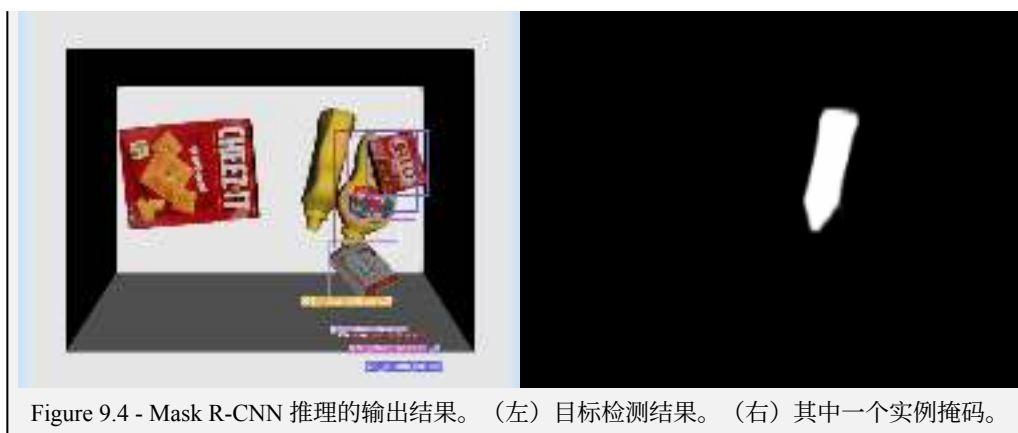
(训练 Notebook)

训练这么大的网络（它在磁盘上大约会占用 150MB）并不快。我强烈建议你在训练时，立即运行训练单元之后的那个单元，这样即使你的 Colab 会话关闭，权重也会被保存并下载下来。但当你完成后，你应该会得到一个崭新的网络，用于对料箱中的 YCB 物体进行实例分割！

我还提供了第二个 notebook，你可以用它来加载并评估训练好的模型。如果你不想等待自己的模型完成训练，也可以查看我已经训练好的模型！

[Open in Colab](#)

(推理 Notebook)



9.3 整合起来

我们可以在操作任务中使用 Mask R-CNN 推理，从料箱中进行选择性抓取……

9.4 变体与扩展

9.4.1 使用自监督学习进行预训练

9.4.2 利用大规模模型

这些笔记的目标之一，是考虑“开放世界”操作——构建一条能够在此前未见过的环境中、面对未见过的模型时执行有用任务的操作流水线。我们怎么可能为机器人将来需要操作的每一个物体都提供带标签的实例呢？

开放世界推理中最引人注目的例子，来自所谓的“基础模型”[\[13\]](#)。在机器人学研究中被最快采用的基础模型，是大型视觉 + 文本模型 [CLIP \[14\]](#)。

更多内容即将推出……

9.5 练习

Exercise 9.1 (标签生成)

在本练习中，你将研究一种简单技巧，用于为 Mask-RCNN 自动生成训练数据。你将完全在 中完成工作。你需要完成以下步骤：

- 从预处理后的点云中自动生成掩码标签。
- 分析该方法在更复杂场景中的适用性。
- 应用数据增强技术生成更多训练数据。

Exercise 9.2 (分割 + 对向抓取)

在本练习中，你将使用 Mask-RCNN 以及我们之前开发的对向抓取策略，在给定点云的情况下选择一个抓取。你将完全在 中完成工作。你需要完成以下步骤：

- a. 自动过滤点云，只保留对应于我们目标抓取物体的点。
- b. 分析多相机设置的影响。
- c. 思考为什么过滤点云是该抓取流水线中的一个有用步骤。
- d. 讨论我们可以如何改进这条抓取流水线。

Exercise 9.3 (视觉-语言分割)

在本练习中，你将探索如何结合视觉-语言模型（VLM）和 Segment Anything Model（SAM）来实现由语言驱动的目标分割。你将完全在 中完成工作。你需要完成以下步骤：

- a. 分析 SAM 的分割能力，并理解其在目标识别方面的局限性。
- b. 使用视觉-语言模型根据自然语言提示生成边界框。
- c. 将 VLM 生成的边界框与 SAM 结合，为指定物体生成精确的分割掩码。

REFERENCES

1. Olga Russakovsky and Jia Deng and Hao Su and Jonathan Krause and Sanjeev Satheesh and Sean Ma and Zhiheng Huang and Andrej Karpathy and Aditya Khosla and Michael Bernstein and others, "Imagenet large scale visual recognition challenge", *International journal of computer vision*, vol. 115, no. 3, pp. 211--252, 2015.
2. Tsung-Yi Lin and Michael Maire and Serge Belongie and James Hays and Pietro Perona and Deva Ramanan and Piotr Dollár and C Lawrence Zitnick, "Microsoft coco: Common objects in context", *European conference on computer vision*, pp. 740--755, 2014.
3. Bryan C Russell and Antonio Torralba and Kevin P Murphy and William T Freeman, "LabelMe: a database and web-based tool for image annotation", *International journal of computer vision*, vol. 77, no. 1-3, pp. 157--173, 2008.
4. Pat Marion and Peter R. Florence and Lucas Manuelli and Russ Tedrake, "A Pipeline for Generating Ground Truth Labels for Real RGBD Data of Cluttered Scenes", *International Conference on Robotics and Automation (ICRA)*, Brisbane, Australia, May, 2018. [[link](#)]
5. Thomas Whelan and Renato F Salas-Moreno and Ben Glocker and Andrew J Davison and Stefan Leutenegger, "ElasticFusion: Real-time dense SLAM and light source estimation", *The International Journal of Robotics Research*, vol. 35, no. 14, pp. 1697--1716, 2016.
6. Curtis G Northcutt and Anish Athalye and Jonas Mueller, "Pervasive label errors in test sets destabilize machine learning benchmarks", *arXiv preprint arXiv:2103.14749*, 2021.
7. Alexander Kirillov and Eric Mintun and Nikhila Ravi and Hanzi Mao and Chloe Rolland and Laura Gustafson and Tete Xiao and Spencer Whitehead and Alexander C Berg and Wan-Yen Lo and others, "Segment anything", *arXiv preprint arXiv:2304.02643*, 2023.
8. Jonathan Long and Evan Shelhamer and Trevor Darrell, "Fully convolutional networks for semantic segmentation", *Proceedings of the IEEE conference on computer vision and pattern recognition*, pp. 3431--3440, 2015.
9. Ross Girshick and Jeff Donahue and Trevor Darrell and Jitendra Malik, "Rich feature hierarchies for accurate object detection and semantic segmentation", *Proceedings of the IEEE conference on computer vision and pattern recognition*, pp. 580--587, 2014.

10. Ross Girshick, "Fast r-cnn", *Proceedings of the IEEE international conference on computer vision* , pp. 1440--1448, 2015.
11. Shaoqing Ren and Kaiming He and Ross Girshick and Jian Sun, "Faster r-cnn: Towards real-time object detection with region proposal networks", *Advances in neural information processing systems* , pp. 91--99, 2015.
12. Kaiming He and Georgia Gkioxari and Piotr Dollár and Ross Girshick, "Mask R-CNN", *Proceedings of the IEEE international conference on computer vision* , pp. 2961--2969, 2017.
13. Rishi Bommasani and Drew A Hudson and Ehsan Adeli and Russ Altman and Simran Arora and Sydney von Arx and Michael S Bernstein and Jeannette Bohg and Antoine Bosselut and Emma Brunskill and others, "On the opportunities and risks of foundation models", *arXiv preprint arXiv:2108.07258*, 2021.
14. Alec Radford and Jong Wook Kim and Chris Hallacy and Aditya Ramesh and Gabriel Goh and Sandhini Agarwal and Girish Sastry and Amanda Askell and Pamela Mishkin and Jack Clark and others, "Learning transferable visual models from natural language supervision", *International Conference on Machine Learning* , pp. 8748--8763, 2021.

CHAPTER 10

用于操作的深度感知

在上一章中，我们讨论了用于目标检测和（实例级）分割的深度学习方法；这些都是处理 RGB 图像的通用任务，在计算机视觉中被广泛使用。仅凭检测和分割，就可以与几何感知结合起来，例如，只在分割出的点云中估计某个已知物体的位姿，而不是在整个场景中估计；或者只在分割出的点云上运行我们的点云抓取选择算法，以便抓取感兴趣的物体。

深度学习用于感知时最令人惊叹的特性之一是：我们可以在另一个数据集（如 ImageNet 或 COCO）上预训练，甚至可以在另一个任务上预训练，然后再在我们特定领域的数据集或任务上进行微调。但是，对于操作任务而言，什么才是合适的感知任务？目标检测和分割是一个很好的起点，但为了操作场景或物体，我们通常还想了解更多信息。这正是本章的主题。

这个问题有一个潜在答案，我们会推迟到后面的章节再讨论：学习端到端的“视觉运动（visuomotor）”策略，有时也亲切地称为“从像素到力矩（pixels to torques）”。在这里，我希望我们首先思考的是：如何将基于深度学习的感知系统，与我们一直在构建的、用于规划和控制的强大现有（基于模型的）工具结合起来。

我将从一个我们已经考虑过的感知任务的深度学习版本开始：物体位姿估计。

10.1 位姿估计

我们在[几何感知章节](#)中较为详细地讨论了位姿估计，并得到了一些要点。最重要的是，几何方法只能非常有限地利用 RGB 值；但这些 RGB 信息对于确定一个位姿来说极其有价值。仅靠几何并不能讲完整个故事。另一个微妙的经验是，ICP 损失虽然在概念上非常清晰，但并不能充分捕捉诸如非穿透约束和自由空间约束这类更丰富的概念。随着二维计算机视觉中最初的核心问题开始显得已经被“解决”，我们看到计算机视觉社区对三维感知的兴趣和活动迅速增长，这对机器人学来说是一件好事！

这个问题在概念上最简单的版本是：我们希望从单张 RGB 图像中估计一个已知物体的位姿。那么，我们应该如何训练一个专门针对芥末瓶（例如）的深度网络，使它输入一张 RGB 图像，并输出一个位姿估计？当然，如果我们能够做到这一点，我们也可以将这个想法应用到例如由目标识别/实例分割系统输出的边界框裁剪出的图像上。

10.1.1 位姿表示

我们再次必须面对一个问题：如何最好地表示位姿。许多早期架构将位姿空间离散化为若干区间，并将位姿估计表述为一个分类问题；但这一趋势最终转向了位姿回归[\[1, 2\]](#)。回归三个数来表示 x - y - z 平移似乎很清楚，但对于如何表示三维朝向，我们有很多选择（例如欧拉角、单位四元数、旋转矩阵，……），而我们的选择会影响学习和泛化性能。

对于输出单个位姿的情况，像[\[3\]](#)和[\[4\]](#)这样的工作认为，许多旋转参数化方式都存在不连续性问题。他们建议让网络为旋转输出 6 个数（然后通过 Gram-Schmidt 正交化投影到 $SO(3)$ ），或者输出旋转矩阵完整的 9 个数，并通过[SVD 正交化](#)投影回 $SO(3)$ 。

也许更实质性的是，许多工作指出，输出一个单一的“正确”位姿根本是不够的[\[5, 6\]](#)。当物体具有旋转对称性，或者被严重遮挡时，输出整个姿态分布要合适得多。对于离散化表示来说，表示一个类别分布是非常自然的；但我们该如何表示连续位姿上的分布呢？一个非常优雅的选择是

Bingham 分布，它给出了应用于单位四元数的高斯分布的自然推广 [7, 8, 6]。

10.1.2 损失函数

无论网络输出的位姿表示和真实标签采用哪一种形式，都必须选择合适的损失函数。基于四元数的损失函数可以用来计算两个朝向之间的测地距离，并且当然应该比例如欧拉角上的最小二乘度量更合适。更昂贵、但可能更合适的方法是，将损失函数写成重建误差的形式，这样网络就不会因为例如它根本不可能处理的对称性而受到人为惩罚 [9]。

训练一个网络来输出完整的位姿分布，会引出关于损失函数选择的更多有趣问题。虽然可以仅根据带有真实位姿标注的数据统计量来训练该分布（同样需要在最大似然损失与均方误差之间进行选择），但也可以利用我们对对称性的理解来提供更直接的监督。例如，[5] 使用图像差异来高效地（但以启发式方式）估计每个样本的真实协方差。

10.1.3 位姿估计基准

6D 物体位姿估计基准 (BOP) [9]。

10.1.4 局限性

尽管位姿估计是一个自然的任务，并且将估计出的位姿接入我们的许多机器人流程也很直接，但我非常强烈地认为，这通常并不是将感知连接到规划和控制的正确选择。虽然已经有人尝试将位姿推广到物体类别上 [10]，但位姿估计仍然与已知物体紧密绑定，这让人感觉很受限制。准确估计一个物体的位姿是困难的，而且对于操作来说往往并不是必要的。

10.2 抓取选择

在 [11, 12] 中，J.J. Gibson 提出了他影响深远的 *可供性理论 (theory of affordances)*，其中他将感知的作用描述为服务于行为（而行为则由感知控制）。在我们的情形中，可供性描述的不是物体/环境是什么，也不是它的位姿，而是它为机器人提供了哪些行动机会。重要的是，人们有可能在不显式估计位姿的情况下估计可供性。

其中两个最早且最成功的例子是 [13, 14] 和 [15, 16]，它们尝试直接从原始 RGB-D 输入中学习抓取可供性。[13] 使用了一种抓取选择策略，这种策略与我们在对向抓取工作流中介绍的 *抓取采样器* 非常相似（并且在一定程度上启发了它）。但除了几何启发式方法之外，[13] 还训练了一个小型 CNN 来预测抓取成功率。该 CNN 的输入是抓取附近像素和局部几何估计的体素化摘要，输出则是对抓取成功或失败的二分类预测。类似地，[15] 学习了一个“抓取质量”CNN，它以抓取坐标系中的深度图像（加上相机坐标系中的抓取深度）作为输入，并输出抓取成功的概率。

Transporter 网络 [17]

10.3 (语义) 关键点

用于类别级机器人操作的关键点可供性 (kPAM) [18, 19]。

10.4 稠密对应

[20], DynoV2, ...

10.5 场景流

10.6 任务级状态

10.7 其他感知任务 / 表示

上面的覆盖范围必然是不完整的，而且这个领域发展很快。 这里快速“致意”一下其他几个非常相关的想法。

更多内容即将推出.....

10.8 练习

Exercise 10.1 (Deep Object Net 与对比损失)

在这个问题中，你将进一步探索 Dense Object Nets，它们已在课堂中介绍过。 Dense Object Nets 能够快速学习用于视觉理解和操作的一致像素级表示。 Dense Object Nets 之所以强大，是因为它们预测的表示既适用于刚性物体，也适用于非刚性物体。 它们还可以泛化到同一类别中的新物体，并且可以通过自监督学习进行训练。 在这个问题中，你将在 中进行操作： 首先实现用于训练 Dense Object Nets 的损失函数， 然后使用训练好的 Dense Object Net 预测图像之间的对应关系。

REFERENCES

1. Siddharth Mahendran and Haider Ali and René Vidal, "3d pose regression using convolutional neural networks", *Proceedings of the IEEE International Conference on Computer Vision Workshops*, pp. 2174--2182, 2017.
2. Yu Xiang and Tanner Schmidt and Venkatraman Narayanan and Dieter Fox, "Posecnn: A convolutional neural network for 6d object pose estimation in cluttered scenes", *arXiv preprint arXiv:1711.00199*, 2017.
3. Yi Zhou and Connelly Barnes and Jingwan Lu and Jimei Yang and Hao Li, "On the continuity of rotation representations in neural networks", *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pp. 5745--5753, 2019.
4. Jake Levinson and Carlos Esteves and Kefan Chen and Noah Snavely and Angjoo Kanazawa and Afshin Rostamizadeh and Ameesh Makadia, "An analysis of svd for deep rotation estimation", *Advances in Neural Information Processing Systems*, vol. 33, pp. 22554--22565, 2020.
5. Kunimatsu Hashimoto* and Duy-Nguyen Ta* and Eric Cousineau and Russ Tedrake, "KOSNet: A Unified Keypoint, Orientation and Scale Network for Probabilistic 6D Pose Estimation", *Under review*, 2020. [[link](#)]
6. Haowen Deng and Mai Bui and Nassir Navab and Leonidas Guibas and Slobodan Ilic and Tolga

- Birdal, "Deep bingham networks: Dealing with uncertainty and ambiguity in pose estimation", *International Journal of Computer Vision*, vol. 130, no. 7, pp. 1627--1654, 2022.
7. Jared Marshall Glover, "The Quaternion Bingham Distribution, 3D Object Detection, and Dynamic Manipulation", PhD thesis, Massachusetts Institute of Technology, May, 2014.
 8. Valentin Peretroukhin and Matthew Giamou and David M Rosen and W Nicholas Greene and Nicholas Roy and Jonathan Kelly, "A smooth representation of belief over so (3) for deep rotation learning with uncertainty", *arXiv preprint arXiv:2006.01031*, 2020.
 9. Tomás Hodan and Martin Sundermeyer and Bertram Drost and Yann Labbé and Eric Brachmann and Frank Michel and Carsten Rother and Jiri Matas, "BOP Challenge 2020 on 6D Object Localization", *European Conference on Computer Vision Workshops (ECCVW)*, 2020.
 10. He Wang and Srinath Sridhar and Jingwei Huang and Julien Valentin and Shuran Song and Leonidas J Guibas, "Normalized object coordinate space for category-level 6d object pose and size estimation", *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pp. 2642--2651, 2019.
 11. James J Gibson, "The theory of affordances", *Hilldale, USA*, vol. 1, no. 2, pp. 67--82, 1977.
 12. James J Gibson, "The ecological approach to visual perception", Psychology press, 1979.
 13. Andreas ten Pas and Marcus Gualtieri and Kate Saenko and Robert Platt, "Grasp pose detection in point clouds", *The International Journal of Robotics Research*, vol. 36, no. 13-14, pp. 1455--1473, 2017.
 14. Andreas Ten Pas and Robert Platt, "Using geometry to detect grasp poses in 3d point clouds", *Robotics Research: Volume 1*, pp. 307--324, 2018.
 15. Jeffrey Mahler and Jacky Liang and Sherdil Niyaz and Michael Laskey and Richard Doan and Xinyu Liu and Juan Aparicio Ojea and Ken Goldberg, "Dex-net 2.0: Deep learning to plan robust grasps with synthetic point clouds and analytic grasp metrics", *arXiv preprint arXiv:1703.09312*, 2017.
 16. Jeffrey Mahler and Ken Goldberg, "Learning deep policies for robot bin picking by simulating robust grasping sequences", *Conference on robot learning*, pp. 515--524, 2017.
 17. Andy Zeng and Pete Florence and Jonathan Tompson and Stefan Welker and Jonathan Chien and Maria Attarian and Travis Armstrong and Ivan Krasin and Dan Duong and Vikas Sindhwani and others, "Transporter networks: Rearranging the visual world for robotic manipulation", *Conference on Robot Learning*, pp. 726--747, 2021.
 18. Lucas Manuelli* and Wei Gao* and Peter Florence and Russ Tedrake, "kPAM: KeyPoint Affordances for Category-Level Robotic Manipulation", *arXiv e-prints*, pp. arXiv:1903.06684, Mar, 2019. [[link](#)]
 19. Wei Gao and Russ Tedrake, "kPAM 2.0: Feedback control for generalizable manipulation", *IEEE Robotics and Automation Letters*, 2020. [[link](#)]
 20. Peter R. Florence* and Lucas Manuelli* and Russ Tedrake, "Dense Object Nets: Learning Dense Visual Object Descriptors By and For Robotic Manipulation", *Conference on Robot Learning (CoRL)*, October, 2018. [[link](#)]

CHAPTER 11

强化学习

如今，强化学习（RL）引起了很多关注，也已经有大量相关文献。人们可能认为属于强化学习算法范畴的内容，其范围也已经显著扩大。Sutton 和 Barto 的经典著作（如今已经更新）[1] 仍然是强化学习最好的入门读物。若想了解更偏理论的视角，可以参考 [2, 3]。此外，也有一些很棒的在线课程：[Stanford CS234](#)、[Berkeley CS285](#)、[DeepMind x UCL](#)。

本章的目标是提供足够的基础知识，让大家站在同一个起点上，但主要聚焦于那些与机器人操作尤其相关的强化学习思想（以及示例）。而机器人操作是强化学习的绝佳试验场，因为它需要丰富的感知能力，需要在多样化环境中运行，并且可能涉及复杂的接触力学。许多核心的应用研究成果，都是由操作任务示例所驱动，并在这些示例上得到展示的！

11.1 强化学习软件

现在网上已经有大量种类繁多的强化学习工具箱，它们在质量和复杂程度上差异很大。但有一个标准已经明确胜出，成为默认接口：为了连接各种强化学习算法，应该用它来封装自己的仿真器，这就是 [Gymnasium](#)（通常仍被简称为“Gym”）。

值得花一分钟体会一下 [Gymnasium 环境](#)（`gymnasium.Env`）接口与 [Drake System](#) 接口之间的差别；我认为这非常能说明问题。我的目标（在 [Drake](#) 中）是为你提供一个丰富而优雅的接口，用于表达和优化动力系统，并暴露、利用控制方程中的所有可能结构。而 [Gym](#) 的目标则是暴露绝对最少的细节，从而能够轻松地把任何可能的系统封装到同一个通用接口之下（不管它是机器人、Atari 游戏，甚至是一个[编译器](#)都无所谓）。几乎从定义上讲，你可以把任何 [Drake](#) 系统封装成一个 [Gym](#) 环境。

Example 11.1 (将 Drake 仿真用作 Gym 环境)

[Gym](#) 环境是对仿真器的一个极其简单的封装，它提供了一个[非常基础的接口](#)，最典型地包括 `reset()`、`step()`、`render()`。`step()` 方法会返回当前观测和单步奖励（以及一些额外的终止条件）。

你可以使用以下代码，将任何 [Drake](#) 仿真封装到 [OpenAI Gym](#) 环境中：

```
from pydrake.gym import DrakeGymEnv
```

[DrakeGymEnv](#) [构造函数](#)接受一个 [Simulator](#)，以及一个与动作相关联的输入端口、一个与观测相关联的输出端口等。对于奖励，你可以将其实现为 [Simulator Context](#) 的一个简单函数，或者实现为另一个输出端口。

[DrakeGymEnv](#) 是围绕一个 [Simulator](#)（而不仅仅是一个 [System](#)）构建的，或者围绕一个能够生成随机 [Simulator](#) 的函数构建，因为你可能希望控制积分器参数，或者让每次 rollout 都包含同一个机器人但处于不同环境中，并且可能有不同数量的物体。这意味着底层的 [System](#) 可能具有不同数量的状态/端口。能够生成仿真器的函数这一概念被称为 [SimulatorFactory](#)，它是 [Drake](#) 中[随机系统建模框架](#)的核心。

你也可以在 [Drake](#) 生态系统中使用任何 [Gym](#) 环境；只是无法应用 [Drake](#) 所提供的一些更高级算法。当然，我认为你也应该在自己的强化学习工作中使用 [Drake](#)（很多人确实这样做），因为它提

供了丰富的动力系统库，这些系统都经过严格编写和测试，其中包括一个非常适合处理接触问题的优秀物理引擎，并且保留了将强化学习方法与更偏模型驱动的替代方案进行正面对比的可能性。我承认我可能有一点偏见。无论如何，这就是我们在这些笔记中将采用的方法。

有些人可能会认为，你越是深思熟虑地为系统建模，就越会把更多假设固化进去，从而让自己更容易受到“sim2real”差距的影响；但我认为事实并非如此。深思熟虑的建模包括构建不确定性模型，使其能够覆盖我们希望探索的或窄或宽的一类系统；当我们能够让不确定性模型本身具有结构性时，就会产生很好的效果。我认为，在强化学习与控制的交叉领域，等待我们解决的最根本挑战之一，是更深入地理解这样一类模型：它们既要足够丰富，能够描述我们在操作任务中探索的问题所具有的多样性和复杂性（例如以 RGB 相机作为输入），同时又要提供一定程度的结构，使我们能够利用更强大的算法。重要的是，这些模型应该随着数据不断扩展和改进。

另一个观点：强化学习方法之所以成功，部分原因在于它很容易上手。它所需训练较少，因此吸引了更多人和资金投入其中。这非常有价值。另一方面，控制作为一个领域，却在某种程度上有意让自己显得晦涩难懂，也许部分是出于傲慢。这或许才是更苦涩的教训。

Gymnasium 提供的是强化学习环境接口，但并不提供实际强化学习算法的实现。算法方面也有大量流行的代码仓库。在撰写本文时，我推荐 [Stable Baselines 3](#)：它在 PyTorch 中提供了一套非常优秀且文档编写得很用心的实现。

- env 接口现在已经标准化了，但 policy 接口还没有。每个人都自己实现，且大多假设以 nn.Module 作为基类。- SB3 允许你定义自定义的“policy”（实际上是 policy + value function），但目前不支持动态 policy（LSTM 曾在某个时候被移除；希望之后能恢复）- 我非常不喜欢把 value function 和 Q function 的概念直接应用到 observations 上。（SB3 是这样，其他大多数包也是如此）。这不是好的做法。

还有另一类算法与强化学习非常相关，尽管它们并不是专门为强化学习设计的，那就是黑盒优化算法。我非常喜欢 [Nevergrad](#)，这里也会使用它。

我们在 anzu 中有一些相关的 nevergrad $\Leftrightarrow$ drake（例如 mathematicalprogram）代码。

11.2 策略梯度方法

11.2.1 黑盒优化

Example 11.2 (黑盒优化。)

即将在 deepnote 上提供（我还需要处理依赖项）。该示例可在 `rl/black_box.ipynb` 中找到。

11.2.2 随机最优控制

11.2.3 使用策略的梯度，而不是环境的梯度

你可以在[这里](#)找到关于这些算法推导过程及一些基础分析的更多细节。

11.2.4 REINFORCE、PPO、TRPO

Example 11.3 (将盒子翻起。)

即将在 `deepnote` 上提供（我还需要处理依赖项）。该示例可在 `rl/box_flipup.ipynb` 中找到。

11.2.5 操作任务的控制应该是容易的

现在是理论强化学习与控制领域发展的绝佳时期：控制领域的专家正在接纳来自机器学习的新技术和新洞见，反之亦然。举一个简单的例子，我们越来越清楚地认识到，尽管许多经典控制问题（如线性二次调节器）的代价函数景观相对于典型的策略参数并不是凸的，但我们现在已经理解，梯度下降在这些问题上仍然有效（不存在局部极小值），而且我们能够作出此类结论的问题类别/参数化方式也正在迅速扩大。

11.3 基于价值的方法

11.4 基于模型的强化学习

11.5 练习

Exercise 11.1 (随机优化)

在本练习中，你将实现一种不需要精确解析梯度的随机优化方案。你将完全在 `中` 完成工作。你需要完成以下步骤：

- 使用精确解析梯度实现梯度下降。
- 使用近似梯度实现随机梯度下降。
- 证明随机更新的期望值不会因基线而改变。
- 使用基线实现随机梯度下降。

Exercise 11.2 (REINFORCE)

在本练习中，你将在一个推箱子任务上实现原始的 REINFORCE 算法。你将完全在 `中` 完成工作。你需要完成以下步骤：

- 实现策略损失函数。
- 实现价值损失函数。
- 实现优势函数。

Exercise 11.3 (使用强化学习分析箱子翻转)

在本练习中，你将分析一个训练用来翻转箱子的 PPO 策略的行为。与 REINFORCE 类似，PPO 是一种策略梯度方法，它直接优化策略参数以最大化价值函数。为了得到一个更容易分析的问题，我们将使用第 8 章中的箱子翻起示例。我们的机器人将是一个简单的点接触手指，目标是将箱子翻转过来。你可以在[这里](#)找到用于训练该策略的代码。

- a. 查看用于生成环境的代码。令 θ 表示箱子相对于竖直方向的角度， ω 表示箱子的角速度， q_f 表示观测到的手指位置， v_f 表示手指速度， u_f 表示给手指的指令位置。这里用于训练策略的奖励函数是什么？请用数学形式写出它（使用取模运算符来处理角度的环绕问题）。奖励函数中的各个项分别表示什么？为什么这些项是合理的？
- b. 虽然这里我们不会深入介绍 PPO 的具体工作细节，但它的工作方式与 REINFORCE 相当类似，只是同时使用了：（i）一个学习得到的价值函数来降低方差；以及（ii）一个近似目标函数，并通过裁剪每个样本的损失来施加信赖域约束，从而确保每一步中策略不会被更新得过多。请简要解释你认为：（a）为什么 PPO 可能比 REINFORCE 更稳定且样本效率更高；以及（b）如果裁剪限制设置得过小或过大，你预计 PPO 在箱子翻转任务中的表现会如何。
- c. 我们已经训练了一个基于 PPO 的策略，用于翻转箱子，共训练 3,000,000 步（关于该策略在各个训练步数下的表现视频，见 [这里](#)）。随着训练步数增加，该策略表现如何？请定性描述策略如何随时间变化，以及奖励函数中的哪些部分在各个阶段产生了最大的影响。

请注意，即使对于像二维箱子翻转这样简单的操作问题——只有一个点接触手指且奖励较为稠密——训练出一个可用策略也需要花费相当多的时间。更困难的操作问题（例如拾取与放置）如果天真地使用强化学习进行训练，可能会变得极具挑战性，尤其是在奖励稀疏的情况下，例如典型拾取与放置任务中，只有当物体被拾起或被放置到正确位置时才会获得奖励。另一方面，强化学习在接触丰富的场景中可能表现良好（如箱子翻转示例）；关于强化学习用于解决接触丰富操作任务的示例，可参见[用单手通过强化学习还原魔方](#)（注意，这也高度依赖领域随机化、课程学习、大规模计算等技术）。相比之下，运动控制中的情况似乎非常不同，也许是因为在仿真中更容易设计稠密奖励并自动完成重置。

REFERENCES

1. Richard S. Sutton and Andrew G. Barto, "Reinforcement Learning: An Introduction", MIT Press, 2018.
2. Alekh Agarwal and Nan Jiang and Sham M. Kakade and Wen Sun, "Reinforcement Learning: Theory and Algorithms", Online Draft, 2020.
3. Csaba Szepesvari, "Algorithms for Reinforcement Learning", Morgan and Claypool Publishers, 2010.

CHAPTER 12

软体机器人与触觉感知

到目前为止，在这些笔记中，我们一直假设相机和本体感觉（例如关节）传感器是我们获取世界信息的主要来源。还有另一种显而易见的传感器模态必须讨论——触觉。本章的第一个目标是探索近年来在传感硬件方面的进展，以及能够利用这些信息的计算框架。（听觉——即感知声音的能力——很可能是人类在操作任务中下一个最重要的感知模态，但目前这一领域仍相对缺乏探索。）

操作研究中的另一个重要趋势，是设计和制造本质上具有柔性的机器人。操作任务需要与环境发生丰富的接触交互。到目前为止我们讨论的大多数机器人都相对刚硬；但它们通常都会在指尖等部位使用类似橡胶垫的结构（这是我们传统上预期会与环境发生接触的唯一位置）。本章的第二个目标是探索软体机器人硬件方面的进展，以及能够处理软体机器人的计算框架。

这看起来像是两个可以分开的概念；那么为什么我要把它们放在同一章中讨论呢？事实证明，柔性能够促进触觉感知。甚至可以说，实现最丰富形式的触觉感知需要柔性结构。反过来，也可以认为，对于具有柔性表皮的机器人而言，触觉感知会成为一种自然的感知模态——它是本体感觉的自然延伸。因此，这两个主题之间存在着紧密联系！

12.1 为什么需要柔性？

12.2 软体机器人硬件

12.3 软体物体仿真

有限元法（FEM）、物质点法（MPM）等。

这里是一篇[近期论文的视频](#)，介绍了在 Drake 中实现高性能、可靠的 FEM 软体仿真的一些最新进展（包括与刚体的交互）^[1]。解说中有一句细微但重要的评论，提到接触问题在每一个时间步都会被求解直到收敛——这与游戏引擎级别的物理仿真形成了鲜明对比，后者为了性能通常会在计算上做大量近似和简化。令人惊讶的是，我们现在已经能够在单个 CPU 核心上以实时速度完成这一过程。

12.4 触觉感知

12.4.1 我们希望/需要获得什么信息？

12.4.2 视觉-触觉感知

12.4.3 全身感知

支持全身触觉皮肤的最有力理由之一，来自接触估计领域。通过一系列优秀的研究工作，我们已经相当清楚如何利用关节力矩传感来估计机器人手臂上发生接触的位置。但这个问题本质上是不适定的。特别是在存在多个接触点的情况下，仅依赖关节力矩传感作为传感方式显得远远不足

[2].

12.4.4 触觉传感器仿真

12.5 基于触觉传感器的感知

12.6 基于触觉传感器的控制

REFERENCES

1. Xuchen Han and Joseph Masterjohn and Alejandro Castro, "A Convex Formulation of Frictional Contact between Rigid and Deformable Bodies", *arXiv preprint arXiv:2303.08912*, 2023.
2. Tao Pang and Jack Umenberger and Russ Tedrake, "Identifying External Contacts from Joint Torque Measurements on Serial Robotic Arms and Its Limitations", *Under Review.*, May, 2021. [[link](#)]

APPENDIX A

空间代数

在这些笔记中，我们已经介绍了空间代数的基本规则。我发现自己经常会回过头来查阅它们！为了方便参考，我在这里再次将它们整理汇总。这些内容大量使用了我们的“[单字母记号法 \(monogram notation\)](#)”。

Drake 文档中也有一份非常好的总结，介绍了[多体系统中的量 \(multibody quantities\)](#)以及它们如何映射到单字母记号法。

A.1 位置、旋转与位姿

正如我们在[这里](#)介绍的那样，我们使用 ${}^B p_C^A$ 表示点或坐标系 A 相对于点或坐标系 B 的位置，并且该位置在坐标系 C 中表达。我们使用 ${}^B R^A$ 表示从坐标系 B 测量得到的坐标系 A 的朝向；与向量不同，纯旋转不需要额外的“在哪个坐标系中表达”的坐标系下标。类似地，我们使用 ${}^B X^A$ 表示从坐标系 B 测量得到的坐标系 A 的位姿/变换。对于位姿，我们不使用“表达于”的坐标系下标；我们始终希望位姿是在参考坐标系中表达的。

空间代数的基本规则如下：

- 当位置在同一个表达坐标系中表达，并且它们的参考符号与目标符号匹配时，可以相加：

$${}^A p_F^B + {}^B p_F^C = {}^A p_F^C. \quad (1)$$

加法满足交换律，并且其加法逆定义明确：

$${}^A p_F^B = -{}^B p_F^A. \quad (2)$$

这些规则应该都比较直观；请务必自己验证一下。

- 与旋转相乘可以用来改变“表达于”的坐标系：

$${}^A p_G^B = {}^G R^F A {}^B p_F^B. \quad (3)$$

你可能会惊讶，仅靠旋转就足以改变表达坐标系，但事实确实如此。表达坐标系的位置不会影响两个点之间的相对位置。

- 当旋转的参考符号与目标符号匹配时，旋转可以相乘：

$${}^A R^B {}^B R^C = {}^A R^C. \quad (4)$$

其逆运算也有简单定义：

$$\left[{}^A R^B \right]^{-1} = {}^B R^A. \quad (5)$$

当旋转被表示为旋转矩阵时，这实际上就是矩阵求逆；并且由于旋转矩阵是正交归一的，我们还有 $R^{-1} = R^T$ 。

- 当位置是相对于某个坐标系测量的（并且也在同一个坐标系中表达）时，变换将这些关系整合为一种统一而方便的记号：

$${}^G p^A = {}^G X^{FF} {}^F p^A = {}^G p^F + {}^F p_G^A = {}^G p^F + {}^G R^{FF} {}^F p^A. \quad (6)$$

- 变换可以复合：

$${}^A X^{BB} X^C = {}^A X^C, \quad (7)$$

并且存在逆变换：

$$\left[{}^A X^B \right]^{-1} = {}^B X^A. \quad (8)$$

请注意，对于变换而言，我们通常 **并不** 有 X^{-1} 等于 X^T ，尽管它仍然具有一种简单的形式。

A.2 空间速度

正如我们在[这里](#)介绍的那样，我们使用一个六维向量来表示位姿的变化率，即 **空间速度** (*spatial velocity*)：

$${}^A V_C^B = \begin{bmatrix} {}^A \omega_C^B \\ {}^A \mathbf{v}_C^B \end{bmatrix}. \quad (9)$$

${}^A V_C^B$ 表示坐标系 B 相对于坐标系 A 测量得到、并在坐标系 C 中表达的空间速度（也称为“twist”）； ${}^A \omega_C^B \in \mathbb{R}^3$ 表示 **角速度** (*angular velocity*)（即坐标系 B 相对于坐标系 A 的角速度，并在坐标系 C 中表达）；而 ${}^A \mathbf{v}_C^B \in \mathbb{R}^3$ 表示 **平移速度** (*translational velocity*)（其简写规则与位置向量相同）。空间速度能够很好地融入我们的空间代数体系：

- 当速度在同一个坐标系中表达时，可以相加：

$${}^A \mathbf{v}_F^B + {}^B \mathbf{v}_F^C = {}^A \mathbf{v}_F^C, \quad {}^A \omega_F^B + {}^B \omega_F^C = {}^A \omega_F^C, \quad (10)$$

并且存在加法逆元， ${}^A V_F^C = -{}^C V_F^A$ 。

- 旋转可以用来改变“表达于”的坐标系：

$${}^A \mathbf{v}_G^B = {}^G R^F {}^A \mathbf{v}_F^B, \quad {}^A \omega_G^B = {}^G R^F {}^A \omega_F^B. \quad (11)$$

- 平移速度在不同坐标系之间满足如下组合关系：

$${}^A \mathbf{v}_F^C = {}^A \mathbf{v}_F^B + {}^B \mathbf{v}_F^C + {}^A \omega_F^B \times {}^B p_F^C. \quad (12)$$

- 这表明，平移速度的加法逆元 并不能简单地通过交换参考坐标系与测量坐标系得到；它会稍微复杂一些：

$$-{}^A \mathbf{v}_F^B = {}^B \mathbf{v}_F^A + {}^A \omega_F^B \times {}^B p_F^A. \quad (13)$$

A.3 空间力

正如我们在[这里](#)介绍的那样，我们使用单字母记号法定义一个六维向量来表示 **空间力** (*spatial force*)：

$$F_{\text{name},C}^{B_p} = \begin{bmatrix} \tau_{\text{name},C}^{B_p} \\ f_{\text{name},C}^{B_p} \end{bmatrix} \quad \text{或者, 如果你更喜欢} \quad \left[F_{\text{name}}^{B_p} \right]_C = \begin{bmatrix} \left[\tau_{\text{name}}^{B_p} \right]_C \\ \left[f_{\text{name}}^{B_p} \right]_C \end{bmatrix}. \quad (14)$$

$F_{\text{name},C}^{B_p}$ 表示施加在点（或坐标系原点） B_p 上、并在坐标系 C 中表达的具名空间力。其中，“名称 (name)”是可选的；如果未指定表达坐标系，则默认是在世界坐标系中表达。特别对于力而言，建议我们在点 B_p 的符号中包含该力所施加到的刚体 B ，因为我们经常会遇到大小相等、方向相反的作用力与反作用力。

空间力能够整齐地融入我们的空间代数体系：

- 当空间力作用于同一个刚体、并在同一个坐标系中表达时，可以相加，例如：

$$F_{\text{total},C}^{B_p} = \sum_i F_{i,C}^{B_p}. \quad (15)$$

- 将空间力从一个作用点 B_p 平移到另一个点 B_q 时，需要使用叉乘：

$$f_C^{B_q} = f_C^{B_p}, \quad \tau_C^{B_q} = \tau_C^{B_p} + B_q p_C^{B_p} \times f_C^{B_p}. \quad (16)$$

- 与所有空间向量一样，旋转可以用来在不同的“表达于”坐标系之间进行转换：

$$f_D^{B_p} = {}^D R^C f_C^{B_p}, \quad \tau_D^{B_p} = {}^D R^C \tau_C^{B_p}. \quad (17)$$

APPENDIX B

Drake

DRAKE 是我们在本教材中使用的主要软件工具箱，而它事实上在很大程度上源自 MIT 的 [Underactuated Robotics \(欠驱动机器人学\)](#) 课程。**DRAKE** 官方网站是获取信息与文档的主要来源。本章的目标是提供一些额外的信息，帮助你顺利运行这些笔记中的示例与练习。

B.1 PYDRAKE

DRAKE 本质上是一个 C++ 库，具有严格的编码规范，并且成熟度足以支持工业界中的专业级应用。为了提供一个更平缓的入门体验，同时方便快速原型开发，我在这些笔记中完全使用 Python 编写，通过 Drake 的 Python 绑定 (pydrake) 来使用它。这些绑定相比 C++ 后端还不够成熟；非常欢迎你的反馈（甚至代码贡献）。它目前仍在快速改进之中。

特别是，虽然 C++ API 文档已经非常完善，但自动生成的 Python 文档仍然在持续建设中。我目前推荐先使用 [C++ 文档](#) 来查找所需内容，然后只有在需要了解某个类或方法在 pydrake 中的命名方式时，再查看 [Python 文档](#)。

DRAKE 中还提供了许多 [教程](#)，可以帮助你快速上手。

B.2 在线 JUPYTER NOTEBOOK

我会以 [Jupyter Notebook](#) 的形式提供几乎所有示例与练习，这样我们就能够利用当前非常优秀且相对新近出现的（免费）云计算资源。

B.2.1 在 Deepnote 上运行

我们将使用 Deepnote 作为本课程的主要平台。在点击章节中的任意链接之后，你需要：

1. 登录（免费账户已经足够用于本课程）
2. “Duplicate（复制）”该文档。该按钮位于右上角、Login 按钮旁边。
3. 运行所有单元格（可以使用该单元格上方的“Run notebook”图标）
4. 许多 notebook 使用 [MeshCat](#) 进行交互式可视化。点击“StartMeshcat”下方打印出的 URL（通常是 notebook 的第二个代码单元）即可打开 MeshCat 窗口。

B.2.2 启用商业求解器

如果你拥有许可证（大多数求解器对学术用户免费），你可以为 `MathematicalProgram` 启用更强大的求解器。请参阅 [这个教程 notebook](#) 获取相关说明。

B.3 在你自己的机器上运行

随着你逐渐深入学习，你很可能会希望在自己的机器上运行（并扩展）这些示例。在 [Drake 支持的平台 / 配置](#) 上（使用系统默认 Python 版本的最新两个 Mac 和 Ubuntu 版本），最简单的安装方式是通过 `pip`；我通常会推荐使用虚拟环境：

 复制代码

```
python3 -m venv venv
source venv/bin/activate
pip install manipulation[all] --extra-index-url https://drake-packages.csail.mit.edu/whl/nig
```

然后你可以直接从 Deepnote 下载 notebook，并在本地运行它们（或者开发你自己的 notebook）。

DRAKE 网站还提供了许多其他 [安装选项](#)，包括预编译二进制文件和 Docker 实例。如果你从源码构建（太棒了！），那么请确保同时构建 Python 绑定。最后，请确保 Drake 安装路径位于你的 `PYTHONPATH` 中。

如果你想一次性下载所有 notebook，可以 `git clone` [课程笔记仓库](#)。你很可能需要从该仓库的根目录开始操作。然后使用以下命令启动 notebook：

 复制代码

```
jupyter notebook
```

每个包含示例的章节，其示例都会放在一个 `.ipynb` 文件中，并与该章节的 `html` 文件位于同一目录；而 notebook 练习则都位于 `exercises` 子目录中。

B.4 获取帮助

如果你在使用 **DRAKE** 时遇到问题，请遵循[这里](#)的建议。如果你在使用 `manipulation` 仓库时遇到问题，可以在[这里](#)查看已知问题（也可以提交新的 issue）。

APPENDIX C

DrakeGym 环境

[DrakeGym](#) 可以轻松地将 Drake Systems 封装到 [Gymnasium](#) 环境中。这也许是探索 Drake 仿真的最简单方式，也是将 Drake 仿真与机器学习库（如 [Stable Baselines 3](#)）对接的自然方式。

我在这些笔记中整理了许多与操作相关的环境，并且会定期添加更多内容。

C.1 盒子翻起

...

APPENDIX D

搭建你自己的“操作实验平台 (Manipulation Station)”

仿真是一种极其强大的工具，而我们能够免费提供它这一事实，使其成为一种强有力的技术平权手段。就在几年前，还没有人相信你能够在仿真中开发一个操作系统，并期望它在现实世界中正常工作。然而如今，我们已经持续看到证据表明，我们的仿真保真度已经足够高：只要能够在仿真中实现**鲁棒**的性能，我们就有理由乐观地认为它能够迁移到现实世界。

当然，没有什么能够替代亲眼看到自己的想法/代码在真实物理机器人上“活起来”。本章的目标，是尽可能推动机器人硬件基础设施的商品化与普及，包括提供物料清单（bill of materials）、底层驱动程序，以及基础配置说明，帮助你快速搭建并运行属于你自己的操作实验平台。

D.1 消息传递

DRAKE 支持多种消息传递子系统（包括 ROS1），总体来说，我对你应该使用哪一种并没有特别强烈的偏好。我们在这门课程中使用 **LCM**，原因有两点：（1）它作为依赖项非常轻量（如果要求 **DRAKE** 强制依赖 ROS，将会非常痛苦且限制较多）；（2）对于机器人最低层级的通信，我们更偏好使用组播 UDP 而不是 TCP/IP。

因此，我们下面列出/提供的主要驱动程序都是基于 LCM 的。我会尽量在可能的情况下给出对应的 ROS 实现。如果你已经建立了成熟的 ROS 工作流，但仍希望使用我们的驱动程序，那么也可以使用简单的 lcm2ros/ros2lcm “桥接”可执行程序，例如将 LCM 消息发布到 ROS 话题上（用于日志记录等）。

D.2 KUKA LBR IIWA + SCHUNK WSG 夹爪

我们已经广泛使用 iiwa7 R800 和 iiwa14 R820 两种型号；**DRAKE** 为这两个模型都提供了 URDF 文件，并且驱动程序适用于两者。我们在本课程实验中使用 iiwa7，因为它体积稍小，而且我们并不需要较大的末端执行器负载能力。

- [IIWA LCM 驱动程序](#)

我们一直对 Schunk WSG 50 夹爪非常满意，它是一种可靠且精确的双指夹爪解决方案。

- [Schunk LCM 驱动程序](#)

注意：我们正在评估 Franka Panda 作为替代平台（并且很可能很快会在这里提供对应驱动程序）。

D.3 FRANKA PANDA

Franka Panda 系列机器人是一种非常流行的 iiwa 替代方案。我们使用一个通过 LCM 直接与其通信的驱动程序。

- [Franka Panda 驱动程序](#)

当然，[直接使用 ROS 驱动程序](#)也是可以的。

D.4 INTEL REALSENSE D415 深度相机

D415 一直是我们的首选相机，这是因为它的最小工作距离相对较好，并且当多台相机观察同一场景时，它们彼此之间不会相互干扰。

- [LCM 驱动程序](#)
- [相机 标定工具箱](#)。

APPENDIX E

Miscellaneous

E.1 HOW TO CITE THESE NOTES

Thank you for citing these notes in your work. Please use the following citation:

```
@book{manipulation,  
  title       = "Robotic Manipulation",  
  subtitle    = "Perception, Planning, and Control",  
  howpublished = "Course Notes for MIT 6.421",  
  author      = "Tedrake, Russ",  
  year        = 2024,  
  url         = "http://manipulation.mit.edu",  
}
```

E.2 ANNOTATION TOOL ETIQUETTE

My primary goal for the annotation tool is to host a completely open dialogue on the intellectual content of the text. However, it has turned out to serve an additional purpose: it's a convenient way to point out my miscellaneous typos and grammatical blips. The only problem is that if you highlight a typo, and I fix it 10 minutes later, your highlight will persist forevermore. Ultimately this pollutes the annotation content.

There are two possible solutions:

- you can make a public editorial comment, but must promise to delete that comment once it has been addressed.
- You can [join my "editorial" group](#) and post your editorial comments [using this group "scope"](#).

Ideally, once I mark a comment as "done", I would appreciate it if you can delete that comment.

I highly value both the discussions and the corrections. Please keep them coming, and thank you!

E.3 SOME GREAT FINAL PROJECTS

Each semester students put together a final project using the tools from this class. So many of them are fantastic! Here is a small sample that captures some of the diversity.

Fall 2023 Outstanding Project Awards:

- [OrderBot: Food Preparation Manipulation Bot for Fast Food Environments](#) by Shreya Chaudhary,
- [APPLE-E: A Compliant Apple Picking Robot](#) by Udayan Bulchandani,
- [Guitar Hero Bot: Motion Planning to Solve Rhythm Games with Robotic Manipulation](#) by Ryan Hourican, John Readlinger, and Noah Fisher,
- [An Autonomous Pool-Playing Robot](#) by Jinger Chong and Eugenia Feng,
- [Dreidel Spinning Bot: Exploring Contact Models in Simulation](#) by Lily Yeazell,
- [The Robot Librarian: Mobile Manipulation in a Library](#) by Shiva Sreeram and Peter Holderrieth,
- [CatchingBot: A Robotic System for Catching Objects in Flight](#) (including online estimation/replanning) by Bartłomiej Cieslar and Michael Zeng,
- [ShelveBot: Fast Pick-and-Place in Human Form Factors](#) by Julianna Schneider and Shayan Pardis and Anna Yang.

And [here is a playlist](#) of all the 2023 projects which opted in to being public.

Fall 2022 Outstanding Project Awards ([playlist](#)):

- [ChessBot](#) by Ethan Chung,
- [Rock Skipping Robot](#) by Michael Burgess and Nicholas Ramirez,
- [How we made a robot arm juggle!](#) by Richard Li, David Jin, and Shao Yuan Chew Chia,
- [SpeedCuber Bot](#) by Bin Pham,
- [Michaelangelo - Robot Painter 2.0](#) by Vainavi Mukkamala and Akila Saravanan,
- [Ping Pong Bot 2023](#) by Jenny Zhang and Daniel Klahn,
- [TetrisBot](#) by Pranav Arunandhi, Ashley Ke, and Sadhana Lolla,
- [Multiple-finger Grasping: Generating Antipodal Grasping with Allegro Hand](#) by Yuxiang Ma,
- [Exploring 6DOF Pose Estimation Approaches Using RGB\(D\)](#) by Amin Heyrani Nobari.

Fall 2021:

- [YouTube playlist](#) containing all projects that opted in.
- [Playing Piano with a Robotic Hand](#) by Seong Ho Yeon
- [Table Cleaning through Task and Motion Planning with Force Control](#) by William Shen
- [RallyBot: Exploring Physics-Based Approaches to Robotic Table Tennis](#) by Dylan Zhou and Chaitanya Ravuri
- [Robust Stacking of Convex Polytopes](#) by Richard Li and Gabriel Margolis
- [Simulation of Discrete Lattice Robot Assemblies](#) by Miani Smith
- [Da Vinci Robot](#) by Maisy Lam and Jose A. Muguira
- [Certifying Convex Decompositions of Free Configuration Space](#) by Alexandre Amice, Peter Werner, Annan Zhang
- [Robotic Arm Weightlifting via Trajectory Optimization](#) by Timur Garipov
- [An Efficient Implementation of Pressure Field Models](#) by Vincent Huang, Franklyn Wang, Eric Zhang
- [Joint Perception and Manipulation System for Multi-Shape Peg-in-Hole Tasks](#) by Portia Gaitskell and Julia Wyatt

Fall 2020:

- [Solving Simple Tiling Puzzles with an End-to-End Robotic System](#) by Anubhav Guha
- [Throwing in Drake](#) by Daniel Yang and Tony Wang
- [Robot Juggler](#) by Ryan Shubert and Matt Beveridge
- [A Geometric Approach to Recycling Cans via Vision-Based Manipulation](#) by Susan Ni and George Chen
- [Interactive Perception: Scene Segmentation Through Physical Perturbation](#) by Lukas Lao Beyer and John Keszler
- [William Shakebot](#) by Carson Smithbot and Clemente Ocejó

E.4 PLEASE GIVE ME FEEDBACK!

I'm very interested in your feedback. The annotation tool is one mechanism, but you can also comment directly on the YouTube lectures, or even add issues to the [github repo](#) that hosts these course notes.

I will also post a proper survey questionnaire here once the notes/course has existed long enough for that to make sense.